

rubrapack 매뉴얼

MSI·MSIX 패키지를 만들고 서명하기 - 그리고 그 형식이 어떻게 짜였는가

판 0.7.0

<https://github.com/rubidus-api/rubrapack>
<https://rubidus-api.github.io/rubrapack/>

차례

제1부 - 따라 하며 배우기

1. 시작하기 전에	2
1.1 설치 파일이 하는 일	2
1.2 무엇을 쓰고 무엇이 만들어지나	2
1.3 rubrapack 받기	3
1.4 터미널을 처음 쓰는 사람을 위해	3
1.5 작업 폴더	3
1.6 이 튜토리얼에서 쓰는 말	3
2. 첫 설치 파일: 프로그램 하나	5
2.1 원본	5
2.2 빌드	6
2.3 설치하고 지우기	6
2.4 안에서 무슨 일이 일어났나	6
2.5 빌드하지 않고 검사하기	7
2.6 편집기 대신 질문으로	7
3. 판과 업그레이드	8
3.1 모든 판에 원본 하나	8
3.2 Windows 는 두 판을 어떻게 다루나	8
3.3 판 번호	9
3.4 두 식별자	9
3.5 사용자보다 먼저 업그레이드를 확인하기	9
3.6 다른 도구로 만든 옛 판	10
4. 더 많은 파일과 폴더	11
4.1 폴더	11
4.2 원본	11
4.3 폴더: [dir.ID]	12
4.4 파일 하나: [file.ID]	13
4.5 여러 파일: [files.ID]	13
4.6 빈 폴더: [folder.ID]	13
4.7 치우기: [remove.ID]	13
4.8 두 번째 사본: [copy.ID]	13
4.9 만날 수 있는 키 두 개 더	13
4.10 안에서 무슨 일이 일어났나	14
5. 바로가기, 아이콘, "설치된 앱" 항목	15
5.1 원본	15
5.2 바로가기: [shortcut.ID]	16
5.3 "설치된 앱" 항목: [arp]	16
5.4 다른 글자로 된 이름	16
5.5 해 보기	16
5.6 안에서 무슨 일이 일어났나	16
6. 대화창, 약관, 그리고 프로그램 시작하기	18
6.1 원본	18
6.2 페이지 세트 고르기: ui	19
6.3 약관: license	19
6.4 설치 폴더	19
6.5 위쪽의 그림: banner	19
6.6 내 말로: [ui-text.ID]	19
6.7 끝에 프로그램 시작하기: launch	19
6.8 해 보기	20

6.9 안에서 무슨 일이 일어났나	22
7. 여러 언어	23
7.1 원본	23
7.2 언어 더하기: <code>languages</code>	24
7.3 언어마다 약관: <code>license-xx</code>	24
7.4 언어마다 글: <code>text-xx</code>	24
7.5 영어와 한국어 말고 다른 언어	24
7.6 번역되지 않는 것	25
7.7 해 보기	25
7.8 안에서 무슨 일이 일어났나	25
8. 선택 구성요소: 기능과 사용자의 선택	26
8.1 기능	26
8.2 원본	26
8.3 기능 선언하기: <code>[feature.ID]</code>	27
8.4 파일을 기능에 넣기: <code>feature</code>	28
8.5 사용자가 보는 것	28
8.6 명령줄에서 고르기	28
8.7 나중에 바꾸기	29
8.8 안에서 무슨 일이 일어났나	29
9. 조건과 검색	30
9.1 속성	30
9.2 원본	30
9.3 내 속성: <code>[property.NAME]</code>	31
9.4 이럴 때만 설치하기: <code>when</code>	32
9.5 설치 거부하기: <code>[require.ID]</code>	32
9.6 먼저 둘러보기: <code>[search.ID]</code>	32
9.7 설치 폴더 기억하기	32
9.8 안에서 무슨 일이 일어났나	32
10. 누구를 위해 설치하나, 그리고 보호되는 폴더	34
10.1 세 가지 범위	34
10.2 원본	34
10.3 겸용(dual) 패키지	35
10.4 보호되는 설치 폴더: <code>guard</code>	35
10.5 해 보기	35
10.6 안에서 무슨 일이 일어났나	35
11. 레지스트리, 환경 변수, INI 파일, 파일 형식과 링크	37
11.1 2분 만에 보는 레지스트리	37
11.2 원본	37
11.3 레지스트리 값: <code>[registry.ID]</code>	39
11.4 환경 변수: <code>[env.ID]</code>	40
11.5 INI 파일: <code>[ini.ID]</code>	40
11.6 파일 형식: <code>[assoc.ID]</code>	40
11.7 링크: <code>[protocol.ID]</code>	40
11.8 해 보기	40
11.9 안에서 무슨 일이 일어났나	41
12. 서비스, 글꼴, 권한	42
12.1 원본	42
12.2 서비스: <code>[service.ID]</code>	43
12.3 글꼴: <code>[font.ID]</code>	43
12.4 접근 권한: <code>[permission.ID]</code>	43
12.5 다시 시작하기: <code>reboot</code>	44

12.6 해 보기	44
12.7 안에서 무슨 일이 일어났나	44
13. 설치하는 동안 내 프로그램 실행하기	45
13.1 왜 그냥 프로그램을 실행하면 안 되나?	45
13.2 원본	45
13.3 [action.ID]	45
13.4 무엇이 언제 도나	46
13.5 해 보기	46
13.6 안에서 무슨 일이 일어났나	46
14. 나만의 대화창 페이지	47
14.1 원본	47
14.2 페이지: [dialog.ID]	49
14.3 페이지 위의 자리	50
14.4 컨트롤: [dialog-control.ID]	50
14.5 모든 값에는 기본값이 있어야 한다: [property.NAME]	50
14.6 해 보기	50
14.7 안에서 무슨 일이 일어났나	51
15. 캐비닛, 압축, 큰 패키지, 여러 아키텍처	52
15.1 폴더	52
15.2 원본	52
15.3 캐비닛과 압축	53
15.4 여러 아키텍처: --arch	53
15.5 매번 같은 바이트: --reproducible	53
15.6 Mac 에서 온 이름: --nfc	54
15.7 명령줄의 변수: -D	54
15.8 안에서 무슨 일이 일어났나	54
16. 서명과 타임스탬프	55
16.1 서명이란	55
16.2 키 파일로 서명하기	55
16.3 타임스탬프: 인증서보다 오래 사는 서명	56
16.4 Windows 인증서 저장소의 키	56
16.5 하드웨어 토큰: PKCS#11	56
16.6 서명 확인하기: verify	57
16.7 외부 캐비닛은 서명되지 않는다	57
16.8 안에서 무슨 일이 일어났나	57
17. MSIX 패키지와 번들	58
17.1 MSI 인가 MSIX 인가?	58
17.2 폴더	58
17.3 원본	59
17.4 패키지의 정체: [msix]	60
17.5 애플리케이션: [msix-app.ID]	60
17.6 덤: [msix-extension.ID]	60
17.7 다른 표들은 무엇이 되나	60
17.8 시험용 빌드: --unsigned-test	61
17.9 서명하기	61
17.10 모든 아키텍처를 파일 하나에: 번들	61
17.11 압축	61
17.12 안에서 무슨 일이 일어났나	62
18. 검사하고 들여다보기	63
18.1 원본 검사: lint	63
18.2 패키지 검사: lint <파일>	64

18.3 표 보기: <code>inspect</code>	64
18.4 파일과 스트림	65
18.5 설치하지 않고 풀기: <code>extract</code>	65
18.6 서명: <code>verify</code>	66
18.7 MSIX 패키지	66
18.8 더 빠른 시험 빌드: <code>--compress none</code>	66
18.9 GUID: <code>guid</code>	66
18.10 명령줄의 도움말	66
19. 빨리 시작하기와 자동화	68
19.1 답으로 원본 만들기: <code>new</code>	68
19.2 원본 고치기: <code>edit</code>	69
19.3 릴리스 스크립트	70
19.4 빌드 서버에서	70
19.5 AI 도우미와 함께	70
19.6 여기서 어디로	70

제2부 - 참조

20. 원본 파일과 <code>rubrapack</code> 명령	73
20.1 시작하기	74
20.2 예	76
20.3 TOML 부분집합	76
20.4 표	77
20.5 명령줄	87
20.6 진단 코드	90

제3부 - 기초 지식

21. 비트, 바이트, 16진수	92
21.1 비트	92
21.2 2진수	92
21.3 바이트	92
21.4 16진수	93
21.5 16진 덤프 읽기	93
21.6 단위: KB, MB 와 KiB, MiB	94
21.7 쓰이는 곳	94
22. 파일 속의 수, 그리고 비트 플래그	95
22.1 1, 2, 4, 8 바이트	95
22.2 바이트 순서: <code>little-endian</code>	95
22.3 음수: 2의 보수	96
22.4 비트 플래그	96
22.5 비트 확인하기와 켜기: AND, OR	97
22.6 쓰이는 곳	97
23. 글자: 문자, 코드 포인트, 인코딩	98
23.1 문자와 코드 포인트	98
23.2 ASCII	98
23.3 코드 페이지: 한 바이트, 256 문자	99
23.4 UTF-8	99
23.5 UTF-16	99
23.6 바이트 순서 표시	100
23.7 유니코드 정규화	100
23.8 글자가 이상하게 보일 때	100
23.9 쓰이는 곳	100
24. GUID 와 해시	102

24.1 GUID	102
24.2 해시	103
24.3 해시로 만드는 GUID: <code>guid --from</code>	104
24.4 쓰이는 곳	104
25. 압축	105
25.1 데이터를 줄일 수 있는 까닭	105
25.2 LZ77: "앞에서 베껴 와라"	105
25.3 허프만 부호: 흔한 기호에 짧은 부호	106
25.4 deflate, 비트 하나하나	106
25.5 MSZIP: 캐비닛 속의 deflate	107
25.6 MSIX 속의 deflate	107
25.7 쓰이는 곳	107
26. 파일 속의 파일	108
26.1 디스크가 파일을 저장하는 방법	108
26.2 MSI 는 파일 속의 파일 시스템이다	108
26.3 MSIX 는 묶음 파일이다	109
26.4 왜 두 설계인가?	110
26.5 쓰이는 곳	110
27. 표, 키, 참조	111
27.1 표, 행, 열	111
27.2 기본 키: 행을 하나로 만드는 것	111
27.3 외래 키: 행을 가리키는 행	112
27.4 내가 주는 키, rubrapack 이 만드는 키	112
27.5 문자열은 한 번만: 문자열 풀	112
27.6 표에 대한 표	112
27.7 IDT 텍스트 형식	113
27.8 쓰이는 곳	113
28. 키, 인증서, 서명	114
28.1 열쇠 하나 대신 둘	114
28.2 서명하기와 확인하기	114
28.3 인증서: 이것은 누구의 키인가?	115
28.4 사슬과 신뢰	115
28.5 타임스탬프	116
28.6 키가 사는 곳: 파일과 토큰	116
28.7 쓰이는 곳	116

제4부 - 파일 형식

29. 구현하는 사람을 위한 패키지 형식	118
29.1 사실을 읽는 법	118
29.2 표기	119
29.3 흐름을 한 그림으로	119
30. 복합 파일(Compound File Binary, CFB)	120
30.1 섹터	120
30.2 머리(처음 512바이트) [spec]	120
30.3 FAT, DIFAT, 미니 FAT [spec]	121
30.4 디렉터리 [spec]	121
30.5 단순하고 결정적인 작성기	122
30.6 안전하게 읽기	122
30.7 실제 예: 튜토리얼의 hello.msi	122
31. 복합 파일 속의 MSI 데이터베이스	124
31.1 스트림	124

31.2 문자열 풀: <code>_StringPool</code> 과 <code>_StringData</code>	124
31.3 시스템 표	125
31.4 표 스트림	126
31.5 IDT 보관 파일(<code>MsiDatabaseExport</code> 가 쓰는 것)	126
31.6 안전하게 읽기	126
31.7 실제 예: 튜토리얼의 <code>hello.msi</code>	126
32. 요약 정보	128
32.1 배치 [spec] [observed]	128
32.2 설치 패키지가 정하는 속성	128
32.3 코드 페이지 함정 - 요약 문자열은 ASCII 로 [observed]	129
32.4 실제 예: 튜토리얼의 <code>hello.msi</code>	129
33. 설치·복구·업그레이드·제거가 되는 가장 작은 패키지	130
33.1 표와 열	130
33.2 Property	131
33.3 Directory	131
33.4 구성 요소, 기능, 파일	131
33.5 빈 폴더	131
33.6 Media 와 캐비닛	132
33.7 메이저 업그레이드와 다운그레이드 거부	132
33.8 설치한 프로그램으로 등록·해제하기	132
33.9 레지스트리 값	132
33.10 바로가기	133
33.11 파일 지우기와 복제	133
33.12 환경 변수	133
33.13 INI 파일	133
33.14 검색과 설치 조건	134
33.15 서비스, 글꼴, 권한	134
33.16 캐비닛, 관리 이미지, 광고	134
33.17 사용자별 패키지와 겸용 패키지	134
33.18 사용 중인 파일	134
33.19 순서	135
33.20 대화창	135
33.21 이 조리법이 아직 다루지 않는 것	136
34. 프로그램 파일: 머신 형식과 판	137
34.1 머리 찾기 [spec]	137
34.2 판 정보 리소스 [spec]	137
34.3 패키지에 드는 것 [observed]	137
34.4 안전하게 읽기	138
35. 캐비닛, MSZIP, deflate	139
35.1 캐비닛 배치 [spec: MS-CAB]	139
35.2 MSZIP [spec: MS-MCI]	139
35.3 deflate 인코더 간단히 [spec: RFC 1951]	140
35.4 실제 예: 튜토리얼의 <code>hello.msi</code>	140
36. 결정적인 정체와 재현 가능한 패키지	141
36.1 이름에서 만드는 GUID(UUID 8판) [spec: RFC 9562]	141
36.2 키	141
36.3 재현 가능한 패키지	141
37. Authenticode 서명	142
37.1 PE 파일에서 서명이 사는 곳	142
37.2 PE 다이제스트	142
37.3 CMS SignedData	142

37.4 MSI 패키지	143
37.5 타임스탬프	144
37.6 ECDSA	144
37.7 Windows 가 알려 주는 것	144
37.8 실제 예: 튜토리얼의 hello.msi	144
38. 출력을 Windows 와 대조하기	146
38.1 데이터베이스가 열리고 풀린다	146
38.2 패키지가 설치된다	146
38.3 쓰기 전에 하는 검사	147
38.4 Microsoft 의 ICE 규칙 [관찰]	147
38.5 멀쩡해 보이지만 아닌 것	147
39. MSIX 패키지	148
39.1 ZIP 압축 파일	148
39.2 블록 맵(AppxBlockMap.xml)	148
39.3 [Content_Types].xml	149
39.4 매니페스트(AppxManifest.xml)	149
39.5 가상 레지스트리(Registry.dat, User.dat)	149
39.6 가상 파일 시스템(VFS\...)	150
39.7 확장	150
39.8 묶음(.msixbundle)	151
39.9 서명(AppxSignature.p7x)	151
39.10 서명 없는 패키지 설치하기	152
39.11 실제 예: 튜토리얼의 hello.msix	152
40. 레지스트리 하이브 파일(REGF)	154
40.1 배치	154
40.2 기본 블록	154
40.3 키(nk)	155
40.4 하위 키 목록(lh, ri)	155
40.5 값(vk)과 값 목록	155
40.6 보안 칸(sk)	155
40.7 재현 가능한 하이브	155

제1부

따라 하며 배우기

1 시작하기 전에

이 장의 차례

1.1	설치 파일이 하는 일	2
1.2	무엇을 쓰고 무엇이 만들어지나	2
1.3	rubrapack 받기	3
1.4	터미널을 처음 쓰는 사람을 위해	3
1.5	작업 폴더	3
1.6	이 튜토리얼에서 쓰는 말	3

매뉴얼의 이 부분은 따라 하며 배우는 튜토리얼이다. 가장 작은 설치 파일 - 프로그램 하나를 Program Files 에 복사하는 것 - 에서 시작해, 장마다 하나씩 더해 가며 끝에는 rubrapack 의 모든 기능을 한 번씩 써 본다. 장마다 앞 장에 이어서 쌓아 가므로 차례대로 읽는 것이 좋다. 컴퓨터를 쓸 줄 안다는 것 말고는 아무것도 가정하지 않는다. 바깥의 개념(GUID, 해시, 레지스트리 키 같은 것)이 필요하면 그 자리에서 설명하거나 "기초 지식" 부를 가리킨다.

1.1 설치 파일이 하는 일

내 컴퓨터에 있는 프로그램은 그저 폴더 안의 파일들이다. 이것을 다른 사람에게 주려면 폴더를 그대로 보내도 된다 - 하지만 그러면 받는 사람이 둘 곳을 정하고, 시작 메뉴 항목을 직접 만들고, 나중에는 모든 것을 손으로 찾아 지워야 한다. 설치 파일이 그 일을 대신한다:

- 파일을 알맞은 자리(보통 C:\Program Files\<프로그램>)에 복사한다.
- 바로가기, 레지스트리 값, 파일 형식 연결 등 프로그램이 필요로 하는 것을 만든다.
- 설정 > 앱 > 설치된 앱에 프로그램을 등록해, 거기서 지울 수 있게 한다.
- 2판을 내면 1판을 깨끗하게 바꿔 넣는다.
- 도중에 무언가 잘못되면 모든 것을 원래대로 되돌린다.

Windows 에는 이 일을 하는 엔진이 들어 있다: **Windows Installer**. 설치하는 프로그램을 짜는 것이 아니라, 무엇을 설치할지 기술한 파일인 패키지를 만들면 Windows Installer 가 그대로 실행한다. 패키지 형식은 둘이다:

형식	파일	설치하는 것	주로 쓰는 곳
Windows Installer	.msi	Windows Installer(msiexec), Windows 2000 부터	모든 종류의 데스크톱 프로그램; 회사는 관리 도구로 MSI 를 배포한다
MSIX	.msix, .msixbundle	Windows 10·11 의 앱 설치 관리자	새 방식의 패키지 앱, Microsoft Store

rubrapack 은 같은 기술에서 둘 다 만든다. 튜토리얼은 대부분 MSI 를 다루고, 17장에서 같은 예제를 MSIX 로 바꾼다.

1.2 무엇을 쓰고 무엇이 만들어지나

짧은 글 파일 하나, 원본을 TOML 로 쓴다 - 키 = 값 꼴의 간단한 형식이다. rubrapack 이 그것을 읽어 패키지를 쓴다:

```
hello.toml (내가 쓴다)
|
|   rubrapack build hello.toml -o hello.msi
v
```

```
hello.msi (사용자에게 준다)
|
|   두 번 누르기, 또는: msixexec /i hello.msi
V
C:\Program Files\Hello\hello.exe (Windows Installer 가 거기 둔다)
```

MSI 파일은 그 자체로 표 수집 개와 식별자와 압축 보관 파일이 든 작은 데이터베이스다 - 이 책의 제4부가 그것을 바이트 단위로 보여 준다. 그 표를 직접 쓸 일은 없다: rubrapack 이 원본에서 알아서 만든다.

1.3 rubrapack 받기

rubrapack 은 따로 설치할 것 없는 프로그램 파일 하나다. 릴리스 페이지 <https://github.com/rubidus-api/rubrapack/releases> 에서 받는다:

- Windows: rubrapack-<판>-windows-x64.exe. 이름을 rubrapack.exe 로 바꾼다.
- Linux: rubrapack-<판>-linux-x86_64. 이름을 rubrapack 으로 바꾸고 실행할 수 있게 한다: `chmod +x rubrapack`.

PATH 에 들어 있는 폴더에 두면(어디서나 명령이 된다) 편하고, 작업 폴더에 그냥 두었다면 rubrapack 대신 .\rubrapack.exe(Windows)나 ./rubrapack(Linux)으로 친다. 이 튜토리얼은 그냥 rubrapack 으로 쓴다.

패키지는 Linux 에서도 만들 수 있지만, 설치해 보려면 Windows 가 있어야 한다. Windows 가상 머신이면 충분하다.

1.4 터미널을 처음 쓰는 사람을 위해

rubrapack 에는 창이 없다: 명령을 쳐서 쓴다. Windows 에서 Windows 키를 누르고 terminal(또는 cmd)을 쳐서 Enter 를 누르면 프롬프트가 있는 창이 열린다. 예를 들어:

```
C:\Users\you>
```

> 앞의 부분이 지금 폴더다. 경로를 주지 않으면 명령은 그 폴더의 파일을 대상으로 한다. 필요한 명령은 셋이다:

명령	하는 일
<code>cd C:\work\hello</code>	C:\work\hello 폴더로 들어간다
<code>dir (Linux: ls)</code>	지금 폴더의 파일을 보인다
<code>mkdir dist</code>	여기에 dist 폴더를 만든다

해 보자: rubrapack version 은 판을 찍고, rubrapack help 는 명령 목록을 보인다.

```
C:\work\hello> rubrapack version
rubrapack 0.7.0 (proven_c_lib-v0.1.1)
```

1.5 작업 폴더

모든 장이 같은 배치를 쓴다. 프로젝트용 폴더를 만들고, 그 안에 배포할 파일만 담은 dist("distribution", 배포) 폴더를 둔다:

```
C:\work\hello\
  dist\
    hello.exe   프로그램 - 자기 프로그램 아무것이나 된다
    hello.toml  원본(다음 장에서 쓴다)
```

마땅한 프로그램이 없으면 아무 .exe 로 배워도 된다. 설치 파일은 그것을 실행하지 않는다.

1.6 이 튜토리얼에서 쓰는 말

말	뜻
패키지	사용자에게 주는 .msi(또는 .msix) 파일

말	뜻
원본	내가 쓰는 .toml 파일; rubrapack 이 여기서 패키지를 만든다
설치 폴더	사용자 컴퓨터에서 파일이 들어갈 곳, 예: C:\Program Files\Hello
제품	Windows 가 보는 내 프로그램: 이름, 판, 제조사
업그레이드 코드	제품의 모든 판을 하나로 묶는 고정된 식별자(2장)
기능(feature)	사용자가 설치할지 말지 고를 수 있는 제품의 한 부분(8장)
컴포넌트 (component)	Windows Installer 의 설치 단위; rubrapack 이 알아서 만든다

2 첫 설치 파일: 프로그램 하나

이 장의 차례

2.1	원본	5
2.2	빌드	6
2.3	설치하고 지우기	6
2.4	안에서 무슨 일이 일어났나	6
2.5	빌드하지 않고 검사하기	7
2.6	편집기 대신 질문으로	7

목표: hello.exe 를 C:\Program Files\Hello 에 복사하고, "설치된 앱"에 나타나고, 깨끗하게 지워지는 설치 파일.

2.1 원본

C:\work\hello 폴더(앞 장 참고)에 아무 편집기로 - 메모장이면 된다 - hello.toml 이라는 글 파일을 만들고 이렇게 쓴다:

```
# tutorial 02: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "1.0.0"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

한 줄씩 보면:

- # 으로 시작하는 줄은 주석이다. rubrapack 은 무시한다.
- [package] 는 표를 시작한다: 그 뒤의 줄은 다음 [...] 까지 이 표에 속한다. package 표는 제품이 무엇인지 말한다.
 - ▶ name 은 사용자가 보는 이름이다: "설치된 앱"에서, 설치 창의 제목에서.
 - ▶ manufacturer 는 나나 회사다. "설치된 앱"이 이름 아래에 보인다.
 - ▶ version 은 수 셋(또는 넷)이다: 주판, 부판, 빌드. 3장이 판 바꾸기를 다룬다.
 - ▶ arch 는 프로그램이 만들어진 프로세서다: x64(요즘 거의 모든 PC), x86(32비트 프로그램), arm64. rubrapack 은 hello.exe 가 정말 x64 프로그램인지 확인한다.
 - ▶ upgrade-code 는 GUID - 16진수 32자리를 중괄호로 쓴 128비트 수 - 로, 내 제품에만 있는 것이다. rubrapack guid 로 자기 것을 만들어(아래) 여기에 한 번 쓰고 **절대 바꾸지 않는다**. 이후 모든 판이 같은 코드를 지녀야 한다. 그러지 않으면 Windows 가 다른 제품으로 보고 옛것 옆에 따로 설치한다.
- [dir.INSTALLDIR] 는 ID 가 INSTALLDIR 인 폴더를 선언한다. ID 는 다른 표에서 그것을 가리키려고 내가 정하는 이름이다(영문자, 숫자, _). path 는 폴더가 어디인지 말한다: ProgramFiles/Hello 는 영어판이든 한국어판이든 Windows 에서 C:\Program Files\Hello 다 - 알려진 폴더 ProgramFiles 는 사용자 컴퓨터에서 정해진다.

- `[file.Hello]` 는 파일 하나를 ID `Hello` 로 설치한다. `dir` 은 들어갈 폴더를 ID 로 가리키고, `source` 는 지금 파일이 있는 곳으로, `hello.toml` 기준의 상대 경로를 `/` 로 쓴다.

자기 업그레이드 코드를 받는다 - 칠 때마다 새로 나온다:

```
C:\work\hello> rubrapack guid
{F1B1ED21-3061-4A65-8D77-FBB5ACCB1642}
```

예제의 코드 자리에 붙여 넣는다. (예제의 코드를 그대로 쓰면 Windows 가 내 제품과 이 책을 읽은 다른 사람의 제품을 "같은 제품"으로 본다.)

2.2 빌드

```
C:\work\hello> rubrapack build hello.toml -o hello.msi
```

잘되면 아무것도 찍지 않고, 원본 옆에 `hello.msi`(약 28 KB)가 생긴다. 무언가 틀리면 `rubrapack` 은 어디가 왜 틀렸는지 말하고 아무것도 쓰지 않는다. 예를 들어 `source` 에 오타가 있으면:

```
hello.toml:14:1: error[RP1507]: source file 'dist/hello.ex' not found
```

수는 줄과 열이고, 대괄호 안의 코드(`RP1507: RP15xx` 는 "설치할 파일")는 참조 부의 진단 코드가 설명한다. 그 줄을 고치고 다시 빌드한다.

2.3 설치하고 지우기

`hello.msi` 를 Windows 컴퓨터에 복사해 두 번 누른다. 모든 사용자의 것인 Program Files 에 설치하므로 Windows 가 관리자 허락을 묻는다(사용자 계정 컨트롤 창). 그다음 진행 막대가 있는 작은 창이 잠깐 나타난다 - 이 패키지에는 아직 자기 대화창이 없다(6장에서 더한다). 끝나면:

- `C:\Program Files\Hello\hello.exe` 가 있다.
- 설정 > 앱 > 설치된 앱에 **Hello**, 판 1.0.0, Example Software 가 보인다.

"설치된 앱"에서(... 메뉴, 제거) 지우거나 터미널에서 지운다:

```
msiexec /x hello.msi
```

파일과 폴더가 다시 사라진다. 같은 패키지를 창 없이 설치할 수도 있는데, 회사가 소프트웨어를 배포하는 방식이다(`/qn` 은 "조용히, 사용자 화면 없이"라는 뜻; 터미널을 관리자로 연다):

```
msiexec /i hello.msi /qn
msiexec /x hello.msi /qn
```

Windows 에서 무언가 잘못되면 기록이 이유를 알려 준다: `msiexec /i hello.msi /l*v install.log` 는 Windows Installer 가 한 일을 모두 `install.log` 에 쓴다.

2.4 안에서 무슨 일이 일어났나

`rubrapack inspect` 는 설치하지 않고 패키지 안을 보여 준다. `--files` 는 파일과 그 자리를 보인다:

```
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello C_185f8db32271fe25f561 1.2.3.4 1033
```

`ProgramFiles64Folder` 는 64비트 Program Files 의 Windows Installer 이름이고, `17920` 은 바이트 단위 크기다. `1.2.3.4` 와 `1033`(미국 영어)은 `hello.exe` 안에 적힌 판과 언어다(Windows 는 파일을 바꿔 넣을 때 이것을 비교한다). MSI 는 데이터베이스이고 `inspect` 는 어떤 표든 찍을 수 있다. `File` 표에는 파일 하나에 행 하나가 있다:

```
C:\work\hello> rubrapack inspect hello.msi File
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 l255 i4 S72 S20 I2 i4
File File
Hello C_185f8db32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 1
```

첫 줄은 열 이름, 둘째 줄은 열의 형식(s72 는 72자까지의 문자열, i4 는 4바이트 정수, 대문자는 비어 있어도 되는 열), 셋째 줄은 표 이름과 키 열이다. Hello 는 내 파일 ID, C_185f... 는 rubrapack 이 이 파일을 위해 만든 컴포넌트, 512 는 "필수"(이 파일을 쓰지 못하면 설치가 실패한다)라는 뜻이다. 제품의 정체는 Property 표에 있다:

```
C:\work\hello> rubrapack inspect hello.msi Property
...
ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
ProductName Hello
ProductVersion 1.0.0
UpgradeCode {3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}
...
```

ProductCode 는 이 판을 가리키는 또 하나의 GUID 다. rubrapack 이 업그레이드 코드, 아키텍처, 판에서 끌어내므로 판이 바뀌면 저절로 바뀐다. 직접 쓰지 않는다. 이 표들이 파일에 어떻게 저장되는지는 제4부가 설명한다.

2.5 빌드하지 않고 검사하기

rubrapack lint hello.toml 은 빌드가 하는 검사를 모두 하되 아무것도 쓰지 않는다 - 고친 뒤 확인하기 좋다. rubrapack lint hello.msi 는 다 만든 패키지를 검사하며, 다른 도구로 만든 것도 된다:

```
C:\work\hello> rubrapack lint hello.msi
hello.msi: 0 errors, 0 warnings
```

2.6 편집기 대신 질문으로

rubrapack new hello.toml 은 이름, 판, 파일이 든 폴더 등을 물은 뒤 위와 같은 원본을 쓴다(업그레이드 코드도 만든다). 19장이 이 명령과 짝인 rubrapack edit 를 설명한다. 여기서처럼 원본을 한 번 손으로 써 보는 것이 각 줄이 하는 일을 익히는 가장 좋은 방법이다.

3 판과 업그레이드

이 장의 차례

3.1	모든 판에 원본 하나	8
3.2	Windows 는 두 판을 어떻게 다루나	8
3.3	판 번호	9
3.4	두 식별자	9
3.5	사용자보다 먼저 업그레이드를 확인하기	9
3.6	다른 도구로 만든 옛 판	10

목표: Hello 1.1.0 을 내서, 설치하면 1.0.0 을 바꿔 넣고, 그 뒤에 1.0.0 을 다시 설치하려 하면 거부되게 한다.

3.1 모든 판에 원본 하나

릴리스 전마다 version 을 손으로 고쳐도 되지만 잊기 쉽다. 대신 판을 기본값이 있는 변수로 두고, 빌드할 때 진짜 번호를 준다:

```
# tutorial 03: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
downgrade-message = "더 새 Hello 가 이미 설치되어 있습니다. 이 판을 원하면 먼저 그것을 지우십시오."

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

- [define] 에는 변수를 둔다. 문자열 안 어디서든 \${VERSION} 은 변수 값으로 바뀐다. 여기서 기본값은 1.0.0 이다.
- 명령줄의 -D 이름=값 은 그 빌드 한 번에서 변수를 덮어쓴다:

```
C:\work\hello> rubrapack build hello.toml -o hello-1.0.0.msi
C:\work\hello> rubrapack build hello.toml -o hello-1.1.0.msi -D VERSION=1.1.0
```

- downgrade-message 는 사용자가 더 새 판 위에 옛(또는 같은) 판을 설치하려 할 때 보는 안내문이다. 없으면 rubrapack 은 "The same or a newer version of [ProductName] is already installed." 를 쓴다 ([ProductName] 은 이름으로 바뀐다).

3.2 Windows 는 두 판을 어떻게 다루나

hello-1.0.0.msi 를 설치하고 이어서 hello-1.1.0.msi 를 설치한다. Windows Installer 는 새 패키지가 같은 업그레이드 코드에 더 높은 판임을 보고, 1.0.0 을 지운 뒤 1.1.0 을 설치한다 - "설치된 앱"에는 항목 하나가 남

고 이제 1.1.0 을 보인다. 이것을 메이저 업그레이드라고 하며, rubrapack 이 만드는 업그레이드는 이것뿐이다. 새 판은 늘 완전한 패키지이고, 새 판을 설치하는 데 옛 판이 필요하지 않다.

이제 hello-1.0.0.msi 를 다시 설치해 본다: Windows 가 다운그레이드 안내문을 보이고 아무것도 바꾸지 않는다. hello-1.1.0.msi 를 한 번 더 실행하는 것도 오류가 아니다. 설치된 판과 제품 코드가 같으므로(아래 참고) Windows 는 제품이 이미 설치되어 있다고 알아보고 두 번 설치하지 않는다. 설치된 판을 고치려면 - 지워진 파일을 되살리는 등 - msixexec /fa hello-1.1.0.msi 를 쓴다.

판을 올리지 않고 바뀐 파일로 다시 빌드한 패키지는 설치된 것과 제품 코드는 같지만 같은 패키지가 아니어서, Windows 가 자체 안내문으로 거부한다: "Another version of this product is already installed"(오류 1638). 그러니 파일이 바뀌면 판을 올린다 - 설치된 것을 바꿔 넣는 것은 더 높은 판뿐이다.

rubrapack 이 이를 어떻게 꾸미는지는 Upgrade 표에 보인다:

```
C:\work\hello> rubrapack inspect hello-1.1.0.msi Upgrade
UpgradeCode VersionMin VersionMax Language Attributes Remove ActionProperty
s38 S20 S20 S255 i4 S255 s72
Upgrade UpgradeCode VersionMin VersionMax Language Attributes
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 1 RP_OLDER_FOUND
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 258 RP_NEWER_FOUND
```

첫 행은 1.1.0 보다 낮은 설치된 판을 찾고(지우려고), 둘째 행은 1.1.0 이상을 찾아 다운그레이드 안내문과 함께 설치를 멈춘다. 속성 수는 비트 플래그다 - 258 은 256 + 2, "최솟값 포함"에 "찾기만 하고 지우지 않음"을 더한 것이다. 비트 플래그는 제3부의 비트 플래그, 이 표는 제4부의 설치하는 표들이 설명한다.

3.3 판 번호

- 수 셋이나 넷: 주판.부판.빌드 또는 주판.부판.빌드.리비전. Windows Installer 는 앞의 셋에 255.255.65535 까지 허용한다.
- **Windows 는 앞의 셋만 비교한다.** 1.0.0.1 과 1.0.0.2 는 같은 판이다: 둘째는 "같은 것"으로 거부된다. 릴리스마다 셋째 수(또는 그 위의 수)를 올린다.
- 틀린 판은 빌드를 멈춘다:

```
hello.toml:5:1: error[RP1308]: version '1.0' must be a.b.c or a.b.c.d with a, b <= 255 and c, d <= 65535
```

3.4 두 식별자

	업그레이드 코드	제품 코드
가리키는 것	모든 판에 걸친 제품	판 하나
직접 쓰나	upgrade-code 에 한 번; 절대 바꾸지 않는다	아니다: rubrapack 이 끌어낸다
바뀌나	바뀌지 않는다	판(앞의 수 셋)이 바뀔 때마다

```
1.0.0: ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
1.1.0: ProductCode {9CC17977-2023-8B8F-8BB0-2FA82C5B248F}
```

끌어낸 제품 코드는 같은 판을 빌드할 때마다, 어느 컴퓨터에서나 같다. [package] 의 product-code 는 이것을 손으로 정한다 - 앞선 판을 다른 도구가 자기 제품 코드로 만든 제품을 이어 갈 때만 필요하다.

3.5 사용자보다 먼저 업그레이드를 확인하기

어떤 실수는 빌드를 멈추지 않지만 사용자 컴퓨터에서 업그레이드를 망가뜨린다: 바뀐 업그레이드 코드, 높은 판, Windows 가 따라갈 수 없게 자리를 옮긴 파일. 새 패키지를 그 앞의 것과 대조한다:

```
C:\work\hello> rubrapack lint hello-1.1.0.msi --previous hello-1.0.0.msi
hello-1.1.0.msi: 0 errors, 0 warnings (against hello-1.0.0.msi)
```

실수가 어떻게 보이는지 보려고 같은 패키지를 자기 자신과 대조하면:

```
C:\work\hello> rubrapack lint hello-1.0.0.msi --previous hello-1.0.0.msi
hello-1.0.0.msi: error[RP2302]: ProductVersion 1.0.0 is not higher than the previous 1.0.0 in its first three
fields: Windows Installer does not upgrade
hello-1.0.0.msi: warning[RP2303]: the ProductCode is the previous version's: that is a minor upgrade
(REINSTALLMODE), not the major upgrade rubrapack builds
hello-1.0.0.msi: 1 error, 1 warning (against hello-1.0.0.msi)
```

낸 .msi 는 모두(예: releases 폴더에) 보관해, 다음 판을 그것과 대조할 수 있게 한다.

3.6 다른 도구로 만든 옛 판

내 제품이 전에 다른 도구의 패키지로 설치되었더라도, 업그레이드 코드가 같으면 rubrapack 의 업그레이드가 그것을 바꿔 넣는다. 그것이 잘 안 될 때 도움이 되는 키가 둘 있다:

- `refuse-upgrade-below = "0.60.0"`: 이 판보다 낮은 판은 업그레이드하지 않고 거부하며, 사용자에게 먼저 지우는 방법을 알려 준다 - 업그레이드 도중에 지우면 망가지는 것으로 알려진 옛 패키지를 위한 것이다.
- `refuse-upgrade-message = "..."`: 그 안내문의 앞부분(지우는 명령은 뒤에 붙는다).

4 더 많은 파일과 폴더

이 장의 차례

4.1	폴더	11
4.2	원본	11
4.3	폴더: [dir.ID]	12
4.4	파일 하나: [file.ID]	13
4.5	여러 파일: [files.ID]	13
4.6	빈 폴더: [folder.ID]	13
4.7	치우기: [remove.ID]	13
4.8	두 번째 사본: [copy.ID]	13
4.9	만날 수 있는 키 두 개 더	13
4.10	안에서 무슨 일이 일어났나	14

목표: 프로그램 폴더 전체를 설치한다 - 프로그램, 다른 이름으로 둘 읽어보기, 사용자가 고칠 수 있는 설정 파일, 문서 폴더, 하위 폴더에 든 예제, 데이터를 위한 빈 폴더 - 그리고 프로그램이 쓰는 기록 파일을 치운다.

4.1 폴더

```
C:\work\hello\
  dist\
    hello.exe
    readme.txt
    settings.ini
    docs\
      guide.txt
    samples\
      sample1.txt
      sub\
        sample2.txt
  hello.toml
```

4.2 원본

```
# tutorial 04: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.2.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.DocsDir]
path = "INSTALLDIR/docs"

[dir.SamplesDir]
```

```

path = "INSTALLDIR/samples"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"
name = "Read me.txt"
vital = false

[file.Settings]
dir = "INSTALLDIR"
source = "dist/settings.ini"
keep = true

[files.DocFiles]
dir = "DocsDir"
glob = "dist/docs/**"

[files.SampleFiles]
dir = "SamplesDir"
glob = "dist/samples/**/*.txt"

[folder.Data]
dir = "INSTALLDIR"
name = "data"
keep = true

[remove.Logs]
dir = "INSTALLDIR"
name = "*.log"
on = "uninstall"

[copy.ReadmeInDocs]
source = "file:Readme"
dir = "DocsDir"
name = "readme.txt"

```

빌드하고 파일을 본다:

```

C:\work\hello> rubrapack build hello.toml -o hello.msi
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\Read me.txt 8 Readme
[ProgramFiles64Folder]\Hello\docs\guide.txt 6 F_6d1508f52f6615459567
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello
[ProgramFiles64Folder]\Hello\samples\sample1.txt 7 F_1dc2b5e85973ee91d955
[ProgramFiles64Folder]\Hello\samples\sub\sample2.txt 7 F_c9c5c4da73214a6abf37
[ProgramFiles64Folder]\Hello\settings.ini 23 Settings

```

(실제 출력에는 열이 더 있다. 여기서는 경로, 크기, ID 만 보였다.)

4.3 폴더: [dir.ID]

dir 의 path 는 알려진 폴더나 다른 dir 에서 시작해 그 아래 폴더 이름을 잇는다:

path	사용자 컴퓨터에서(흔한 경우)
ProgramFiles/Hello	C:\Program Files\Hello (x86 패키지는 C:\Program Files (x86)\Hello)
INSTALLDIR/docs	C:\Program Files\Hello\docs - 사용자가 어디에 두든 dir INSTALLDIR 아래

path	사용자 컴퓨터에서(흔한 경우)
CommonAppData/Hello	C:\ProgramData\Hello - 모든 사용자가 함께 쓰는 데이터
LocalAppData/Hello	C:\Users\<이름>\AppData\Local\Hello

ProgramFiles/Hello 를 되풀이하지 않고 INSTALLDIR 위에 쌓는 것은 사용자가 설치 폴더를 고를 수 있게 되면(6장) 중요해진다: 그러면 docs 가 사용자가 고른 폴더를 따라간다. 알려진 폴더 전체 목록은 참조 부의 표에 있다.

4.4 파일 하나: [file.ID]

- name 은 파일을 다른 이름으로 설치한다 - 여기서는 readme.txt 가 Read me.txt 가 된다. 이름에는 어느 언어의 글자든 쓸 수 있다. Windows 가 파일 이름에 금지하는 것(< > : " / \ | ? *)은 거부한다.
- vital = false: 이 파일을 쓰지 못해도 설치하는 그 파일 없이 이어진다. 기본은 모든 파일이 필수다.
- keep = true: 제품을 지워도 파일이 남고, 사용자가 한 번 고친 뒤에는 복구나 다음 판이 덮어쓰지 않는다. 사용자가 고치는 설정에 쓴다.

4.5 여러 파일: [files.ID]

glob 은 와일드카드가 든 경로다: * 는 폴더 이름 하나 안의 아무 글자들, ? 는 글자 하나, ** 는 몇 단계든 폴더다. 맞는 파일이 모두 설치되고, 첫 와일드카드 아래의 폴더는 dir 아래에 다시 만들어진다:

glob	dist\samples\sub\sample2.txt 가 가는 곳
SamplesDir 로 dist/samples/**/*.txt	...\Hello\samples\sub\sample2.txt
SamplesDir 로 dist/samples/*.txt	설치되지 않음: * 는 sub 안으로 들어가지 않는다

맞은 파일은 이름 순으로 정렬되므로, 파일 시스템이 어떤 순서로 보이든 패키지는 같다. 아무것도 맞지 않는 그룹은 빌드를 멈춘다(RP1503) - 대개 경로의 오타다. rubrapack 은 맞은 파일마다 ID 를 따로 준다(F_ 와 경로에서 끌어낸 수).

4.6 빈 폴더: [folder.ID]

[folder.Data] 는 파일이 하나도 들어가지 않아도 INSTALLDIR 안에 data 를 만든다 - 프로그램이 쓸 자리다. keep = true 는 제품을 지울 때 그 폴더(와 프로그램이 거기 쓴 것)를 남긴다.

4.7 치우기: [remove.ID]

프로그램이 스스로 만드는 파일 - 기록, 캐시 - 은 설치 파일의 것이 아니어서 Windows Installer 가 남겨 두고, 그 때문에 폴더도 남는다. [remove.Logs] 는 제품을 지울 때(on = "uninstall") INSTALLDIR 의 *.log 를 지운다. on = "install" 은 설치할 때(옛 판이 남긴 파일), on = "both" 는 둘 다에서 지운다. name 이 없으면 폴더가 비었을 때 폴더 자체를 지운다. 설치 전부터 있던 폴더는 절대 지우지 않고, 설치가 실패하면 지운 파일을 되살린다.

4.8 두 번째 사본: [copy.ID]

[copy.ReadmeInDocs] 는 파일 Readme 의 사본을 하나 더 DocsDir 에 readme.txt 로 설치한다. 사본은 원래 파일과 함께 오고 함께 간다.

4.9 만날 수 있는 키 두 개 더

- 파일의 any-arch = true: rubrapack 은 모든 프로그램 파일(.exe, .dll)이 패키지의 arch 로 만들어졌는지 확인한다. 실수로 32비트 DLL 을 복사하는 x64 설치 파일은 망가진 프로그램을 설치하기 때문이다. x64 제품 안의 32비트 도우미처럼 일부러 그런 것이면 any-arch = true 로 밝힌다.
- 파일의 component-guid = "{...}": rubrapack 은 모든 컴포넌트의 GUID 를 제품과 파일 자리에서 끌어내므로, 판이 바뀌어도 같게 유지되고 업그레이드는 그것에 기댄다. 앞선 패키지가 다른 GUID 로 만든 컴포넌트를 이어 갈 때만 손으로 준다.

4.10 안에서 무슨 일이 일어났나

Windows Installer 는 컴포넌트를 설치한다: 함께 설치되고 함께 지워지는 작은 자원 묶음으로, 저마다 GUID 와 키 경로(그것이 있으면 "설치됨"을 뜻하는 것)를 가진다. rubrapack 은 파일마다 컴포넌트 하나를 만드는데 - 안전한 규칙이다 - 필요한 폴더, 지우기, 사본에도 하나씩 더 만든다:

```
C:\work\hello> rubrapack inspect hello.msi Component
...
C_74a883a037bc227f9189 {51A68DD4-96FA-83C0-86AA-F20D51339238} INSTALLDIR 272 Settings
C_cec3a9b89b2e391393d0 {615660C7-986E-8FB8-87AF-0616B544A6F9} Data 272
...
```

속성 열의 272 는 $256 + 16$ 이다: 256 은 64비트 컴포넌트, 16 은 "영구" - keep = true 다. 제4부가 이 표들을 설명한다. RemoveFile, CreateFolder, DuplicateFile 에 지우기, 빈 폴더, 사본이 들어 있다.

5 바로가기, 아이콘, "설치된 앱" 항목

이 장의 차례

5.1	원본	15
5.2	바로가기: [shortcut.ID]	16
5.3	"설치된 앱" 항목: [arp]	16
5.4	다른 글자로 된 이름	16
5.5	해 보기	16
5.6	안에서 무슨 일이 일어났나	16

목표: 시작 메뉴에 "Example Software" 폴더를 만들어 Hello 와 그 읽어보기를 넣고, 자기 아이콘을 가진 바탕화면 바로가기를 두고, "설치된 앱"의 Hello 에 아이콘과 링크를 단다.

원본 옆에 아이콘 파일 hello.ico 가 있어야 한다(아무 .ico 나 된다. 많은 그림 편집기가 저장할 수 있다).

5.1 원본

```
# tutorial 05: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.3.0"

[arp]
icon = "hello.ico"
help = "https://example.com/hello/help"
about = "https://example.com/hello"
no-repair = true

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.MenuFolder]
path = "Programs/Example Software"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"

[shortcut.StartMenu]
dir = "MenuFolder"
name = "Hello"
target = "file:Hello"
description = "Says hello"
working-dir = "INSTALLDIR"
```

```
[shortcut.ReadmeLink]
dir = "MenuFolder"
name = "Hello - read me"
target = "file:Readme"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
args = "--greet \"[ProductName] user\""
icon = "hello.ico"
```

5.2 바로가기: [shortcut.ID]

- `dir` 은 바로가기가 들어갈 곳이다: 내 `dir`, 또는 바로 `Programs`(시작 메뉴의 프로그램 목록), `StartMenu`, `Desktop`, `Startup`(로그인할 때마다 실행). 여기서 `dir MenuFolder` 는 시작 메뉴 프로그램 안의 "Example Software" 폴더라서, 시작 메뉴에 "Example Software" 와 그 안의 항목 둘이 보인다.
- `name` 은 바로가기 아래의 글이다.
- `target` 은 바로가기가 여는 파일이다: `file:` 과 이 패키지의 `[file.*]` ID.
- `description` 은 도움말 풍선, `working-dir`(`dir` ID)은 프로그램이 시작하는 폴더다.
- `args` 는 프로그램의 인자다. MSI 서식 문자열이어서 바로가기를 만들 때 `[ProductName]` 이 "Hello" 가 된다. TOML 문자열 안의 큰따옴표는 `\` 로 쓴다.
- `icon` 은 바로가기에 자기 `.ico` 를 준다. 없으면 Windows 는 프로그램 자체의 아이콘을 보인다.

rubrapack 이 바로가기를 위해 만든 시작 메뉴 폴더는 바로가기와 함께 제품을 지울 때 사라진다.

5.3 "설치된 앱" 항목: [arp]

("ARP" 는 이 목록의 옛 이름 Add or Remove Programs(프로그램 추가/제거)다.)

키	효과
<code>icon = "hello.ico"</code>	목록의 아이콘
<code>help = "https://..."</code>	Windows 가 제품과 함께 보이는 지원 링크
<code>about = "https://..."</code>	제품의 웹 페이지
<code>no-repair = true</code>	이 제품에 복구가 없다
<code>no-modify = true</code>	수정/변경이 없다(패키지에 바꿀 것이 없을 때 쓸모 있다. 8장이 바꿀 것을 더한다)

5.4 다른 글자로 된 이름

`name` 은 어느 언어로든 쓸 수 있다(`name = "안녕"`). 그런데 `.msi` 파일 자체의 속성(자세히 탭)은 ASCII 글자만 보일 수 있어서, 그런 이름에는 ASCII 형태도 준다: `[package]` 의 `summary-name = "Annyeong"`. 없으면 rubrapack 이 경고하고(RP1203) 속성에는 "rubrapack package" 가 보인다.

5.5 해 보기

빌드하고 설치한 뒤 시작 메뉴를 연다: "Example Software" 에 "Hello" 와 "Hello - read me" 가 있고, 바탕 화면에는 준 아이콘을 단 "Hello" 가 있다. "설치된 앱"에는 아이콘과 함께 Hello 가 보인다. 지우면 바로가기와 시작 메뉴 폴더도 사라진다.

5.6 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi Shortcut
Shortcut Directory_ Name Component_ Target Arguments Description Hotkey
```

```
...
Desktop DesktopFolder Hello C_185f8db32271fe25f561 [#Hello] --greet "[ProductName] user"
ReadmeLink MenuFolder HELLO~1\Hello - read me C_44ff26388205705fb806 [#Readme]
StartMenu MenuFolder Hello C_185f8db32271fe25f561 [#Hello] Says hello
```

- 바로가기는 대상 파일의 컴포넌트에 속한다: 파일과 함께 설치되고 지워진다.
 - [#Hello] 는 "파일 Hello 가 설치된 경로"를 뜻하는 Windows Installer 의 표기다.
 - HELLO~1\Hello - read me 에는 이름이 둘 들어 있다: 옛 방식의 짧은 이름(8자, 점, 3자 - MS-DOS 시절부터)과 긴 이름. Windows Installer 는 모든 이름에 둘 다를 원하고, 짧은 이름은 rubrapack 이 지어낸다.
- 아이콘은 패키지 안(Icon 표)에 저장되고, "설치된 앱" 설정은 Property 표에 들어 있다:

```
ARPHHELPLINK https://example.com/hello/help
ARNOREPAIR 1
ARPPRODUCTICON RpIcon1.ico
ARPURLINFOABOUT https://example.com/hello
```

6 대화창, 약관, 그리고 프로그램 시작하기

이 장의 차례

6.1	원본	18
6.2	페이지 세트 고르기: ui	19
6.3	약관: license	19
6.4	설치 폴더	19
6.5	위쪽의 그림: banner	19
6.6	내 말로: [ui-text.ID]	19
6.7	끝에 프로그램 시작하기: launch	19
6.8	해 보기	20
6.9	안에서 무슨 일이 일어났나	22

목표: 페이지가 있는 설치 파일 - 환영, 사용자가 동의해야 하는 약관, 폴더 고르기, 진행, 완료 - 을 만들고, 끝에 Hello 를 시작하겠냐고 묻게 한다.

6.1 원본

원본 옆에 LICENSE.txt 와 banner.bmp(약 493 x 58 픽셀 그림)가 있다.

```
# tutorial 06: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installdir"
license = "LICENSE.txt"

[define]
VERSION = "1.4.0"

[ui]
banner = "banner.bmp"
launch = "file:Hello"
launch-args = "--first-run"
launch-checked = true

[ui-text.WelcomeText]
text = "This will install [ProductName] [ProductVersion]. Close Hello if it is running."

[ui-text.ExitText]
text = "[ProductName] is ready. Thank you for installing it."

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
```

```
dir = "Programs"
name = "Hello"
target = "file:Hello"
```

6.2 페이지 세트 고르기: ui

[package] 의 ui 는 내장 대화창 세트 하나를 고른다:

ui	설치할 때	설치 뒤 다시 실행할 때
(없음)	Windows Installer 자체의 작은 진행 창만	같음
basic	진행, 그다음 완료(또는 오류)	진행, 완료
minimal	환영, 약관(있으면), 진행, 완료	복구 또는 제거
installldir	환영, 약관, 설치 폴더(폴더 찾아보기 포함), 준비, 진행, 완료	복구 또는 제거
features	installldir 과 같고, 기능 트리과 그것이 필요로 하는 디스크 공간이 더해진다(8장)	복구, 변경, 제거

어느 세트에나 무슨 일이 생겼을 때 Windows 가 필요로 하는 페이지도 있다: "정말 취소하겠습니까?", 오류 메시지, "이 프로그램들이 바뀌어야 할 파일을 쓰고 있습니다"(사용 중인 파일), "디스크 공간 부족". 대화창이 있는 패키지도 /qn 으로 조용히 설치된다: 모든 페이지는 기본값이 있는 값만 모으기 때문이다.

6.3 약관: license

license = "LICENSE.txt" 는 환영 페이지 뒤에 약관 페이지를 더한다. "I accept the terms of the license agreement"(사용권 계약에 동의합니다)에 표시하기 전에는 다음 단추가 꺼져 있다. 파일은:

- .txt 또는 .md - 평문으로 보인다. 한국어, 다른 글자, 이모지도 그대로다.
- .rtf - 글꼴과 서식을 그대로 보인다(워드패드나 Word 로 만든다).

약관 글은 빌드할 때 패키지 안에 저장되므로 사용자에게 그 파일이 필요하지 않다.

6.4 설치 폴더

installldir 이나 features 에서는 설치 폴더와, 폴더 찾아보기를 여는 변경 단추가 있는 페이지가 보인다. 사용자가 고른 폴더가 INSTALLDIR 이 되고, INSTALLDIR 위에 쌓은 dir 은 모두 함께 옮겨 간다. 다른 dir 을 바꾸게 하려면 이름을 낸다: [ui] 의 install-dir = "AppDir". 명령줄에서는 폴더를 속성으로 준다:

```
msiexec /i hello.msi INSTALLDIR="D:\Tools\Hello"
```

6.5 위쪽의 그림: banner

banner = "banner.bmp" 는 안쪽 페이지마다 위쪽 띠에 그림을 넣는다(Windows Installer 는 BMP 를 그린다. 약 493 x 58 픽셀). 없으면 띠는 그냥 흰색이다.

6.6 내 말로: [ui-text.ID]

내장 페이지의 모든 글에는 ID 가 있고 [ui-text.ID] 가 하나를 바꾼다. ID 목록은 참조 부의 대화창에 있다. WelcomeText 와 ExitText 는 첫 페이지와 마지막 페이지의 문장이다. 글은 서식 문자열이다: [ProductName], [ProductVersion], [Manufacturer] 는 바뀌고, 대괄호 자체는 [\] 로 쓴다. 단추 글에서 & 는 Alt 와 함께 쓰는 글자를 표시한다: &Next 는 Alt+N 이다.

6.7 끝에 프로그램 시작하기: launch

launch = "file:Hello" 는 완료 페이지에 "Launch Hello"(Hello 실행) 확인란을 넣는다(launch-checked = false 가 아니면 표시된 채로). launch-args 는 프로그램의 인자다. 표시되어 있으면 마침 단추가 Hello 를 시작한다 - 설치 파일의 관리자 권한이 아니라 설치를 실행한 사용자의 권한으로 - 첫 설치나 업그레이드 뒤에만이고, 복구나 제거 뒤에는 아니며, 조용한 설치에서는 절대 아니다.

6.8 해 보기

hello.msi 를 두 번 누른다: 환영(내 문구와 배너), 약관(확인란을 표시할 때까지 다음이 회색), 설치 폴더, 준비, 진행, "Launch Hello" 가 표시된 완료. 설치한 뒤 다시 실행하면 복구와 제거를 원하는 페이지가 나온다.

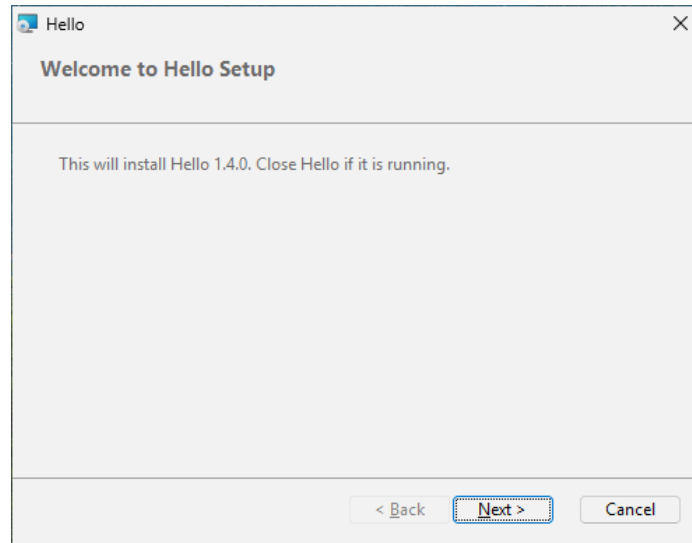


그림 1: 환영: [ui-text.WelcomeText] 의 글

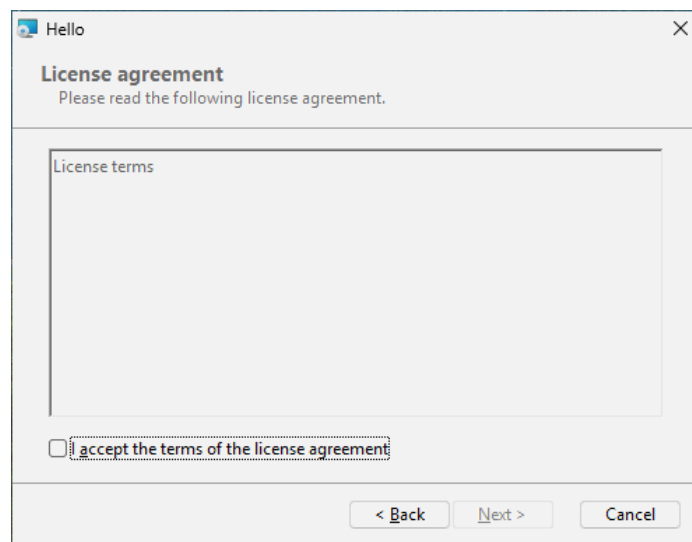


그림 2: 약관: 확인란을 표시할 때까지 다음은 회색이다

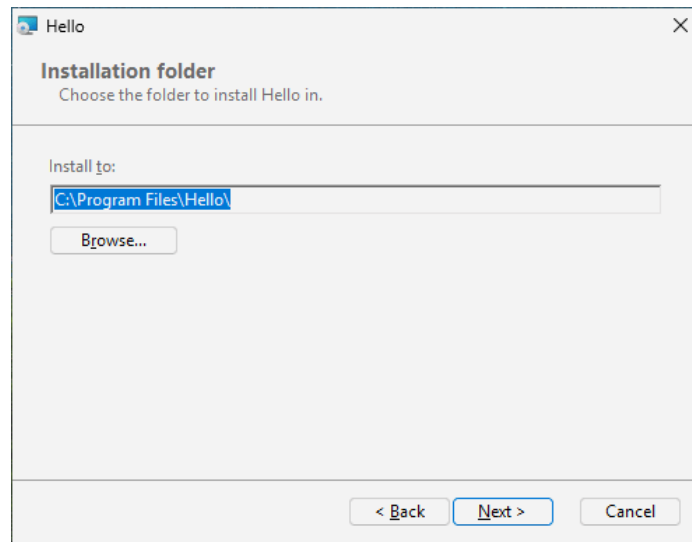


그림 3: 설치 폴더: 바꿀 수 있다

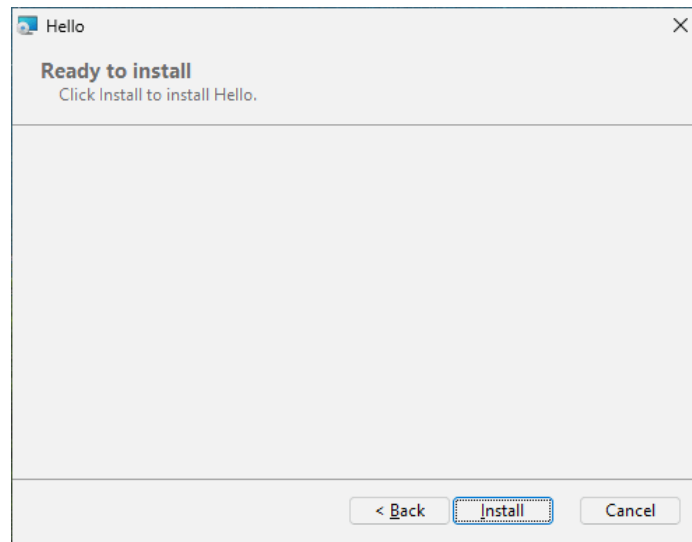


그림 4: 설치 준비

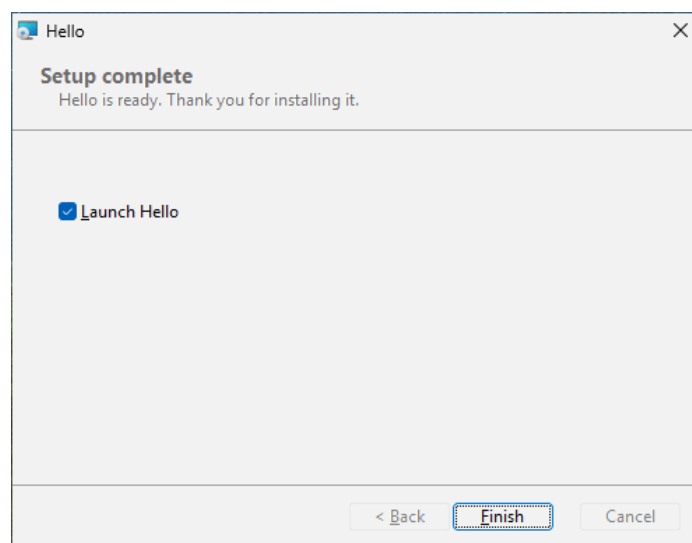


그림 5: "Launch Hello" 가 표시된 완료 페이지

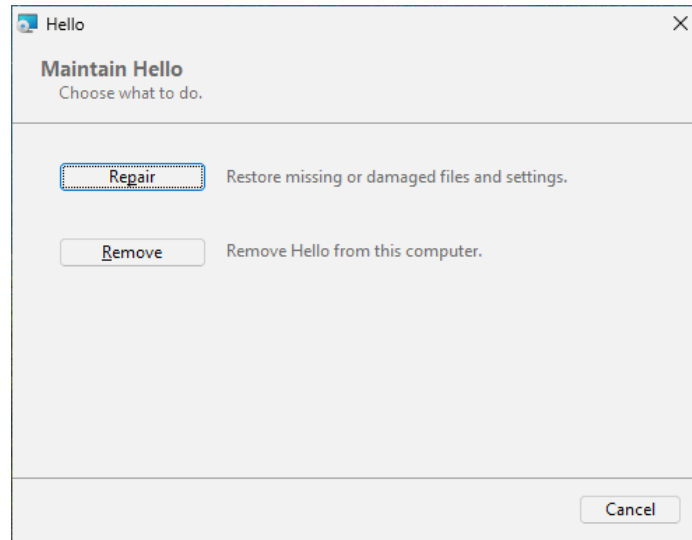


그림 6: 설치한 뒤 다시 실행: 복구 또는 제거

이 장의 원본에는 한국어가 없으므로 페이지는 영어다. 한국어 페이지는 7장에서 더한다.

6.9 안에서 무슨 일이 일어났나

대화창이 있는 패키지는 그것을 표로 지닌다 - Dialog, Control, ControlEvent 등 - 이 표들이 모든 창, 단추, 글상자를 위치와 눌렀을 때 일어나는 일과 함께 기술한다. rubrapack 은 내장 세트에서 이것을 쓴다:

```
C:\work\hello> rubrapack inspect hello.msi Dialog
...
RpBrowseDlg RpCancelDlg RpErrorDlg RpExitDlg RpFatalDlg RpInstallDirDlg RpLicenseDlg
RpMaintenanceDlg RpOutOfDiskDlg RpProgressDlg RpReadyDlg RpUserExitDlg RpWelcomeDlg FilesInUse
```

(여기서는 첫 열만 보았다.) InstallUISequence 표는 페이지가 언제 나오는지 말한다:

```
RpWelcomeDlg NOT Installed 1230
RpMaintenanceDlg Installed AND NOT RESUME AND NOT Preselected 1240
RpProgressDlg 1280
RpExitDlg -1
RpUserExitDlg -2
RpFatalDlg -3
```

가운데 열은 조건이다: 환영 페이지는 제품이 아직 설치되지 않았을 때만, 유지 관리 페이지는 설치되어 있을 때 나온다. 음수는 특별하다: 성공한 끝(-1), 사용자가 취소한 끝(-2), 실패한 끝(-3)에 보이는 페이지다. 9장에서 내 조건을 쓴다.

7 여러 언어

이 장의 차례

7.1	원본	23
7.2	언어 더하기: languages	24
7.3	언어마다 약관: license-xx	24
7.4	언어마다 글: text-xx	24
7.5	영어와 한국어 말고 다른 언어	24
7.6	번역되지 않는 것	25
7.7	해 보기	25
7.8	안에서 무슨 일이 일어났나	25

목표: 영어나 한국어로 말하는 패키지 하나를 만든다. 약관도 언어마다 따로 두고, 한국어 Windows에서는 한국어가 미리 골라진다.

7.1 원본

한국어 약관 LICENSE-ko.txt 가 LICENSE.txt 옆에 있다.

```
# tutorial 07: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${(VERSION)}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installdir"
license = "LICENSE.txt"
downgrade-message = "A newer Hello is already installed."

[define]
VERSION = "1.5.0"

[ui]
languages = ["ko"]
license-ko = "LICENSE-ko.txt"
launch = "file:Hello"

[ui-text.WelcomeText]
text = "This will install [ProductName] [ProductVersion]."
text-ko = "[ProductName] [ProductVersion]을(를) 설치합니다."

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"
```

7.2 언어 더하기: languages

페이지는 늘 영어다. [ui]의 languages = ["ko"]는 같은 패키지에 한국어를 더하고, 그러면:

- 첫 페이지가 어떤 언어를 쓸지 묻는다 - English 또는 한국어.
- 지역 형식(없으면 시스템 언어)이 한국어인 Windows에서는 한국어가 이미 골라져 있다. 다른 곳에서는 English.
- 그 뒤의 모든 페이지 - 환영, 약관, 폴더, 준비, 진행, 완료, 그리고 취소, 오류, 사용 중인 파일, 디스크 공간, 유지 관리 페이지 - 가 고른 언어로 말한다.

영어만 있으면 언어 페이지가 없다. 조용한 설치(/qn)는 페이지를 보이지 않으므로 고를 필요가 없다. 명령줄의 RPLANGUAGE=ko는 한국어를 미리 고른다 - 그래도 맨 먼저 나오는 언어 페이지에서, 그리고 조용한 설치에서:

```
msiexec /i hello.msi RPLANGUAGE=ko
```

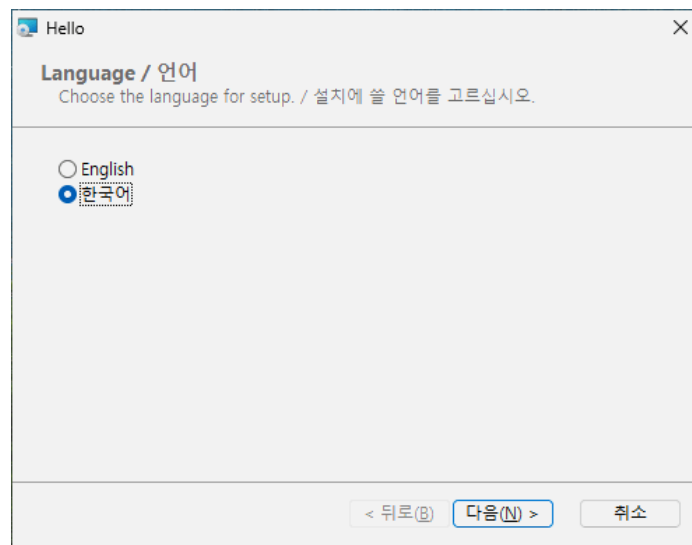


그림 7: 한국어 Windows의 언어 페이지: 한국어가 골라져 있다

7.3 언어마다 약관: license-xx

license-ko = "LICENSE-ko.txt"는 한국어를 골랐을 때 보이는 약관이다. 영어(와 자기 약관이 없는 더한 언어)는 [package] license를 보인다. .txt / .md / .rtf 규칙은 같다.

7.4 언어마다 글: text-xx

[ui-text.ID]에서 text는 모든 언어의 글, text-ko는 한국어 글이다. 내장 글은 모두 이미 영어와 한국어로 있다. text를 준 곳에만 text-ko를 주어 두 언어가 같은 말을 하게 한다.

7.5 영어와 한국어 말고 다른 언어

rubrapack의 내장 페이지는 영어와 한국어로 쓰여 있다. 다른 언어 - ja, de, fr 등 - 는 모든 글을 text-xx로 직접 준다. 빠진 것은 빌드가 알려 준다:

```
hello.toml:16:1: error[RP1202]: language 'ja' has no built-in texts: give [ui-text.ID] text-ja for all of them (64 missing, the first is Back)
```

그런 언어에는 [ui]의 키 셋이 더 도움이 된다: name-ja(언어 페이지에 보일 이름), font-ja(그 페이지들의 글꼴), langid-ja(그 언어를 미리 고르게 하는 Windows 언어 번호 - 수 하나나 목록). 흔한 언어(ja, zh, de, fr, es, it, pt, nl, pl, ru, uk, tr, vi, th)에는 이 셋이 내장되어 있다. 한국어 페이지는 맑은 고딕, 영어 페이지는 Segoe UI 글꼴을 쓴다.

7.6 번역되지 않는 것

- Windows Installer 자체의 말 - 기능 트리의 크기와 메뉴, "남은 시간", 오류 메시지 - 은 패키지가 아니라 Windows 에서 온다.
- 다운그레이드 안내문(3장)과 `refuse-upgrade-below` 의 제거 안내문은 글이 하나씩이다. 두 언어가 다 필요하면 두 언어로 쓴다("A newer Hello is already installed. / 더 새 Hello 가 설치되어 있습니다.").
- `[package] language = "ko-KR"` 는 패키지 자체의 언어 코드만 정한다. 페이지를 한국어로 만드는 것은 `languages` 다.

7.7 해 보기

한국어 Windows 에서 `hello.msi` 를 두 번 누른다: 언어 페이지에 한국어가 골라져 있고, 그 뒤 모든 페이지가 한국어 약관과 함께 한국어다. English 를 고르면 같은 페이지가 `LICENSE.txt` 와 함께 영어로 나온다. 영어 Windows 에서 `msiexec /i hello.msi RPLANGUAGE=ko` 도 해 본다.

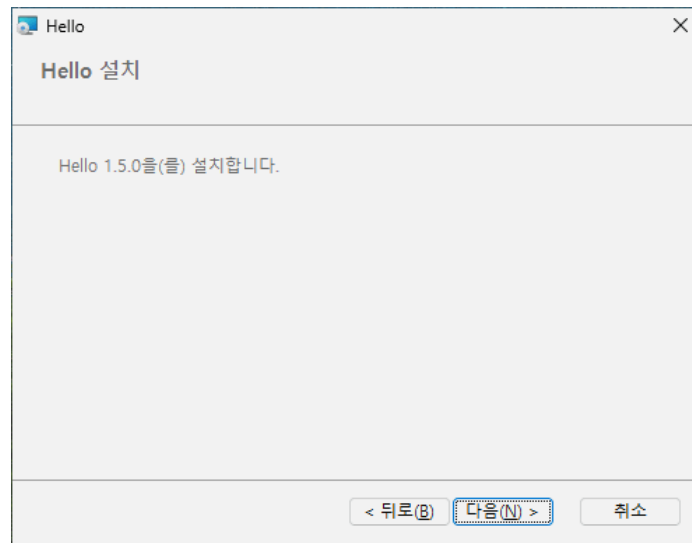


그림 8: 한국어를 고른 환영 페이지: `[ui-text.WelcomeText]` 의 `text-ko`

7.8 안에서 무슨 일이 일어났나

Windows Installer 패키지에는 "여러 언어"라는 개념이 따로 없다: 페이지의 글은 그 표에 고정되어 있다. 그래서 `rubrapack` 은 모든 글을 속성 - 이름 붙은 값 - 으로 만들고 페이지에는 그 속성을 보인다:

```
C:\work\hello> rubrapack inspect hello.msi Property
...
RpT_WelcomeText This will install Hello 1.5.0.
RpT_WelcomeText_en This will install Hello 1.5.0.
RpT_WelcomeText_ko Hello 1.5.0을(를) 설치합니다.
...
```

첫 페이지 전에 작은 동작들이 고른 언어의 글을 보이는 속성에 복사한다. 각 동작은 조건이 참일 때만 돈다:

```
C:\work\hello> rubrapack inspect hello.msi InstallUISequence
...
RpL_ko_26 RPLANGUAGE = "ko" 21
...
C:\work\hello> rubrapack inspect hello.msi CustomAction
...
RpL_ko_26 51 RpT_WelcomeText [RpT_WelcomeText_ko]
```

형식 51 은 "속성 정하기"다: `RpT_WelcomeText` 가 `RpT_WelcomeText_ko` 의 값이 된다. 글은 패키지에 UTF-8(코드 페이지 65001)로 저장된다. 글자 인코딩은 제3부의 글자, 문자열 저장 방식은 제4부의 MSI 데이터베이스가 설명한다.

8 선택 구성요소: 기능과 사용자의 선택

이 장의 차례

8.1	기능	26
8.2	원본	26
8.3	기능 선언하기: [feature.ID]	27
8.4	파일을 기능에 넣기: feature	28
8.5	사용자가 보는 것	28
8.6	명령줄에서 고르기	28
8.7	나중에 바꾸기	29
8.8	안에서 무슨 일이 일어났나	29

목표: 무엇을 설치할지 사용자가 고르게 한다. 프로그램 자체는 늘 설치하고, 문서는 보여 주며 표시된 채로 두고, 예제는 보여 주되 표시하지 않고, 진단 부분은 숨겨 두었다가 요청할 때만 설치한다.

8.1 기능

기능(feature)은 사용자가 고를 수 있는 제품의 한 부분이다: 설치 파일의 트리에서 제목, 설명, 표시가 있는 한 줄이다. 모든 파일은 정확히 한 기능에 속하고, 한 기능에는 파일이 몇 개든 들어갈 수 있다.

8.2 원본

```
# tutorial 08: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "features"
license = "LICENSE.txt"

[define]
VERSION = "1.6.0"

[feature.Main]
title = "Hello"
description = "The program itself."
required = true

[feature.Documentation]
title = "Documentation"
description = "The guide, in the docs folder."

[feature.Samples]
title = "Samples"
description = "Example files to try Hello with."
level = 2

[feature.MoreSamples]
title = "More samples"
description = "Samples in sub folders."
parent = "Samples"
```

```

follow-parent = true

[feature.Diagnostics]
title = "Diagnostics"
hidden = true
level = 3

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
feature = "Main"

[dir.DocsDir]
path = "INSTALLDIR/docs"
feature = "Documentation"

[dir.SamplesDir]
path = "INSTALLDIR/samples"
feature = "Samples"

[dir.MoreSamplesDir]
path = "INSTALLDIR/samples/sub"
feature = "MoreSamples"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[files.DocFiles]
dir = "DocsDir"
glob = "dist/docs/**"

[files.SampleFiles]
dir = "SamplesDir"
glob = "dist/samples/*.txt"

[files.MoreSampleFiles]
dir = "MoreSamplesDir"
glob = "dist/samples/sub/*.txt"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"
feature = "Diagnostics"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

```

8.3 기능 선언하기: [feature.ID]

키	뜻
title	트리의 줄
description	그 줄을 골랐을 때 보이는 글
level	1(기본): 기본으로 설치. 2 이상: 보여 주지만 고를 때만 설치
required = true	늘 설치: 트리가 "사용할 수 없음"을 제안하지 않는다
parent = "Samples"	트리에서 다른 기능 아래의 줄
follow-parent = true	부모가 설치될 때 정확히 함께 설치

키	뜻
hidden = true	트리에 아예 보이지 않음

기능 ID 는 dir, 파일과 이름 공간을 함께 쓴다: 기능 Documentation 과 폴더 DocsDir 는 ID 가 달라야 한다(아니면 rubrapack 이 RP1301 로 알린다).

8.4 파일을 기능에 넣기: feature

- dir 에: 그 폴더의 모든 파일이 그 기능에 속한다 - DocsDir 는 guide.txt 를 Documentation 에, SamplesDir 는 sample1.txt 를 Samples 에 넣는다.
- 파일이나 파일 묶음에: 그 파일(묶음)이 폴더와 상관없이 그 기능에 속한다 - Readme 는 Main 의 폴더인 INSTALDIR 에 있지만 Diagnostics 에 속한다.

dir 은 자기 기능을 아래 dir 에 물려주지 않는다: MoreSamplesDir 는 자기 기능을 따로 낸다. 원본이 기능을 하나라도 선언하면 설치되는 모든 것에 기능이 필요하다. 기능이 없는 것은 rubrapack 이 RP1202 로 짚고 멈춘다.

8.5 사용자가 보는 것

ui = "features" 에서는 약관과 설치 폴더 다음에 "Choose features"(기능 고르기)가 나온다: 트리 하나

```
[x] Hello          (끌 수 없음)
[x] Documentation
[ ] Samples
    [ ] More samples (Samples 를 따라감)
```

줄마다 메뉴가 있다: "Will be installed on local hard drive"(로컬 하드 드라이브에 설치), "Entire feature will be installed on local hard drive"(하위까지 모두 설치), "Entire feature will be unavailable"(모두 사용할 수 없음). 줄을 고르면 설명과 필요한 디스크 공간이 보이고, "Disk Usage" 는 드라이브마다 공간을 보인다. (이 글들은 Windows 에서 오프로 영어로 남는다.) Diagnostics 는 보이지 않는다.

기본 설치(두 번 누르고 다음, 다음, ...)는 Hello 와 문서를 설치한다.

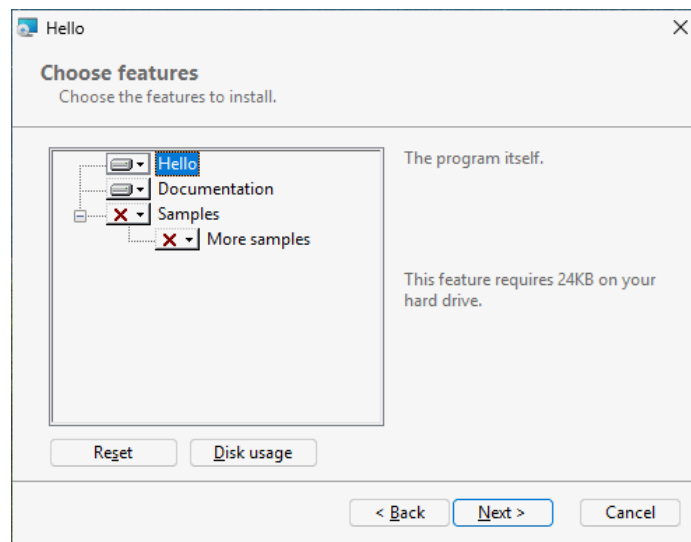


그림 9: 기능 고르기: Samples 와 More samples 는 제안되지만 골라져 있지 않고, Diagnostics 는 숨어 있다

8.6 명령줄에서 고르기

기능은 속성으로 고르므로 조용한 설치도 고를 수 있다:

명령	설치하는 것
msiexec /i hello.msi /qn	기본: Main, Documentation

명령	설치하는 것
<code>msiexec /i hello.msi /qn ADDLOCAL=Samples</code>	기본과 예제(따라가는 More samples 포함)
<code>msiexec /i hello.msi /qn ADDLOCAL=ALL</code>	숨긴 것까지 모든 기능
<code>msiexec /i hello.msi /qn INSTALLLEVEL=3</code>	level 이 3 이하인 모든 기능: 여기서는 전부
<code>msiexec /i hello.msi /qn REMOVE=Documentation</code>	문서를 뺀 기본

기능 ID 는 대소문자를 가리고 쉼표로 나눈다: ADDLOCAL=Samples,Diagnostics.required 는 트리만 묶는다: 명령줄의 REMOVE=Main 은 여전히 따른다(Windows Installer 가 그렇게 동작한다).

8.7 나중에 바꾸기

설치한 뒤 "설치된 앱"은 Hello 에 수정을 보인다. 설치 파일의 유지 관리 페이지가 열리고 **변경, 복구, 제거**가 있다. 변경은 설치된 것이 표시된 같은 트리를 보이고, 사용자는 부분을 더하거나 뺀다. 설치된 제품에 명령줄로는:

```
msiexec /i hello.msi ADDLOCAL=Samples
msiexec /i hello.msi REMOVE=Samples
```

8.8 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi Feature
Feature Feature_Parent Title Description Display Level Directory_ Attributes
...
Diagnostics Diagnostics 0 3 0
Documentation Documentation The guide, in the docs folder. 3 1 0
Main Hello The program itself. 1 1 16
MoreSamples Samples More samples Samples in sub folders. 7 1 2
Samples Samples Example files to try Hello with. 5 2 0
```

- Display 는 트리의 순서다(원본 순서의 홀수). 0 은 기능을 숨긴다.
- Level 은 수준이다. Windows 는 수준이 속성 INSTALLLEVEL(1)보다 높지 않은 기능을 기본으로 설치한다.
- Attributes 는 비트 플래그다: 16 은 "없앨 수 없음"(required), 2 는 "부모를 따라감".

FeatureComponents 표는 컴포넌트 - 파일 - 마다 그 기능을 잇는다:

```
C:\work\hello> rubrapack inspect hello.msi FeatureComponents
...
Documentation C_a10be66379e69940a2e0
Main C_185f8db32271fe25f561
Samples C_a5674982edf0797da30f
...
```

9 조건과 검색

이 장의 차례

9.1	속성	30
9.2	원본	30
9.3	내 속성: [property.NAME]	31
9.4	이럴 때만 설치하기: when	32
9.5	설치 거부하기: [require.ID]	32
9.6	먼저 둘러보기: [search.ID]	32
9.7	설치 폴더 기억하기	32
9.8	안에서 무슨 일이 일어났나	32

목표: 무엇을 설치할지 사용자 컴퓨터에서 정한다. 64비트 Windows 에서만 설치하고, 메모장이 있는 곳에서만 한 부분을 권하고, 바탕화면 바로가기를 사용자가 끌 수 있게 하고, 다음 판은 사용자가 이번 판에 고른 폴더에 설치한다.

9.1 속성

Windows Installer 는 실행하는 동안 속성이라는 이름 붙은 값을 지닌다: `ProductName`, `INSTALLDIR`, `VersionNT64`(64비트 Windows 에서 정해짐), 그리고 내 것들. 이름이 모두 대문자인 속성은 공개다: 명령줄 (`msiexec /i hello.msi DESKTOP_SHORTCUT=0`)과 대화창이 정할 수 있다. 조건은 속성에 대한 작은 식으로, 참이거나 거짓이다:

조건	참일 때
<code>VersionNT64</code>	속성이 정해져 있을 때(비어 있지 않음): 여기서는 64비트 Windows
<code>NOT Installed</code>	제품이 아직 설치되지 않았을 때(첫 설치)
<code>DESKTOP_SHORTCUT = "1"</code>	속성 값이 정확히 1 일 때
<code>VersionNT >= 603</code>	Windows 판 번호가 6.3 이상일 때
<code>A AND (B OR NOT C)</code>	생각하는 대로의 조합

9.2 원본

```
# tutorial 09: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${(VERSION)}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "features"

[define]
VERSION = "1.7.0"

[property.DESKTOP_SHORTCUT]
value = "1"
secure = true

[require.Windows64]
```

```

condition = "VersionNT64"
message = "[ProductName] needs 64-bit Windows."

[search.Notepad]
property = "NOTEPAD_PATH"
kind = "file"
path = "Windows"
file = "notepad.exe"

[search.PreviousDir]
property = "INSTALLDIR"
kind = "registry"
root = "HKLM"
key = "Software\\Example Software\\Hello"
name = "InstallDir"

[feature.Main]
title = "Hello"
required = true

[feature.NotepadHelper]
title = "Open notes in Notepad"
description = "Only offered where Notepad is installed."
when = "NOTEPAD_PATH"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
feature = "Main"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Settings]
dir = "INSTALLDIR"
source = "dist/settings.ini"
feature = "NotepadHelper"

[registry.RememberDir]
root = "HKLM"
key = "Software\\Example Software\\Hello"
name = "InstallDir"
value = "[INSTALLDIR]"
feature = "Main"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
when = "DESKTOP_SHORTCUT = \"1\""

```

9.3 내 속성: [property.NAME]

[property.DESKTOP_SHORTCUT] 은 기본값 1 로 속성을 만든다. 공개가 되려면 이름이 모두 대문자여야 한다. Windows 나 rubrapack 이 스스로 쓰는 이름(ALLUSERS, ARP..., MSI... 등)은 거부한다.

- `secure = true`: 모든 사용자를 위한 설치에 두 부분으로 돈다 - 사용자 부분(대화창)과, 컴퓨터를 바꾸는 권한 높은 부분. 첫 부분이나 명령줄에서 정한 값은 속성이 보안(secure)일 때만 둘째 부분에 닿는다. `when` 조건이 보는 속성은 모두 보안으로 한다. (내 대화창 페이지의 속성과 검색 결과는 rubrapack 이 알아서 그렇게 한다.)
- `hidden = true`: 값을 설치 기록에 쓰지 않는다 - 암호와 키를 위한 것이다.

9.4 이럴 때만 설치하기: `when`

기능, 파일이나 파일 묶음, 바로가기, 레지스트리 값, 환경 변수, INI 값에 `when` 을 달면 첫 설치 때 조건이 참일 때만 설치한다:

- 바탕화면 바로가기의 `when = "DESKTOP_SHORTCUT = \"1\""`: 기본으로는 만들고, `msiexec /i hello.msi DESKTOP_SHORTCUT=0` 이면 만들지 않는다. 14장에서 이 속성을 내 페이지의 확인란으로 바꾼다.
- 기능의 `when = "NOTEPAD_PATH"`: 조건이 거짓인 곳에서는 그 기능을 설치하지도 트리에 보이지도 않는다.

복구는 설치된 것을 그대로 두고, 메이저 업그레이드(다음 판)는 다시 정한다.

9.5 설치 거부하기: `[require.ID]`

요구 조건은 `condition` 이 거짓이면 첫 설치를 `message` 와 함께 멈춘다 - 여기서는 32비트 Windows 에서: "Hello needs 64-bit Windows." 복구와 제거는 절대 막지 않는다.

9.6 먼저 둘러보기: `[search.ID]`

검색은 다른 무엇보다 먼저 돌고, 찾은 것을 속성에 넣는다(못 찾으면 비움). 조건이 그것을 볼 수 있다:

kind	찾는 것	키
file	파일의 전체 경로	path(알려진 폴더와 상대 경로), file, min-version(프로그램용)
dir	폴더	path
registry	레지스트리의 값	root, key, name, view(32비트 보기는 32)
component	다른 제품 컴포넌트의 키 파일	component-guid

`[search.Notepad]` 는 Windows 폴더에서 `notepad.exe` 를 찾아 경로를 `NOTEPAD_PATH` 에 넣는다.

9.7 설치 폴더 기억하기

사용자가 Hello 를 `D:\Tools\Hello` 에 설치했다면 다음 판도 Program Files 로 돌아가지 말고 그리 가야 한다. 두 조각으로 한다:

1. `[registry.RememberDir]` 는 설치할 때 폴더를 레지스트리에 적는다: 값 안의 `[INSTALLDIR]` 은 폴더로 바뀐다 (레지스트리는 11장).
2. `[search.PreviousDir]` 는 그것을 다시 읽는다. `property` 가 `dir ID INSTALLDIR` 이므로 찾은 것이 그 `dir` 의 기본값이 된다 - 그 폴더가 아직 있을 때만이고, 명령줄로 준 폴더가 여전히 이긴다.

9.8 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi LaunchCondition
Installed OR (VersionNT64) [ProductName] needs 64-bit Windows.
C:\work\hello> rubrapack inspect hello.msi AppSearch
NOTEPAD_PATH Notepad
RpFound_PreviousDir PreviousDir
C:\work\hello> rubrapack inspect hello.msi Condition
NotepadHelper 0 NOT Installed AND NOT (NOTEPAD_PATH)
```

(머리줄은 뺐다.) rubrapack 은 요구 조건에 `Installed OR` 를 붙여, 설치된 제품은 늘 복구하거나 지울 수 있게 한다. 기능의 `when` 은 `Condition` 표의 한 행이 되어 조건이 거짓이면 수준을 0 - "설치 안 함, 안 보임" - 으로 만

든다. NOT Installed 는 속성이 사라진 제거 때 이 일이 일어나지 않게 막는다. 바탕화면 바로가기는 컴포넌트를 따로 받았고, 그 Condition 열에 when 이 들어 있다:

```
C_2ea6d5bfd9b4a9073b30 {BCBFF063-...} DesktopFolder 260 DESKTOP_SHORTCUT = "1" R_7f1795ed8aa7ac1a1c60
```

10 누구를 위해 설치하나, 그리고 보호되는 폴더

이 장의 차례

10.1	세 가지 범위	34
10.2	원본	34
10.3	겸용(dual) 패키지	35
10.4	보호되는 설치 폴더: guard	35
10.5	해 보기	35
10.6	안에서 무슨 일이 일어났나	35

목표: 관리자 권한이 없는 사용자도 자기만을 위해 Hello 를 설치하게 하되, 관리자는 여전히 모든 사용자를 위해 설치할 수 있게 한다 - 그리고 남이 미리 만들어 둔 설치 폴더는 거부한다.

10.1 세 가지 범위

지금까지의 패키지는 모두 컴퓨터 전체에 설치했다: Program Files 에, 컴퓨터의 모든 사용자를 위해. 그래서 관리자 권한(사용자 계정 컨트롤 창)이 필요했다. [package] 의 scope 가 이것을 바꾼다:

scope	설치 대상	관리자 필요	ProgramFiles/Hello 는	시작 메뉴, 바탕화면	레지스트리 HKMU
machine(기본)	모든 사용자	예	C:\Program Files\Hello	모든 사용자의 것	HKLM
user	지금 사용자	아니요	%LOCALAPPDATA%\Programs\Hello	사용자의 것	HKCU
dual	기본은 사용자, 고르면 모든 사용자	모든 사용자일 때만	위의 둘 중 하나	둘 중 하나	둘 중 하나

%LOCALAPPDATA% 는 사용자 자신의 C:\Users\<이름>\AppData\Local 이다. 사용자별 설치의 허락이 필요 없고 컴퓨터의 다른 사용자에게 보이지 않는다 - 작은 도구에 흔히 알맞다.

어떤 것은 컴퓨터 전체에만 있어서 scope = "machine" 이 필요하다: 서비스, 글꼴, 권한, 그리고 Windows, System, Fonts, CommonAppData 폴더. rubrapack 은 user 나 dual 패키지에서 이것들을 거부한다.

10.2 원본

```
# tutorial 10: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
scope = "dual"
ui = "installldir"

[define]
VERSION = "1.8.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
guard = true
```

```
[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"
```

10.3 겸용(dual) 패키지

scope = "dual" 에 대화창이 있으면, 약관 다음 페이지가 묻는다:

```
( ) Just me           (기본, 나만)
( ) Everyone on this computer (이 컴퓨터의 모든 사용자)
```

"모든 사용자"는 관리자 권한을 묻는다. 설치 폴더, 시작 메뉴 항목, "설치된 앱" 항목이 선택을 따른다. 대화창이 없으면 겸용 패키지는 지금 사용자를 위해 설치되고, 관리자 터미널에서 이렇게 하면 모든 사용자를 위해 설치된다:

```
msiexec /i hello.msi ALLUSERS=1 MSIINSTALLPERUSER=""
```

(MSIINSTALLPERUSER="" 는 "비움"이다: 속성을 지운다.) user 패키지는 모든 사용자를 위한 설치를 거부한다.

10.4 보호되는 설치 폴더: guard

어떤 프로그램은 다른 프로그램이 불러 쓰거나 높은 권한으로 돈다 - 입력기, 셸 확장, 서비스. 이런 프로그램에게 이미 있는 설치 폴더는 위험하다: 그것을 만든 누군가가 파일을 남겨 두었을 수 있다(심어 둔 DLL 은 프로그램의 권한으로 불러 온다). dir 에 guard = true 를 달면, 그 폴더가 다음과 같을 때 첫 설치가 파일을 하나도 두기 전에 멈춘다:

- 이미 있고, 주인이 SYSTEM, Administrators, TrustedInstaller 가 아니다.
- 교차점(junction)이나 다른 링크를 거쳐 닿는다(그 폴더든 위의 어느 폴더든).

아직 없는 폴더는 통과한다 - 설치 파일이 만든다. 복구, 제거, 그리고 옛 판이 만든 폴더로의 업그레이드는 영향받지 않는다. 설치 는 DirGuardText 안내문(대화창에서 고른 언어로)과 함께 끝나며, 조용한 설치에서도 마찬가지이고, 기록에는 찾은 주인이 적힌다. 검사는 패키지가 싣고 가는 rubrapack 의 작은 도우미 DLL 이 한다.

10.5 해 보기

일반 사용자(관리자가 아님)로 hello.msi 를 두 번 누르고 "Just me" 를 그대로 둔다: 허락 창이 없다. Hello 는 %LOCALAPPDATA%\Programs\Hello 에 들어가고, 시작 메뉴와 "설치된 앱"에서 나에게만 보인다. 지운 뒤 "Everyone" 을 골라 다시 설치하면 Windows 가 관리자 권한을 묻고 Program Files 에 설치한다.

폴더 보호를 보려면: 일반 사용자로 C:\Users\Public\Hello 폴더를 만든다(그러면 주인이 그 사용자다). 관리자 터미널에서 그 폴더에 모든 사용자용으로 설치한다:

```
msiexec /i hello.msi ALLUSERS=1 MSIINSTALLPERUSER="" INSTALLDIR="C:\Users\Public\Hello\"
```

설치가 보호 안내문과 함께 멈추고 아무것도 설치하지 않는다. 폴더를 지우고 같은 명령을 다시 하면 이번에는 설치 파일이 폴더를 직접 만들고 설치가 끝까지 간다.

10.6 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi Property
...
ALLUSERS 2
MSIINSTALLPERUSER 1
```

```
RP_GUARD  INSTALDIR
...
```

ALLUSERS = 2 와 MSIINSTALLPERUSER = 1 의 작은 Windows Installer 의 "둘 다를 위한 단일 패키지"다: 따로 말하지 않으면 사용자별이다. 컴퓨터 전체 패키지는 ALLUSERS = 1 이다. RP_GUARD 는 도우미 DLL 에게 보호할 dir 을 알린다. 묻는 페이지는 Dialog 표의 RpScopeDlg 다.

11 레지스트리, 환경 변수, INI 파일, 파일 형식과 링크

이 장의 차례

11.1	2분 만에 보는 레지스트리	37
11.2	원본	37
11.3	레지스트리 값: [registry.ID]	39
11.4	환경 변수: [env.ID]	40
11.5	INI 파일: [ini.ID]	40
11.6	파일 형식: [assoc.ID]	40
11.7	링크: [protocol.ID]	40
11.8	해 보기	40
11.9	안에서 무슨 일이 일어났나	41

목표: 다른 프로그램이 찾는 설정을 Hello 에 준다 - 모든 형식의 레지스트리 값, 환경 변수, INI 파일의 한 줄 - 그리고 Windows 가 .hello 파일과 hello: 링크를 Hello 로 열게 한다.

11.1 2분 만에 보는 레지스트리

Windows 레지스트리는 폴더처럼 짜인 설정 데이터베이스다: 키는 다른 키와 값을 담고, 값에는 이름, 형식, 데이터가 있다. 맨 위 키("루트")는:

루트	담는 것
HKLM(HKEY_LOCAL_MACHINE)	컴퓨터 전체의 설정; 쓰려면 관리자 권한이 필요하다
HKCU(HKEY_CURRENT_USER)	지금 사용자의 설정
HKCR(HKEY_CLASSES_ROOT)	파일 형식과 링크(둘을 합쳐 보인 것)
HKMU	rubrapack(과 Windows Installer)의 "모든 사용자용이면 HKLM, 한 사용자용이면 HKCU"

Windows 에서 regedit 를 실행하면 둘러볼 수 있다(모르는 것은 바꾸지 않는다).

11.2 원본

```
# tutorial 11: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.9.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Settings]
```

```

dir = "INSTALLDIR"
source = "dist/settings.ini"

[registry.InstallDir]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "InstallDir"
value = "[INSTALLDIR]"

[registry.Greeting]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Greeting"
value = "Hello, world"
with = "file:Hello"

[registry.Runs]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "MaxRuns"
type = "dword"
value = 100

[registry.BigNumber]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Limit"
type = "qword"
value = "0x100000000"

[registry.Colors]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Colors"
type = "multi"
value = ["red", "green", "blue"]

[registry.LogPath]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "LogPath"
type = "expand"
value = "%TEMP%\hello.log"

[registry.Legacy32]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Mode"
value = "compatible"
view = "32"

[registry.OldKey]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello Old'
remove = true

[registry.UserChoice]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "FirstRun"
type = "dword"
value = 1

```

```

keep = true

[env.HelloHome]
name = "HELLO_HOME"
value = "[INSTALLDIR]"

[env.Path]
name = "PATH"
value = "[INSTALLDIR]"
mode = "append"

[ini.InstalledVersion]
dir = "INSTALLDIR"
file = "settings.ini"
section = "hello"
key = "installed"
value = "[ProductVersion]"

[ini.Plugins]
dir = "INSTALLDIR"
file = "settings.ini"
section = "hello"
key = "plugins"
value = "core"
mode = "add"

[assoc.HelloDoc]
extension = ".hello"
prog-id = "Example.HelloDocument"
description = "Hello document"
target = "file:Hello"
icon = "file:Hello"
args = "--open \"%1\""

[protocol.HelloLink]
name = "hello"
description = "Hello link"
target = "file:Hello"

```

TOML 메모: 작은따옴표 문자열은 쓴 그대로 쓰이므로 'SOFTWARE\Example Software\Hello' 에는 백슬래시를 겹쳐 쓸 필요가 없다. 큰따옴표 안에서는 백슬래시가 이스케이프를 시작한다("%TEMP%\hello.log").

11.3 레지스트리 값: [registry.ID]

type	TOML 값	레지스트리 형식
string(기본)	문자열	REG_SZ
expand	%변수% 가 든 문자열	REG_EXPAND_SZ - 값을 읽을 때 Windows 가 %TEMP% 를 펼친다
dword	정수 0 .. 4294967295	REG_DWORD, 32비트
qword	정수, 또는 "0x" 와 16진수 16자리까지	REG_QWORD, 64비트
binary	16진수, "01ff"	REG_BINARY
multi	문자열 배열	REG_MULTI_SZ

- name 이 없으면: 키의 기본값(regedit 에서 "(기본값)").
- value 는 서식 문자열이다: [INSTALLDIR] 은 설치 폴더, [#Hello] 는 파일 Hello 의 경로가 된다.
- 모든 값은 제품과 함께 다시 지워진다. keep = true 는 남긴다(여기서는 프로그램이 바꿀 수 있는 FirstRun).
- with = "file:Hello" 는 값을 그 파일에 묶는다: 파일과 함께 설치되고 지워진다(Greeting).

- view = "32": 64비트 Windows 는 32비트 프로그램을 위한 레지스트리를 따로 둔다. SOFTWARE\Example Software\Hello 를 읽는 32비트 프로그램은 그쪽을 본다. x64 패키지의 값은 view = "32" 가 아니면 64비트 쪽으로 간다.
- remove = true 는 설치하는 동안 name 이 가리키는 값을 - name 이 없으면 키 전체를 - 지운다(OldKey: 옛 판이 남긴 것).

Windows Installer 는 REG_QWORD 를 스스로 쓰지 못한다. qword 값을 위해 rubrapack 은 작은 도우미 DLL 을 패키지에 더하고, 이것이 값을 쓰고 설치가 실패하면 이전 값을 되돌린다.

11.4 환경 변수: [env.ID]

컴퓨터 전체의 변수(사용자별 설치에서는 사용자의 것)다. mode = "set"(기본)은 정하고, append 는 있는 것의 끝에 ;값 을, prepend 는 앞에 값; 을 더한다. 제품을 지우면 정확히 그만큼 되돌린다: HELLO_HOME 은 지워지고, PATH 에서는 ;C:\Program Files\Hello\ 만 빠진다. keep = true 는 남긴다. 설치 뒤에 시작한 프로그램이 바뀐 것을 본다(이미 열려 있던 터미널은 못 본다).

11.5 INI 파일: [ini.ID]

INI 파일은 [섹션] 과 키=값 줄로 된 글 파일이다. mode = "set"(기본)은 섹션에 키=값 을 쓰고, add 는 값을 쉼표로 나눈 목록에 더하고(plugins=core, 또는 plugins=extra 가 있었다면 plugins=extra,core), remove 는 키를 지운다 - 파일을 설치하기 전에 하므로 옛 판이 남긴 INI 파일을 치운다. 제거는 set 과 add 가 쓴 것을 빼낸다. file 은 dir 안의 파일 이름이며, 패키지가 설치하는 파일이 아니어도 된다.

설치 뒤 settings.ini 는:

```
[hello]
greeting=Hello
installed=1.9.0
plugins=core
```

11.6 파일 형식: [assoc.ID]

[assoc.HelloDoc] 는 Windows 가 .hello 파일을 Hello 로 열게 한다:

- extension 은 점을 포함한 소문자 파일 끝이다.
- prog-id 는 문서 종류의 이름이다(회사.종류). 여러 확장자가 하나를 함께 쓸 수 있다.
- description 은 탐색기가 그 형식을 부르는 이름("Hello document")이다.
- target 은 프로그램, args 는 인자다("%1" 은 파일 경로이며 기본값이다).
- icon(file:ID)은 문서가 보일 첫 아이콘을 가진 파일이다.

사용자가 .hello 에 이미 다른 프로그램을 골라 두었다면 Windows 는 그 선택을 지킨다.

11.7 링크: [protocol.ID]

[protocol.HelloLink] 는 hello: 방식을 등록한다. 브라우저나 실행 상자의 hello:world 링크가 링크 전체를 인자로 Hello 를 시작한다.

11.8 해 보기

설치한 뒤:

- regedit: HKEY_LOCAL_MACHINE\SOFTWARE\Example Software\Hello 에 값들이 있고, ...\WOW6432Node\Example Software\Hello 에 Mode(32비트 쪽)가 있다.
- 새 터미널: echo %HELLO_HOME% 과 echo %PATH%.
- C:\Program Files\Hello\settings.ini: 새 두 줄.
- 바탕화면의 test.hello 파일이 Hello 로 열리고, Win+R 에 hello:world, Enter 로 Hello 가 시작된다.

Hello 를 지우면 FirstRun 말고는 모두 사라진다.

11.9 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi Registry
...
Colors 2 SOFTWARE\Example Software\Hello Colors [~]red[~]green[~]blue[~] C_f023...
Greeting 2 SOFTWARE\Example Software\Hello Greeting Hello, world C_185f...
HelloDoc.Ext 0 .hello Example.HelloDocument C_185f...
LogPath 2 SOFTWARE\Example Software\Hello LogPath #%%TEMP%\hello.log C_2e80...
Runs 2 SOFTWARE\Example Software\Hello MaxRuns #100 C_97ef...
...
```

둘째 열은 루트다(0 HKCR, 1 HKCU, 2 HKLM, -1 HKMU). 값 열은 Windows Installer 가 원하는 대로 첫 글자에 형식을 싣는다: #100 은 DWORD, %% 는 펼치는 문자열, [~] 는 여러 문자열을 가르키고, #x 는 이진 데이터를 시작한다. 파일 형식과 링크는 프로그램 컴포넌트 안의 HKCR 아래 레지스트리 값일 뿐이다. QWORD 값은 이 표에 없다: 도우미 DLL 을 위한 속성 RP_QWORDS 에 실려 간다. 환경 변수는 Environment 표에 있다:

```
HelloHome =-*HELLO_HOME [INSTALLDIR] C_0fa9...
Path =-*PATH [~];[INSTALLDIR] C_82df...
```

이름에서 = 는 "정하기", - 는 "제거 때 지우기", * 는 "시스템 변수"다. 값의 [~] 는 원래 있던 것을 뜻한다.

12 서비스, 글꼴, 권한

이 장의 차례

12.1	원본	42
12.2	서비스: [service.ID]	43
12.3	글꼴: [font.ID]	43
12.4	접근 권한: [permission.ID]	43
12.5	다시 시작하기: reboot	44
12.6	해 보기	44
12.7	안에서 무슨 일이 일어났나	44

목표: 스스로 시작하는 백그라운드 서비스, 모든 프로그램이 쓸 수 있는 글꼴, 접근 권한을 내가 정한 데이터 폴더를 설치한다 - 컴퓨터 전체 패키지만 할 수 있는 세 가지다.

12.1 원본

dist\hellosvc.exe 는 서비스 프로그램, dist\HelloSans.ttf 는 글꼴 파일이다.

```
# tutorial 12: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
reboot = "suppress"

[define]
VERSION = "1.10.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.DataDir]
path = "CommonAppData/Hello"

[dir.FontsDir]
path = "Fonts"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Service]
dir = "INSTALLDIR"
source = "dist/hellosvc.exe"

[file.Font]
dir = "FontsDir"
source = "dist/HelloSans.ttf"

[service.HelloService]
file = "file:Service"
name = "HelloService"
```

```

display-name = "Hello background service"
description = "Keeps Hello's greetings up to date."
start = "auto"
account = "LocalService"
args = "--service"
start-on-install = true

[font.HelloSans]
file = "file:Font"
title = "Hello Sans"

[permission.DataFolder]
target = "dir:DataDir"
sddl = "D:PAI(A;OICI;FA;;;SY)(A;OICI;FA;;;BA)(A;OICI;0x1301bf;;;BU)"

```

12.2 서비스: [service.ID]

서비스는 Windows 가 창 없이 뒤에서, 흔히 누군가 로그인하기도 전에 돌리는 프로그램이다(서비스 관리 도구 services.msc 가 목록을 보인다).

키	뜻
file	서비스 프로그램: file: 과 [file.*] ID
name	서비스의 내부 이름(sc start HelloService 에 쓴다)
display-name, description	서비스 관리 도구가 보이는 것
start	auto(Windows 가 시작할 때마다), demand(무언가가 시작할 때), disabled
account	누구로 도나: LocalSystem(모든 권한), LocalService(적은 권한 - 이것을 권한다), NetworkService
args	명령줄 인자
start-on-install = true	설치 끝에 시작

설치 파일은 서비스의 파일을 바꾸기 전(업그레이드나 복구)과 제거 때 서비스를 멈추고, 제거 때 서비스를 지운다. 프로그램은 정말 서비스여야 한다 - Windows 서비스 관리자에 응답하는 것. 보통 프로그램이면 시작이 실패하고, 그러면 설치도 실패한다.

12.3 글꼴: [font.ID]

글꼴은 파일을 글꼴 폴더에 두고 - path = "Fonts", 알려진 폴더만 쓴 [dir.FontsDir] - [font.ID] 로 등록해 설치한다. title 은 Windows 가 보이는 이름이다. 없으면 Windows 가 글꼴 파일에서 이름을 읽는다. 설치 뒤 모든 프로그램이 그 글꼴을 보고, 제거는 등록을 풀고 파일을 지운다.

12.4 접근 권한: [permission.ID]

Windows 의 모든 파일, 폴더, 레지스트리 키에는 누가 읽고 바꿀 수 있는지 말하는 접근 목록이 있다. [permission.ID] 는 SDDL(Security Descriptor Definition Language)로 쓴 그 목록을, 패키지가 만드는 폴더(dir:ID), 패키지의 파일 (file:ID), 패키지가 쓰는 레지스트리 값(registry:ID)에 정한다. 권한을 준 폴더는 파일이 없어도 만들어진다.

위의 SDDL 문자열을 조각조각 보면:

조각	뜻
D:	접근 목록(DACL)이 이어진다
PAI	보호됨(부모 폴더에서 물려받지 않음), 그리고 안에 든 것에 물려줌
(A;OICI;FA;;;SY)	허용(Allow), 파일과 하위 폴더에(Object-Container Inherit), 모든 권한(Full Access), SYSTEM 에게

조각	뜻
(A;0ICI;FA;;;BA)	같은 것을 기본 제공 Administrators 에게
(A;0ICI;0x1301bf;;;BU)	기본 제공 Users 는 읽고 쓰고 지울 수 있지만 권한은 바꾸지 못함

그래서 C:\ProgramData\Hello 는 모든 사용자가 쓸 수 있지만, 누가 쓸 수 있는지는 관리자만 바꾼다. 손으로 설정한 폴더의 문자열은 `icaccls` 나 PowerShell 의 `(Get-Acl C:\경로).Sddl` 로 얻는다.

12.5 다시 시작하기: reboot

어떤 파일은 Windows 를 다시 시작해야 바꿀 수 있다. `reboot = "suppress"`(기본)는 스스로 다시 시작하지 않는다: 설치가 종료 코드 3010("다시 시작 필요")으로 끝나고 사용자가 나중에 다시 시작한다. `reboot = "allow"`는 필요할 때 Windows Installer 가 다시 시작을 묻게(/qn 에서는 다시 시작하게) 한다.

12.6 해 보기

(관리자로) 설치한 뒤: `services.msc` 에 "Hello background service" 가 돌고 있고, 글 편집기가 "Hello Sans" 글꼴을 권하고, C:\ProgramData\Hello 의 보안 탭에 항목 셋이 보인다. Hello 를 지우면 서비스, 글꼴, 폴더가 사라진다.

12.7 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi ServiceInstall
HelloService HelloService Hello background service 16 2 1 NT AUTHORITY\LocalService --service C_d677...
Keeps Hello's greetings up to date.
C:\work\hello> rubrapack inspect hello.msi ServiceControl
HelloService HelloService 163 1 C_d677...
```

16 은 "자기 프로세스의 서비스", 2 는 "자동 시작", 1 은 "오류를 보통으로 알림"이다. `ServiceControl` 의 163 은 비트 플래그의 합이다: 1(설치 때 시작) + 2(설치 때 멈춤) + 32(제거 때 멈춤) + 128(제거 때 삭제). 이런 합이 어떻게 되는지는 제3부의 비트 플래그가 설명한다. 권한이 있는 패키지는 Windows Installer 5.0 을 밝힌다 (요약 정보의 스키마, `rubrapack inspect hello.msi --summary` 의 14 500). `MsiLockPermissionsEx` 표가 나머지도 다 새것이기 때문이다.

13 설치하는 동안 내 프로그램 실행하기

이 장의 차례

13.1	왜 그냥 프로그램을 실행하면 안 되나?	45
13.2	원본	45
13.3	[action.ID]	45
13.4	무엇이 언제 도나	46
13.5	해 보기	46
13.6	안에서 무슨 일이 일어났나	46

목표: 파일이 자리 잡은 뒤 `hello.exe --register` 를 실행해 Hello 가 어딘가(다른 프로그램, 플러그인 호스트, 시스템)에 자신을 등록하게 하고, 제거하기 전에는 `hello.exe --unregister` 를 실행한다. 그리고 무언가 실패하면 모두 되돌린다.

13.1 왜 그냥 프로그램을 실행하면 안 되나?

설치는 트랜잭션이다: 40단계가 실패하면 Windows Installer 는 1~39단계를 되돌려 컴퓨터를 원래대로 한다. 내가 실행한 프로그램은 그 밖에 있다 - Windows 는 그 프로그램이 한 일을 어떻게 되돌릴지 모른다. 그래서 `rubrapack` 은 명령을 둘 받는다. 하는 것과 되돌리는 것. 그리고 일이 어떻게 흘러가든 알맞은 것이 돌게 꾸민다.

13.2 원본

```
# tutorial 13: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.11.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[action.Register]
run = "file:Hello"
do = "--register"
undo = "--unregister"
```

13.3 [action.ID]

키	뜻
run	프로그램: <code>file:</code> 과 이 패키지가 설치하는 <code>.exe</code> 의 ID
do	설치 뒤의 인자 - 복구 때 다시

키	뜻
undo	제거 때, 파일을 지우기 전의 인자
check	받아들이지만 이 판의 rubrapack 은 쓰지 않는다

인자는 쓴 그대로 넘어간다: 서식 문자열이 아니어서 [INSTALLDIR] 은 글자 [INSTALLDIR] 그대로 도착한다. 프로그램은 자기 자리(자기 경로)를 알고, 대개 그것이면 충분하다.

두 명령 모두:

- 아무것도 묻지 않고 끝나야 한다: 창 없이 돌고, 대답해 줄 사람이 없다.
- 두 번 실행해도 안전해야 한다(이미 등록된 것을 등록: 괜찮음).
- 실패를 0 이 아닌 종료 코드로 알린다 - 그러면 설치가 실패하고 되돌려진다.

컴퓨터 전체 패키지에서는 설치 파일의 관리자 권한으로, 사용자별 패키지에서는 사용자로 돈다.

13.4 무엇이 언제 도나

상황	도는 것
첫 설치	파일 설치 뒤 do
복구	do 다시
제거	undo, 그다음 파일이 사라짐
새 판으로 업그레이드	옛 판의 undo(옛 판 제거), 그다음 새 판의 do
do 가 돈 뒤 설치 실패	되돌리기의 일부로 undo
undo 가 돈 뒤 제거 실패	do, 그리고 파일이 돌아옴

13.5 해 보기

이런 일을 하는 진짜 프로그램은 인자를 읽고, 예를 들어 레지스트리 값을 쓰거나 등록 인터페이스를 부른다. 순서를 보려면 hello.exe 가 인자를 기록 파일에 쓰게 하고, 설치, 복구, 제거를 한 뒤 기록을 읽는다.

13.6 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect hello.msi CustomAction
RP_Register_Do 3090 Hello --register
RP_Register_DoRollback 3410 Hello --unregister
RP_Register_RedoRollback 3410 Hello --register
RP_Register_Undo 3090 Hello --unregister
RP_Register_UndoRollback 3410 Hello --register
```

사용자 지정 동작이 다섯이다. 형식 수는 비트 플래그를 더한 것이다: 18("이 패키지가 설치한 .exe 실행") + 1024 ("자연": 설치 스크립트 안에서 파일 복사와 차례를 맞춰 돈다) + 2048("가장 없음": 설치 파일의 권한으로) = 3090. 되돌리기 짝은 256("되돌릴 때만 실행")과 64("종료 코드 무시")를 더한다. InstallExecuteSequence 는 이것들을 파일 동작 둘레에 두고, 각각 hello.exe 컴포넌트의 상태에 대한 조건을 단다:

```
RP_Register_UndoRollback $C_185f...=2 AND ?C_185f...=3 3400
RP_Register_Undo $C_185f...=2 AND ?C_185f...=3 3401
RemoveFiles 3500
InstallFiles 4000
RP_Register_DoRollback $C_185f...>2 AND ?C_185f...<>3 4001
RP_Register_RedoRollback $C_185f...>2 AND ?C_185f...=3 4002
RP_Register_Do $C_185f...>2 4003
```

\$C 는 컴포넌트가 갈 상태(2 없음, 3 설치됨), ?C 는 지금 상태다. "설치되어 있다가 사라짐"이면 undo, "들어옴"이면 do 가 돈다.

14 나만의 대화창 페이지

이 장의 차례

14.1	원본	47
14.2	페이지: [dialog.ID]	49
14.3	페이지 위의 자리	50
14.4	컨트롤: [dialog-control.ID]	50
14.5	모든 값에는 기본값이 있어야 한다: [property.NAME]	50
14.6	해 보기	50
14.7	안에서 무슨 일이 일어났나	51

목표: 설치 폴더 페이지 뒤에 "선택 사항" 페이지를 더한다. 사용자는 거기서 인사 방식을 고르고, 이름을 입력하고, 목록에서 언어를 고르고, 바탕화면 바로가기를 만들지 정한다. 그리고 그 답을 레지스트리에 저장한다.

14.1 원본

```
# tutorial 14: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${(VERSION)}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installDir"

[define]
VERSION = "1.12.0"

[ui]
languages = ["ko"]

[dialog.Options]
after = "RpInstallDirDlg"
title = "Options"
title-ko = "선택 사항"
description = "Choose how Hello greets you."
description-ko = "Hello 가 인사하는 방식을 고르십시오."

[dialog-control.ModeLabel]
dialog = "Options"
type = "text"
x = 20
y = 55
width = 330
height = 12
text = "Greeting &style:"
text-ko = "인사 방식(&S):"

[dialog-control.Mode]
dialog = "Options"
type = "radio"
x = 20
y = 70
```

```

width = 200
height = 42
property = "GREETING_STYLE"
values = ["friendly", "formal", "silent"]
labels = ["&Friendly", "F&ormal", "S&ilent"]
labels-ko = ["친근하게(&F)", "정중하게(&O)", "조용히(&I)"]

[dialog-control.NameLabel]
dialog = "Options"
type = "text"
x = 20
y = 122
width = 330
height = 12
text = "&Your name:"
text-ko = "이름(&Y):"

[dialog-control.Name]
dialog = "Options"
type = "edit"
x = 20
y = 136
width = 200
height = 18
property = "GREETING_NAME"

[dialog-control.LanguageLabel]
dialog = "Options"
type = "text"
x = 20
y = 164
width = 330
height = 12
text = "&Greeting language:"
text-ko = "인사 언어(&G):"

[dialog-control.Language]
dialog = "Options"
type = "combo"
x = 20
y = 178
width = 200
height = 16
property = "GREETING_LANGUAGE"
values = ["en", "ko"]
labels = ["English", "Korean"]
labels-ko = ["영어", "한국어"]

[dialog-control.DesktopBox]
dialog = "Options"
type = "checkbox"
x = 20
y = 204
width = 330
height = 16
property = "DESKTOP_SHORTCUT"
text = "Create a &desktop shortcut"
text-ko = "바탕화면 바로가기 만들기(&D)"

[property.GREETING_STYLE]
value = "friendly"

```

```

[property.GREETING_NAME]
value = "friend"

[property.GREETING_LANGUAGE]
value = "en"

[property.DESKTOP_SHORTCUT]
value = "1"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[registry.StyleValue]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Style"
value = "[GREETING_STYLE]"

[registry.NameValue]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Name"
value = "[GREETING_NAME]"

[registry.LanguageValue]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Language"
value = "[GREETING_LANGUAGE]"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
when = "DESKTOP_SHORTCUT"

```

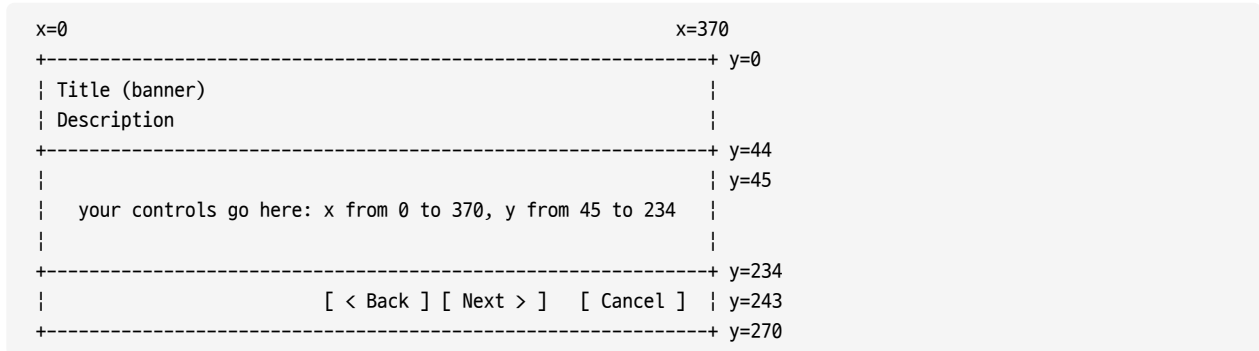
14.2 페이지: [dialog.ID]

키	뜻
after	이 페이지가 뒤따를 페이지: RpWelcomeDlg, RpLicenseDlg(약관이 있을 때), RpInstallDirDlg(installdir, features), RpCustomizeDlg(features), 또는 내가 만든 다른 페이지
title, description	배너의 제목과 그 아래 한 줄(제목 기본값: 제품 이름)
title-xx, description-xx	[ui] languages 의 다른 언어로 쓴 같은 것

페이지는 내장 페이지와 같은 배너와 뒤로 / 다음 / 취소 단추를 받고, 본문은 내가 채운다. 같은 페이지 뒤에 오는 페이지가 여럿이면 ID 순서로 온다. `minimal`에서는 마지막 페이지의 단추가 "설치"가 되고, 다른 세트에서는 늘 "준비" 페이지가 마지막에 온다. 페이지는 `minimal`, `installdir`, `features`에서 쓸 수 있다.

14.3 페이지 위의 자리

위치와 크기는 픽셀이 아니라 대화창 단위다. Windows 가 글꼴에 맞춰 늘이고 줄이므로, 페이지는 어느 화면 크기에서도 같게 보인다. 모든 페이지는 370 x 270 단위다:



그림의 이름은 틀의 컨트롤 ID 그대로다: Title 은 제목, Description 은 설명, Back / Next / Cancel 은 뒤로 / 다음 / 취소 단추다.

Tab 키는 내 컨트롤을 위에서 아래로, 그다음 왼쪽에서 오른쪽으로 옮겨 다닌다.

14.4 컨트롤: [dialog-control.ID]

컨트롤마다 자기 dialog, type, 그리고 x, y, width, height 를 적는다:

type	보이는 것	속성
text	글자(text)	-
checkbox	글자가 붙은 확인란	표시하면 1, 표시를 지우면 속성이 없어진다
edit	입력하는 한 줄 칸	입력한 글
radio	값마다 단추 하나, 그중 하나를 고름	values 중 하나
combo	펼쳐지는 목록	values 중 하나

- values(문자열 1~32개)는 속성이 받는 값이고, labels 는 사용자가 보는 글로 같은 순서다(기본값: 값 그대로). labels-xx 와 text-xx 가 번역이다.
- 라디오 묶음은 값 하나에 높이 12단위쯤이 필요하다.
- 글자 안의 & 는 Alt 키를 표시한다: edit 앞의 text 에 두면 Alt+Y 가 그 입력 칸으로 쏜다. 한국어 글에서는 이름(&Y): 처럼 괄호 안에 쓰는 것이 Windows 의 관례다.
- 컨트롤 ID Banner, Title, Description, BannerLine, BottomLine, Back, Next, Cancel 은 틀의 것이고, Rp 로 시작하는 페이지 ID 는 rubrapack 의 것이다. 한 페이지에 컨트롤은 64개까진다.

14.5 모든 값에는 기본값이 있어야 한다: [property.NAME]

대화창은 값을 모을 뿐이다. 조용한 설치(/qn)는 페이지를 하나도 보이지 않고 속성을 있는 그대로 쓴다. 그래서 컨트롤이 정하는 속성마다 기본값을 담은 [property.*] 가 있고, radio 와 combo 의 기본값은 values 중 하나여야 한다. 같은 속성은 명령줄에서도 줄 수 있다:

```
msiexec /i hello.msi /qn GREETING_STYLE=formal GREETING_NAME=Ada DESKTOP_SHORTCUT=""
```

rubrapack 은 내 컨트롤의 속성을 스스로 보안 속성으로 만들어, 설치의 관리자 권한 부분까지 전해지게 한다: [registry.StyleValue] 가 [GREETING_STYLE] 을 쓰고, 바탕화면 바로가기의 when = "DESKTOP_SHORTCUT" 이 확인란을 본다.

14.6 해 보기

설치한다: 폴더 페이지 뒤에 내 컨트롤이 든 "Options" 가 온다. 뒤로는 폴더 페이지로, 다음은 "준비"로 간다. 언어 페이지에서 한국어를 고르면 페이지 제목과 글자가 한국어("선택 사항")다. 설치한 뒤 regedit 을 열면

HKLM\SOFTWARE\Example Software\Hello 아래에 Style, Name, Language 가 있다. 바탕화면 바로가기는 확인란을 표시했을 때만 있다.

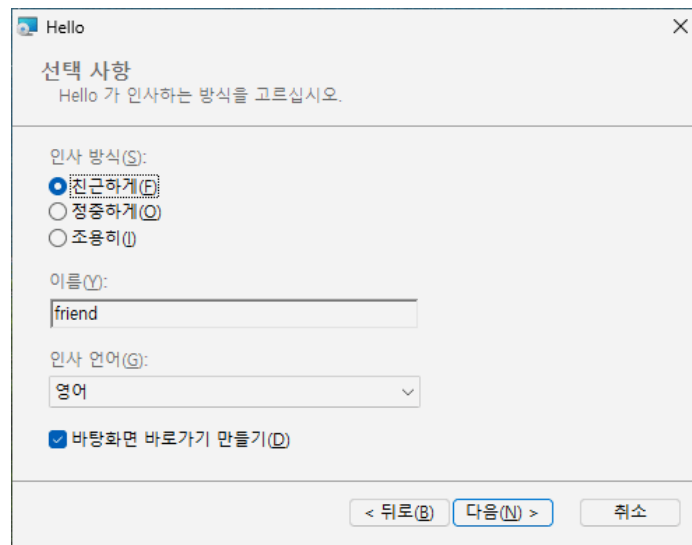


그림 10: 한국어로 본 "선택 사항" 페이지

14.7 안에서 무슨 일이 일어났나

페이지는 Control 표의 행들 - 틀의 것과 내 것 - 이고, 단추는 ControlEvent 의 행들이다:

```
C:\work\hello> rubrapack inspect hello.msi ControlEvent
...
RpInstallDirDlg Next NewDialog Options 1 1
Options Back NewDialog RpInstallDirDlg 1 1
Options Next NewDialog RpReadyDlg 1 1
RpReadyDlg Back NewDialog Options 1 1
...
```

"폴더 페이지에서 다음을 누르면 Options 를 보여라". Options 의 뒤로와 다음은 이웃 페이지로 이어진다. rubrapack 은 내 페이지가 들어가도록 내장 페이지들의 연결을 바꾼다. Control 표의 속성 수는 여기서도 비트 플래그다(1 보임, 2 사용 가능, 65536 투명, ...).

15 캐비닛, 압축, 큰 패키지, 여러 아키텍처

이 장의 차례

15.1	폴더	52
15.2	원본	52
15.3	캐비닛과 압축	53
15.4	여러 아키텍처: --arch	53
15.5	매번 같은 바이트: --reproducible	53
15.6	Mac 에서 온 이름: --nfc	54
15.7	명령줄의 변수: -D	54
15.8	안에서 무슨 일이 일어났나	54

목표: 파일을 어떻게 담을지 - 압축, 패키지 파일 하나 또는 패키지와 캐비닛들 - 를 정하고, 원본 하나로 Hello 를 x64, x86, Arm64 용으로 빌드하고, 빌드가 바이트 하나까지 되풀이되게 한다.

15.1 폴더

이제 프로그램은 처리기(프로세서) 종류마다 하나씩, 세 벌로 빌드되어 있다:

```
dist\
  x64\hello.exe
  x86\hello.exe
  arm64\hello.exe
  docs\guide.txt
```

15.2 원본

```
# tutorial 15: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
upgrade-code-x86 = "{7D1C2B3A-4E5F-4061-9728-3A4B5C6D7E8F}"
upgrade-code-arm64 = "{0E9F8D7C-6B5A-4948-8372-6150F4E3D2C1}"
compress = "mszip:9"
cab = "external"
cab-max-size = 500

[define]
VERSION = "2.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/${ARCH}/hello.exe"

[files.DocFiles]
```

```
dir = "INSTALLDIR"
glob = "dist/docs/**"
```

15.3 캐비닛과 압축

MSI 안의 파일은 하나하나 따로 저장되지 않는다: 캐비닛(.cab)에 담긴다. 캐비닛은 Windows 가 1990년대부터 써 온 압축 묶음 형식이다. [package] 의 키:

키	값	효과
compress	mszip:6(기본값), mszip:0 .. mszip:9, mszip(= 6), none	얼마나 세게 압축할지. 9 가 가장 작고 느리며, 1 이 가장 빠르다. none 은 파일을 그대로 저장한다
cab	embed(기본값), external	캐비닛을 .msi 안에 둘지, 옆에 hello-x64.cab 으로 둘지
cab-max-size	MiB 수	데이터가 그만큼 차면 새 캐비닛을 시작한다(-1.cab, -2.cab, ...)

대부분의 프로그램에는 기본값이 맞다: 모든 것을 담은 .msi 하나. 외부 캐비닛은 아주 큰 제품을 위한 것이다: **2 GiB 이상인 MSI 는 Windows Installer 가 열지 못한다.** 그래서 그만큼 큰 제품은 cab = "external" 을 써야 하고(rubrapack 은 넣어 두는 형태를 RP1516 으로 거부한다), 그때는 .msi 와 .cab 파일들을 늘 함께 - 같은 폴더, 같은 공유 폴더에 - 두어야 한다. cab-max-size 가 없으면 외부 캐비닛은 하나가 2 GiB 아래가 되게 나뉜다.

MiB(메비바이트)는 1024 x 1024 바이트다. MB 와 GiB 같은 단위는 제3부의 단위에서 설명한다.

압축은 컴퓨터의 모든 처리기에서 동시에 돈다. --jobs 2 는 많아야 둘만 쓴다. 처리기 16개로 200 MB 프로그램을 mszip:6 으로 압축하면 5초쯤 걸리고 105 MB 가 된다.

15.4 여러 아키텍처: --arch

arch = "x64" 는 원본 자신의 아키텍처다. --arch x86 은 같은 원본을 다른 아키텍처로 빌드한다:

```
C:\work\hello> rubrapack build hello.toml -o out-x64\hello-x64.msi --arch x64
C:\work\hello> rubrapack build hello.toml -o out-x86\hello-x86.msi --arch x86
C:\work\hello> rubrapack build hello.toml -o out-arm64\hello-arm64.msi --arch arm64
```

- \$(ARCH) 는 내장 변수로, 지금 빌드하는 아키텍처인 x64, x86, arm64 중 하나다. 여기서는 x86 패키지에 dist/x86/hello.exe 를 고른다. rubrapack 은 프로그램마다 이것과 맞는지 확인한다.
- ProgramFiles 도 따라간다: x64 와 Arm64 는 C:\Program Files, x86 은 C:\Program Files (x86).
- **아키텍처마다 업그레이드 코드가 따로 있어야 한다** - upgrade-code-x86, upgrade-code-arm64(원본 자신의 arch 가 다른 것이라면 upgrade-code-x64 도). Windows 의 업그레이드 논리는 아키텍처를 구별하지 못한다: 코드 하나를 함께 쓰면 x86 패키지를 설치할 때 x64 패키지가 "옛 판"으로 지워진다. 그것들이 없으면 --arch 는 거부된다.

외부 캐비닛은 패키지 이름을 따서 이름이 붙고 서로 부딪치면 안 되므로, 여기서는 출력마다 폴더를 따로 두었다.

15.5 매번 같은 바이트: --reproducible

보통의 빌드는 패키지마다 새 무작위 패키지 코드(파일 요약 정보에 든 GUID)를 주므로, 같은 원본을 두 번 빌드해도 결과가 다르다. --reproducible 은 그것을 내용에서 끌어낸다: 같은 원본과 파일은 어느 컴퓨터에서든, 처리기가 몇 개든 바이트 하나까지 같은 .msi 를 낸다:

```
C:\work\hello> rubrapack build hello.toml -o r1.msi --reproducible --jobs 1
C:\work\hello> rubrapack build hello.toml -o r2\r1.msi --reproducible
C:\work\hello> fc /b r1.msi r2\r1.msi
FC: 다른 점이 없습니다.
```

(`fc /b` 는 두 파일을 바이트 단위로 비교하는 Windows 명령이다.) 그러면 누구나 공개된 패키지가 공개된 원본에서 빌드되었는지 확인할 수 있다. 패키지 코드는 `rubrapack inspect hello.msi --summary` 가 보여 준다(속성 9).

15.6 Mac 에서 온 이름: `--nfc`

유니코드는 한 이나 `é` 같은 글자를 두 가지로 쓸 수 있다 - 코드 포인트 하나로, 또는 글자와 결합 표시들로. macOS 파일 시스템은 이름을 두 번째 형태로 저장한다. `--nfc` 는 파일과 폴더 이름을 Windows 가 쓰는 첫 번째 형태(NFC)로 바꾼다. `lint` 는 NFC 가 아닌 이름을 경고한다(RP2105). 유니코드 정규화는 제3부가 설명한다.

15.7 명령줄의 변수: `-D`

`-D NAME=VALUE` 는 어느 `[define]` 변수든 정하고, 몇 번이든 줄 수 있다: `-D VERSION=2.0.1 -D EDITION=Pro. $(` 는 글자 그대로의 `$(` 를 쓴다.

15.8 안에서 무슨 일이 일어났나

```
C:\work\hello> rubrapack inspect out-x64\hello-x64.msi Media
1 2  hello-x64.cab
C:\work\hello> rubrapack inspect out-x86\hello-x86.msi --summary
...
7 Intel;1033
```

`Media` 표는 캐비닛을 나열한다: 디스크 1 이 파일 1~2 를 `hello-x64.cab` 에 담는다(패키지 안에 넣은 캐비닛은 `#cab1.cab` 으로 쓴다. `#` 은 "이 파일 안에"라는 뜻이다). 요약 정보의 템플릿(속성 7)은 플랫폼(x64, Arm64, x86 은 Intel)과 언어를 적는다. 제4부가 캐비닛 형식 - CFHEADER, CFFOLDER, CFDATA - 과 MSZIP 압축을 바이트 단위로 보여 준다.

16 서명과 타임스탬프

이 장의 차례

16.1	서명이란	55
16.2	키 파일로 서명하기	55
16.3	타임스탬프: 인증서보다 오래 사는 서명	56
16.4	Windows 인증서 저장소의 키	56
16.5	하드웨어 토큰: PKCS#11	56
16.6	서명 확인하기: verify	57
16.7	외부 캐비닛은 서명되지 않는다	57
16.8	안에서 무슨 일이 일어났나	57

목표: hello.msi 에 서명해, Windows 가 게시자를 "알 수 없는 게시자" 대신 "Example Software" 로 보이게 하고, 누구도 몰래 고치지 못하게 한다 - 키 파일로, Windows 인증서 저장소의 키로, 또는 하드웨어 토큰으로.

16.1 서명이란

디지털 서명은 파일 내용과 비밀인 개인 키로 계산한 수다. 짝이 되는 공개 키로 누구나 그것을 확인할 수 있다. 공개 키는 인증서 안에 든다. 인증서는 "이 공개 키는 Example Software 의 것이다"라고 적은 작은 문서로, 그 자체가 인증 기관(CA)의 서명을 받은 것이다. Windows 는 믿는 CA 목록을 가지고 있고, 그중 한 곳의 인증서면 사용자 계정 컨트롤 창에 내 이름이 나온다. 패키지의 바이트 하나만 바뀌어도 서명은 더 이상 맞지 않는다. 해시와 키, 인증서, 서명은 제3부가 차근차근 설명한다.

인증서를 얻는 곳:

- **실제 사용자에게 배포할 때:** CA 에서 사는 코드 서명 인증서. 2023년부터 그 개인 키는 하드웨어 - USB 토큰이나 클라우드 서비스 - 안에만 있어야 하고 파일로 둘 수 없다. rubrapack 은 아래의 PKCS#11 토큰이나 Windows 인증서 저장소를 통해 그런 키로 서명한다.
- **배우고 시험할 때:** 직접 만든 인증서. Windows 는 그것을 믿지 않지만(시험 컴퓨터의 신뢰 목록에 더하지 않는 한 창에는 여전히 "알 수 없는 게시자"가 나온다), 나머지는 모두 똑같이 된다. Windows 의 PowerShell 에서:

```
PS> New-SelfSignedCertificate -Type CodeSigningCert -Subject "CN=Example Software" -CertStoreLocation Cert:
\CurrentUser\My
```

인증서의 지문(thumbprint) - 인증서를 가리키는 16진수 40자리 - 이 출력된다.

16.2 키 파일로 서명하기

키 파일은 개인 키와 인증서를 비밀번호로 보호해 담는다: Windows 가 내보내는 .pfx(.p12 라고도 한다. Export-PfxCertificate 에 -CryptoAlgorithmOption AES256_SHA256), 또는 PEM 키. 비밀번호는 명령줄에 절대 적지 않는다(다른 프로그램이 거기서 읽을 수 있다): 환경 변수나 파일에서 온다. 비밀번호를 받을 법한 옵션은 그렇게 말하려고만 있다:

```
C:\work\hello> rubrapack sign hello.msi --key signer.pfx --pass secret
rubrapack: error[RP0004]: a password is never taken on the command line (others can see it); use --pass-env or --pass-file
```

(토큰 PIN 의 --pin 도 같은 답을 한다.)

```
C:\work\hello> set SIGN_PASS=...비밀번호...
C:\work\hello> rubrapack sign hello.msi --key signer.pfx --pass-env SIGN_PASS -o hello-signed.msi
rubrapack: warning[RP0011]: 'hello.msi': no --timestamp: the signature stops being valid when the certificate expires
signed hello-signed.msi
```

-o 가 없으면 파일 자체에 서명한다. 이미 서명된 파일은 거부한다 - 서명 안 된 파일에 다시 서명한다. 인증서는 코드 서명용이어야 하고, CA 의 인증서가 아니어야 하며, 오늘 유효해야 한다. --pass-file pass.txt 는 비밀번호를 파일에서 읽는다. --cert chain.pem 은 키 파일에 없는 인증서들(CA 의 중간 인증서 - Windows 가 내 인증서를 믿는 루트까지 잇는 데 필요하다)을 더한다.

빌드하면서 서명하면 한 번에 된다:

```
C:\work\hello> rubrapack build hello.toml -o hello.msi --key signer.pfx --pass-env SIGN_PASS
```

sign 은 프로그램(.exe, .dll)과 MSIX 패키지(다음 장)에도 서명한다. 빌드 전에 hello.exe 에 서명해 두면 프로그램 자체에도 내 이름이 붙는다.

16.3 타임스탬프: 인증서보다 오래 사는 서명

인증서는 1~3년 동안 유효하다. 위의 경고가 그 뒤의 일을 말한다: 서명이 더는 효력이 없다. 타임스탬프가 이것을 해결한다: 타임스탬프 서버(CA 들이 운영하고, 무료로 쓸 수 있다)가 "이 서명은 이 시각에 있었다"고 서명하면, 그 서명은 그 뒤로 계속 유효하다.

```
C:\work\hello> rubrapack sign hello.msi --key signer.pfx --pass-env SIGN_PASS --timestamp http://timestamp.digicert.com
```

배포하는 것에는 늘 타임스탬프를 넣는다. 특별한 네트워크를 위한 옵션: --proxy http://host:port, --tls-trust (https:// 타임스탬프 서버에 믿을 인증서), --system-roots(그 연결에 컴퓨터 자신의 루트 인증서를 믿는다), --tsa-trust(타임스탬프 자체에 믿을 인증서).

16.4 Windows 인증서 저장소의 키

위에서 만든 인증서는 내 인증서 저장소에 있다. 지문으로 그것을 써서 서명한다 - Windows 에서만 된다:

```
C:\work\hello> rubrapack keys list
CurrentUser\My thumbprint=1a2b... RSA publisher=CN=Example Software until=...
C:\work\hello> rubrapack sign hello.msi --key-store 1a2b...
```

--machine-store 는 내 저장소 대신 컴퓨터의 저장소(LocalMachine)에서 찾는다. 드라이버가 Windows 에 연결되는 USB 토큰의 키도 여기에 보인다.

16.5 하드웨어 토큰: PKCS#11

토큰 제조사는 PKCS#11 모듈 - 토큰 대신 말하는 DLL(Linux 에서는 .so) - 을 준다. rubrapack 은 그것과 직접 말한다:

```
C:\work\hello> rubrapack keys list --pkcs11 C:\token\pkcs11.dll --pin-env TOKEN_PIN
C:\work\hello> rubrapack sign hello.msi --pkcs11 C:\token\pkcs11.dll --key-label "Code Signing" --pin-env TOKEN_PIN --timestamp http://timestamp.digicert.com
```

--key-label 은 이름표로 키를 고르고, --token-label 은 토큰이 여럿 꽂혀 있을 때 토큰을 고르며, --pin-env/--pin-file 이 PIN 을 준다(명령줄에는 절대 적지 않는다).

16.6 서명 확인하기: verify

```
C:\work\hello> rubrapack verify hello-signed.msi --trust ca.pem
hello-signed.msi
  structure: ok
  digest:    ok
  signature: ok
  chain:     trusted
  revocation: not-checked
  timestamp: not-present
```

- **structure:** 서명이 있어야 할 자리에 있다. **digest:** 내용이 바뀌지 않았다. **signature:** 인증서의 키로 만든 서명이다.
- **chain:** 인증서가 내가 믿는 인증서까지 이어진다 - --trust 로 주거나, 컴퓨터가 믿는 루트는 --system-roots 로. 둘 다 없으면 verify 는 판단할 수 없다고 말한다(종료 코드 4).
- **timestamp:** 있고 유효한지, 없는지.

서명 안 된 파일은 structure: invalid ... the package is not signed 라고 나온다.

16.7 외부 캐비닛은 서명되지 않는다

서명은 .msi 만 덮는다. cab = "external"(15장)이면 누군가 .cab 파일을 몰래 바꿔치기할 수 있으므로 sign 은 그런 패키지를 거부한다:

```
C:\work\hello\out-x64> rubrapack sign hello-x64.msi --key signer.pfx --pass-env SIGN_PASS
rubrapack: error[RP0011]: 'hello-x64.msi': the package uses cabinets outside itself, which a signature of the .msi
does not cover; build it with embedded cabinets, or sign anyway with --allow-unsigned-cabs
```

--allow-unsigned-cabs 는 경고와 함께 .msi 에만 서명한다. 서명할 패키지에는 넣어 둔 캐비닛을 쓰는 편이 낫다.

16.8 안에서 무슨 일이 일어났나

MSI 의 서명은 숨은 스트림 \005DigitalSignature 로, PKCS#7 구조를 담는다: 다른 모든 스트림의 해시, 내 인증서 사슬, 그리고 그 위의 서명. 제4부(Authenticode)가 해시를 어떻게 계산하고 각 부분이 어디에 있는지 보여 준다.

17 MSIX 패키지와 번들

이 장의 차례

17.1	MSI 인가 MSIX 인가?	58
17.2	폴더	58
17.3	원본	59
17.4	패키지의 정체: [msix]	60
17.5	애플리케이션: [msix-app.ID]	60
17.6	덤: [msix-extension.ID]	60
17.7	다른 표들은 무엇이 되나	60
17.8	시험용 빌드: --unsigned-test	61
17.9	서명하기	61
17.10	모든 아키텍처를 파일 하나에: 번들	61
17.11	압축	61
17.12	안에서 무슨 일이 일어났나	62

목표: 같은 원본으로 Hello 를 MSIX 패키지 - Windows 의 더 새로운 패키지 형식 - 로도 빌드하고, 이어서 세 아키텍처를 모두 담은 번들 하나로 빌드한다.

17.1 MSI 인가 MSIX 인가?

	MSI (.msi)	MSIX (.msix)
설치	패키지가 말하는 곳 어디든, 시키는 대로 컴퓨터를 바꾼다	Windows 가 관리하는 봉인된 폴더에. 앱이 쓰는 파일과 레지스트리는 따로 보 관된다
제거	패키지가 만든 만큼 깨끗하게	늘 완전히: 아무것도 남지 않는다
서명	권장	필수: Windows 는 서명 안 된 MSIX 를 설치하지 않는다(아래의 시험용은 예 외)
할 수 있는 것	이 튜토리얼의 모든 것	파일, 바로가기, 파일 형식, 레지스트리, 글꼴, 명령줄 별칭, 로그인할 때 시작
할 수 없는 것	-	서비스, 사용자 지정 동작, 환경 변수, INI 파일, 권한, 조건, 대화창

많은 제품이 둘 다 낸다. rubrapack 은 원본 하나로 어느 쪽이든 빌드한다: 출력 파일의 확장자가 정한다.

17.2 폴더

원본 옆 assets\ 에 로고 셋을 둔다: 정확히 150x150, 44x44, 50x50 픽셀인 PNG 그림이다. 시작 메뉴와 설정 앱이 이것을 보인다. 없으면 패키지는 단색의 밋밋한 로고를 받는다 - 셋을 모두 주거나 하나도 주지 않는다.

```
C:\work\hello\
  assets\
    Square150x150.png
    Square44x44.png
    StoreLogo.png
  dist\
    x64\hello.exe
    x86\hello.exe
    arm64\hello.exe
```

```
docs\guide.txt
hello.toml
```

17.3 원본

```
# tutorial 17: hello.toml

[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
upgrade-code-x86 = "{7D1C2B3A-4E5F-4061-9728-3A4B5C6D7E8F}"
upgrade-code-arm64 = "{0E9F8D7C-6B5A-4948-8372-6150F4E3D2C1}"

[define]
VERSION = "2.1.0"

[msix]
identity-name = "ExampleSoftware.Hello"
publisher = "CN=Example Software"
publisher-display-name = "Example Software"
min-version = "10.0.17763.0"

[msix-app.Hello]
executable = "Hello"
display-name = "Hello"
description = "Says hello."
logo-150 = "assets/Square150x150.png"
logo-44 = "assets/Square44x44.png"
store-logo = "assets/StoreLogo.png"

[msix-extension.Command]
kind = "alias"
alias = "hello.exe"

[msix-extension.AtSignIn]
kind = "startup-task"
display-name = "Hello"
enabled = false

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/${ARCH}/hello.exe"

[files.DocFiles]
dir = "INSTALLDIR"
glob = "dist/docs/**"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

[assoc.HelloDoc]
extension = ".hello"
prog-id = "ExampleSoftware.HelloDocument"
description = "Hello document"
```

```
target = "file:Hello"

[registry.Greeting]
root = "HKMU"
key = "Software\\Example Software\\Hello"
name = "Greeting"
value = "Hello"

[env.HelloHome]
name = "HELLO_HOME"
value = "[INSTALLDIR]"
msi-only = true
```

17.4 패키지의 정체: [msix]

키	뜻
identity-name	Windows 가 부르는 패키지 이름. A-Z a-z 0-9 . - 로 된 3~50자이고, 관례는 회사.제품 이다. MSI 의 업그레이드 코드처럼 절대 바꾸지 않는다: 이름과 게시자가 같은 새 판이 옛 판을 대신 한다.
publisher	서명할 인증서의 주체(subject)를 Windows 가 쓰는 방식 그대로(다음 절)
publisher-display-name	사람들이 보는 게시자 이름. 기본값은 [package] manufacturer
min-version	설치할 수 있는 가장 오래된 Windows, 10.0.<빌드>.0 꼴. 기본값 10.0.17763.0 은 Windows 10 버전 1809 다

판 번호는 [package] version 에 네 번째 부분을 붙인 것이다: 2.1.0 은 2.1.0.0 이 된다.

17.5 애플리케이션: [msix-app.ID]

MSIX 는 자기가 담은 애플리케이션 - 시작 메뉴의 항목 - 을 나열한다. executable 은 그것을 시작하는 [file.*] 를 가리키고, 그 파일의 폴더(여기서는 INSTALLDIR)가 패키지 자신의 폴더가 된다. display-name 과 description 의 기본값은 패키지 이름이다. [msix-app.*] 표가 여럿이면 항목도 여럿이 된다.

17.6 덤: [msix-extension.ID]

- kind = "alias": 콘솔에서 hello.exe 라고 치면 어느 폴더에서든 애플리케이션이 시작된다 - 프로그램을 PATH 에 두는 MSIX 식 방법이다.
- kind = "startup-task": 사용자가 로그인할 때 애플리케이션이 시작된다(한 번 실행한 뒤부터). enabled = false 는 사용자가 작업 관리자의 시작 앱에서 켜 때까지 꺼 둔다. 그 목록에는 display-name 이 보인다. task-id 는 Windows 가 부를 작업 이름이고, 기본값은 표의 ID 다(아래 매니페스트에 보이듯 여기서는 AtSignIn).

[msix-app.*] 표가 여럿이면 app = "Hello" 로 덤이 어느 애플리케이션의 것인지 밝힌다. 기본값은 첫 번째다. MSI 빌드는 이 표들을 뺀다.

17.7 다른 표들은 무엇이 되나

- Programs 에 있고 애플리케이션의 실행 파일을 가리키는 [shortcut.StartMenu] 는 애플리케이션의 시작 메뉴 항목 그 자체다. Desktop 의 바로가기도 min-version = "10.0.19645.0" 부터 된다. 다른 곳의 바로가기, 인수 나 icon 이 있는 바로가기는 MSI 전용이다(RP1613).
- [assoc.HelloDoc] 는 애플리케이션의 파일 형식 연결이 된다. 그 파일 형식은 애플리케이션의 로고로 보인다.
- [registry.Greeting] 은 패키지의 가상 레지스트리에 들어간다: 애플리케이션은 값이 실제 레지스트리에 있는 것처럼 보이지만, 컴퓨터의 레지스트리는 그대로다. Software 아래의 키만 들어갈 수 있다(RP1612).
- [env.HelloHome]: MSIX 는 환경 변수를 정하지 못한다. msi-only = true 가 없으면 빌드가 멈춘다:

```
C:\work\hello> rubrapack build hello.toml -o hello.msix --unsigned-test
hello.toml:65:1: error[RP1605]: [env.HelloHome] cannot go into an MSIX; add msi-only = true to build the MSIX
without it
```

msi-only = true 는 그 표를 MSI 에는 두고 MSIX 에서는 뺀다. rubrapack 은 아무것도 몰래 빼지 않는다: MSIX 가 담지 못하는 것은 내가 뜻을 밝힐 때까지 오류다.

기능, 속성, 대화창, [arp] 는 Windows Installer 에만 해당하므로 쓰이지 않는다. rubrapack lint hello.toml --target msix 는 빌드하지 않고 원본을 MSIX 기준으로 검사한다.

17.8 시험용 빌드: --unsigned-test

```
C:\work\hello> rubrapack build hello.toml -o hello.msix --unsigned-test
C:\work\hello> rubrapack build hello.toml -o hello.msi
```

Windows 는 패키지가 시험용이라고 밝히고 설치하는 쪽도 그것을 받아들인다고 할 때만 서명 안 된 MSIX 를 설치한다. --unsigned-test 가 그 표시를 더한다(게시자가 바뀌므로 서명한 패키지와는 다른 패키지다). 프로그램이 들어 있으니 관리자 권한 PowerShell 에서 설치한다:

```
PS C:\work\hello> Add-AppxPackage -Path hello.msix -AllowUnsigned
PS C:\work\hello> hello.exe
PS C:\work\hello> Get-AppxPackage ExampleSoftware.Hello | Remove-AppxPackage
```

이런 패키지는 내 시험 컴퓨터에서만 쓴다.

17.9 서명하기

다른 사람이 설치하게 하려면 서명한다(16장). 이때 MSIX 의 publisher 는 인증서의 주체와 정확히 같아야 한다. 다르면 서명이 멈추고 써야 할 주체를 알려 준다:

```
C:\work\hello> rubrapack build hello.toml -o hello.msix --key signer.pfx --pass-env SIGN_PASS
rubrapack: error[RP0011]: 'hello.msix': the package's publisher "CN=Example Software" is not the certificate's
subject "O=Example Software Ltd, CN=Example Software"; set [msix] publisher to it (without --unsigned-test)
```

그 주체를 publisher 에 옮겨 적는다. Windows 는 주체의 부분들을 마지막 것부터 거꾸로 쓰므로, 어떤 도구가 보여 주는 순서와 다를 수 있다.

17.10 모든 아키텍처를 파일 하나에: 번들

.msixbundle 로 끝나는 출력은 15장처럼 --arch 의 아키텍처마다 원본을 한 번씩 빌드해 패키지들을 파일 하나에 담는다:

```
C:\work\hello> rubrapack build hello.toml -o hello.msixbundle --arch x64,x86,arm64 --unsigned-test
C:\work\hello> rubrapack inspect hello.msixbundle --files
AppxMetadata\AppxBundleManifest.xml 1616 deflate
ExampleSoftware.Hello_2.1.0.0_x64.msix 11009 stored
ExampleSoftware.Hello_2.1.0.0_x86.msix 11096 stored
ExampleSoftware.Hello_2.1.0.0_arm64.msix 11012 stored
```

Windows 는 그중 자기 처리기에 맞는 패키지를 설치한다. --key 를 주면 번들과 그 안의 패키지 모두에 서명한다.

17.11 압축

파일은 deflate 로 압축되고, 그림처럼 이미 압축된 파일은 그대로 저장된다. --msix-compress store 는 모두 그대로 저장한다. 패키지는 커지지만(여기서는 11009 에서 29961 바이트로) 여는 것은 빨라진다. MSIX 에는 날짜도 시각도 들어 있지 않다: 같은 원본은 어느 컴퓨터에서든 같은 바이트를 낸다.

17.12 안에서 무슨 일이 일어났나

MSIX 는 ZIP 압축 파일이다. 파일 목록을 본다:

```
C:\work\hello> rubrapack inspect hello.msix --files
hello.exe 17920 deflate
guide.txt 6 deflate
Assets\Square150x150.png 301 stored
Assets\Square44x44.png 111 stored
Assets\StoreLogo.png 117 stored
Registry.dat 8192 deflate
AppxManifest.xml 2537 deflate
```

Registry.dat 가 가상 레지스트리 - 레지스트리 하이버 파일 - 다. AppxManifest.xml 은 패키지를 설명하고, --manifest 가 그것을 출력한다:

```
C:\work\hello> rubrapack inspect hello.msix --manifest
...
<Identity Name="ExampleSoftware.Hello" Publisher="CN=Example Software,
OID.2.25.311729368913984317654407730594956997722=1" Version="2.1.0.0" ProcessorArchitecture="x64" />
...
<Application Id="Hello" Executable="hello.exe" EntryPoint="Windows.FullTrustApplication">
...
    <uap3:FileTypeAssociation Name="examplesoftware.hellodocument" Parameters="&quot;%1&quot;">
...
    <desktop:StartupTask TaskId="AtSignIn" Enabled="false" DisplayName="Hello" />
...
    <desktop:ExecutionAlias Alias="hello.exe" />
...
</rescap:Capability Name="runFullTrust" />
```

게시자의 OID.2.25...=1 부분이 --unsigned-test 가 더한 시험 표시다. runFullTrust 는 프로그램이 스토어 앱의 제한된 샌드박스가 아니라 여느 데스크톱 프로그램처럼 돈다는 뜻이다. --files 가 보이지 않지만 모든 패키지에 두 항목이 더 있다: 모든 파일의 64 KB 블록마다 해시를 적은 AppxBlockMap.xml(서명이 덮는 것이 이것이다) 과, 파일마다 형식을 적은 [Content_Types].xml. 서명한 패키지에는 서명인 AppxSignature.p7x 도 있다. 제4부가 ZIP 구조와 블록 맵을 바이트 단위로 보여 준다.

18 검사하고 들여다보기

이 장의 차례

18.1	원본 검사: lint	63
18.2	패키지 검사: lint <파일>	64
18.3	표 보기: inspect	64
18.4	파일과 스트림	65
18.5	설치하지 않고 풀기: extract	65
18.6	서명: verify	66
18.7	MSIX 패키지	66
18.8	더 빠른 시험 빌드: --compress none	66
18.9	GUID: guid	66
18.10	명령줄의 도움말	66

목표: 사용자보다 먼저 실수를 잡는다 - 원본이나 완성된 패키지를 검사하고, 그 안의 모든 표와 파일과 스트림을 보고, 설치하지 않고 풀어 보고, 서명을 확인한다. 이 명령들은 다른 도구로 만든 패키지에도 쓸 수 있다.

이 장은 17장의 hello.toml 과 그것으로 빌드한 hello.msi 를 쓴다.

18.1 원본 검사: lint

lint 는 build 가 하는 모든 검사를 하고, 아무것도 쓰지 않는다:

```
C:\work\hello> rubrapack lint hello.toml
C:\work\hello> rubrapack lint hello.toml --target msix
C:\work\hello> rubrapack lint hello.toml --arch x86 -D VERSION=2.2.0
```

아무 출력 없이 종료 코드 0 이면 문제가 없다는 뜻이다. --target msix 는 원본을 MSI 대신 MSIX 기준으로 검사하고, --arch 와 -D 는 같은 옵션을 준 build 가 빌드할 모습 그대로 검사한다.

경고는 빌드를 멈추지 않는다: 아마 뜻한 바가 아닌 것이 있지만 패키지는 동작한다. --strict 는 경고도 실패로 만든다 - 자동 빌드가 원하는 것이 이것이다. 제품 이름이 한국어인데 summary-name 이 없다고 하자(5장):

```
C:\work\hello> rubrapack lint ko.toml
ko.toml:1:1: warning[RP1203]: name is not ASCII and summary-name is missing; the summary Subject will read
'rubrapack package'
C:\work\hello> echo %errorlevel%
0
C:\work\hello> rubrapack lint ko.toml --strict
ko.toml:1:1: warning[RP1203]: name is not ASCII and summary-name is missing; the summary Subject will read
'rubrapack package'
C:\work\hello> echo %errorlevel%
5
```

%errorlevel% 은 cmd 에서 마지막 명령의 종료 코드를 보는 방법이다. PowerShell 에서는 \$LASTEXITCODE 다. 종료 코드:

코드	뜻
0	성공
1	원본의 오류
2	rubrapack 이 알아듣지 못하는 명령줄

코드	뜻
3	파일을 읽거나 쓰지 못함
4	서명 또는 서명 확인
5	lint 가 문제를 찾음(--strict 면 경고도)
6	네트워크(타임스탬프 서버)

18.2 패키지 검사: lint <파일>

```
C:\work\hello> rubrapack lint hello.msi
hello.msi: 0 errors, 0 warnings
C:\work\hello> rubrapack lint hello.msix
hello.msix: 7 files, 0 errors, 0 warnings
C:\work\hello> rubrapack lint hello.msixbundle
hello.msixbundle: 4 files, 0 errors, 0 warnings
```

MSI 는 Windows Installer 가 적용하는 규칙으로 표를 검사한다: 설치를 멈추게 할 것은 오류, 나머지는 경고다. 다른 도구가 만든 패키지에는 경고가 있을 수 있다. 여기서는 악센트 글자를 두 조각으로 저장하는 프로그램에서 제품 이름을 입력했다(제3부의 유니코드 정규화 참고):

```
C:\work\hello> rubrapack lint cafe.msi
cafe.msi: warning[RP2105]: lint: Feature row 'Main': column Title is not in NFC (1 row); build --nfc puts folder,
file and shortcut names in NFC
cafe.msi: warning[RP2105]: lint: Property row 'ProductName': column Value is not in NFC (1 row); build --nfc puts
folder, file and shortcut names in NFC
cafe.msi: 0 errors, 2 warnings
```

MSIX 는 Windows 가 설치 전에 검사하는 것을 검사한다: ZIP 압축 파일, 블록 맵, 모든 블록의 해시, 그리고 매니페스트가 가리키는 파일이 실제로 있는지. lint new.msi --previous old.msi 는 업그레이드를 검사한다(3장).

18.3 표 보기: inspect

MSI 는 작은 데이터베이스다. inspect 에 표 이름을 주면 그 표를 출력한다:

```
C:\work\hello> rubrapack inspect hello.msi File
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 l255 i4 S72 S20 I2 i4
File File
F_6d1508f52f6615459567 C_a10be66379e69940a2e0 guide.txt 6 512 1
Hello C_185f8db32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 2
```

이것은 Windows Installer 자신의 표 텍스트 형식(IDT)으로, 그 도구들도 읽는다. 처음 세 줄이 머리다:

1. 열 이름들
2. 열마다 형식: s72 는 72자까지의 문자열, l255 는 지역화할 수 있는 문자열, i4 는 4바이트 정수, i2 는 2바이트 정수. 대문자(S72, I2)는 그 열이 비어 있어도 된다는 뜻이다.
3. 표 이름, 그리고 키 열들 - 행을 하나로 구별하는 열.

그 뒤의 행이 파일 하나씩이다. hello.exe 는 판 정보 리소스가 있는 프로그램이라 판 번호와 언어(1033 은 영어(미국))가 있고, guide.txt 에는 없다. Attributes 512 는 "필수" 비트다(4장: 파일을 쓰지 못하면 설치가 실패한다). 제4부가 모든 형식과 행이 저장되는 방식을 설명한다.

표 이름 없이 부르면 inspect 는 모든 표를 행과 열 수와 함께 나열하고, Windows 가 파일 속성에 보이는 몇 가지 사실인 요약 정보도 보인다:

```
C:\work\hello> rubrapack inspect hello.msi
code page: 65001
```

```
strings: 156
tables: 19
  AdminExecuteSequence rows=8 columns=3
  ...
  File rows=2 columns=8
  ...
summary:
  2 = Installation Database
  3 = Hello
  4 = Example Software
  5 = Installer
  7 = x64;1033
  9 = {7FC806A2-F359-4941-A159-26A77B78F839}
  14 = 200
  15 = 2
  18 = rubrapack 0.7.0
```

요약 정보의 수는 속성 ID 다: 2 제목, 3 주제, 4 작성자, 5 키워드, 7 플랫폼과 언어, 9 패키지 코드(빌드마다 새 GUID), 14 필요한 Windows Installer 판(200 = 2.0), 15 단어 플래그(2 = 압축됨), 18 만든 프로그램. --summary 는 이것만 출력한다.

18.4 파일과 스트림

```
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\guide.txt 6 F_6d1508f52f6615459567 C_a10be66379e69940a2e0
2b6739281f9372442df30e784b7dbdc7
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello C_185f8db32271fe25f561 1.2.3.4 1033
```

파일마다 한 줄: 설치되는 곳, 크기, 파일 ID, 컴포넌트, 판, 언어, 그리고 판 번호 없는 파일을 바꿀지 Windows 가 정할 때 쓰는 MD5 해시(프로그램은 판 번호로 비교하므로 없다).

```
C:\work\hello> rubrapack inspect hello.msi --streams
table 40 File
stream 6821 cab1.cab
table 14 Media
table 728 _Columns
...
stream 332 !SummaryInformation
```

스트림은 패키지 안의 파일이다(제4부: 패키지는 파일 안의 파일 시스템이다). 표 하나가 스트림 하나이고, cab1.cab 에 압축된 파일들이, !SummaryInformation 에 요약 정보가 든다.

캐비닛도 따로 볼 수 있다 - 15장의 외부 캐비닛:

```
C:\work\hello> rubrapack inspect out-x64\hello-x64.cab
6 F_6d1508f52f6615459567
17920 Hello
```

18.5 설치하지 않고 풀기: extract

```
C:\work\hello> rubrapack extract hello.msi -d unpacked
unpacked: 2 files, 17926 bytes
```

파일은 설치될 모습 그대로, 표준 폴더 이름 아래에 나온다:

```
unpacked\ProgramFiles64Folder\Hello\guide.txt
unpacked\ProgramFiles64Folder\Hello\hello.exe
```

extract 는 .msi, .cab, .msix, .msixbundle 에 쓸 수 있고, 어디서 온 패키지에도 안전하다: 대상 폴더는 새것이거나 비어 있어야 하고, 모든 파일을 크기와 해시로 확인하고, 폴더 밖에 쓰게 될 이름(.., 드라이브, con)은 거부하며, 패키지 전체가 통과할 때까지 아무것도 쓰지 않는다. 디스크를 채우려고 만든 패키지에 대비한 한도도 있다:

```
C:\work\hello> rubrapack extract hello.msi -d small --limit-entries 2
rubrapack: error[RP0008]: 'hello.msi': 4 entries, more than the limit 2 (--limit-entries); nothing was extracted
C:\work\hello> rubrapack extract hello.msi -d small --limit-bytes 1000
rubrapack: error[RP0008]: 'hello.msi': more than 1000 bytes to write (--limit-bytes); nothing was extracted
```

기본값은 항목 100,000개와 16 GiB 다. (항목 넷은 파일 둘과 폴더 둘, ProgramFiles64Folder 와 Hello 다.)

18.6 서명: verify

16장에서 verify 를 보았다. 배포하기 직전에 돌릴 검사다:

```
C:\work\hello> rubrapack verify hello.msi --system-roots
```

18.7 MSIX 패키지

```
C:\work\hello> rubrapack inspect hello.msix
C:\work\hello> rubrapack inspect hello.msix --files
C:\work\hello> rubrapack inspect hello.msix --manifest
```

첫 번째는 모든 블록의 해시를 확인한 뒤 정제, 실행 파일, 파일들을 보인다. 나머지 둘은 17장에서 보았다.

18.8 더 빠른 시험 빌드: --compress none

빌드 시간의 대부분은 압축이다. 이것저것 해 보는 동안에는 압축을 뺀다:

```
C:\work\hello> rubrapack build hello.toml -o fast.msi --compress none
```

패키지는 커지지만(여기서는 32768 바이트 대신 45056 바이트) 똑같이 동작한다.

18.9 GUID: guid

```
C:\work\hello> rubrapack guid
{98AE4FED-BF0B-4C77-B712-E0649EB47178}
C:\work\hello> rubrapack guid --from hello
{B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}
```

첫 번째는 무작위 - 매번 새것 - 로, 업그레이드 코드에 쓴다. 두 번째는 rubrapack 이 글에서 끌어낸 GUID 로, 어느 컴퓨터에서든 언제나 같다. rubrapack 은 컴포넌트 GUID 를 이렇게 끌어내므로 판이 바뀌어도 같게 유지된다. 제3부가 그 계산을 보여 준다.

18.10 명령줄의 도움말

```
C:\work\hello> rubrapack help
rubrapack 0.7.0 - build Windows Installer (.msi) and MSIX (.msix) packages

usage: rubrapack <command> [arguments]

commands:
  build    build a package from a source file
  sign     sign a PE file, an MSI package, an MSIX package or bundle
  ...

C:\work\hello> rubrapack help sign
usage: rubrapack sign <file.exe|.dll|.msi|.msix|.msixbundle> (--key <key.pfx|.pem> ...
```

```
C:\work\hello> rubrapack version
rubrapack 0.7.0 (proven_c_lib-v0.1.1)
```

rubrapack --help 는 rubrapack help 와 같다. help <명령> 은 그 명령의 옵션을 출력한다 - 모든 옵션과 뜻을 담은 제2부의 짧은 판이다.

19 빨리 시작하기와 자동화

이 장의 차례

19.1	답으로 원본 만들기: new	68
19.2	원본 고치기: edit	69
19.3	릴리스 스크립트	70
19.4	빌드 서버에서	70
19.5	AI 도우미와 함께	70
19.6	여기서 어디로	70

목표: rubrapack 이 첫 원본을 대신 쓰게 하고, 편집기를 열지 않고 그것을 고친다. 그리고 이 튜토리얼의 모든 단계를 스스로 릴리스를 빌드하고 검사하고 서명하는 스크립트로 만든다 - 내 컴퓨터에서, 빌드 서버에서, 또는 AI 도우미를 통해.

19.1 답으로 원본 만들기: new

프로그램 파일이 dist\ 에 있는 폴더에서:

```
C:\work\hello2> rubrapack new hello.toml
rubrapack new: a few questions make the source (Enter takes the value in [ ]).
Product name [hello]: Hello
Manufacturer (shown in Installed apps) [Hello authors]: Example Software
Version [1.0.0]:
Folder with the files to install [dist]:
  2 files and 2 folders there
Main program, for shortcuts (- for none) [hello.exe]:
Architecture (x64, x86, arm64) [x64]:
Folder name under Program Files [Hello]:
Install for (machine, user, dual) [machine]:
  sub folders: docs samples
Optional parts the user may tick (sub folders, commas; - for none) [-]: samples
Dialogs (none, basic, minimal, installdir, features) [features]:
License to accept (.txt, .md, .rtf; - for none) [LICENSE.txt]:
Korean dialogs as well as English? [y/N]: y
Start menu shortcut? [Y/n]:
Desktop shortcut? [y/N]: y
wrote hello.toml
the same without questions:
  rubrapack new "hello.toml" --name "Hello" --manufacturer "Example Software" --version "1.0.0" --arch "x64" --
  dist "dist" --main "hello.exe" --install-dir "Hello" --scope "machine" --ui "features" --license "LICENSE.txt" --
  languages "ko" --optional "samples" --shortcuts "start,desktop"
  lint: no problems
```

질문 하나하나가 이 튜토리얼의 어느 장에서 설명한 것이고, [] 안의 값은 Enter 가 받는 값이다. 제안은 앞의 답을 따라간다: 이름을 Hello 로 하면 제조사로 Hello authors 가 제안되고, 선택 구성요소가 생기면 대화창으로 features 가 제안된다.

만들어진 원본은 보통의 원본이다 - 각 줄이 무엇을 하는지 적은 주석, 새 업그레이드 코드, 폴더 맨 위의 파일마다 [file.*], 하위 폴더마다 글루브, 선택 구성요소마다 [feature.*] 가 들어 있다:

```
[feature.Main]
title = "Hello"
required = true                # always installed
```

```
[feature.samples]
title = "samples"
level = 2                # offered, not ticked by default
...
[files.samples_files]
dir = "samples_dir"
glob = "dist/samples/**"
```

여기서부터는 2~18장처럼 손으로 고치거나 `edit` 로 고친다. `new` 는 있는 파일을 절대 바꾸지 않는다: `new hello.toml` 을 한 번 더 하면 `edit` 을 쓰라고 한다.

다른 형태 셋:

- `new` 가 마지막에 출력하는 줄 - `rubrapack new hello.toml --name ... --shortcuts ...` - 은 묻지 않고 같은 답을 준다. 스크립트용이다. 빠진 옵션은 질문이 제안했을 기본값을 받는다.
- `rubrapack new hello(.toml 없는 이름, 옵션 없음)`는 아무것도 묻지 않고 짧은 시작용 원본을 쓴다. 손으로 채울 때 쓴다.
- `rubrapack new hello -i(또는 --interactive)`는 질문을 한다. 질문은 오류 출력으로 가고 답은 한 줄에 하나씩 읽으므로, 답을 적은 파일을 흘려 넣을 수 있다: `rubrapack new hello.toml < answers.txt`. 마지막 질문에 답이 떨어지면 아무것도 쓰지 않는다.

19.2 원본 고치기: `edit`

옵션 없는 `edit` 은 메뉴를 보인다:

```
C:\work\hello> rubrapack edit hello.toml
hello.toml: "Hello" 1.0.0, x64, machine, dialogs features
  1 name, manufacturer, version, architecture  2 install folder, who it installs for
  3 dialogs, license, languages                4 optional parts
  5 shortcuts                                6 files: match the program folder
  7 any key                                  v view  l lint  s save  q quit
Choose [q]:
```

고르면 그 질문들을 지금 값을 제안하며 다시 묻는다. 7 은 어느 표의 어느 키든 바꾼다. v 는 원본을 보이고, l 은 검사하고, s 는 저장하고 검사하며, q 는 나간다(바뀐 것이 있으면 먼저 묻는다). 바꾼 값만 다시 쓴다: 내 주석과 순서, 손으로 쓴 표는 그대로 남는다.

옵션을 주면 `edit` 은 아무것도 묻지 않는다:

```
C:\work\hello> rubrapack edit hello.toml --set define.VERSION=1.1.0 --set package.summary-name=Hello --unset package.license
wrote hello.toml
lint: no problems
```

- `--set 표.키=값` 은 키를 정한다. TOML 로 쓴 값 - "글", 수, true, ["ko"] - 은 그대로 쓰고, 그 밖의 것은 글로 받는다. 없는 표는 더한다.
- `--unset 표.키` 는 키를 지운다.

프로그램이 바뀐 뒤 - 여기서는 `readme.txt` 가 없어지고 `changes.txt` 가 새로 생겼다 - `--sync` 는 파일 목록을 폴더에 맞춘다:

```
C:\work\hello> rubrapack edit hello.toml --sync
removed [file.readme_txt]
added [file.changes_txt]
wrote hello.toml
lint: no problems
```

19.3 릴리스 스크립트

지금까지의 모든 것은 원본 밖에 상태가 없는 명령이므로, 릴리스는 짧은 스크립트가 된다. hello.toml 옆의 PowerShell 스크립트 release.ps1:

```
param([Parameter(Mandatory)] [string] $Version)
$ErrorActionPreference = "Stop"
function Run { & rubrapack @args; if ($LASTEXITCODE -ne 0) { throw "rubrapack $args failed ($LASTEXITCODE)" } }

Run lint hello.toml --strict -D VERSION=$Version
foreach ($arch in "x64", "x86", "arm64") {
    Run build hello.toml -o "out\hello-$Version-$arch.msi" -D VERSION=$Version --arch $arch --reproducible `
        --key-store $env:SIGN_THUMBPRINT --timestamp http://timestamp.digicert.com
    Run lint "out\hello-$Version-$arch.msi" --strict --previous "released\hello-$arch.msi"
    Run verify "out\hello-$Version-$arch.msi" --system-roots
}
Run build hello.toml -o "out\hello-$Version.msixbundle" -D VERSION=$Version --arch x64,x86,arm64 `
    --key-store $env:SIGN_THUMBPRINT --timestamp http://timestamp.digicert.com
```

```
PS C:\work\hello> .\release.ps1 -Version 2.2.0
```

released\ 에는 바로 전 판의 패키지가 아키텍처마다 하나씩 있어서, --previous 가 각 업그레이드를 검사한다 (3장). 스크립트는 실패한 첫 명령에서 멈춘다. rubrapack 의 종료 코드가 0 이 아니기 때문이다(18장). 지문은 환경 변수에서 옴으로 스크립트에는 비밀이 없고, 원본과 함께 보관해도 된다.

19.4 빌드 서버에서

rubrapack 은 Linux 에서도 돌고, --reproducible 이면 거기서도 같은 바이트를 빌드하므로 빌드 서버(GitHub Actions, GitLab CI, Jenkins)에 Windows 가 없어도 된다. Linux 러너에서:

```
V=0.7.0
curl -sLo rubrapack "https://github.com/rubidus-api/rubrapack/releases/download/v$V/rubrapack-$V-linux-x86_64"
chmod +x rubrapack
./rubrapack lint hello.toml --strict
./rubrapack build hello.toml -o "hello-$VERSION.msi" -D VERSION="$VERSION" --reproducible \
    --key signer.pfx --pass-env SIGN_PASS --timestamp http://timestamp.digicert.com
```

키와 그 비밀번호는 서버의 비밀 저장소(예를 들어 GitHub Actions 저장소의 secrets)에서 이 단계 동안만 파일과 환경 변수로 온다. 절대 저장소에 넣지 않는다. 하드웨어 토큰이나 클라우드 서명 서비스의 키는 대신 --pkcs11 로 쓴다(16장).

19.5 AI 도우미와 함께

원본은 짧은 일반 텍스트이고 모든 오류 메시지가 어디서 왜 틀렸는지 말하므로, AI 코딩 도우미가 위의 일을 모두 할 수 있다: 이 매뉴얼(책, <https://rubidus-api.github.io/rubrapack/>)을 주고 무엇을 설치할지 말한다. 도우미는 원본을 쓰고, 깨끗해질 때까지 rubrapack lint 를 돌리고, 빌드하고, inspect 와 extract 로 결과를 확인하고, 릴리스 스크립트를 쓸 수 있다. 둘은 내 몫으로 남는다:

- 첫 판의 업그레이드 코드를 이후 모든 판에 그대로 둔다(3장).
- 배포하기 전에 실제 Windows 에 패키지를 한 번 설치해 보고, 전 판에서의 업그레이드도 해 본다.

19.6 여기서 어디로

rubrapack 의 모든 표와 모든 옵션을 써 보았다. 이제부터는:

- 제2부 참조는 모든 표의 모든 키와 모든 옵션을 한곳에 모았다 - 원본을 쓰는 동안 열어 둘 페이지다.
- 제3부 기초 지식은 이 모든 것 밑의 개념 - 비트와 바이트, 문자 인코딩, GUID, 해시, 압축, 서명 - 을 처음부터 설명한다.

- **제4부 파일 형식**은 내가 빌드한 파일을 열어 모든 바이트를 보여 준다: MSI 의 파일 시스템, 표와 문자열, 캐비닛, MSIX 의 ZIP 과 블록 맵, 그리고 서명.

제2부

참조

20 원본 파일과 rubrapack 명령

이 장의 차례

20.1	시작하기	74
	첫 패키지	74
	약관 페이지와 선택 구성요소	75
	나머지를 설명하는 곳	76
20.2	예	76
20.3	TOML 부분집합	76
20.4	표	77
	MSIX 패키지	78
	설치한 프로그램으로 등록하기: [action.ID]	79
	레지스트리 값: [registry.ID]	79
	바로가기: [shortcut.ID]	79
	파일 형식과 링크: [assoc.ID], [protocol.ID]	80
	MSIX 전용: [msix-extension.ID]	80
	지우기와 복사: [remove.ID], [copy.ID]	80
	환경 변수: [env.ID]	80
	INI 파일: [ini.ID]	80
	검색과 요구: [search.ID], [require.ID]	80
	서비스, 글꼴, 권한	81
	사용자별 패키지와 검용 패키지: scope	81
	캐비닛	81
	설치된 앱 항목과 속성	82
	명령줄로 설치하기	82
	창 없이 설치하기	82
	대화창: ui	82
	여러 언어	83
	아이콘, 완료 페이지, 설치 범위 페이지	84
	나만의 대화창 페이지: [dialog.ID], [dialog-control.ID]	84
	아키텍처와 업그레이드 계열	85
	먼저 지워야 하는 판	85
	와일드카드: [files.ID]	85
	빈 폴더: [folder.ID]	85
	기능(feature)	85
	조건: when	85
	제거할 때 파일 남기기: keep	86
	변수	86
	프로그램 파일	86
	설치 폴더 지키기	86
	경로	86
20.5	명령줄	87
20.6	진단 코드	90

원본 파일 하나가 패키지 하나를 기술한다. 형식은 [TOML 1.0](#)의 엄격한 부분집합이고 이름은 *.toml 이라서, 편집기와 GitHub, AI 도우미가 TOML 로 알아본다(예전 이름 *.rpk 도 똑같이 읽는다). 모든 원본은 올바른 TOML 이지만, 부분집합 밖의 TOML 기능은 무시하지 않고 오류로 거부한다.

20.1 시작하기

rubrapack 은 프로그램 파일 하나다. 릴리스 페이지에서 rubrapack-<판>-windows-x64.exe(원하면 rubrapack.exe 로 이름을 바꾼다)나 rubrapack-<판>-linux-x86_64(chmod +x 한다)를 내려받아 그 자리에서 실행하거나 PATH 에 둔다. 따로 설치할 것은 없다: 런타임도, SDK 도, 라이브러리도 필요 없다. 같은 프로그램이 Windows 에서도 Linux 에서도 같은 패키지를 만든다.

이 절은 설치 파일을 아는 독자를 위한 짧은 판이다. **제1부 튜토리얼**은 같은 것을 처음부터 한 걸음씩 가르치고, 모든 표와 옵션까지 나아간다: 시작하기 전에부터 읽는다.

첫 패키지

배포할 것을 dist/ 폴더에 둔다: 프로그램, 그리고 그것이 필요로 하는 파일은 dist/files/ 에(하위 폴더도 그대로 간다). 그다음 rubrapack 에게 시작할 원본을 쓰게 한다:

```
rubrapack new app.toml      # 묻고 나서 app.toml 을 쓰고 검사한다
```

제품 이름, 판, 파일이 든 폴더, 주 프로그램(아키텍처는 파일에서 읽는다), Program Files 아래 폴더, 누구를 위해 설치하는지, 어떤 하위 폴더가 선택 구성요소인지, 대화창, 약관(옆에 LICENSE.txt, .md, .rtf 가 있으면 권한다), 한국어 대화창, 바로가기를 묻는다. Enter 는 대괄호 안의 값을 쓴다. 끝에는 같은 답을 한 줄 명령(rubrapack new app.toml --dist dist --ui features ...)으로 보여 주는데, 스크립트나 AI 도우미는 묻지 않고 이것을 돌리면 된다. 이름 없이 rubrapack new 만 주면 파일 이름까지 묻고, rubrapack new app(확장자 없이)은 대신 고정된 틀을 쓴다.

어느 쪽이든 app.toml 은 읽고 고칠 수 있는 평문이다 - 아무 편집기로나, 또는 rubrapack edit app.toml 으로. edit 는 같은 질문을 지금 값을 기본으로 보여 주는 메뉴다(판, 설치 폴더, 대화창과 약관, 선택 구성요소, 바로가기, 그리고 프로그램 폴더가 바뀐 뒤 파일 목록 맞추기). 그 값만 바꾸고 다른 줄, 주석, 표는 그대로 둔다. 스크립트에서는: rubrapack edit app.toml --set define.VERSION=1.1.0 --sync. 아래 원본은 프로그램과 파일을 Program Files\My App 에 설치하고, 시작 메뉴에 넣고, 사용자가 폴더를 바꿀 수 있는 대화창을 보인다:

```
[package]
name = "My App"
manufacturer = "My Company"          # 설정 > 설치된 앱에 보인다
version = "${(VERSION)}"
arch = "x64"                          # x64, arm64, x86
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}" # `new` 가 준 것; 영원히 간직한다
ui = "installdir"

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/My App"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[shortcut.StartMenu]
dir = "Programs"
name = "My App"
target = "file:App"
```

```
rubrapack build app.toml -o app-1.0.0.msi
rubrapack build app.toml -o app-1.0.1.msi -D VERSION=1.0.1  # 다음 판
```

이것으로 완전한 설치 파일이다. "설치된 앱"에 나타나고, 스스로 복구하고, 설치한 것을 모두 지운다. 더 높은 판은 설치된 판을 대체하고, 같거나 낮은 판은 안내문과 함께 거부된다. 컴포넌트 GUID, 파일 키, 캐비닛과 표는 원본에서 끌어내므로, 판에서 판으로 기억해야 할 것은 업그레이드 코드뿐이다.

약관 페이지와 선택 구성요소

설치 파일이라면 흔히 바라는 두 가지: 설치 전에 사용자가 약관에 동의하는 것, 그리고 일부 구성요소를 고를 수 있는 것. license 는 약관 페이지를 더하고("동의합니다"에 표시하기 전에는 다음 단추가 꺼져 있다), ui = "features" 는 사용자가 설치할 기능을 고르는 트리를 더한다. 아래 원본은 프로그램을 늘 설치하고, 예제는 보여 주되 고르지 않은 상태로 둔다:

```
[package]
name = "My App"
manufacturer = "My Company"
version = "${(VERSION)}"
arch = "x64"
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}"
ui = "features"                # welcome, license, folder, feature tree, ready
license = "LICENSE.txt"        # Next stays off until "I accept" is ticked

[define]
VERSION = "1.0.0"

[feature.Main]
title = "My App"
description = "The program itself."
required = true                # always installed: the tree does not offer to leave it out

[feature.Samples]
title = "Samples"
description = "Example documents to try the program with."
level = 2                      # offered in the tree, not selected by default

[dir.INSTALLDIR]
path = "ProgramFiles/My App"
feature = "Main"

[dir.SamplesDir]
path = "INSTALLDIR/samples"
feature = "Samples"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[files.Samples]
dir = "SamplesDir"
glob = "dist/samples/**"

[shortcut.StartMenu]
dir = "Programs"
name = "My App"
target = "file:App"
```

약관 글은 LICENSE.txt(.txt, .md, .rtf)에, 예제 파일은 dist/samples/ 에 둔다. level = 2 인 기능은 사용자가 표시하지 않으면 설치되지 않고, required 는 트리가 그 기능을 빼자고 제안하지 못하게 한다. 나중에 사용자는 "설치된 앱"의 "변경"에서 고른 것을 바꿀 수 있고, 명령줄에서도 대화창 없이 같은 일을 한다: msixexec /i

app.msi /qn ADDLOCAL=Samples 는 예제를 더하고, REMOVE=Samples 는 뺀다(required 는 트리에만 적용되고 명령 줄은 묵지 않는다). 나머지는 기능(feature)과 조건: when에 있다.

나머지를 설명하는 곳

하려는 일	읽을 곳
msiexec 로 설치, 업그레이드, 제거, 기록 남기기	첫 설치 파일, 판과 업그레이드
오류를 이해하고 패키지 안 보기(lint, inspect, extract, 종료 코드)	검사하고 들여다보기, 진단 코드
패키지에 서명하기	서명과 타임스탬프
스크립트나 CI 작업, 또는 AI 도우미로 빌드하기	빨리 시작하기와 자동화

20.2 예

```
[define]
VERSION = "1.4.0"

[package]
name = "Example App"
manufacturer = "Example"
version = "${VERSION}"
arch = "x64" # x64, arm64, x86 가운데 하나 - 기본값 없음
upgrade-code = "{0B9A6C1E-3D2F-4A5B-8C7D-6E5F4A3B2C1D}" # 한 번 만들어 영원히 쓴다
language = "en-US" # 또는 "ko-KR"

[dir.INSTALLDIR]
path = "ProgramFiles/Example App"

[dir.Docs]
path = "INSTALLDIR/docs"

[file.MainExe]
dir = "INSTALLDIR"
source = "dist/app.txt"

[file.Guide]
dir = "Docs"
source = "dist/guide.txt"
name = "User guide.txt"
```

```
rubrapack build example.toml -o example.msi -D VERSION=1.4.1
rubrapack inspect example.msi File
```

20.3 TOML 부분집합

받는 것: [kind]·[kind.ID] 표, 맨 키(A-Z a-z 0-9 _ -), TOML 이스케이프가 되는 기본 문자열 "...", 리터럴 문자열 '...' (이스케이프 없음 - 백슬래시와 따옴표를 쓸 때 편하다: 'SOFTWARE\Example'), 십진수와 0x 정수, true/false, 한 가지 형만 담은 배열, # 주석. BOM 이 있거나 없는 UTF-8, 또는 BOM 이 있는 UTF-16LE. 줄 끝은 LF 나 CRLF.

오류로 거부하는 것: 여러 줄 문자열, 인라인 표, 표 배열, 점으로 이은 키와 따옴표 키, 세 부분 이상의 표 이름, 부동소수, 날짜, 숫자 속 _, 8진·2진 수, 빈 배열이나 섞인 배열, 첫 표보다 앞의 키. 표와 키의 순서는 아무 의미가 없다.

20.4 표

표	키(굵은 것은 반드시)
[package]	name , manufacturer , version (a.b.c 또는 a.b.c.d), arch , upgrade-code , upgrade-code-x64 / -arm64 / -x86, product-code, summary-name(ASCII), language, scope(machine, user, dual), ui(none, basic, minimal, installdir, features), license(.txt, .md, .rtf), reboot(suppress/allow), downgrade-message, compress(none, mszip, mszip:0..mszip:9; 기본 mszip:6), cab(embed 또는 external), cab-max-size(MiB), refuse-upgrade-below, refuse-upgrade-message
[define]	변수: NAME = "value"
[feature.ID]	title , description, level(1-32767), hidden, parent, required, follow-parent, when
[dir.ID]	path = 기준/상대/경로, feature, guard(true: 설치 폴더 지키기 참고)
[file.ID]	dir , source , name, vital(기본 true), any-arch, feature, component-guid, keep, when
[files.ID]	dir , glob , vital, any-arch, feature, keep, when
[folder.ID]	dir , name , keep, feature
[arp]	no-modify, no-repair, help(URL), about(URL), icon(.ico) - "설치된 앱"에 제품이 어떻게 보이는가
[property.ID]	value , secure, hidden - 대문자 이름의 공개 속성
[action.ID]	run (이 패키지의 .exe 를 가리키는 file:ID), do , undo , check
[registry.ID]	root (HKLM, HKCU, HKCR, HKMU), key , name, value, type, remove, keep, view, with, feature, when
[remove.ID]	dir , name(* 와 ?; 없으면 폴더 자체), on (install, uninstall, both), feature
[ini.ID]	dir , file , section , key , value, mode(set, add, remove), feature, when
[require.ID]	condition , message
[search.ID]	property (또는 dir ID), kind (registry: root, key, name, view; file: path, file, min-version; dir: path; component: component-guid)
[service.ID]	file (.exe 를 가리키는 file:ID), name , display-name, description, start(auto, demand, disabled), account(LocalSystem, LocalService, NetworkService), args, start-on-install
[assoc.ID]	extension (.ext, 소문자), prog-id , target (.exe 를 가리키는 file:ID), description, icon(file:ID), args(기본 "%1")
[protocol.ID]	name (스킴, 소문자), target (.exe 를 가리키는 file:ID), description, args(기본 "%1")
[font.ID]	file (path = "Fonts" 인 dir 에 드는 파일의 file:ID), title
[permission.ID]	target (dir:ID, file:ID, registry:ID), sddl
[env.ID]	name , value , mode(set, append, prepend), keep, feature, when
[copy.ID]	source (file:ID), dir , name(기본: 원본 파일 이름)
[ui]	install-dir(dir ID; 기본 INSTALLDIR), banner(.bmp), launch(file:ID), launch-args, launch-checked, languages(영어에 덧붙일 언어, 예 ["ko"]), license-xx, name-xx, font-xx, langid-xx - 여러 언어 참고
[ui-text.ID]	text 또는 text-xx - 내장 대화창 문구 하나를 바꾼다
[dialog.ID]	after (내장 페이지 또는 다른 [dialog.*]), title, description, title-xx, description-xx
[dialog-control.ID]	dialog , type (text, checkbox, edit, radio, combo), x , y , width , height , text, property, values, labels, text-xx, labels-xx
[shortcut.ID]	dir (dir ID, 또는 Programs, Desktop, StartMenu, Startup), name , target (file:ID), args, description, working-dir(dir ID), icon(.ico), when
[msix]	identity-name , publisher , publisher-display-name, min-version - MSIX 패키지 참고
[msix-app.ID]	executable ([file.*] ID), display-name, description, logo-150, logo-44, store-logo

표	키(굵은 것은 반드시)
[msix-extension.ID]	kind (alias: alias ; startup-task: task-id, display-name, enabled), app([msix-app.*] ID; 기본은 첫째) - MSIX 전용

dir 경로의 기준은 다른 dir ID 이거나 다음 가운데 하나다: ProgramFiles(x64/arm64 는 64비트, x86 은 32비트), ProgramFiles32, CommonFiles, AppData, LocalAppData, CommonAppData, StartMenu, Programs, Desktop, Startup, Windows, System, Fonts, Temp. 알려진 폴더 하나만 쓴 경로(path = "Fonts")는 그 폴더 바로 안에 드는 파일에 쓴다.

ID 는 [A-Za-z_][A-Za-z0-9_]* 풀이고(최대 72자, feature 는 38자) dir·파일·feature 를 통틀어 서로 달라야 한다.

MSIX 패키지

출력 이름이 .msix 로 끝나면 같은 원본이 MSIX 패키지가 된다: 완전 신뢰로 도는 데스크톱 앱 하나, 아키텍처 하나. MSIX 에만 필요한 것은 표 두 개가 더 말한다:

```
[msix]
identity-name = "Example.App"           # 3~50자: A-Z a-z 0-9 . -
publisher = "C=KR, O=Example, CN=Example" # 서명 인증서의 주체, 마지막 부분부터
publisher-display-name = "Example"       # 기본: [package] manufacturer
min-version = "10.0.17763.0"             # 설치되는 가장 오래된 Windows(이것이 기본값)

[msix-app.Main]
executable = "MainExe"                  # 앱을 시작하는 [file.*]
display-name = "Example App"             # 기본: [package] name
description = "An example"               # 기본: 표시 이름
logo-150 = "assets/Square150x150.png"    # PNG, 150x150
logo-44 = "assets/Square44x44.png"       # PNG, 44x44
store-logo = "assets/StoreLogo.png"      # PNG, 50x50
```

- 판은 [package] version 을 네 부분으로 늘린 것(1.2.3 은 1.2.3.0), 아키텍처는 [package] arch, 언어는 [package] language 다.
- 패키지 자신의 폴더는 첫 [msix-app.*] 실행 파일이 드는 폴더 - ProgramFiles/Example App 처럼 알려진 자리에 붙은 dir - 다. 다른 알려진 자리의 파일은 패키지의 가상 파일 시스템(VFS)에 들어가고, 앱은 그것을 늘 보던 자리에서 본다: Program Files(VFS\ProgramFilesX64, x86 패키지와 ProgramFiles32 는 X86), CommonFiles, System, Windows, CommonAppData(ProgramData). 사용자의 AppData·LocalAppData 와 Temp 에는 가상 폴더가 없어 거기 드는 파일은 오류다(RP1609). Fonts 의 글꼴은 그 [font.*] 로 들어가고, 시작 메뉴·Programs·바탕 화면 폴더는 바로가기([shortcut.*])로, Startup 은 시작 작업([msix-extension.*])으로 쓴다.
- [msix-app.*] 표가 여럿이면 한 패키지에 항목이 여럿 생긴다(최대 100개). 패키지 로고는 첫째 것이다.
- [registry.*] 값은 패키지의 가상 레지스트리에 들어간다. 앱은 그것을 실제 레지스트리와 합쳐 보고, 컴퓨터의 레지스트리는 건드리지 않는다: Software 아래의 HKLM(과 HKMU)은 Registry.dat (view = "32" 면 32비트 보기)에, Software 아래의 HKCU 는 User.dat 에 든다. MSIX 가 담을 수 없는 것은 오류다(RP1612): HKCR(파일 형식과 프로토콜은 [assoc.*]·[protocol.*] 로 쓴다), Software 밖의 키, remove 와 keep, 그리고 Windows Installer 가 설치 때 채우는 부분이 든 값 ([INSTALLDIR], [#File] 등. 이스케이프 [\] 와 [\] 는 랜찮다).
- 로고는 셋 다 주거나 하나도 주지 않는다. 주지 않으면 한 가지 색의 로고가 들어간다. 로고는 크기가 정확해야 한다(RP1608).
- MSIX 가 할 수 없는 것은 조용히 빠지 않고 오류로 알린다(RP1605): 사용자 지정 동작, 서비스, 환경 변수, INI 파일, 권한, 설치 조건과 검색, 설치 때 지우거나 복사하는 파일, 빈 폴더. 그런 표(또는 [file.*]/[files.*])에 msi-only = true 를 달면 MSI 에는 들어가고 MSIX 는 그것 없이 만들어진다. feature·속성·대화창·[arp] 은 Windows Installer 에만 해당하므로 MSIX 에는 쓰지 않는다.
- --unsigned-test 는 서명 없는 패키지를 시험 삼아 설치할 때 Windows 가 요구하는 속성을 더한다 (Add-AppxPackage -AllowUnsigned, 프로그램이 들어 있으면 관리자). 그런 패키지는 배포용이 아니고 정체(identity)도 서명판과 다르다. 대신 --key 를 주면 패키지(또는 묶음과 그 안의 패키지)에 서명한다. 아래 sign 참고.

- 파일은 압축한다(--msix-compress store 로 끈다). 그림, 압축 파일처럼 이미 압축된 파일은 저장만 한다. MSIX 에는 시각이 들어가지 않아, 같은 원본은 Linux 와 Windows 에서 같은 바이트가 된다.
- 출력 이름이 .msixbundle 로 끝나면 묶음이다: --arch 의 아키텍처마다 원본을 한 번씩 짓고 (목록, --arch x64,x86,arm64; 없으면 원본의 아키텍처), 각 패키지는 안에서 <identity-name>_<version>_<arch>.msix 라는 이름을 갖는다. Windows 는 그 가운데 자기 아키텍처의 패키지를 설치한다(x64 컴퓨터는 x86 보다 x64 를 고른다). \$(ARCH) 로 빌드마다 제 프로그램을 준다: source = "bin/\$(ARCH)/app.exe". 묶음의 판은 패키지들의 판이다.

설치한 프로그램으로 등록하기: [action.ID]

```
[action.Tip]
run = "file:MainExe"      # 이 패키지가 설치하는 .exe
do = "--register"          # 파일을 설치한 뒤, 그리고 복구할 때 다시 실행
undo = "--unregister"     # 제거할 때, 파일을 지우기 전에 실행
```

rubrapack 은 이 한 쌍을 관리자 권한의 지연(deferred) 동작과 그 되돌림(rollback) 짝으로 바꾼다. 그래서 제거와 업그레이드는 전부 되거나 전혀 되지 않는다: 뒤에서 무엇이 실패하면(또는 do/undo 자체가 0 아닌 값으로 끝나면) 파일이 되돌아오고 다른 명령이 등록을 되살린다. 두 명령은 두 번 실행해도 안전해야 하고, 아무것도 묻지 않고 끝나야 한다 - 창 없이 실행되고 사용자를 기다리는 것은 없다. 인자는 쓴 그대로 넘어간다(서식 문자열이 아니다). check 는 받아들이지만 이 판에서는 쓰지 않는다.

레지스트리 값: [registry.ID]

```
[registry.InstallDir]
root = "HKLM"
key = 'SOFTWARE\Example'      # 리터럴 문자열은 백슬래시를 그대로 둔다
name = "InstallDir"          # 없으면 키의 기본값
value = "[INSTALLDIR]"        # MSI 서식 문자열: [PROPERTY], [#FileID], "[" 는 [\[
```

type 은 string(기본), expand, dword(0~0xFFFFFFFF 정수), qword(정수, 또는 "0x" 와 16자 이하의 16진수), binary(16진수), multi(문자열 배열)이다. Windows Installer 는 REG_QWORD 를 스스로 쓰지 못한다. 그래서 rubrapack 이 작은 도우미 DLL 을 패키지에 넣어 쓰게 하고, 설치가 실패하면 그 DLL 이 이전 값을 되살린다. 값 하나가 구성 요소(component) 하나이고 제거할 때 지워진다(keep = true 면 남는다). with = "file:ID" 는 대신 그 파일의 구성 요소에 넣는다. 64비트 패키지에서 값은 64비트 레지스트리 보기로 가고, view = "32" 는 32비트 보기에 쓴다. remove = true(value 없이는) 설치 중에 이름 붙은 값을 - name 이 없으면 키 전체를 - 지운다.

바로가기: [shortcut.ID]

```
[dir.Menu]
path = "Programs/Example"     # 시작 메뉴 > Example

[shortcut.Settings]
dir = "Menu"
name = "Example settings"     # ".lnk" 는 붙여 준다
target = "file:MainExe"
args = "--settings \"[INSTALLDIR]\""
```

바로가기는 대상 파일(과 그 feature)에 딸린다. 바로가기 때문에 만든 폴더는 제거할 때 지운다. 컴퓨터 전체 패키지에서 Programs 와 Desktop 은 모든 사용자의 시작 메뉴와 공용 바탕화면이다.

MSIX 에서 바로가기는 앱을 시작한다(대상은 [msix-app.*] 의 실행 파일이어야 한다): Programs 나 StartMenu 의 것은 그 앱 자신의 시작 메뉴 항목이고(args 없음), Desktop 의 것은 매니페스트에 적히며 min-version = "10.0.19645.0" 이상이 필요하다(RP1614). 다른 폴더, working-dir, args 속 [...] 는 거기서 오류다(RP1613). Startup 은 시작 작업으로 쓴다.

파일 형식과 링크: [assoc.ID], [protocol.ID]

```
[assoc.Doc]
extension = ".exdoc"
prog-id = "Example.Document"      # 같은 prog-id 의 표들은 한 종류의 문서를 뜻한다
description = "Example document"
target = "file:MainExe"
args = "--open \"%1\""           # 기본값은 "%1"

[protocol.Link]
name = "example"                  # example:... 가 프로그램을 연다
target = "file:MainExe"
```

MSI에서는 프로그램의 구성 요소에 든 HKEY_CLASSES_ROOT 아래 레지스트리 값이다: 확장자의 기본값은 prog-id, prog-id에는 설명과 DefaultIcon(icon, 없으면 프로그램의 첫 아이콘), shell\open\command가 있고, 스킴에는 URL Protocol이 붙는다. HKEY_CLASSES_ROOT는 설치 방식을 따른다: 컴퓨터 전체면 HKLM\Software\Classes, 사용자별이면 HKCU\Software\Classes에 들고, 제거하면 없어진다. 이 프로그램만 맡겠다는 형식은 곧바로 이 프로그램으로 열리고, 사용자가 다른 프로그램을 골라 두었으면 Windows는 그 선택을 지킨다.

MSIX에서는 대상 실행 파일을 가진 앱의 매니페스트에 들어가고(파일 형식 연결, 프로토콜) args는 평문이어야 한다. icon은 쓰지 않는다: 앱의 로고가 파일 형식을 나타낸다.

MSIX 전용: [msix-extension.ID]

```
[msix-extension.Cli]
kind = "alias"
alias = "example.exe"             # 콘솔에서 이 이름을 치면 앱이 시작된다

[msix-extension.Boot]
kind = "startup-task"             # 앱을 한 번 실행한 뒤로 Windows와 함께 시작된다
display-name = "Example"         # 작업 관리자에 보이는 이름; task-id 기본값은 표 ID
enabled = true
```

MSI 빌드는 이것들을 뺀다. MSIX에서 글꼴([font.*])은 패키지의 Fonts 폴더에서 다른 앱과 나눠 쓰며(uap4:SharedFonts) title은 쓰지 않는다.

지우기와 복사: [remove.ID], [copy.ID]

[remove.ID]는 dir에서 name에 맞는 파일을 지운다 - 예를 들어 옛 판이 남긴 *.log를 on = "install"로, 프로그램이 실행 중에 만드는 파일을 on = "uninstall"로. name이 없으면 비어 있는 폴더 자체를 지운다. 설치가 실패하면 지운 파일은 되돌아온다. [copy.ID]는 이 패키지의 파일을 다른 폴더에 한 벌 더 설치하고, 원본과 함께 들어오고 함께 나간다. 설치 전부터 있던 폴더는 절대 지우지 않는다.

환경 변수: [env.ID]

시스템(컴퓨터 전체) 변수다. mode = "set"(기본)은 값을 바꾸고, append/prepend는 기존 값 끝에 ;value를, 앞에 value;를 붙인다. 제거는 정확히 그만큼 되돌린다: set한 변수는 지우고, 붙인 부분은 떼어 내고 나머지는 둔다. keep = true면 남긴다.

INI 파일: [ini.ID]

mode = "set"(기본)은 파일의 [section]에 key=value를 쓰고, add는 쉼표 목록에 값을 덧붙이며(a가 a,b로), remove는 설치 중에 키를 지운다. 제거하면 set과 add가 쓴 것을 뺀다. remove는 파일 설치보다 먼저 실행되므로 옛 판이 남긴 INI 파일을 위한 것이다.

검색과 요구: [search.ID], [require.ID]

검색은 무엇보다 먼저 실행되고 찾은 것을 공개 속성에 넣는다(못 찾으면 비어 있다): 레지스트리 값의 데이터, 파일의 전체 경로(path = 알려진 폴더와 상대 경로, 예 System이나 ProgramFiles/Example; 프로그램 파일은 min-version), 폴더, 또는 다른 제품 구성 요소의 키 파일. 요구 사항은 condition이 거짓이면 그 메시지로 첫

설치를 멈춘다(복구와 제거는 막지 않는다). 조건은 Windows Installer 의 문법을 쓰고 (VersionNT >= 603, FOUND_TOOL, NOT OLDSETTING) 검색 결과를 볼 수 있다.

```
[search.Tool]
property = "FOUND_TOOL"
kind = "file"
path = "System"
file = "tool.exe"

[require.Tool]
condition = "FOUND_TOOL"
message = "[ProductName] needs tool.exe."
```

registry 나 dir 검색은 속성 대신 dir 을 가리킬 수 있다. 그러면 찾은 폴더가 그 dir 의 기본값이 된다. 사용자가 고른 폴더를 패키지가 기억하는 방법이 이것이다: 폴더를 적어 두고, 다음 판에서 찾아본다. 레지스트리 값은 그 폴더가 아직 있을 때만 쓰이고, 명령줄에 준 폴더(msiexec /i app.msi INSTALLDIR=D:\Apps\Example\)가 여전히 이긴다.

```
[registry.RememberDir]
root = "HKLM"
key = "Software\\Example"
name = "InstallDir"
value = "[INSTALLDIR]"

[search.PreviousDir]
property = "INSTALLDIR"           # dir: 레지스트리에 있는 폴더가 실제로 있으면 그것이 기본값
kind = "registry"
root = "HKLM"
key = "Software\\Example"
name = "InstallDir"
```

서비스, 글꼴, 권한

[service.ID] 는 패키지의 .exe 가 돌리는 서비스를 설치한다: 파일이 바뀌기 전과 제거할 때 멈추고, 제거할 때 지우며, start-on-install = true 면 설치 뒤에 시작한다. [font.ID] 는 패키지가 Fonts 폴더에 설치하는 글꼴 파일을 등록한다. title 이 없으면 Windows 가 TrueType/OpenType 파일에서 이름을 읽는다. [permission.ID] 는 패키지가 만드는 폴더, 그 파일, 또는 그 레지스트리 값에 SDDL 보안 서술자를 건다(이것이 있으면 패키지는 Windows Installer 5.0 을 요구한다).

사용자별 패키지와 겸용 패키지: scope

scope = "machine"(기본)은 모든 사용자에게 설치하고 관리자 권한이 필요하다. scope = "user" 는 권한 상승 없이 현재 사용자에게 설치한다: ProgramFiles 는 %LOCALAPPDATA%\Programs 가 되고, Programs 와 Desktop 은 그 사용자의 것, 레지스트리 값은 HKCU(또는 HKMU), 환경 변수는 사용자의 것이며 do/undo 동작은 그 사용자로 실행된다. 컴퓨터 전체 설치를 요청하면 거부한다. scope = "dual" 은 기본으로 사용자별로 설치하고, 관리자 명령창에서 msiexec /i x.msi ALLUSERS=1 MSIINSTALLPERUSER="" 로 하면 컴퓨터 전체로 설치한다. 그 레지스트리 값은 설치된 방식에 따라 HKLM 이나 HKCU 가 되는 HKMU 를 쓴다. 서비스, 글꼴, 권한과 컴퓨터 폴더(Windows, System, Fonts, CommonAppData)는 scope = "machine" 이 필요하다.

캐비닛

파일은 패키지에 넣는 캐비닛 하나로 압축한다. cab-max-size = N 은 파일이 N MiB 를 넘을 때마다 새 캐비닛을 시작하고, cab = "external" 은 캐비닛을 패키지 옆에 <name>.cab(또는 <name>-1.cab, <name>-2.cab, ...)으로 쓴다. 그 캐비닛은 패키지와 함께 다녀야 하고, cab-max-size 가 없으면 캐비닛마다 2 GiB 전에 나눈다. Windows Installer 는 2 GiB 이상인 패키지를 열지 못하므로, 그만큼 큰 내장 캐비닛은 cab = "external" 을 권한하는 오류(RP1516)다. rubrapack 은 있는 캐비닛을 덮어쓰지 않고 패키지를 맨 나중에 쓴다: 패키지는 <out>.rp-map 에서 만들어지고(바이트가 메모리가 아니라 그 파일로 바로 간다) 모든 일이 성공했을 때만 제 이름으로 바뀌므로, 실패한 빌드는 패키지를 남기지 않는다. 압축은 모든 프로세서에서 돈다(덜 쓰려면 --jobs N); 어느 쪽이든

바이트는 같다. 모든 패키지에는 관리 설치(`msiexec /a`, 압축을 푼 네트워크 이미지)와 광고(`msiexec /jm`) 순서도 들어 있다.

MSI 서식 문자열로 해석되는 곳은 이것뿐이다: 레지스트리 value(와 multi 항목), 바로가기 args, 환경 변수 value, INI value, 요구 사항 message, 서비스 args. 나머지는 적은 그대로 쓴다.

설치된 앱 항목과 속성

[arp] 는 제품이 "설정 > 설치된 앱"에 어떻게 보이는지 정한다(no-modify, no-repair, help, about). [property.NAME] 은 공개 속성을 더한다. `secure = true` 는 그 값이 설치의 관리자 권한 부분까지 가게 하고, `hidden = true` 는 로그에 값이 남지 않게 한다. 설치 엔진이나 rubrapack 이 스스로 쓰는 이름(ARP*, MSI*, RP_*, ALLUSERS, REBOOT 등)은 거부한다.

명령줄로 설치하기

대화창이 고르는 것은 모두 `msiexec` 에 줄 수 있다.

속성	하는 일
INSTALLDIR=D:\Apps\Example\	설치 폴더(대문자 ID 의 dir 모두)
ADDLOCAL=Core,Extra	이 기능들을 설치(ADDLOCAL=ALL: 모든 기능)
REMOVE=Extra	설치된 제품에서 이 기능들을 제거(REMOVE=ALL: 전부)
INSTALLLEVEL=3	level 이 3 이하인 기능을 모두 설치
RPLANGUAGE=ko	대화창 언어([ui] languages 가 있을 때)
ALLUSERS=1 MSIINSTALLPERUSER=""	검용 패키지를 모든 사용자에게(기본: 현재 사용자만)
DESK=1, APP_MODE=server	원본이 정한 속성: when 조건, 대화창 값

창 없이 설치하기

rubrapack 이 만드는 패키지는 모두 명령줄에서 사용자 화면 없이 설치·복구·업그레이드·제거된다:

```
msiexec /i example.msi /qn /l*v install.log
msiexec /x {ProductCode} /qn
```

종료 코드 0 은 성공, 3010 은 다시 시작이 필요한 성공(rubrapack 은 컴퓨터를 스스로 다시 시작하지 않는다), 그 밖의 값은 실패이며 그 뒤 컴퓨터는 설치 전과 같다.

대화창: ui

[package] 의 ui 가 내장 대화창 세트 하나를 고른다. 없으면(none) Windows Installer 자신의 진행 막대만 보인다.

ui	설치할 때	이미 설치되어 있을 때
basic	진행, 끝(또는 오류)	진행, 끝
minimal	환영, 사용권(있으면), 진행, 끝	복구 또는 제거
installldir	환영, 사용권, 설치 폴더(폴더 찾아보기 포함), 준비, 진행, 끝	복구 또는 제거
features	installldir 에 더해 기능 트리과 필요한 디스크 공간	복구 또는 제거

모든 세트에는 취소 확인, 오류 대화창, 사용 중인 파일 목록, 디스크 공간 부족 경고도 있다. 대화창은 모두 기본값이 있는 값만 모으므로 /qn 은 여전히 창 없이 설치한다.

```
[package]
ui = "installldir"
license = "LICENSE.txt"           # 동의할 때까지 설치/다음 단추가 꺼져 있다

[ui]
install-dir = "APPDIR"           # 사용자가 바꿀 수 있는 dir(기본 INSTALLDIR)
banner = "banner.bmp"           # 위쪽 띠; 약 493 x 58 화소
```

```
[ui-text.WelcomeText]
text = "This will install [ProductName]. Close other programs first."
```

.txt 나 .md 사용권은 평문으로 보여 준다(한글, 이모지 같은 어떤 글자도 그대로 둔다). .rtf 사용권은 그대로 쓴다. banner 가 없으면 띄는 흰색이다. [ui-text.ID] 는 문구 하나를 바꾼다. 문구는 MSI 서식 문자 열이라 [ProductName] 은 치환되고 [자체는 [\[] 로 쓴다. ID 는 다음과 같다: Back, Next, Cancel, Install, Finish, OK, Yes, No, Retry, Ignore, Abort, Exit, Browse, WelcomeTitle, WelcomeText, LicenseTitle, LicenseText, LicenseAccept, DirTitle, DirText, DirLabel, BrowseTitle, BrowseText, BrowseLookIn, BrowseFolder, BrowseUp, BrowseNew, CustomizeTitle, CustomizeText, Reset, DiskCost, DiskCostTitle, DiskCostText, ReadyTitle, ReadyText, ProgressTitle, ProgressText, ProgressStatus, ExitTitle, ExitText, UserExitTitle, UserExitText, FatalTitle, FatalText, CancelText, FilesInUseTitle, FilesInUseText, OutOfDiskTitle, OutOfDiskText, MaintTitle, MaintText, Repair, RepairText, Remove, RemoveText, LanguageTitle, LanguageText, DirGuardText(아래 가드의 메시지). 단추 문구에서 & 는 바로 가기 키를 표시한다(&Next 는 Alt+N).

여러 언어

대화창은 영어다. [ui] languages 는 같은 패키지에 다른 언어를 덧붙인다. 그러면 첫 페이지가 언어를 묻고, 그 뒤의 모든 페이지 - 환영, 사용권, 폴더, 사용자 페이지, 준비, 진행, 완료, 그리고 취소·오류·사용 중 파일·디스크 공간·유지보수 페이지 - 가 고른 언어로 나온다. 미리 골라 두는 언어는 사용자의 지역 형식 (UserLanguageID), 그다음 시스템 로캘(SystemLanguageID)이 LANGID 에 드는 첫 덧붙인 언어이고, 없으면 영어다. 명령줄의 RPLANGUAGE=ko 는 그 선택을 대신 정한다(언어 페이지는 그대로 나오고 그것이 골라져 있다). 무인 설치(/qn)는 아무것도 보여 주지 않으니 고를 것도 없다. 영어만 있으면 언어 페이지도 없다.

```
[ui]
languages = ["ko"]           # 영어는 늘 있고 기본이다
license-ko = "LICENSE-ko.txt" # 언어별 사용권(기본: [package] license)

[ui-text.WelcomeText]
text-ko = "[ProductName]을(를) 설치합니다." # text = 모든 언어, text-xx = 한 언어

[dialog.Options]
after = "RpInstallDirDlg"
title = "Options"
title-ko = "선택 사항"

[dialog-control.Mode]
# ...
labels = ["&Typical", "&Portable"]
labels-ko = ["표준(&T)", "휴대용(&P)"]
```

- 내장 문구는 영어와 한국어(ko)로 있다. 다른 언어(ja, de, ...)는 내장 문구를 모두 text-xx 로 주고 (빠진 것은 lint 가 이름을 댄다), name-xx(언어 페이지에 보일 이름), font-xx(그 언어 대화창의 글꼴), langid-xx(그 언어를 고르게 하는 LANGID: 숫자 하나나 배열)를 줄 수 있다. 흔한 언어는 이것들이 내장되어 있다: ja, zh, de, fr, es, it, pt, nl, pl, ru, uk, tr, vi, th.
- 한국어 대화창은 맑은 고딕, 영어는 Segoe UI 를 쓴다.
- [ProductName], [Manufacturer], [ProductVersion] 밖의 속성을 쓰는 문구는 언어를 고를 때 서식이 풀리므로 255자 안이어야 한다.
- Windows Installer 자신의 문구 - 기능 트리 메뉴, 크기, 남은 시간, 오류 메시지 - 는 패키지 것이 아니다: 앞의 것들은 영어로 남고, 오류 메시지는 Windows 를 따른다.
- [package] language 는 패키지의 언어(ProductLanguage, 요약 정보)만 정한다. 0.2 부터는 대화창을 한국어로 만들지 않는다: languages = ["ko"] 를 넣는다(lint 가 RP1317 로 경고한다).

아이콘, 완료 페이지, 설치 범위 페이지

- [arp] icon = "app.ico" 는 "설치된 앱"에 보일 제품 아이콘이다. [shortcut] icon 은 바로가기에 따로 .ico 를 준다(없으면 프로그램 자신의 아이콘). 둘 다 빌드할 때 읽어 패키지 안에 넣는다.
- [ui] launch = "file:App" 은 완료 페이지에 "[ProductName] 실행"을 둔다(launch-checked = false 가 아니면 체크된 채로; launch-args 는 그 인자). "마침"을 누르면 첫 설치나 업그레이드 뒤에 설치를 실행한 사용자의 권한으로(설치 엔진의 권한이 아니라) 프로그램을 띄운다. 복구·제거 뒤에는 띄우지 않고, /qn 에서 는 결코 띄우지 않는다.
- 대화창이 있는 scope = "dual" 패키지는 사용권 다음에 "나만"(기본) / "이 컴퓨터의 모든 사용자"(관리자 권한 필요) 페이지를 둔다. 설치 폴더도 그에 따라 %LOCALAPPDATA%\Programs 나 Program Files 로 옮긴다.

나만의 대화창 페이지: [dialog.ID], [dialog-control.ID]

minimal, installdir, features 에서는 내장 흐름에 페이지를 더할 수 있다. 더한 페이지도 다른 페이지와 같은 띠와 뒤로/다음/취소 단추를 갖고, 본문에 컨트롤을 놓는다.

```
[dialog.Options]
title = "Options"           # 띠의 제목; 없으면 [ProductName]
description = "Choose how to install."
after = "RpInstallDirDlg"   # 이 페이지 바로 다음에 나온다

[dialog-control.ModeLabel]
dialog = "Options"
type = "text"
x = 20
y = 55
width = 330
height = 12
text = "Installation &mode:"

[dialog-control.Mode]
dialog = "Options"
type = "radio"
x = 20
y = 70
width = 200
height = 42                # 값 하나에 12 이상
property = "APP_MODE"
values = ["typical", "portable", "server"]
labels = ["&Typical", "&Portable", "&Server"]

[property.APP_MODE]
value = "typical"          # 기본값, 창 없는 설치에서도 쓰인다
```

- after 는 고른 세트의 페이지 - RpWelcomeDlg, RpLicenseDlg(사용권이 있을 때), RpInstallDirDlg(installdir, features), RpCustomizeDlg(features) - 또는 직접 만든 다른 페이지다. 같은 페이지 뒤에 여러 페이지가 오면 ID 순서를 따른다. minimal 에서는 마지막 페이지의 단추가 설치이고, 다른 세트에서는 준비 페이지가 늘 마지막이다.
- 좌표는 370 x 270 페이지 위의 대화창 단위다. 컨트롤은 본문, y = 45 에서 y = 234 사이에 놓는다. Tab 키 는 컨트롤을 위에서 아래로, 그다음 왼쪽에서 오른쪽으로 옮겨 다닌다.
- text 는 글씨를 보인다(& 는 바로 가기 키를 표시하고, 글씨 컨트롤에서는 다음 컨트롤로 옮긴다). checkbox 는 체크하면 속성을 1 로, 풀면 속성을 없앤다. edit 는 사용자가 속성 값을 입력하게 한다. radio 와 combo 는 values(64바이트 이하 문자열 1~32개) 가운데 하나를 고르게 하고, 적은 순서대로 labels(기본: 값 자체)로 보인다.
- radio 와 combo 에는 그 값 가운데 하나를 값으로 가진 [property.*] 가 있어야 한다: 대화창은 값을 모을 뿐 이고 창 없는 설치(/qn)는 기본값을 쓴다. 어떤 속성이든 명령줄에서 줄 수도 있다: msixexec /i example.msi /qn APP_MODE=server.

- 속성은 직접 정한 공개 이름(대문자)이고 컨트롤 하나에 하나이며, 설치의 관리자 권한 부분까지 가므로 [registry.*], [ini.*], [env.*], 조건 등에서 [APP_MODE] 로 쓸 수 있다.
- Rp 로 시작하는 페이지 ID 와 틀의 컨트롤 ID(Banner, Title, Description, BannerLine, BottomLine, Back, Next, Cancel)는 예약되어 있다. 페이지 하나에는 컨트롤이 64개까지 든다.

아키텍처와 업그레이드 계열

arch 는 원본 자신의 아키텍처, upgrade-code 는 그 업그레이드 계열이다. 같은 원본을 다른 아키텍처로 지으려면 (--arch) 그 아키텍처에 upgrade-code-x86, upgrade-code-arm64, upgrade-code-x64 로 제 계열을 주어야 하고, 없으면 빌드를 거부한다. Windows Installer 의 업그레이드 감지는 아키텍처를 가리지 못해서, 코드를 같이 쓰면 한 아키텍처를 설치할 때 다른 아키텍처가 지워진다. MSIX 에는 업그레이드 코드가 없으므로 .msix.msixbundle 빌드에는 필요 없다.

먼저 지워야 하는 판

refuse-upgrade-below = "1.0.0" 은 설치된 1.0.0 미만 판의 업그레이드를 거부하고 먼저 지우라고 알린다. 메시지(refuse-upgrade-message, 또는 기본 문구)는 늘 그 일을 하는 명령으로 끝난다: msiexec /x {ProductCode} /qn MSIRESTARTMANAGERCONTROL=Disable. 옛 판을 다른 도구가 MSIRESTARTMANAGERCONTROL=Disable 없이 만들었을 때 쓴다: 그런 판을 업그레이드 도중에 지우면 그 파일을 불러 쓰는 프로그램을 모두 닫으려 한다(formats/msi-package.md 의 "사용 중인 파일" 참고).

와일드카드: [files.ID]

glob 은 *(한 폴더 안의 아무 글자들), ?(한 글자), **(폴더 여러 단계)가 든 원본 경로다. 맞는 것은 이름순으로 정렬하므로 결과가 파일 시스템에 따라 달라지지 않는다. 첫 와일드카드 아래의 폴더는 dir 아래에 다시 만든다: glob = "dist/layouts/**/*.*.jmt" 는 dist/layouts/de/x.jmt 를 <dir>/de/x.jmt 로 설치한다. 맞는 것이 없거나, 심볼릭 링크이거나, 출력 파일 자신이 맞으면 오류다.

빈 폴더: [folder.ID]

파일이 하나도 들지 않아도 dir 안에 name 폴더를 만든다. keep = true 면 제거한 뒤에도 폴더가 남는다(프로그램이 쓰는 데이터를 위해).

기능(feature)

[feature.*] 표가 없으면 모든 것이 숨은 기능 하나에 든다. 기능을 하나라도 선언하면 모든 파일에 기능이 있어야 한다: 파일 자신의 feature 키, 또는 그 dir 의 feature.level 이 1 보다 큰 기능은 기본으로 설치하지 않는다. ui = "features" 면 사용자가 트리에서 기능을 고르고, 나중에 "설치된 앱"에서 바꿀 수 있다(유지보수 페이지의 "변경").

- required = true: 트리에서 "설치하지 않음"을 고를 수 없다.
- follow-parent = true: 부모가 설치되는 대로 따라간다.
- when = "<조건>": 조건이 참이 아니면 그 기능은 꺼진다 - 설치하지도 보여 주지도 않는다(아래).

조건: when

when 은 Windows Installer 조건(VersionNT64, DESK = "1", NOT OLDVERSION)을 받는다. 기능, 파일과 파일 묶음, 레지스트리 값(파일과 with 로 묶인 값은 안 된다: 파일 쪽에 둔다), 바로가기, 환경 변수, INI 값에 쓸 수 있다. 조건이 참일 때만 그것을 설치한다. 나만의 대화창 페이지에서 정한 속성, 명령줄에 준 속성, 검색 결과를 볼 수 있다. 조건은 그것을 처음 설치할 때(메이저 업그레이드 포함) 따진다 - 복구는 있는 그대로 둔다. MSIX 는 모두 설치하므로 when 을 거부한다.

```
[dialog-control.Desk]           # 나만의 페이지의 체크박스
dialog = "Options"
type = "checkbox"
x = 20
y = 60
width = 300
height = 16
```

```
text = "바탕화면 바로 가기 만들기(&D)"
property = "DESK"

[shortcut.Desk]
dir = "Desktop"
name = "Example"
target = "file:App"
when = "DESK"
```

제거할 때 파일 남기기: keep

파일(또는 파일 묶음)에 `keep = true` 를 두면 제품을 제거해도 그 파일은 남는다 - 사용자가 바꿨을 수 있는 설정 파일용이다. 한 번 바뀐 파일은 복구나 다음 판이 덮어쓰지 않는다(판 없는 파일에 대한 Windows Installer의 규칙). MSIX 는 파일을 모두 지우므로 `keep` 을 거부한다.

변수

문자열 값 속 `$(NAME)` 은 한 번 치환된다. 명령줄의 `-D NAME=value` 가 먼저, 그다음 `[define]` 이다. 치환 결과는 다시 읽지 않는다(`$(X)` 가 든 값은 그대로 남는다). `$$` 는 `$` 한 글자다. 정의되지 않은 이름은 오류다. `$(ARCH)` 는 `-D` 나 `[define]` 이 정의하지 않는 한 내장이다: 짓고 있는 아키텍처(`x64`, `arm64`, `x86`) - 그래서 원본 하나가 `source = "bin/$(ARCH)/app.exe"` 처럼 아키텍처마다 제 프로그램을 가리킬 수 있다.

프로그램 파일

PE 파일(`.exe`, `.dll` 등)은 검사한다: 머신 형식이 `arch` 와 맞아야 하고(`x64` 패키지의 `x86` 도우미는 `any-arch = true` 가 필요하다), 판 정보 리소스가 패키지 안 파일의 판이 된다. Windows Installer 는 그것을 보고 설치된 파일을 바꿀지 정한다.

설치 폴더 지키기

다른 프로그램이 그 파일을 불러 쓰는 프로그램 - 입력기, 셸 확장, 서비스 - 은 남이 미리 만들어 둔 폴더에 설치되면 안 된다: 거기 있던 파일은 그대로 남고, 프로그램 옆에 심어 둔 DLL 이 그 프로그램의 권한으로 실리기 때문이다. `dir` 에 `guard = true` 를 두면, 그 폴더가 다음과 같을 때 첫 설치가 파일을 하나도 놓기 전에 멈춘다.

- 이미 있고 주인이 `SYSTEM`, `Administrators`, `TrustedInstaller` 가 아니다.
- 연결 지점(junction)이나 다른 리파스 포인트를 거쳐 닿는다(폴더 자신이나 그 위의 어느 폴더든).

아직 없는 폴더는 통과한다(설치 엔진이 만들고, `[permission.*]` 로 잠글 수 있다). 복구, 제자리 업그레이드, 제거는 검사하지 않는다. 메이저 업그레이드는 새 판의 첫 설치라 똑같이 검사한다(이전 판이 만든 폴더는 통과한다). 설치는 `DirGuardText` 메시지([1] 은 그 폴더; 대화창에서 고른 언어) 뒤에 1603 으로 끝나고, `/qn` 에서도 같으며, 로그에 찾은 주인이 적힌다. 검사는 rubrapack 의 도우미 DLL 이 하고, 패키지가 그 DLL 을 실행한다(`REG_QWORD` 값과 같다). 보는 것은 주인이다: 관리자 소유라도 권한이 누구나 쓸 수 있게 되어 있는 폴더는 거부하지 않는다(`[permission.*]` 로 잠근다).

```
[dir.INSTALLDIR]
path = "ProgramFiles/Example"
guard = true
```

경로

원본 경로는 원본 파일 기준의 상대 경로이고 `/` 를 쓴다. 절대 경로, `\`, 심볼릭 링크, 없는 파일은 오류다. 설치될 이름에는 Windows 가 금하는 것(`< > : " / \ | ? * , 제어 문자, 끝의 점이나 공백, con` 같은 장치 이름)만 빼고 어떤 유니코드 글자든 쓸 수 있고, 한 폴더 안의 두 이름이 대소문자만 달라서는 안 된다. Windows Installer 는 이름을 8.3 짧은 이름과 함께 UTF-16 255단위에 저장하므로, 긴 이름은 짧은 이름이 남긴 만큼만 쓸 수 있다: 세 글자 확장자의 `name.ext` 는 242단위, 확장자 없는 이름은 246단위(NTFS 는 255 를 받는다). 더 긴 이름은 RP1514 로 거부한다.

20.5 명령줄

```

rubrapack build <src.toml> -o <out.msi|out.msix|out.msixbundle> [-D NAME=VALUE]...
    [--arch x64|arm64|x86 | --arch <목록> (.msixbundle)]
    [--compress none|mszip|mszip:N] [--jobs N] [--nfc] [--reproducible]
    [<키>] [--cert <chain.pem>]
    [--timestamp <URL>] [--tsa-trust <인증서>] [--tls-trust <인증서>] [--system-roots]
    [--proxy <URL>]] [--allow-unsigned-cabs]]
    [--unsigned-test] [--msix-compress deflate|store]      (.msix, .msixbundle)
rubrapack inspect <file.msi> [table | --summary | --files | --streams]
rubrapack inspect <file.msix|file.msixbundle> [--files | --manifest]
rubrapack inspect <file.cab>
rubrapack new [msi] <name>
rubrapack new [<file>.toml] [-i]
rubrapack new <name> [--dist <folder>] [--name ...] [--ui ...] [--optional ...] ...
rubrapack edit <file>.toml [--set <table>.<key>=<value>]... [--unset <table>.<key>]... [--sync]
rubrapack guid [--from <text>]
rubrapack lint <src.toml> [-D NAME=VALUE]... [--arch x64|arm64|x86] [--target msi|msix] [--nfc] [--strict]
rubrapack lint <file.msi> [--previous <old.msi>] [--strict]
rubrapack lint <file.msix|file.msixbundle> [--strict]
rubrapack extract <file.msi|file.msix|file.msixbundle|file.cab> -d <새 폴더> [--limit-entries N] [--limit-bytes N]
rubrapack sign <file.exe|.dll|.msi|.msix|.msixbundle> <키> [--cert <chain.pem>]
    [--timestamp <URL>] [--tsa-trust <인증서>] [--tls-trust <인증서>] [--system-roots]
    [--proxy <URL>]] [--allow-unsigned-cabs] [-o <out>]
rubrapack keys list [--pkcs11 <모듈>] [--token-label <이름>] [--pin-env VAR | --pin-file FILE]]
rubrapack verify <file.exe|.dll|.msi|.msix|.msixbundle> [--trust <인증서>]... [--system-roots] [--tsa-trust <인증서>]...
rubrapack version | help [command]

```

<키> 는 다음 가운데 하나:

--key <key.pfx .pem> [--pass-env VAR --pass-file FILE]	키 파일
--pkcs11 <모듈> --key-label <이름> [--token-label <이름>]	
[--pin-env VAR --pin-file FILE]	PKCS#11 토큰 안의 키
--key-store <SHA-1 지문> [--machine-store]	Windows 저장소의 키

- 명령줄 옵션은 원본보다 앞선다.
- --reproducible 은 패키지 코드를 내용에서 끌어낸다: 같은 원본은 Linux 에서든 Windows 에서든, 어느 폴더에서든, 옵션 순서가 어떻게 같은 바이트가 된다. 없으면 패키지 코드는 무작위다 - Windows Installer 가 서로 다른 패키지 파일에 기대하는 대로. MSI 에는 시각이 전혀 들지 않으므로(요약 정보에 만든·저장한 날짜가 없고 캐비닛 항목은 1980-01-01) SOURCE_DATE_EPOCH 는 결과를 바꾸지 않는다. 원본의 경로나 지은 컴퓨터·사용자의 이름도 들지 않는다. 원본 파일은 한 번만 읽고, 그 바이트를 해시하고 판을 읽고 담는다. 짓는 동안 바뀐 파일은 빌드를 멈춘다(RP1515).
- --nfc 는 패키지가 폴더·파일·바로그가에 붙이는 이름을 유니코드 NFC(조합형)로 바꾼다. macOS 에서 온 파일은 흔히 이름이 분해형이다 - 한글 음절 U+D55C 가 자모 셋 U+1112 U+1161 U+11AB 로 - 그리고 Windows 는 이름을 있는 그대로 설치하므로 같은 낱말이 서로 다른 두 파일이 될 수 있다. 원본 파일의 이름은 그대로 두고, 문구와 레지스트리 값도 적은 대로 둔다. lint 는 NFC 가 아닌 문구를 경고하고(RP2105), 다른 이름과 같아지는 이름은 거부한다(RP1511). 정규화는 Unicode 17.0 자료로 rubrapack 이 직접 한다.
- inspect <file.msi> <table> 은 표를 Windows Installer 의 IDT 형식으로 찍는다. --files 는 파일마다 탭으로 나눈 한 줄을 찍는다: 설치 경로, 크기, File 키, 구성 요소, 판, 언어, MD5(MsiFileHash 에서; 없으면 비어 있다). --streams 는 스트림과 그 크기를 늘어놓는다. inspect <file.cab> 은 캐비닛 속 파일을 늘어놓는다. inspect <file.msix> 는 (블록 해시를 모두 확인한 뒤) 정제, 실행 파일, 파일을 보인다. --files 는 경로, 크기, 압축 여부를, --manifest 는 AppxManifest.xml 을 찍는다. 묶음은 정제와 패키지(하나하나 패키지로 열어 확인한 것)와 AppxBundleManifest.xml 을 보이고, extract 는 패키지를 꺼내 쓴다.
- new <name> 은 <name>.toml 을 쓴다. 프로그램 파일을 dist/ 에 넣기만 하면 빌드되는 원본이고 upgrade-code 는 새로 만든다. new <file>.toml, 이름 없는 new(파일 이름도 묻는다), new <name> -i 는 대신 첫 패키지의 질문을 한다(질문은 표준 오류로, 답은 표준 입력에서 한 줄씩 받으므로 파이프로 줄 수 있다; 마지막 답 전

에 입력이 끝나면 아무것도 쓰지 않는다). `new <name>` 에 옵션을 주면 묻지 않고 같은 답을 받는다: `--name, --manufacturer, --version, --dist`(기본 `dist`), `--main <파일>|-`, `--arch, --install-dir, --scope, --optional <폴더,...>|-`, `--ui, --license <파일>|-`, `--languages ko|-`, `--shortcuts start,desktop|none`; 빠진 것은 질문이 권했을 기본값을 쓴다. 원본은 폴더 최상위 파일을 하나씩 적고(바로가기는 파일을 가리키고, 그룹은 파일을 뺄 수 없다) 파일이 든 하위 폴더는 그룹으로 적는다. 선택 하위 폴더는 필수 Main 옆에 `level = 2` 기능이 된다. 쓴 뒤에는 `lint` 처럼 검사한다(문제가 있으면 종료 코드 1). 있는 파일은 절대 바꾸지 않는다. 이미 있는 `app.toml` 에 `new app.toml` 을 주면 `edit` 를 쓰라고 알린다.

- `edit <file>.toml` 은 원본을 그 자리에서 고친다. 옵션이 없으면 메뉴를 보인다: 1 이름, 제조사, 판(`version = "$(VERSION)"` 이면 `[define] VERSION`), 아키텍처; 2 Program Files 아래 폴더와 설치 범위; 3 대화창, 약관, 한국어 대화창; 4 선택 구성요소로 할 하위 폴더(처음 켜면 필수 기능 Main 을 더하고, 그때부터 기능이 필요한 모든 것에 준다); 5 시작 메뉴·바탕화면 바로가기와 그것이 여는 프로그램; 6 프로그램 폴더와 파일 목록 맞추기 - 사라진 파일과 하위 폴더는 빼고 새것은 더한다; 7 아무 표.키; v 는 글을 보이고, l 은 `lint` 처럼 검사하고, s 는 저장하고 검사하며, q 는 나간다(바뀐 것이 있으면 먼저 묻는다). 질문마다 지금 값을 권한다. 바꾼 값만 다시 쓰고 주석, 순서, 손으로 쓴 표는 그대로다. 구문이 깨질 변경은 하지 않는다. 옵션을 주면 묻지 않고 차례로 적용해 저장한다: `--set package.version=1.2.0`(TOML 값 - "글", 수, true, ["ko"]) - 이면 그대로, 아니면 문자열로 쓰고, 없는 표는 더한다), `--unset package.license, --sync`(묻지 않는 6). UTF-16 원본은 거부한다: 먼저 UTF-8 로 저장한다.
- `guid` 는 무작위 GUID(4판)를 찍는다. `guid --from <text>` 는 rubrapack 이 글에서 끌어내는 GUID 를 어느 컴퓨터에서나 같게 찍는다: `guid` 의 32비트 리틀엔디언 길이와 그 4바이트, 32비트 리틀엔디언 수 1, 글의 32비트 리틀엔디언 길이와 UTF-8 바이트에 대한 SHA-256. 해시의 앞 16바이트에 판 니블을 8 로, 변종 비트를 10 으로 넣어(RFC 9562 UUIDv8) 대문자 {XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX} 로 쓴다. 예를 들어 `guid --from hello` 는 {B52FE8EF-B68D-84B5-91AF-E6E01BEC2773} 이다.
- 진단은 `example.toml:12:3: error[RP1201]: unknown key 'nmae' in [package] (did you mean 'name'?)` 꼴이다.
- 쓰기 전에 `build` 는 완성된 표를 검사한다(RP20xx 진단, 종료 코드 5). 이 검사는 rubrapack 자신을 지키는 것이라, RP1xxx 검사를 통과한 원본은 여기에 걸리지 않아야 한다.
- `lint <src.toml>` 는 `build` 가 하는 검사를 모두 하고 아무것도 쓰지 않는다. 원본을 MSI 기준으로 검사하며, `--target msix` 를 주면 MSIX 기준으로 검사한다([`msix`] 표, 그리고 `keep` 이나 `when` 처럼 MSIX 가 답을 수 없는 것). `lint <file.msi>` 는 어떤 도구가 만든 패키지든 같은 표 규칙으로 검사한다: 설치를 멈추게 하는 것은 오류(종료 코드 5) - 열에 맞지 않는 값, 가리키는 행이 없음, 중복 키, 깨진 대화창 Tab 순서(RP2101), 대화창의 첫/기본/최소 컨트롤이 없음(RP2102), ListBox 표 없는 사용 중 파일 대화창(RP2103), ErrorText/ErrorIcon 없는 오류 대화창(RP2104) - 이고 나머지는 경고다. `--strict` 는 경고도 실패로 친다. 65001 이 아닌 코드 페이지의 패키지에는 알림(RP2100)이 붙는다: 그 글자는 UTF-8 로 검사할 수 없다. 표준 출력의 마지막 줄은 오류와 경고의 수다. `lint <file.msix>` 는 Windows 가 읽는 방식대로 - ZIP, 블록 맵, 블록마다의 해시 - 검사하고, 매니페스트에 정체가 있는지, 매니페스트가 가리키는 파일이 패키지에 있는지 본다(RP2201, RP2202). 묶음은 매니페스트의 블록 맵, 그리고 패키지마다 매니페스트가 말하는 자리·크기·정체와 패키지 자체를 본다. 아키텍처 하나에 패키지 하나다.
- `lint new.msi --previous old.msi` 는 `new.msi` 가 `old.msi` 를 깨끗하게 업그레이드하는지도 본다. 오류: 다른 UpgradeCode(RP2301: 새 패키지가 옛것을 대체하지 않는다), 앞의 세 자리가 높지 않은 버전(RP2302: Windows 는 그 세 자리만 비교한다). 경고: 같은 ProductCode(RP2303: 업그레이드에는 새 것이 필요하다), GUID 는 그대로인데 키 경로(파일, 폴더, 레지스트리 값)가 바뀐 컴포넌트(RP2304), 64비트 표시가 바뀐 컴포넌트(RP2305), 없어진 컴포넌트(RP2306: 그 자원은 옛 판과 함께 지워진다), 없어진 기능(RP2307: 패치나 설치된 기능 바꾸기에서 사라진다). 마지막 줄은 이전 패키지를 밝힌다.
- `extract` 는 패키지를 설치되는 모습대로 푼다: 폴더는 Directory 트리의 긴 이름으로(ProgramFiles64Folder 같은 표준 폴더는 그 이름 그대로), 파일은 안팎의 캐비닛에서, 또는 압축하지 않은 패키지라면 옆의 원본 폴더에서 가져온다. 파일마다 크기와 MsiFileHash 를 확인한다. 대상은 새 폴더거나 비어 있어야 하고, 패키지 전체가 통과할 때까지 아무것도 쓰지 않는다: ., /, \, :, 드라이브, 예약 장치 이름(CON, COM1 등), 끝의 점이나 공백이 든 이름, 대소문자만 다른 두 경로는 거부한다. 기본 한도는 항목 100,000개와 16 GiB 다. .cab 은 안의 이름대로 푼다. .msix 는 블록이 모두 해시와 맞은 뒤 파일(ZIP 자신의 [Content_Types].xml 과 AppxBlockMap.xml 은 빼고)을 푼다.

- `sign` 은 PE 파일, MSI 패키지, MSIX 패키지나 묶음(안의 패키지를 먼저 서명한다)에 Authenticode 서명 (SHA-256; RSA 또는 ECDSA P-256/P-384)을 제자리에서, 또는 `-o` 로 붙인다. `build --key` 는 짓는 김에 (같은 코드로) 서명하고, 서명한 `--reproducible` 빌드도 매번 같은 바이트다(서명에 시각이 없다) - 타임스탬프가 붙지 않는 한: 타임스탬프에는 서버의 시각과 일련번호가 들어 매번 다르고, 서명 안의 패키지만 같다. 캐비닛이 밖에 있는 MSI 는 거부한다: 서명이 그것을 덮지 못한다 - 캐비닛을 안에 넣거나, `--allow-unsigned-cabs` 로 경고와 함께 .msi 만 서명한다. 키 파일은 PKCS#12 파일 (.pfx/.p12: PBKDF2-HMAC-SHA256 의 AES, SHA-256 MAC - 요즘 Windows 와 OpenSSL 이 내보내는 것)이거나, 평문 또는 PBES2 로 암호화한 PKCS#8 PEM/DER 키다. `--cert` 는 키 파일에 없는 인증서를 더한다. 인증서는 코드 서명용이어야 하고, CA 가 아니어야 하며, 지금 유효해야 한다. 암호는 환경 변수나 파일에서 받고 명령줄에서는 절대 받지 않는다. 이미 서명된 파일은 거부한다. MSIX 의 [msix] publisher 는 Windows 가 쓰는 방식의 인증서 주체 - 부분을 마지막부터 처음으로, O=Example Ltd, CN=Example 처럼 - 와 같아야 하고, 다르면 서명을 멈추고 써야 할 주체를 알려 준다. `--unsigned-test` 와 키는 함께 쓰지 않는다.
- 하드웨어 밖으로 나가서는 안 되는 키 - 2023년부터 공개 신뢰 코드 서명 인증서에 요구되는 것 - 는 거기 때문이다: rubrapack 이 해시, CMS 구조, 타임스탬프 요청을 스스로 계산하고 서명 연산 하나만 맡긴다. `--pkcs11 <모듈>` 은 토큰의 PKCS#11 라이브러리를 불러오고(rubrapack 이 불러오는 유일한 라이브러리이고, 이름 준 것만 불러온다), `--pin-env` 나 `--pin-file` 의 PIN 으로(명령줄은 절대 아님) 로그인해, `--key-label` 이라는 이름표의 개인 키와 토큰에 같은 ID 로 저장된 인증서를 쓴다(토큰에 인증서가 없으면 `--cert` 가 그것이나 사슬의 나머지를 준다). 토큰이 여럿이면 `--token-label` 로 고른다. Windows 에서 `--key-store <지문>` 은 현재 사용자의 개인 저장소(`--machine-store`: 로컬 컴퓨터의 것)에 있고 개인 키에 NCrypt 로 닿을 수 있는 인증서 - 키 저장소 공급자를 거친 스마트카드·토큰·TPM, 또는 소프트웨어 키 - 를 쓰고, Windows 가 만들어 주는 사슬을 가져온다. RSA 키와 ECDSA 키 모두 된다. 서명은 모두 파일에 쓰기 전에 인증서로 확인한다.
- `keys list` 는 서명에 쓸 수 있는 키를 보인다: `--pkcs11` 이면 토큰의 개인 키마다 이름표, ID, 인증서를, Windows 에서 그것 없으면 개인 저장소에서 개인 키가 있는 인증서와 그 지문을 보인다. 인증서마다 그것이 서명하는 MSIX publisher 와 만료일을 찍고, 비밀은 아무것도 찍지 않는다.
- `--timestamp <URL>` 은 RFC 3161 타임스탬프 서버에 서명의 연서명을 청해, 인증서가 만료된 뒤에도 서명이 유효하게 한다. 없으면 `sign` 과 `build --key` 는 그렇지 않을 것이라고 경고한다. 기본 서버는 없고, 서버를 적지 않으면 네트워크에 아무것도 나가지 않는다. 공개 서버로는 <http://timestamp.digicert.com>, <http://timestamp.sectigo.com>, <http://time.certum.pl> 이 있다. `http` 여도 괜찮다: 답 자체가 서명되어 있고 확인하기 때문이다(nonce, 서명의 해시, 서버의 서명과 그 타임스탬프 인증서). `--tsa-trust` 는 서버 인증서의 경로가 준 인증서 가운데 하나에서 끝나야 한다는 조건을 더한다. `https` 서버와는 rubrapack 이 TLS 1.3 을 직접 말하고(AES-GCM, x25519 또는 P-256, RSA-PSS 또는 ECDSA 서버 키; TLS 1.2 는 아님) 서버 인증서를 확인한다 - `--tls-trust` 로 준 루트나 `--system-roots` 로 운영 체제 저장소에서 찾은 루트까지의 경로, 날짜, 서버 이름을 담았는지(subjectAltName; 점을 넘는 와일드카드 없음), TLS 서버용인지. 이 검사를 끄는 방법은 없다. 세 가지 신뢰는 섞이지 않는다: `--tls-trust` 와 `--system-roots` 는 연결을 누구 말로 믿을지, `--tsa-trust` 는 시각을 누구 말로 믿을지를 정하고, 어느 것도 다른 쪽에 쓰이지 않는다. Linux 에서 `--system-roots` 는 `$SSL_CERT_FILE` 이나 배포판의 CA 묶음을 읽고, Windows 에서는 ROOT 저장소를 읽는다. Windows 는 어떤 루트를 무언가가 처음 필요로 할 때에야 채우므로 그때까지 없을 수 있다. `--proxy http://host:port` 는 요청을 프록시로 보낸다(`https` 서버는 `CONNECT` 로, 그래서 프록시는 서버 이름만 본다). 없으면 `https_proxy` 나 `HTTPS_PROXY` (`https` 서버), `http_proxy`(`http`; 대문자 `HTTP_PROXY` 는 읽지 않는다 - CGI 환경에서는 요청 머리가 그것을 정할 수 있다)를 쓰되, `no_proxy` 나 `NO_PROXY` - 이름들, 각각 그 아래 이름까지 포함, 또는 * - 에 서버가 있으면 쓰지 않는다. 암호를 묻는 프록시와 `https://` 프록시는 지원하지 않는다(분명한 오류). 타임스탬프가 실패하면 - 답 없음, 거절, 잘못된 답, 서명 인증서의 유효 기간 밖의 시각 - 아무것도 서명하지 않고 아무것도 쓰지 않는다(종료 코드 6). 서명이 타임스탬프 없이 슬그머니 나가는 일은 없다. 한도: 연결 10초, 전체 60초, 답 4 MiB, 리디렉션 3번, 네트워크 오류나 5xx 답 뒤 재시도 2번.
- `verify` 는 무엇을 확인했는지 찍는다 - 구조, 다이제스트, 서명, `--trust` 로 준 인증서까지의 경로, 폐기 여부(찾아보지 않는다: not-checked), 타임스탬프 - 그리고 모두 맞고 경로가 신뢰하는 인증서에서 끝날 때만 성공한다. `--trust` 가 없으면 아무것도 신뢰한다고 하지 않는다(종료 코드 4). `--system-roots` 는 서명자의 경로에 대해(타임스탬프 쪽은 아님) `--trust` 에 운영 체제의 루트를 더한다. Windows 자신의 평판 검사에 대해서

는 아무것도 말하지 않는다. 타임스탬프 줄은 not-present, <시각>, TSA not-checked (give --tsa-trust), <시각>, TSA trusted, <시각>, TSA untrusted, invalid, unsupported(Set-AuthenticodeSignature -TimestampServer 가 쓰는 옛 Authenticode 타임스탬프로, rubrapack 은 확인하지 않는다) 가운데 하나이고 뒤의 셋은 실패다. 서버가 --tsa-trust 로 신뢰되는 타임스탬프만 - --trust 는 여기에 치지 않는다 - 서명자 인증서가 유효해야 하는 시각을 지금에서 찍힌 시각으로 옮긴다. (시험용으로, 환경 변수 RUBRAPACK_TEST_NOW - 1970년부터의 초 - 가 verify 검사의 "지금"을 대신한다. 그래서 시계를 바꾸지 않고 만료된 인증서를 확인할 수 있다. 그 시각으로 맞춘 시계가 허용하지 않을 것은 아무것도 넓히지 않는다.)

- 종료 코드: 0 성공, 1 원본 오류, 2 사용법, 3 입출력, 4 서명, 5 lint, 6 네트워크.

20.6 진단 코드

rubrapack 이 알리는 모든 문제에는 파일, 줄, 열 다음에 코드가 붙는다: error[RPnnnn] 또는 warning[RPnnnn]. 앞의 두 자리가 문제의 종류를 말한다:

코드	무엇이 잘못됐나	볼 곳
RP00xx	명령줄: 모르는 명령이나 옵션, 읽거나 쓸 수 없는 파일	rubrapack help <명령>
RP10xx	원본 파일의 인코딩: UTF-8(또는 BOM 있는 UTF-16)이 아님, 짝 없는 캐리지 리턴	파일을 UTF-8 로 저장한다
RP11xx	rubrapack 이 읽는 부분집합 밖의 TOML: 여러 줄 문자열, 인라인 표, 두 번 정의한 표	TOML 부분 집합
RP12xx	표와 키: 모르는 표나 키(제안과 함께), 빠진 필수 키나 표, 기능이 생긴 뒤 기능 없는 항목	표
RP13xx	값: ID(모든 표에 걸쳐 유일, 예약어 아님), GUID, 판, 범위를 벗어난 수, 없는 것을 가리키는 참조	그 표의 절
RP14xx	변수: 값 없는 \$(NAME), 닫히지 않은 \$(	변수
RP15xx	설치할 파일: 없음, 폴더임, 링크임, 맞는 것 없는 글롭, 다른 아키텍처의 프로그램, 너무 긴 이름, 빌드 중에 바뀐 파일, 너무 큰 패키지	경로, 프로그램 파일
RP16xx	MSIX: MSIX 패키지에 필요한 것, 담을 수 없는 것	MSIX 패키지
RP19xx	이 rubrapack 이 제공하지 않는 기능	-
RP20xx	완성된 표가 Windows Installer 규칙을 어김. RP1xxx 검사를 통과한 원본은 여기에 걸리지 않아야 한다: 알려 주기 바란다	-
RP21xx	패키지 lint: 대화창, 코드 페이지, 글자 정규화	명령줄의 lint
RP22xx	MSIX 패키지나 묶음의 lint	명령줄의 lint
RP23xx	lint --previous: 이 패키지가 앞 판을 깨끗하게 업그레이드하지 못함	판과 업그레이드

무엇을 고칠지는 메시지가 말한다. 매뉴얼에서 찾을 때는 위의 코드 범위를 쓴다.

제3부

기초 지식

21 비트, 바이트, 16진수

이 장의 차례

21.1	비트	92
21.2	2진수	92
21.3	바이트	92
21.4	16진수	93
21.5	16진 덤프 읽기	93
21.6	단위: KB, MB 와 KiB, MiB	94
21.7	쓰이는 곳	94

이 부는 매뉴얼의 나머지가 덮고 선 개념을 처음부터 설명한다. 컴퓨터를 쓸 줄 안다는 것 말고는 아무것도 가정하지 않는다. 각 장의 끝에는 그 개념이 튜토리얼과 제4부의 어디에 쓰이는지 적었다.

21.1 비트

컴퓨터가 저장하는 모든 것 - 프로그램, 사진, 이 매뉴얼, 설치 파일 - 은 결국 **비트**의 긴 줄이다. 비트는 값이 둘 이고 0 과 1 로 쓴다(하드웨어에서는 전하가 없음과 있음, 자기의 한 방향과 반대 방향). 비트 하나는 예/아니요 질문 하나에 답할 수 있다.

비트 여럿을 모으면 더 많은 값을 구별할 수 있다. 비트 하나를 더할 때마다 가짓수가 두 배가 된다:

비트 수	서로 다른 값	전부
1	2	0 1
2	4	00 01 10 11
3	8	000 001 010 011 100 101 110 111
8	256	00000000 ... 11111111
16	65,536	
32	4,294,967,296	

n 비트로는 $2 \times 2 \times \dots \times 2(n \text{ 번}) = 2^n$ 가지 값이 있다.

21.2 2진수

비트의 줄은 10진수 숫자를 읽듯 수로 읽을 수 있다. 10진수에서는 한 자리가 오른쪽 자리의 열 배다: 305 는 $3 \times 100 + 0 \times 10 + 5 \times 1$ 이다. **2진수**(밑이 2)에서는 한 자리가 오른쪽 자리의 두 배다:

bit:	1	0	1	1	0	0	1	0
worth:	128	64	32	16	8	4	2	1
	---		--	--			-	
	128	+	32	16		+	2	= 178

그래서 2진수 10110010 은 10진수로 178 이다. 값이 1 인 맨 오른쪽 비트를 비트 0, 이 여덟 개 중 값이 128 인 맨 왼쪽 비트를 비트 7 이라고 부른다. 비트 k 의 값은 2^k 다.

21.3 바이트

비트는 여덟 개씩 묶여 **바이트**가 된다. 바이트는 파일을 이루는 가장 작은 조각이다: 28,672 바이트인 파일 - 튜토리얼의 첫 hello.msi 크기 - 은 $28,672 \times 8 = 229,376$ 비트다. 바이트 하나는 0 부터 255 까지 256 가지 값 중 하나를 담는다.

바이트가 무엇을 뜻하는지는 전적으로 프로그램이 그것을 어떻게 읽느냐에 달렸다: 같은 바이트 01001000 이 수 72 이기도 하고, 텍스트 파일에서는 글자 H(3장)이기도 하고, 더 큰 수의 일부(2장)이기도 하고, 프로그램 안에 서는 기계 명령의 일부이기도 하다. 파일은 그것을 말해 주지 않는다. 말해 주는 것은 파일의 형식이다. 이 매뉴얼의 제4부가 그런 형식의 설명이다.

21.4 16진수

2진수는 쓰기에 길고 읽기에 어렵다. 10진수는 바이트와 줄이 맞지 않는다. 그래서 바이트는 보통 **16진수**(밑이 16, "hex")로 쓴다: 숫자 0-9 다음에 10 부터 15 를 뜻하는 A-F 를 쓴다. 16진수 한 자리는 정확히 4비트이므로, 바이트 하나는 정확히 16진수 두 자리다:

10진수	2진수	16진수	10진수	2진수	16진수
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

바이트를 16진수로 바꾸려면 여덟 비트를 넷씩 둘로 나눠 각각 찾는다:

```
10110010
1011 0010
  B    2    -> B2
```

거꾸로: 16진수 B2 는 $B \times 16 + 2 = 11 \times 16 + 2 = 178$ 이다. 같은 수를 세 가지로 쓰면 178, 10110010, B2.

수가 16진수임을 밝히려고 프로그래머는 앞에 0x 를 붙이거나(0xB2), 뒤에 h 를 붙이거나(B2h), 그냥 말로 밝힌다. 이 매뉴얼은 바이트를 사이에 빈칸을 둔 16진수 두 자리로(B2 00 FF), 16진수로 적은 수는 0x 를 붙여 쓴다: 0x1000 은 4096 이다.

Windows 계산기의 프로그래머 모드는 10진수, 16진수, 2진수를 서로 바꿔 준다: 하나에 수를 넣고 나머지를 읽는다.

21.5 16진 덤프 읽기

16진 덤프는 파일의 바이트를 한 줄에 열여섯 개씩 16진수로 보인다. 튜토리얼의 첫 hello.msi 의 처음 64 바이트다:

```
offset  bytes                                     as text
00000000  d0 cf 11 e0 a1 b1 1a e1 00 00 00 00 00 00 00 00  .....
00000010  00 00 00 00 00 00 00 00 3e 00 04 00 fe ff 0c 00  .....>.....
00000020  06 00 00 00 00 00 00 00 01 00 00 00 01 00 00 00  .....
00000030  01 00 00 00 00 00 00 00 00 10 00 00 02 00 00 00  .....
```

- 왼쪽의 **오프셋**은 그 줄 첫 바이트의 위치로, 0 부터 세어 16진수로 적는다: 00000010 은 16번째 바이트, 00000030 은 48번째 바이트다. 오프셋 0x1C 의 바이트는 둘째 줄의 열세 번째인 fe 다.
- 그 뒤에 바이트가 16진수 두 자리씩 온다.
- 마지막 열은 글자인 바이트(32~126, 3장)는 글자로, 나머지는 . 으로 보인다. 여기에는 글이 거의 없다. 텍스트 파일이라면 이 열을 읽을 수 있다.

처음 여덟 바이트 d0 cf 11 e0 a1 b1 1a e1 은 모든 .msi 가 쓰는 복합 파일 형식의 서명이다 - 상상력을 조금 보태 읽으면 "DOCFILE". 모든 파일 형식은 이런 고정된 무늬, **매직 넘버**로 시작해서 프로그램이 파일의 정체를 알아볼 수 있다: 프로그램(.exe)은 4d 5a("MZ"), ZIP 압축 파일과 그래서 MSIX 는 50 4b 03 04("PK"), 캐비

닛은 4d 53 43 46("MSCF")로 시작한다. 파일 이름은 거짓말을 할 수 있어도 첫 바이트들은 좀처럼 그러지 않는다.

직접 이렇게 보려면 PowerShell 에서:

```
PS C:\work\hello> Format-Hex hello.msi | Select-Object -First 4
```

(배치가 위와 조금 다르지만 바이트는 같다.)

21.6 단위: KB, MB 와 KiB, MiB

크기는 바이트로 세고, 큰 수에는 접두어를 붙인다. 접두어는 두 벌이 있고, 둘을 헷갈리기 쉽다:

10진(SI)	바이트	2진(IEC)	바이트
1 kB (킬로바이트)	1,000	1 KiB (키비바이트)	$1,024 = 2^{10}$
1 MB (메가바이트)	1,000,000	1 MiB (메비바이트)	$1,048,576 = 2^{20}$
1 GB (기가바이트)	1,000,000,000	1 GiB (기비바이트)	$1,073,741,824 = 2^{30}$
1 TB (테라바이트)	10^{12}	1 TiB (테비바이트)	2^{40}

2진 크기가 쓰이는 것은 메모리와 파일 형식이 2의 거듭제곱으로 일하기 때문이다. 위의 덤프에도 하나 있다: 오프셋 0x1E 에 복합 파일 머리가 0c 00, 곧 수 12 를 저장하고, 그 섹터의 길이는 $2^{12} = 4096$ 바이트(4 KiB, 0x1000) 다 - 형식이 크기가 아니라 지수를 저장한다. MSIX 블록 맵의 한 항목은 64 KiB 를 덮는다. Windows 탐색기는 "KB", "MB" 라고 쓰지만 뜻은 KiB, MiB 다. 디스크 제조사는 10진 단위를 뜻한다. 그래서 "1 TB" 디스크가 931 GB 로 보인다.

rubrapack 은 한도가 2의 거듭제곱일 때 2진 단위를 쓴다: cab-max-size 는 MiB 이고, Windows Installer 의 .msi 한도는 2 GiB(2,147,483,648 바이트)다. 이 매뉴얼이 프로그램 크기를 "MB" 로 쓸 때("200 MB 프로그램")는 대략의 크기이고, 그 차이는 중요하지 않다.

21.7 쓰이는 곳

- 제4부의 모든 장이 파일 내용을 16진 덤프와 오프셋으로 보인다. 첫 장은 복합 파일이다.
- 튜토리얼 15장: MiB 단위의 cab-max-size 와 2 GiB 한도.
- 튜토리얼 18장: inspect --files 의 파일 크기는 바이트다.

22 파일 속의 수, 그리고 비트 플래그

이 장의 차례

22.1	1, 2, 4, 8 바이트	95
22.2	바이트 순서: little-endian	95
22.3	음수: 2의 보수	96
22.4	비트 플래그	96
22.5	비트 확인하기와 켜기: AND, OR	97
22.6	쓰이는 곳	97

파일 형식은 크기, 개수, 위치, 판 번호 같은 수를 정해진 수의 바이트로 저장한다. 이 장은 그 방법과, 수 하나가 예/아니오 설정을 한꺼번에 여럿 담는 방법을 보인다.

22.1 1, 2, 4, 8 바이트

파일에 저장된 수는 형식이 정한 만큼의 바이트를 차지한다:

크기	비트	부호 없는 범위	흔한 이름
1 바이트	8	0 ~ 255	byte, u8, uint8
2 바이트	16	0 ~ 65,535	word, u16, uint16, short
4 바이트	32	0 ~ 4,294,967,295	double word(DWORD), u32, uint32, int
8 바이트	64	0 ~ 약 1.8×10^{19}	quad word(QWORD), u64, uint64

부호 없는(unsigned)은 음수가 없다는 뜻이다. 크기에 비해 너무 큰 수는 저장할 수 없다 - 그래서 i2(2 바이트)로 선언된 MSI 표의 열은 100,000 을 담지 못하고, 위치를 4 KiB 섹터의 4바이트 번호로 적는 복합 파일에는 크기 상한이 있다.

22.2 바이트 순서: little-endian

여러 바이트로 된 수는 바이트로 쪼개진다. 튜토리얼의 hello.exe 크기인 17,920 바이트를 보자. 16진수로는 0x00004600 이고, 큰 자리부터 네 바이트로 쓰면 00 00 46 00 이다. 그런데 Windows 용 파일은 거의 언제나 바이트를 반대로, **작은 자리부터** 저장한다:

```
17920 = 0x00004600
```

```
most significant first (big-endian):  00 00 46 00
least significant first (little-endian): 00 46 00 00
                                     | | | |
                                     | | | +-- x 16,777,216 (256^3) = 0
                                     | | +----- x 65,536      (256^2) = 0
                                     | +----- x 256           = 17,920  (0x46 = 70)
                                     +----- x 1               = 0
```

이 순서를 **little-endian**("작은 끝 먼저")이라 하고, 반대는 **big-endian** 이라 한다. Windows 컴퓨터의 Intel 과 Arm 처리기는 little-endian 으로 일하고, MSI, CAB, ZIP, 프로그램 파일 형식도 그렇다. 네트워크 프로토콜과 인증서(8장)는 big-endian 을 쓴다. 제4부의 표가 "u32 LE" 라고 하면 "부호 없는 4바이트 수, little-endian" 이라는 뜻이다.

덤프에서 little-endian 수를 읽는 법: 그 바이트들을 뒤집어 16진수로 읽는다. 1장의 복합 파일 머리에서:

오프셋	바이트	뒤집으면	값	뜻
0x18	3e 00	00 3e	62	부 판 번호
0x1A	04 00	00 04	4	주 판 번호: 4 KiB 섹터
0x1C	fe ff	ff fe	0xFFFFE	바이트 순서 표시: 0xFFFFE 를 얻은 읽기 프로그램은 거꾸로 읽었음을 안다
0x1E	0c 00	00 0c	12	섹터 크기는 $2^{12} = 4096$
0x38	00 10 00 00	00 00 10 00	4096	이보다 작은 스트림은 미니 스트림에 들어간다

22.3 음수: 2의 보수

음수가 필요한 형식은 2의 보수를 쓴다: 같은 비트를 쓰되 범위의 위쪽 절반을 음수로 읽는다. 1바이트에서 0~127 은 그대로이고, 128~255 가 -128~-1 을 나타낸다:

바이트	부호 없이	부호 있게
00	0	0
7f	127	127
80	128	-128
fe	254	-2
ff	255	-1

규칙: 맨 위 비트가 1 이면 음수이고, 값은 부호 없는 값에서 $2^{\text{비트수}}$ 를 빼 것이다. 4바이트에서 ff ff ff ff 는 -1, fe ff ff ff 는 -2 다. 형식들은 이런 "모두 1" 값을 표시로 즐겨 쓴다: 복합 파일에서 섹터 번호 0xFFFFFFFF(-2) 는 "사슬의 끝", 0xFFFFFFFF(-1)는 "비어 있음"이다.

22.4 비트 플래그

형식이 한 가지에 대해 예/아니오 설정을 여럿 두어야 할 때가 많다. 설정마다 바이트를 하나씩 주는 대신, 설정마다 수의 비트 하나를 주고, 켜진 비트의 값을 더한 것이 그 수가 된다. 비트마다 값이 2의 거듭제곱 - 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, ... - 이므로 어떤 합이든 만드는 방법은 하나뿐이고, 그 수에서 설정을 거꾸로 읽어 낼 수 있다.

튜토리얼에서 이런 수를 여럿 만났다. 여기서 분해해 본다 - 수를 2진수로 쓰면 1 하나하나가 설정 하나다:

수	2진수	켜진 비트	뜻(튜토리얼 장)
258	1 0000 0010	$256 + 2$	Upgrade 행: 최소 판 포함, 찾기만 함(3)
272	1 0001 0000	$256 + 16$	Component: 64비트, 영구(4)
512	10 0000 0000	512	File: 필수(2, 18)
163	1010 0011	$128 + 32 + 2 + 1$	ServiceControl: 제거 때 삭제, 제거 때 멈춤, 설치 때 멈춤, 설치 때 시작(12)
3090	1100 0001 0010	$2048 + 1024 + 16 + 2$	사용자 지정 동작: 가장(impersonation) 없음, 지연, 원본은 File, .exe(13)
3410	1101 0101 0010	$2048 + 1024 + 256 + 64 + 16 + 2$	같은 것에 되돌리기와 종료 코드 무시를 더함(13)

손으로 분해하려면 들어가는 가장 큰 2의 거듭제곱을 빼고 되풀이한다: $3090 - 2048 = 1042$, $1042 - 1024 = 18$, $18 - 16 = 2$, $2 - 2 = 0$. 그래서 $3090 = 2048 + 1024 + 16 + 2$ 다. (Microsoft 문서는 사용자 지정 동작의 $2 + 16$ 을 합쳐 "형식 18" 이라 부른다: 패키지가 설치하는 .exe 를 실행한다.)

16진수로 보면 플래그가 잘 보인다. 16진수 한 자리가 4비트이기 때문이다: $3090 = 0xc12$ 이고, $c = 1100$ 은 2048 과 1024, $1 = 0001$ 은 16, $2 = 0010$ 은 2 다.

22.5 비트 확인하기와 켜기: AND, OR

프로그램은 AND 로 플래그를 확인한다: 두 수에서 모두 1 인 비트만 남긴다.

```
3090 = 1100 0001 0010
AND 1024 = 0100 0000 0000
        = 0100 0000 0000  not zero: "deferred" is on
```

OR(어느 한쪽이라도 1 이면 1)로 비트를 켜고, 반대 수와의 AND 로 끈다. Windows Installer 조건(튜토리얼 9장)에도 같은 확인이 연산자 >< 로 있다: MYFLAGS >< 2 는 속성 MYFLAGS 에서 값이 2 인 비트가 켜져 있으면 참이다. rubrapack 을 쓰는 데 이것들이 필요하지는 않지만 - 이 수들은 rubrapack 이 모두 계산한다 - 수가 왜 그렇게 생겼는지 설명해 준다.

22.6 쓰이는 곳

- 제4부의 모든 "u16", "u32 LE". 첫 예는 복합 파일의 머리와 섹터 사슬이다.
- 튜토리얼 2, 3, 4, 12, 13장: 표 속의 수.

23 글자: 문자, 코드 포인트, 인코딩

이 장의 차례

23.1	문자와 코드 포인트	98
23.2	ASCII	98
23.3	코드 페이지: 한 바이트, 256 문자	99
23.4	UTF-8	99
23.5	UTF-16	99
23.6	바이트 순서 표시	100
23.7	유니코드 정규화	100
23.8	글자가 이상하게 보일 때	100
23.9	쓰이는 곳	100

파일은 바이트를 담고, 바이트는 수다(1장). 글 - 제품 이름, 파일 이름, 약관 - 은 문자마다 수를 주고 그 수를 바이트로 써서 저장한다. 둘 다 방법이 여러 가지이고, 방법을 잘못 고른 패키지는 깨진 글자를 보인다. 이 장은 rubrapack 과 Windows 가 쓰는 방법들을 설명한다.

23.1 문자와 코드 포인트

유니코드는 세계의 문자 목록이다: 모든 문자 체계의 글자, 숫자, 문장 부호, 기호, 이모지 - 약 16만 개. 문자마다 코드 포인트라는 번호가 있고, $u+$ 와 16진수 네 자리 이상으로 쓴다:

문자	코드 포인트	목록의 이름
H	U+0048	LATIN CAPITAL LETTER H
é	U+00E9	LATIN SMALL LETTER E WITH ACUTE
€	U+20AC	EURO SIGN
한	U+D55C	HANGUL SYLLABLE HAN
😊	U+1F600	GRINNING FACE

코드 포인트는 U+10FFFF 까지 있다. 코드 포인트는 어떤 문자인지 말할 뿐, 어떤 바이트를 쓸지는 아직 말하지 않는다. 그것이 인코딩의 일이다.

23.2 ASCII

아직도 어디에나 있는 가장 오래된 인코딩은 ASCII(1963년)다: 문자 128개, 코드 포인트 0~127, 한 문자에 한 바이트. 영어 글자, 숫자, 문장 부호와 제어 문자 몇 가지가 있다:

바이트	문자
00-1f	제어 문자: 09 탭, 0a 줄 바꿈(line feed), 0d 캐리지 리턴
20	빈칸
30-39	0-9
41-5a	A-Z
61-7a	a-z
2e 2f 3a 5c	. / : \

튜토리얼의 dist\docs\guide.txt 는 여섯 바이트다:

```
47 75 69 64 65 0a
G u i d e (new line)
```

유니코드의 처음 128 코드 포인트는 ASCII 의 것이고, 아래의 모든 인코딩이 그것들을 같은 한 바이트로 쓴다. 그래서 영어 글은 거의 어떤 인코딩으로든 똑같이 보이고 - 문제는 다른 글자에서만 드러난다.

Windows 는 줄 끝을 두 바이트 0d 0a(캐리지 리턴, 줄 바꿈)로, Linux 와 macOS 는 0a 하나로 쓴다. 대부분의 프로그램은 둘 다 읽는다.

23.3 코드 페이지: 한 바이트, 256 문자

ASCII 는 바이트 값 128~255 를 쓰지 않는다. 수십 년 동안 언어권마다 그 자리를 다르게 채웠고, 채운 방식 하나하나가 Microsoft 가 번호를 붙인 **코드 페이지**다:

코드 페이지	지역	é	€	한
1252	서유럽	e9	80	-
949	한국어	-	a2 e6	c7 d1
65001	모두: 이것이 UTF-8 이다	c3 a9	e2 82 ac	ed 95 9c

코드 페이지는 자기 지역의 문자만 담을 수 있다(949 는 한글에 두 바이트를 쓴다). 그리고 같은 바이트가 코드 페이지마다 다른 뜻이다: 바이트 e9 는 1252 에서는 é 지만 949 에서는 한국어 문자의 절반이다. 한쪽으로 쓰고 다른 쪽으로 읽은 글은 깨져 나온다. 흔히 "글자 깨짐"이라 하고, 영어로도 일본어 낱말 mojibake 로 부른다.

MSI 패키지는 모든 글에 코드 페이지 하나를 쓴다. rubrapack 은 늘 65001, UTF-8 로 써서 어느 언어든 패키지 하나에 들어간다. 튜토리얼 7장에서 inspect 가 그것을 보였다. 제4부는 함정 하나를 보인다: 요약 정보에는 코드 페이지가 따로 있다.

23.4 UTF-8

UTF-8 은 모든 유니코드 코드 포인트를 1~4 바이트로 쓴다. ASCII 는 한 바이트 그대로이므로 ASCII 파일은 이미 UTF-8 이다. 더 큰 코드 포인트는 정해진 비트 무늬를 따라 바이트를 더 쓴다:

코드 포인트	바이트	비트 무늬(x = 코드 포인트의 비트)
U+0000 - U+007F	1	0xxxxxxx
U+0080 - U+07FF	2	110xxxxx 10xxxxxx
U+0800 - U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000 - U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

한, U+D55C 로 해 보면:

```
D55C in binary:    1101 0101 0101 1100    (16 bits)
split 4 + 6 + 6:   1101  010101  011100
into the pattern:  1110 1101  10 010101  10 011100
in hex:            ED      95      9C
```

그래서 한 은 UTF-8 로 ed 95 9c 다. 이 무늬 덕분에 UTF-8 은 확인하기 쉽다: 10 으로 시작하는 바이트는 결코 문자의 첫 바이트가 아니므로 읽는 쪽은 언제나 문자가 어디서 시작하는지 찾을 수 있고, 다른 인코딩의 바이트가 우연히 올바른 UTF-8 이 되는 일은 거의 없다. rubrapack 은 원본에서 무늬를 깨는 첫 바이트에서 멈춘다 (RP1001: not valid UTF-8). 짐작하지 않는다.

23.5 UTF-16

UTF-16 은 U+FFFF 까지의 코드 포인트를 2바이트 단위 하나로, 나머지는 단위 둘(대리 쌍, surrogate pair)로 쓴다. Windows 는 안에서 이것을 쓴다: 파일 이름, 레지스트리, 복합 파일 디렉터리의 모든 글이 UTF-16 little-endian (2장)이다:

```

H   e   l   l   o
48 00 65 00 6c 00 6c 00 6f 00      "Hello" in UTF-16LE: ASCII letters get a 00 after them

한
5c d5                                U+D55C: the two bytes of D55C, little end first

😊
3d d8 00 de                          U+1F600: too big for one unit, so the pair D83D DE00

```

덤프에서 글자 사이사이에 점이 낀 것처럼 보이는 글 - H.e.l.l.o. - 은 UTF-16 이다.

23.6 바이트 순서 표시

파일은 어떤 인코딩인지 밝히려고 코드 포인트 U+FEFF, **바이트 순서 표시(BOM)**로 시작할 수 있다: UTF-8 은 ef bb bf, UTF-16LE 는 ff fe. Windows 메모장은 예전에 UTF-8 파일에 이것을 붙였다. rubrapack build 와 lint 는 BOM 이 있든 없든 UTF-8 원본을, 그리고 BOM 이 있는 UTF-16 원본을 읽는다. edit 은 UTF-8 원본만 고친다.

23.7 유니코드 정규화

어떤 문자는 유니코드로 쓰는 방법이 둘 이상이다. é 는 코드 포인트 하나 U+00E9 이거나 - 둘이다: 글자 e U+0065 와, 그 앞 글자 위에 얹히는 결합 양음 부호 U+0301. 둘은 화면에서 완전히 똑같아 보인다:

형태	코드 포인트	UTF-8 바이트
합친 꼴(NFC)	U+00E9	c3 a9
푼 꼴(NFD)	U+0065 U+0301	65 cc 81

가장 큰 예가 한국어다. 현대 한글 음절은 모두 합친 코드 포인트가 있고, 자모로 풀어 쓸 수도 있다:

형태	코드 포인트	UTF-8 바이트
NFC	U+D55C 한	ed 95 9c
NFD	U+1112 ㅎ U+1161 ㅏ U+11AB ㄴ	e1 84 92 e1 85 a1 e1 86 ab

(합친 코드 포인트는 자모에서 계산된다: $0xAC00 + (18 \times 21 + 0) \times 28 + 4 = 0xD55C$. 18, 0, 4 는 유니코드 표에서 ㅎ, ㅏ, ㄴ 의 번호다.)

정규화는 글을 합의된 한 형태로 바꾸는 것이다. NFC("합친 꼴")는 Windows, 웹, 거의 모든 자판이 만드는 형태다. NFD("푼 꼴")는 macOS 파일 시스템이 파일 이름을 저장하는 형태다 - 그래서 Mac 에서 복사한 파일의 이름은 Windows 에서 친 같은 이름과 바이트가 다를 수 있다. Windows 에게 둘은 다른 이름이다: 그러면 패키지가 한.txt 를 두 번 설치하거나, 바로가기가 대상을 놓칠 수 있다.

그래서 rubrapack 에는 파일, 폴더, 바로가기 이름을 NFC 로 바꾸는 build --nfc(튜토리얼 15장)가 있고, lint 는 NFC 가 아닌 글을 경고한다(RP2105, 튜토리얼 18장). rubrapack 의 정규화는 유니코드 17.0 표를 따른다.

23.8 글자가 이상하게 보일 때

보이는 것	흔한 원인
é 대신 Ã©	UTF-8 바이트 c3 a9 를 코드 페이지 1252 로 읽었다
한 대신 韓이나 ?	UTF-8 바이트를 코드 페이지 949 로 읽었거나 그 반대
덤프에서 H.e.l.l.o.	UTF-16 을 바이트로 읽었다
똑같아 보이는데 다른 두 이름	하나는 NFC, 하나는 NFD

23.9 쓰이는 곳

- 튜토리얼 7장: UTF-8, 코드 페이지 65001 로 저장한 글.

- 튜토리얼 15장과 18장: --nfc 와 RP2105.
- 제4부: MSI 데이터베이스의 문자열 풀, 복합 파일의 UTF-16 이름, 요약 정보의 코드 페이지.

24 GUID 와 해시

이 장의 차례

24.1	GUID	102
	판과 변형	102
	글과 바이트	102
24.2	해시	103
	튜토리얼 패키지 속의 해시	103
24.3	해시로 만드는 GUID: <code>guid --from</code>	104
24.4	쓰이는 곳	104

패키지 곳곳에 16~32 바이트짜리 수 두 종류가 나온다: 이름이 절대 겹치지 않게 무언가에 이름을 붙이는 **GUID**, 그리고 파일 내용을 요약해 어떤 변화든 드러나게 하는 **해시**. rubrapack 은 둘을 잇기도 한다: 해시로 부터 GUID 를 만든다.

24.1 GUID

GUID(globally unique identifier, 표준에서는 **UUID** 라 부른다)는 128비트 - 16 바이트 - 수로, 16진수 32 자리를 다섯 묶음으로 나누어, 보통 중괄호 안에 쓴다:

```
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}
8 digits 4 4 4 12 digits
```

Windows 는 어디서도 다른 것과 헷갈리면 안 되는 것에 GUID 로 이름을 붙인다: 제품(제품 코드), 판을 넘어 이어지는 제품 가족(업그레이드 코드), 컴포넌트, 패키지 빌드 하나(패키지 코드), COM 개체의 종류. GUID 를 등록하는 곳은 없다. 아무도 나눠 주지 않는다. 너무 많아서 겹치지 않는 것이다: 무작위 GUID 는 2^{122} 가지, 약 5×10^{36} 개다. 컴퓨터 한 대가 초마다 무작위 GUID 10억 개를 만들어도, 그중 둘이 같을 확률이 절반에 이르려면 약 86년이 걸린다.

판과 변형

128비트가 다 자유롭지는 않다. 글 속의 두 자리가 어떤 GUID 인지 말한다:

```
{98AE4FED-BF0B-4C77-B712-E0649EB47178}    rubrapack guid    (random)
      ^      ^
      |      |
      |      | variant: 8, 9, A or B (the bits 10xx)
      |      | version: 4 = random

{B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}    rubrapack guid --from hello    (derived)
      ^      ^
      |      |
      |      | variant: 9
      |      | version: 8 = made by a method of the maker's own choosing
```

판 4 GUID 는 무작위 비트 122개다: `rubrapack guid` 가 업그레이드 코드용으로 하나 만든다. **판 8** GUID 는 만년 쪽이 정한 방법으로 계산한 122비트다 - rubrapack 은 아래처럼 이름에서 계산한다.

글과 바이트

글로 쓴 GUID 는 32자리를 순서대로 적은 것이다. 이진 구조(복합 파일의 디렉터리, 프로그램 파일) 안에서 Windows 는 앞의 세 묶음을 little-endian 수(2장)로, 뒤의 두 묶음을 바이트 그대로 저장한다:

```
text: B52FE8EF - B68D - 84B5 - 91AF - E6E01BEC2773
bytes: ef e8 2f b5 8d b6 b5 84 91 af e6 e0 1b ec 27 73
      (reversed) (rev.) (rev.) (as written)
```

MSI 표는 GUID 를 대문자, 중괄호 붙은 글로 둔다. 섞인 순서는 제4부에서만 중요하다.

24.2 해시

해시 함수는 얼마든지 많은 데이터를 읽어 짧고 크기가 정해진 수, 해시(또는 다이제스트)를 만든다. 좋은 해시 함수에는 세 성질이 있다:

1. **같은 입력, 같은 해시** - 어느 컴퓨터에서든, 언제든.
2. **조금만 바뀌어도 전부 바뀐다.** 입력의 비트 하나를 바꾸면 해시 비트의 절반쯤이 눈에 띄는 무늬 없이 바뀐다.
3. **한 방향.** 해시로부터 그 해시를 내는 입력을 찾을 수 없다 - 끝없이 입력을 시험해 보는 것 말고는.

튜토리얼의 guide.txt, 여섯 바이트 47 75 69 64 65 0a("Guide" 와 줄 바꿈)와, g 를 소문자로 바꿔 비트 하나만 다른 것(47 = 0100 0111, 67 = 0110 0111)을 보자:

입력	MD5 (16 바이트)
Guide\n	2b 67 39 28 1f 93 72 44 2d f3 0e 78 4b 7d bd c7
guide\n	e5 24 11 d5 57 3e b8 9b 5c 31 26 97 79 50 d3 ef

입력	SHA-256 (32 바이트)
Guide\n	32 74 fc ad 88 6c de 4e 2c a8 6b 11 d3 0f d7 c4 48 58 ea df 1c 43 7a 95 83 f3 1e 78 15 db 1a f6
guide\n	90 c3 90 ec 1d e8 06 bf 94 58 85 cd 0a f5 1e 90 c3 cd 8c da 0d 0f f6 76 05 1a 56 c2 08 48 c9 0f

PowerShell 에서 해 본다: `Get-FileHash -Algorithm MD5 dist\docs\guide.txt`.

흔한 해시 함수:

이름	크기	오늘날의 쓰임
MD5	128비트	변화를 알아차리는 데만. 보안에는 깨졌다(MD5 가 같은 다른 입력을 일부러 만들 수 있다)
SHA-1	160비트	마찬가지. 인증서의 지문(8장)으로는 아직 쓰인다
SHA-256	256비트	서명의 표준이고, rubrapack 이 GUID 를 끌어낼 때 쓴다

튜토리얼 패키지 속의 해시

MsiFileHash: 판 번호 없는 파일마다 Windows Installer 는 MD5 를 보관했다가, 복구나 업그레이드 때 디스크의 파일이 아직 패키지가 설치한 그 파일인지 판단한다. `inspect --files` 가 그것을 16진수로 보인다(튜토리얼 18장). 표 자체는 그것을 4바이트 수 넷으로 저장한다 - 16 바이트를 넷씩, little-endian 부호 있는 수로 읽은 것이다(2장):

```
C:\work\hello> rubrapack inspect hello.msi MsiFileHash
File_ Options HashPart1 HashPart2 HashPart3 HashPart4
...
F_6d1508f52f6615459567 0 674850603 1148359455 2014245677 -943882933

2b 67 39 28 -> 0x2839672B = 674850603
1f 93 72 44 -> 0x4472931F = 1148359455
2d f3 0e 78 -> 0x780EF32D = 2014245677
4b 7d bd c7 -> 0xC7BD7D4B = 3351084363, top bit set, minus 2^32 = -943882933
```

- **MSIX 블록 맵**(튜토리얼 17장): 모든 파일의 64 KiB 블록마다 SHA-256.
- **서명**(튜토리얼 16장): 서명하는 것은 패키지가 아니라 패키지의 해시다.
- **--reproducible**(튜토리얼 15장): 패키지 코드를 내용의 해시에서 끌어낸다.

24.3 해시로 만드는 GUID: `guid --from`

컴포넌트의 GUID 는 판이 바뀌어도 같아야 하지만 - 업그레이드가 그것에 기댄다 - 누구도 수백 개를 적어 두고 싶지는 않다. 그래서 rubrapack 은 하나하나를 끌어낸다: 그것을 가리키는 글(제품과 파일의 자리)을 해시해 GUID 로 바꾼다. 같은 글은 언제나 같은 GUID 를, 다른 글은 다른 GUID 를 낸다.

`rubrapack guid --from <글>` 은 어떤 글이든 그 계산의 결과를 보인다. `hello` 로 한 단계씩:

1. 입력 바이트를 만든다.

부분마다 길이를 4바이트 little-endian 수로 쓰고 그 바이트를 잇는다: 낱말 `guid`, 수 1(방법의 판), 그리고 UTF-8 로 쓴 글.

```
04 00 00 00 67 75 69 64 01 00 00 00 05 00 00 00 68 65 6c 6c 6f
length 4      "guid"      1          length 5      "hello"
```

길이를 적으면 부분이 모호하지 않다: `guid + hello` 가 `guidh + ello` 와 헷갈릴 수 없다.

2. SHA-256 으로 해시한다:

```
b5 2f e8 ef b6 8d f4 b5 d1 af e6 e0 1b ec 27 73 a2 8a 56 80 0f 9f 53 1b cd dc ee 7c f5 c8 59 90
```

3. 앞의 16 바이트를 남긴다:

```
b5 2f e8 ef b6 8d f4 b5 d1 af e6 e0 1b ec 27 73.
```

4. 판과 변형을 표시한다.

바이트 6 의 위 네 비트를 1000(판 8)으로: `f4` 가 `84` 가 된다. 바이트 8 의 위 두 비트를 10(변형)으로: `d1` = `1101 0001` 이 `1001 0001` = `91` 이 된다.

```
b5 2f e8 ef b6 8d 84 b5 91 af e6 e0 1b ec 27 73
      ^^      ^^
```

5. GUID 로 쓴다.

바이트를 순서대로, 대문자로:

```
C:\work\hello> rubrapack guid --from hello
{B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}
```

어느 프로그램이든 이것을 되풀이하면 같은 GUID 를 얻는다. 제4부 결정적인 정체가 rubrapack 이 종류마다 해시하는 글을 정확히 적어 두었다.

24.4 쓰이는 곳

- 튜토리얼 2장: `rubrapack guid` 로 만든 업그레이드 코드.
- 튜토리얼 4장과 15장: 끌어낸 컴포넌트 GUID 와 패키지 코드.
- 튜토리얼 16, 17, 18장: 서명, 블록 맵, 파일 해시.
- 제4부: 결정적인 정체, Authenticode, MSIX.

25 압축

이 장의 차례

25.1	데이터를 줄일 수 있는 까닭	105
25.2	LZ77: "앞에서 베껴 와라"	105
25.3	허프만 부호: 흔한 기호에 짧은 부호	106
25.4	deflate, 비트 하나하나	106
25.5	MSZIP: 캐비닛 속의 deflate	107
25.6	MSIX 속의 deflate	107
25.7	쓰이는 곳	107

MSI 의 파일은 캐비닛에, MSIX 의 파일은 ZIP 압축 파일에 담겨 옮겨지고, 둘 다 보통 압축되어 있다: 더 적은 바이트로 쓰이고, 거기서 원래 것이 정확히 되살아난다. 이 장은 둘이 함께 쓰는 방법인 **deflate** 가 그것을 어떻게 하는지를 실제 예의 비트까지 내려가 보인다.

25.1 데이터를 줄일 수 있는 까닭

손실 없는 압축이 되는 것은 실제 데이터가 무작위가 아니기 때문이다. 두 가지가 되풀이된다:

1. **나열.** 프로그램의 기계어는 명령 무늬를 되풀이하고, 글은 낱말을 되풀이하고, 파란 하늘 그림은 같은 색을 되풀이한다.
2. **기호, 고르지 않게.** 영어 글에서 e 는 z 보다 훨씬 흔하고, 프로그램에서는 바이트 00 이 아주 흔하다.

deflate 는 둘을 차례로 쓴다: LZ77 이 되풀이된 나열을 짧은 참조로 바꾸고, 이어서 **허프만 부호**가 흔한 기호를 더 적은 비트로 쓴다.

어느 쪽도 없는 데이터 - 무작위 수, 또는 이미 압축된 데이터(PNG, JPEG 그림, ZIP 파일, 동영상) - 는 줄일 수 없다. 해 보면 시간이 들고 몇 바이트가 늘어난다.

25.2 LZ77: "앞에서 베껴 와라"

LZ77(Lempel 과 Ziv, 1977년)은 데이터를 처음부터 읽다가, 다음 바이트들이 조금 전에 나온 적이 있으면 대신 **뒤 참조**를 쓴다: "거리만큼 뒤로 가서 길이만큼 베껴라". 나머지는 **글자 그대로**(literal) 한 바이트씩 쓴다.

12 바이트 `abcbcabcbabc` 를 보자. deflate 는 이것을 글자 넷과 참조 하나로 쓴다:

```
literal a
literal b
literal c
literal a
copy: distance 3, length 8
```

베끼는 것은 자기가 만드는 것과 겹쳐도 된다: 3 바이트 뒤로 가면 `bca` 에 닿고, 한 바이트씩 베끼면 방금 쓴 바이트를 계속 읽게 된다:

```
written so far: a b c a
                  ^ start 3 back
copy 8:         a b c a b c a b c a b c
                  \_____ / 8 copied bytes
```

그래서 아무리 긴 반복도 짧은 참조 하나다. deflate 는 거리 32,768 바이트(창)까지, 길이 3 에서 258 까지를 쓴다.

25.3 허프만 부호: 흔한 기호에 짧은 부호

LZ77 을 지나면 데이터는 기호의 줄이 된다: 글자 바이트, 길이, 거리. 보통의 바이트는 무엇이든 8비트를 쓴다. **허프만 부호**는 기호마다 비트 수를 따로 준다 - 잦은 것에는 적게 - 그리고 어떤 부호도 다른 부호의 앞부분이 되지 않게 해서, 읽는 쪽이 언제나 한 부호가 어디서 끝나는지 안다.

기호 넷으로 된 작은 예:

기호	빈도	고정 2비트 부호	허프만 부호
A	50%	00	0
B	25%	01	10
C	12.5%	10	110
D	12.5%	11	111

평균하면 기호 하나에 2비트 대신 $0.5 \times 1 + 0.25 \times 2 + 0.125 \times 3 + 0.125 \times 3 = 1.75$ 비트다. ABAD 는 0 10 0 111 이 되고, 0100111 을 왼쪽부터 읽으면 A, B, A, D 로만 읽힌다.

deflate 에는 부호 표가 두 가지다: 표준에 한 번 정해 둔 **고정** 표(작은 데이터에 좋다)와, 블록마다 그 블록의 기호 수를 세어 만들어 블록 앞에 저장하는 **동적** 표(큰 데이터에 좋다).

25.4 deflate, 비트 하나하나

deflate(RFC 1951)는 블록을 쓴다. 블록은 저마다 세 비트짜리 머리로 시작한다:

비트	뜻
1비트: BFINAL	1 = 마지막 블록
2비트: BTYPE	0 = 저장(바이트 그대로), 1 = 고정 허프만 부호, 2 = 동적 허프만 부호

고정 표의 기호:

기호	뜻	부호 길이	부호
0-143	글자 바이트 0-143	8비트	00110000 + 기호
144-255	글자 바이트 144-255	9비트	110010000 + (기호 - 144)
256	블록 끝	7비트	0000000
257-279	길이(와 추가 비트)	7비트	0000001 ...
280-287	길이(와 추가 비트)	8비트	11000000 ...

길이 뒤에는 5비트 거리 부호가 오고, 어떤 부호는 범위 안의 정확한 값을 고르려고 추가 비트를 몇 개 더 붙인다.

abcbabcabc 를 deflate 로 압축하면 이렇다 - 열두 바이트 대신 일곱 바이트:

```
4b 4c 4a 4e 84 21 00
```

deflate 는 바이트마다 가장 낮은 비트(비트 0, 2장)부터 채운다. 그 순서로 풀어 쓰면 비트의 줄은:

```
byte 4b = 01001011 -> bits in order: 1 1 0 1 0 0 1 0
byte 4c = 01001100 -> bits in order: 0 0 1 1 0 0 1 0
...
```

이고, 이렇게 읽힌다:

비트(순서대로)	뜻
1	BFINAL = 1: 마지막 블록
1 0	BTYPE = 1(두 비트를 낮은 비트부터 수로 읽는다): 고정 부호
10010001	글자 a (0x61 = 97. 00110000 + 97 = 10010001)

비트(순서대로)	뜻
10010010	글자 b
10010011	글자 c
10010001	글자 a
0000110	기호 262: 길이 8
00010	거리 부호 2: 거리 3
0000000	기호 256: 블록 끝

$3 + 4 \times 8 + 7 + 5 + 7 = 54$ 비트이고, 일곱째 바이트의 마지막 두 비트는 쓰지 않는다. (히프만 부호만은 deflate 가 부호의 가장 높은 비트부터 쓴다. BTYPE 이나 추가 비트 같은 수는 가장 낮은 비트부터다. 제4부가 모든 표를 적어 두었다.)

25.5 MSZIP: 캐비닛 속의 deflate

캐비닛(.cab)은 파일 데이터를 32 KiB 이하의 블록으로 나눈다. **MSZIP**에서는 블록마다 두 글자 CK(43 4b) 뒤에 deflate 스트림 하나가 오고, 블록은 앞 블록 안을 참조할 수 있다. 원본의 compress = "mszip:6" 은 9 단계 중 6 의 노력으로 하는 deflate 다: 수가 클수록 일치를 더 오래 찾는다 - 결과는 작아지고 시간은 더 든다 - 풀리는 방식은 같다.

25.6 MSIX 속의 deflate

MSIX 는 ZIP 압축 파일이고, ZIP 은 파일마다 그대로("저장", 방법 0) 또는 deflate 로 압축해(방법 8) 담는다. 튜토리얼의 hello.msix(17장):

파일	크기	패키지 안	
hello.exe	17,920	6,706	deflate: 37%
Registry.dat	8,192	430	deflate: 대부분 0 바이트라 5%
AppxManifest.xml	2,537	900	deflate: 글이라 35%
Assets\Square150x150.png	301	301	저장: PNG 는 이미 압축되어 있다
guide.txt	6	14	deflate: 너무 작아 얻는 것이 없다

마지막 줄이 아주 작은 데이터를 압축하는 비용을 보인다: deflate 블록에는 머리 몇 비트와 끝 부호가 있고, 여섯 바이트에는 되풀이할 것이 없다.

25.7 쓰이는 곳

- 튜토리얼 15장: compress, cab, cab-max-size.
- 튜토리얼 17장: --msix-compress.
- 제4부: 캐비닛, MSZIP, deflate, MSIX.

26 파일 속의 파일

이 장의 차례

26.1	디스크가 파일을 저장하는 방법	108
26.2	MSI 는 파일 속의 파일 시스템이다	108
	작은 스트림: 미니 스트림	109
	이름	109
26.3	MSIX 는 묶음 파일이다	109
26.4	왜 두 설계인가?	110
26.5	쓰이는 곳	110

패키지는 여러 파일을 담은 파일 하나다: .msi 는 표, 요약 정보, 캐비닛을 담고, .msix 는 프로그램 파일, 로고, 매니페스트를 담는다. 파일 안에 파일을 넣는 고전적인 방법은 두 가지 - 작은 **파일 시스템**과 **묶음 파일** (archive) - 이고, 두 패키지 형식이 하나씩 쓴다. 이 장은 튜토리얼의 패키지로 둘을 설명한다.

26.1 디스크가 파일을 저장하는 방법

디스크는 같은 크기의 조각, **섹터**(또는 클러스터)로 나뉜다 - 이를테면 하나에 4096 바이트 - 그리고 0, 1, 2, ... 로 번호가 붙는다. 파일이 한 조각에 다 들어가는 일은 드물고, 받은 섹터들이 나란히 있을 필요도 없다. 그래서 파일 시스템은 데이터 말고 두 가지를 더 둔다:

- **디렉터리**: 파일마다 이름, 크기, 그리고 첫 섹터.
- **할당 표**: 섹터마다 같은 파일의 다음 섹터 번호, 또는 "마지막"이나 "비어 있음" 표시.

파일을 읽으려면 디렉터리에서 첫 섹터를 찾고, 표를 따라 섹터에서 섹터로 - **사슬**을 따라 - "마지막" 표시까지 간다:

```
directory: report.txt    size 10,000    first sector 4

allocation table:
sector: 0    1    2    3    4    5    6    7
next:   -    -    -    -    5    7    free end

report.txt = sector 4 -> sector 5 -> sector 7 -> end    (3 x 4096 bytes, the last one partly used)
```

USB 메모리의 FAT 파일 시스템이 이렇게 동작한다(FAT: file allocation table, 파일 할당 표). 파일을 지우면 섹터를 비었다고 표시할 뿐이고, 파일이 커지면 사슬에 섹터를 더 이을 뿐이다.

26.2 MSI 는 파일 속의 파일 시스템이다

.msi 는 **복합 파일**(Compound File, Microsoft 의 "구조화된 저장소")이다: 위의 방식 그대로를 파일 하나 안에 둔 것이다. 튜토리얼의 첫 hello.msi, 28,672 바이트를 뜯어 보면:

```
offset 0x0000 header          (the 512-byte header, padded to one 4096-byte sector)
offset 0x1000 sector 0         the allocation table (FAT)
offset 0x2000 sector 1         the directory
offset 0x3000 sector 2         the mini FAT (see below)
offset 0x4000 sector 3         the mini stream (see below)
offset 0x5000 sector 4         cab1.cab, first part
offset 0x6000 sector 5         cab1.cab, second part
```

머리가 처음 4096 바이트를 차지하므로 섹터 n 은 바이트 $(n + 1) \times 4096$ 에서 시작한다. 머리(1장)가 나머지가 어디 있는지 말한다: 오프셋 0x30 의 01 00 00 00 - 디렉터리는 섹터 1 에서 시작한다. 섹터 0 의 할당 표를 4 바이트 little-endian 수(2장)로 읽으면:

섹터	값	뜻
0	0xFFFFFFFF	이 섹터는 할당 표 자신을 담는다
1	0xFFFFFFFF	사슬의 끝: 디렉터리는 섹터 하나
2	0xFFFFFFFF	사슬의 끝: 미니 FAT 은 섹터 하나
3	0xFFFFFFFF	사슬의 끝: 미니 스트림은 섹터 하나
4	5	다음은 섹터 5
5	0xFFFFFFFF	사슬의 끝: cab1.cab 은 섹터 4 와 5
6 ...	0xFFFFFFFF	비어 있음(섹터가 더 없다)

안에 든 파일을 **스트림**이라 부른다. 디렉터리에는 스트림마다 128 바이트 항목이 하나씩 있다: UTF-16(3장)으로 쓴 이름, 종류, 첫 섹터, 크기. 첫 항목은 언제나 뿌리(root)다:

```
52 00 6f 00 6f 00 74 00 20 00 45 00 6e 00 74 00 72 00 79 00 00 00 ...
R   o   o   t           E   n   t   r   y   (end)
```

작은 스트림: 미니 스트림

4096 바이트 섹터에 20 바이트짜리 표를 넣으면 공간 대부분이 버려진다. 그래서 4096 바이트(머리의 오프셋 0x38 에 있는 수)보다 작은 스트림은 **미니 스트림**이라는 스트림 하나에 함께 두고, 64 바이트 **미니 섹터**로 잘라, 자기 표인 **미니 FAT** 으로 관리한다. 이 패키지에서는 스트림 열아홉 개 - 4~1,392 바이트짜리 표 열여덟과 332 바이트 요약 정보 - 가 섹터 3 의 3,840 바이트 미니 스트림을 나눠 쓴다. 보통 섹터를 쓸 만큼 큰 것은 6,776 바이트 캐비닛뿐이다. 같은 방식을 한 단계 아래에서 한 번 더 쓴 것이다.

이름

MSI 의 스트림 이름은 이상하게 보인다 - 艱難藤桃淵苑論. 이름 하나에 허용되는 31 글자에 더 많은 글자를 넣으려고 Windows Installer 가 표 이름을 한중일 문자 영역의 글자로 눌러 담기 때문이다. rubrapack inspect hello.msi --streams (튜토리얼 18장)는 그것을 풀어 보인다: File, Component, cab1.cab, ... 이름 하나는 평범하다: \005SummaryInformation, 곧 요약 정보로, 복합 파일을 다루는 어느 도구든 읽을 수 있다.

26.3 MSIX 는 묶음 파일이다

묶음 파일은 파일 시스템보다 단순하다: 파일을 하나씩, 저마다 작은 머리 뒤에 차례로 쓰고, 끝의 목록이 각각 어디 있는지 말한다. 파일을 제자리에서 키울 수는 없지만 패키지는 그럴 일이 없다. MSIX 가 쓰는 ZIP 형식은 이렇게 생겼다:

```
"PK 03 04" local header: name, sizes, method + the file's (compressed) bytes hello.exe
"PK 03 04" local header + bytes guide.txt
... (9 files)
"PK 01 02" central directory: one entry per file, with where its local header is
"PK 06 06" end record: how many files, where the central directory starts
```

튜토리얼의 hello.msix(11,009 바이트)는 50 4b 03 04 - ZIP 을 만든 Phil Katz 의 머리글자 "PK" - 로 시작하고 이름 hello.exe 가 뒤따른다. 중앙 디렉터리는 바이트 10,098 에서 시작해 항목 아홉을 나열한다. 읽는 쪽은 먼저 파일 끝으로 가서 끝 기록을 찾고, 거기서 목록으로 간다. (MSIX 는 4 GiB 넘는 묶음을 위해 만든 ZIP64 형태의 끝 기록 PK 06 06 을 쓴다.)

항목마다 방법(5장)을 적는다: deflate 는 8, 저장은 0 - 여기서는 PNG 로고 셋이 0, 나머지는 모두 8 이다.

26.4 왜 두 설계인가?

	복합 파일(MSI)	ZIP(MSIX)
만든 목적	제자리에서 고치는 문서(1990년대의 Word, Excel)	한 번 써서 여러 번 읽는 파일
스트림 바꾸기	그 사슬을 다시 쓴다	묶음 전체를 다시 쓴다
작은 조각	미니 스트림에서 짜다	조각마다 30 바이트 넘는 머리가 붙는다
한 조각 읽기	사슬을 따라간다	중앙 디렉터리에서 찾는다

Windows Installer 는 1999년에 Office 가 쓰던 저장소 위에 만들어졌고, MSIX 는 2018년에 .docx 파일이 쓰는 포장 표준(OPC) 위에 만들어졌다 - 그것이 ZIP 이다. 둘 다 "파일 속의 파일"이다.

26.5 쓰이는 곳

- 튜토리얼 18장: `inspect --streams, extract`.
- 제4부: 복합 파일, MSI 데이터베이스(스트림 이름), MSIX(ZIP 구조).

27 표, 키, 참조

이 장의 차례

27.1	표, 행, 열	111
27.2	기본 키: 행을 하나로 만드는 것	111
27.3	외래 키: 행을 가리키는 행	112
27.4	내가 주는 키, rubrapack 이 만드는 키	112
27.5	문자열은 한 번만: 문자열 풀	112
27.6	표에 대한 표	112
27.7	IDT 텍스트 형식	113
27.8	쓰이는 곳	113

.msi 안(6장)의 스트림은 대부분 **표**: 패키지는 작은 **데이터베이스**다. Windows Installer 는 내가 적은 단계 별 각본을 따르는 것이 아니라, 표를 읽고 무엇을 할지 스스로 알아낸다. 이 장은 표를 읽는 데 필요한 데이터 베이스 개념 몇 가지를 설명한다.

27.1 표, 행, 열

표는 격자다. **열**마다 이름과 형식이 있고, **행** 하나가 하나의 대상 - 파일 하나, 바로가기 하나, 레지스트리 값 하나 - 이다. 튜토리얼 첫 패키지(튜토리얼 2장)의 File 표를 rubrapack inspect hello.msi File 이 출력한 대로 보면:

```
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 l255 i4 S72 S20 I2 i4
File File
Hello C_185f8db32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 1
```

격자로 펼치면:

File	Component_	FileName	FileSize	Version	Language	Attributes	Sequence
Hello	C_185f8db32271fe25f561	hello.exe	17920	1.2.3.4	1033	512	1

글의 둘째 줄이 열마다 **형식**을 준다:

형식	뜻
s72	72자까지의 문자열
l255	255자까지의 지역화할 수 있는(번역이 바꿀 수 있는) 문자열
i2, i4	2 또는 4 바이트 정수(2장)
대문자 S, L, I	같지만 칸이 비어 있어도("null") 된다

표준 표의 열 형식은 Windows Installer 가 정해 두었고, rubrapack 은 모든 값을 그것과 대조한다(lint, 튜토리얼 18장): i2 열에 100,000 은 들어가지 않는다.

27.2 기본 키: 행을 하나로 만드는 것

셋째 줄 File File 은 표 이름과 **기본 키** - 행마다 값이 다른 열(또는 열들) - 를 적는다. File 표에서는 열 File 이다: Hello 가 이 행의 이름이고 다른 어떤 행도 그 이름을 쓸 수 없다. 그래서 rubrapack 의 ID 는 겹치면 안 된다(튜토리얼 2장): [file.Hello] 에 적은 ID 가 이 키가 된다.

어떤 표는 열 둘을 함께 써야 한다. FeatureComponents 는 어느 컴포넌트가 어느 기능에 속하는지 적는다. 기능 하나에 컴포넌트가 여럿이고 컴포넌트 하나가 여러 기능에 들 수 있으므로, 둘의 짝만이 하나뿐이다:

```
Feature_ Component_
s38 s72
FeatureComponents Feature_ Component_
Main C_185f8db32271fe25f561
```

27.3 외래 키: 행을 가리키는 행

이름이 _ 로 끝나는 열은 다른 표의 행의 키를 담는다 - **외래 키**, 곧 참조다. 참조를 따라가면 표들이 하나의 그림으로 이어진다. 파일에서 시작해 참조를 따라가 보면:

```
File          Hello
Component_ --> Component C_185f8db32271fe25f561
                  ComponentId {B80BEC59-F582-8F10-8EB1-638C6687B899}
                  Directory_ --> Directory INSTALLDIR
                              DefaultDir Hello
                              Directory_Parent --> Directory ProgramFiles64Folder
                                              DefaultDir .
                                              Directory_Parent --> TARGETDIR
KeyPath        --> File          Hello (back to the start)
FeatureComponents Main + C_185f8db32271fe25f561
Feature_       --> Feature      Main
                  Level 1 (installed by default)
```

소리 내어 읽으면: 파일 hello.exe 는 한 컴포넌트에 속하고, 그 컴포넌트는 ProgramFiles64Folder(Program Files) 안의 폴더 Hello 에 설치되며, 기능 Main 이 그 컴포넌트를 설치한다. 컴포넌트의 키 경로는 파일 자신이다: hello.exe 가 있으면 그 컴포넌트는 설치된 것으로 친다(튜토리얼 4장).

없는 행을 가리키는 참조는 Windows Installer 가 멈출 오류다 - rubrapack lint 가 어느 도구로 만든 .msi 에 서든 검사하는 것 중 하나다.

27.4 내가 주는 키, rubrapack 이 만드는 키

표에는 원본의 ID 가 그대로 나온다 - Hello, INSTALLDIR, Main. Windows Installer 에는 필요하지만 원본에 자기 표가 없는 행에는 rubrapack 이 키를 만든다: 파일마다 만드는 컴포넌트는 c_ 와 16진수 20자리, 그룹으로 찾은 파일은 F_ 와 16진수 20자리. 이 숫자는 그 대상의 자리의 해시(4장)에서 오므로 빌드마다 같다.

27.5 문자열은 한 번만: 문자열 풀

표에는 되풀이되는 문자열 - INSTALLDIR, 컴포넌트 키, Hello - 이 가득하고, MSI 는 서로 다른 문자열을 한 번씩만 보관한다. 패키지의 모든 문자열이 **문자열 풀**이라는 목록 하나(스트림 둘, _StringPool 과 _StringData)에 있고, 표의 문자열 칸은 그 목록에서의 문자열 번호만 2 바이트로(아주 큰 패키지에서는 3 바이트로) 담는다. 튜토리얼 첫 패키지는 표 16개 전체에 서로 다른 문자열이 111개다:

```
C:\work\hello> rubrapack inspect hello.msi
code page: 65001
strings: 111
tables: 16
```

풀은 문자열마다 몇 칸이 쓰는지도 센다. 그 바이트는 제4부가 보인다.

27.6 표에 대한 표

데이터베이스는 자기를 설명한다. 두 표가 다른 표들을 나열한다: _Tables 는 표마다 한 행, _Columns 는 모든 표의 열마다 형식과 함께 한 행이다. inspect 가 처음 두 줄에 출력하는 열 이름과 형식을 거기서 찾는다. 순서 표

들 (`InstallExecuteSequence`, `InstallUISequence`, 그리고 관리용·광고용 설치를 위한 셋 더)은 설치의 단계를 순서대로, 단계마다 조건과 함께 나열한다(튜토리얼 9장과 13장) - 절차마져 표다.

27.7 IDT 텍스트 형식

`inspect` 가 출력하는 글은 Windows Installer 자신의 **IDT** 형식이다: 머리 세 줄, 그다음 행마다 한 줄, 열은 탭으로 나눈다. Microsoft 의 Windows SDK 도구들이 표를 이 형식으로 들고 내보내므로, `rubrapack inspect` 의 표를 다른 도구가 내보낸 것과 비교할 수 있다.

27.8 쓰이는 곳

- 튜토리얼 2장과 4장: ID, 컴포넌트, 키 경로.
- 튜토리얼 18장: `inspect` 와 `lint`.
- 제4부: MSI 데이터베이스(문자열 풀과 표 인코딩), 설치하는 표들.

28 키, 인증서, 서명

이 장의 차례

28.1	열쇠 하나 대신 둘	114
28.2	서명하기와 확인하기	114
28.3	인증서: 이것은 누구의 키인가?	115
	이름: CN, O, C	115
28.4	사슬과 신뢰	115
	폐지	115
28.5	타임스탬프	116
28.6	키가 사는 곳: 파일과 토큰	116
	지문	116
28.7	쓰이는 곳	116

튜토리얼 16장은 hello.msi 에 서명한다. 이 장은 그것이 무슨 뜻인지 설명한다: 서명이 파일을 누가 만들었고 아무도 바꾸지 않았음을 어떻게 증명하는지, 인증서가 무엇인지, 그리고 타임스탬프가 왜 인증서가 만료된 뒤에도 서명을 유효하게 두는지.

28.1 열쇠 하나 대신 둘

가장 오래된 비밀 글쓰기는 양쪽에 열쇠 하나를 쓴다: 잠글 수 있는 사람은 열 수도 있다. **공개 키 암호**(1970년대)는 대신 **키 쌍**을 쓴다: 함께 만들어지고 서로 짝이 되는 두 수다.

- **개인 키**는 주인에게만 비밀로 남는다 - 비밀번호로 보호한 파일에, 더 좋게는 키를 절대 내보내지 않는 하드웨어 토큰 안에.
- **공개 키**는 모두에게 준다.

한 키가 한 일은 다른 키만 확인할 수 있다. 서명에서는 이런 뜻이다: 서명을 만드는 것은 개인 키만 할 수 있고, 공개 키가 있는 누구나 그것을 확인할 수 있다. 공개 키를 안다고 개인 키를 찾는 데 도움이 되지는 않는다 - 오늘날의 크기로는 우주의 수명 안에 되지 않는다.

코드 서명에는 두 방법이 쓰인다:

방법	흔한 키 크기	참고
RSA	2048 또는 3072 비트	전통의 방법. rubrapack 의 시험 인증서가 RSA 2048 을 쓴다
ECDSA	256 또는 384 비트(P-256, P-384)	타원 곡선: 같은 강도에 훨씬 작은 키

28.2 서명하기와 확인하기

서명은 파일 전체 위에 만들지 않는다 - 200 MB 패키지라면 느리다 - 파일의 **해시**(4장) 위에 만든다:

```

signing (the publisher):
  file ---hash---> digest ---private key---> signature      (stored in the file)

checking (any user's computer):
  file ---hash---> digest'
  signature ---public key---> digest
  digest' = digest ?  yes: unchanged and signed by that key.  no: changed, or another key.
  
```

파일의 비트 하나만 바뀌어도 해시가 완전히 바뀌므로 어떤 수정도 확인을 통과하지 못한다. 그리고 공개 키가 받아들이는 서명은 개인 키만 만들 수 있으므로, 다른 누구도 바뀐 파일에 게시자의 이름으로 서명할 수 없다.

28.3 인증서: 이것은 누구의 키인가?

서명은 어떤 공개 키로 확인된다 - 그런데 누구의 키인가? 누구든 키 쌍을 만들어 "Example Software" 라고 부를 수 있다. **인증서가 그 답이다**: "이 공개 키는 이 이름의 것이다"라고 적은 작은 문서로, 그 이름을 확인한 **인증 기관(CA)**이 서명한 것이다.

rubrapack 자체 시험용으로 만든 시험 인증서를 `openssl x509 -text` 가 보이는 대로 옮기면:

필드	값
주체(Subject, 누구)	CN=rubrapack test signer, O=rubrapack tests
발급자(Issuer, 누가 보증)	CN=rubrapack test root
유효 기간 시작 / 끝	2026년 9월 30일 / 2036년 9월 27일
일련번호	71AB698C19EACB47D6A00D71A711CF7A0ACDB236
공개 키	RSA, 2048 비트
기본 제약	CA:FALSE - 다른 인증서를 보증할 수 없다
키 용도	디지털 서명
확장 키 용도	코드 서명
서명	sha256WithRSAEncryption, 발급자의 개인 키로 만들

rubrapack 은 서명하기 전에 이것들을 확인한다: 인증서는 코드 서명용이어야 하고, CA 의 것이 아니어야 하며, 오늘 유효해야 한다(튜토리얼 16장).

이름: CN, O, C

주체는 부분들의 목록으로, 부분마다 종류와 값이 있다: CN(일반 이름), O(조직), OU(부서), L(도시), S(주/도), C(나라, 두 글자). 인증서는 이것을 어떤 순서로 저장하고, 프로그램마다 쓰는 순서가 다르다. 이 인증서는 CN 다음에 O 순서로 저장되어 있고, Windows 는 부분들을 마지막 것부터 거꾸로 쓴다:

```
as stored:      CN=rubrapack test signer, O=rubrapack tests
as Windows writes: O=rubrapack tests, CN=rubrapack test signer
```

MSIX 의 publisher 는 정확히 Windows 의 형태여야 한다(튜토리얼 17장) - 그래서 둘이 다르면 rubrapack 이 옮겨 적을 글을 출력해 준다.

28.4 사슬과 신뢰

시험 인증서는 "rubrapack test root" 가 발급했고, 그 루트의 인증서는 자기 자신이 서명했다 - 그것이 **루트**다. 실제 인증서는 **사슬**을 이룬다:

```
your code signing certificate   issued by   -> an intermediate CA certificate
the intermediate certificate     issued by   -> a root CA certificate
the root certificate            issued by   -> itself
```

Windows 는 믿는 루트 인증서의 목록, **신뢰할 수 있는 루트 저장소**를 두고, Microsoft 가 관리한다. 사슬이 그중 하나에서 끝나면 서명을 믿는다. 중간 인증서는 서명 안에 함께 실려 가므로(--cert 가 더한다) Windows 는 누구에게 묻지 않고 사슬을 만들 수 있다.

rubrapack verify --trust ca.pem 은 내가 지정한 루트로, --system-roots 는 컴퓨터 자신의 목록으로 확인한다. 직접 만든 인증서(튜토리얼 16장)는 사슬이 Windows 가 믿는 어디에서도 끝나지 않는다 - 그래서 시험에는 괜찮지만 사용자에게는 쓸모가 없다.

폐지

개인 키를 도둑맞으면 CA 는 그 인증서를 **폐지**한다: 일련번호를 목록(CRL)에 공개하거나 그에 대한 질문(OCSP)에 답한다. Windows 는 네트워크에 닿을 때 이것을 확인한다. rubrapack verify 는 revocation: not-checked 라고 알린다: 스스로 온라인에 나가지 않는다.

28.5 타임스탬프

인증서는 만료된다 - 오늘날 코드 서명 인증서는 길어야 39개월 뒤에. 그 인증서로 서명한 파일들은 어떻게 되나? 아무 도움이 없으면 그날부터 서명은 효력이 없다.

타임스탬프가 이것을 해결한다. 서명할 때 rubrapack 은 서명의 해시를 **타임스탬프 기관(TSA)** - CA 가 운영하는 서버 - 에 보낸다. TSA 는 "이 해시는 이 시각에 있었다"를 자기 인증서로 서명해 돌려주고, 그것이 서명 옆에 파일로 들어간다(RFC 3161). 나중에 Windows 는 확인한다: 서명한 인증서가 타임스탬프가 보이는 시각에 유효했는가? 그렇다면 서명은 인증서가 만료된 한참 뒤에도 계속 유효하다.

```
signed 2026-10-01, certificate valid until 2029-10-01
checked 2031-05-12:
  without a timestamp: certificate expired -> signature not valid
  with a timestamp:   valid on 2026-10-01 (the TSA says so) -> signature valid
```

TSA 는 파일을 보지 않는다. 해시만 본다.

28.6 키가 사는 곳: 파일과 토큰

형태	무엇인가
.pfx / .p12 (PKCS #12)	개인 키, 인증서, 흔히 사슬까지 한 파일에 담고 비밀번호로 보호한 것. Windows 가 내보내는 형태
.pem	글: 이진 형태를 base64 로 바꿔 -----BEGIN CERTIFICATE----- / -----END ...----- (또는 PRIVATE KEY) 줄 사이에 둔 것. Linux 도구들이 쓴다
.der, .cer	같은 것의 이진 형태
Windows 인증서 저장소	Windows 가 보관하는 키(certmgr.msc). 지문으로 쓴다(--key-store)
하드웨어 토큰, 클라우드는 HSM	개인 키가 장치를 떠나지 않는다. 프로그램이 장치에 서명을 부탁한다(PKCS #11, --pkcs11)

2023년부터 CA 는 하드웨어 안의 키에만 코드 서명 인증서를 발급하므로, 배포용으로는 마지막 두 줄을 쓰게 된다.

이진(DER) 인증서 자체도 길이가 앞에 붙은 필드들의 등지다. 시험 인증서는 30 82 03 d8 로 시작한다: 30 은 "묶음이 뒤따른다", 82 는 "그 길이가 다음 2 바이트를 차지한다", 그리고 03 d8 이 그 길이 984 다 - 이번에는 big-endian 이다 (2장). 984 바이트에 이 네 바이트를 더하면 인증서 전체 988 바이트가 된다.

지문

인증서의 **지문**(thumbprint)은 그 이진 형태의 SHA-1 해시(4장)다 - 여기서는 2E AB FE 60 19 0A F6 6C 52 E4 29 31 AF A6 01 BE C0 EF 86 0D. 비밀이 아니다. 인증서 하나를 가리키는 짧은 이름이고, Windows 저장소와 rubrapack keys list 가 인증서를 이것으로 부른다.

28.7 쓰이는 곳

- 튜토리얼 16장: sign, verify, 키, 타임스탬프.
- 튜토리얼 17장: MSIX 게시자.
- 제4부: Authenticode(정확히 무엇을 해시하고 서명하는지, Windows 가 받아들이는 구조), Windows 와 대조하기.

제4부

파일 형식

29 구현하는 사람을 위한 패키지 형식

이 장의 차례

29.1	사실을 읽는 법	118
29.2	표기	119
29.3	흐름을 한 그림으로	119

이 문서들은 Windows 가 받아들이는 Windows Installer 패키지(.msi)와 MSIX 패키지·묶음을 만들려면 프로그램이 무엇을 써야 하는지, 그리고 그것들에 어떻게 서명하는지를 설명한다. 공개 명세와 Windows 자체에 대한 실험으로 썼으므로, rubrapack 의 코드를 읽지 않고도 이것만으로 자기 도구를 만들 수 있다 - 또는 rubrapack 의 코드를 이것과 대조해 볼 수 있다.

문서	다루는 것
복합 파일 (Compound File Binary, CFB)	복합 파일(Compound File Binary): 모든 .msi 의 그릇
복합 파일 속의 MSI 데이터베이스	스트림, 문자열 풀, 시스템 표, 표 인코딩, IDT 내보내기
요약 정보	요약 정보 스트림과 그 코드 페이지 함정
설치·복구·업그레이드·제거가 되는 가장 작은 패키지	설치·업그레이드·복구·제거를 하는 표들: 파일, 레지스트리, 바로가기, 서비스, 글꼴, 대화창, 사용자별 패키지, 순서
프로그램 파일: 머신 형식과 판	프로그램 파일: 머신 형식과 판 정보 리소스
캐비닛, MSZIP, deflate	캐비닛 파일, MSZIP 블록, deflate 인코더
결정적인 정체와 재현 가능한 패키지	결정적인 GUID 와 키, 재현 가능한 빌드
Authenticode 서명	PE 파일과 MSI 패키지의 Authenticode 서명: 다이제스트, Windows 가 받는 CMS 구조, ECDSA, RFC 3161 타임스탬프
출력을 Windows 와 대조하기	Windows 자신의 구성 요소로 결과를 확인하는 방법
MSIX 패키지	MSIX: ZIP 배치, 블록 맵, 매니페스트, 가상 레지스트리와 파일 시스템, 확장, 묶음, 서명
레지스트리 하이브 파일(REGF)	MSIX Registry.dat·User.dat 가 담는 레지스트리 하이브 파일(REGF)

29.1 사실을 읽는 법

모든 서술에는 출처 표시가 붙는다:

- **[spec]** - 공개 명세: [MS-CFB], [MS-OLEPS], [MS-CAB], [MS-MCI], RFC 1951, RFC 1321, RFC 3161, RFC 5652, RFC 9562, PKWARE ZIP APPNOTE, ECMA-376 Part 2(OPC), Authenticode PE 명세, 또는 Microsoft Learn 의 Windows Installer·MSIX 참고 문서.
- **[observed]** - Windows 11 에서 켜진 것: MSI 문서는 25H2(빌드 26200)와 Windows Installer 5.0.10011 에서 msi.dll 로 데이터베이스를 만들어 바이트 단위로 다시 읽고 패키지를 설치해서, MSIX·레지스트리 하이브·서명 문서는 빌드 26100 에서 Windows 자신의 패키징 API, 오프라인 레지스트리 라이브러리, 서명

기와 대조하고 패키지를 설치·실행해서 얻었다. 관찰은 그 빌드들에 대한 사실이다. Microsoft 가 형식을 문서로 내지 않은 곳(MSI 표 인코딩, 레지스트리 하이브, MSIX 서명의 해시 기록은 공개되지 않았다)에서 는 관찰이 유일한 출처다.

여기의 어떤 글이나 코드도 다른 MSI·MSIX·캐비닛·레지스트리 구현에서 오지 않았다.

29.2 표기

- 따로 말하지 않으면 모든 정수는 리틀엔디언이다.
- u16, u32, u64 는 부호 없는 수, i16, i32 는 2의 보수 부호 있는 수다.
- 오프셋은 이름 붙은 구조의 처음부터 센 바이트 수다.
- 16진 바이트열은 파일에 놓인 순서로 쓴다: D0 CF 11 E0.

29.3 흐름을 한 그림으로

```
원본 기술 --> 표(문자열/정수/스트림의 행)
            --> MSI 데이터베이스 인코딩(문자열 풀 + 표마다 스트림 하나)
            --> 캐비닛에 담은 파일(저장 또는 MSZIP) --> 스트림 하나 더
            --> 요약 정보 스트림
            --> 모든 스트림을 복합 파일 하나에 쓴다 = .msi
```

MSIX 패키지는 더 단순하다: 페이로드 파일(각각 독립된 64 KiB 조각으로 deflate), AppxManifest.xml, AppxBlockMap.xml(조각마다 해시), [Content_Types].xml 이 ZIP64 압축 파일 하나에 들고, 레지스트리 값은 Registry.dat/User.dat 하이브에 든다. 묶음은 패키지를 나란히 담고, 서명은 AppxSignature.p7x 를 더한다(MSIX 패키지, 레지스트리 하이브 파일(REGF)).

화살표 하나가 위의 문서 하나다. 어느 것도 만드는 데 Windows 가 필요하지 않다: rubrapack 은 Linux 와 Windows 에서 같은 바이트를 만든다.

30 복합 파일(Compound File Binary, CFB)

이 장의 차례

30.1	섹터	120
30.2	머리(처음 512바이트) [spec]	120
30.3	FAT, DIFAT, 미니 FAT [spec]	121
30.4	디렉터리 [spec]	121
30.5	단순하고 결정적인 작성기	122
30.6	안전하게 읽기	122
30.7	실제 예: 튜토리얼의 hello.msi	122

.msi 파일은 복합 파일("구조적 저장소")이다: 파일 하나 안에 이름 붙은 스트림들이 든 작은 파일 시스템이다. 형식은 [MS-CFB]가 정한다. 이 문서는 MSI 작성기에 필요한 부분과 Windows 자신의 작성기가 하는 일을 다룬다.

30.1 섹터

- 파일은 크기가 고정된 섹터의 연속이다. **3판**은 512바이트 섹터(섹터 시프트 9), **4판**은 4096바이트 섹터(시프트 12)를 쓴다. [spec]
- 머리(header)는 오프셋 0 에 있다. 섹터 번호 N 은 두 판 모두 파일 오프셋 $(N + 1) * \text{sector_size}$ 에서 시작한다. 그래서 4판에서는 머리가 4096바이트 섹터 하나를 통째로 차지하고, 머리 뒤의 바이트는 0 이다. [spec]
- msi.dll 은 크기와 상관없이(20 KB 에서 10 MB 까지 해 보았다) 새 데이터베이스를 모두 **4판**으로, 부판(minor version) 0x003E 로 쓴다. 설치할 때는 두 판 다 받는다. rubrapack 은 기본으로 4판을 쓰고 3판도 쓸 수 있다. [observed]

체인과 표 속의 특별한 섹터 번호: [spec]

값	뜻
FFFFFFFA	가장 큰 보통 섹터 번호
FFFFFFFC	DIFAT 섹터(FAT 안에서)
FFFFFFFD	FAT 섹터(FAT 안에서)
FFFFFFFE	체인의 끝
FFFFFFF	빈 섹터 / 스트림 없음

30.2 머리(처음 512바이트) [spec]

오프셋	크기	필드	쓰는 값
0	8	서명	D0 CF 11 E0 A1 B1 1A E1
8	16	CLSID	0
24	2	부판	003E
26	2	주판	3 또는 4
28	2	바이트 순서	FFFE(바이트 FE FF)
30	2	섹터 시프트	9 또는 12
32	2	미니 섹터 시프트	6(64바이트 미니 섹터)
34	6	예약	0

오프셋	크기	필드	쓰는 값
40	4	디렉터리 섹터 수	3판은 0, 4판은 그 수
44	4	FAT 섹터 수	
48	4	첫 디렉터리 섹터	
52	4	트랜잭션 서명	0
56	4	미니 스트림 경계	4096
60	4	첫 미니 FAT 섹터	없으면 FFFFFFFE
64	4	미니 FAT 섹터 수	
68	4	첫 DIFAT 섹터	없으면 FFFFFFFE
72	4	DIFAT 섹터 수	
76	436	처음 109개 FAT 섹터 번호(DIFAT)	안 쓰는 칸은 FFFFFFFF

30.3 FAT, DIFAT, 미니 FAT [spec]

- FAT 는 섹터마다 u32 하나인 배열로, 그 섹터가 속한 체인의 다음 섹터를 준다. FAT 섹터 자신은 FFFFFFFD 로 표시한다.
- FAT 자신의 섹터 번호는 DIFAT 에 적는다: 처음 109개는 머리에, 나머지는 DIFAT 섹터에. DIFAT 섹터 하나는 $\text{sector_size} / 4 - 1$ 개의 번호를 담고, 마지막 u32 에 다음 DIFAT 섹터(끝이면 FFFFFFFE)를 담는다. DIFAT 섹터는 FAT 에서 FFFFFFFC 로 표시한다.
- 4096바이트 섹터에서는 FAT 섹터 109개가 이미 약 457 MB 를 덮으므로, 실제 4판 MSI 파일에는 DIFAT 섹터가 없다. 512바이트 섹터에서는 약 7 MB 부터 DIFAT 가 생긴다.
- 경계(4096바이트) 보다 작은 스트림은 미니 스트림에 산다. 64바이트 미니 섹터로 나뉘고 미니 FAT(FAT 와 같은 배치)로 이어진다. 미니 스트림 자체는 루트 디렉터리 항목의 보통 스트림으로 저장된다.

FAT 는 자기 섹터와 DIFAT 의 섹터까지 덮어야 한다. 작성기는 수가 더 바뀌지 않을 때까지 되풀이해 셈한다.

30.4 디렉터리 [spec]

디렉터리는 128바이트 항목을 담은 섹터 체인이다. 항목 0 이 루트다.

오프셋	크기	필드
0	64	이름, UTF-16LE, NUL 로 끝남(최대 31자 + NUL)
64	2	NUL 을 포함한 이름 길이(바이트)
66	1	종류: 0 안 씀, 1 저장소, 2 스트림, 5 루트
67	1	색: 0 빨강, 1 검정
68	4	왼쪽 형제(FFFFFFF = 없음)
72	4	오른쪽 형제
76	4	자식(저장소와 루트만)
80	16	CLSID
96	4	상태 비트
100	8	만든 시각
108	8	고친 시각
116	4	시작 섹터(크기에 따라 보통 또는 미니)
120	8	스트림 크기(3판에서는 아래 32비트만 센다)

- 루트 항목의 이름은 Root Entry 이고, 그 시작 섹터와 크기는 미니 스트림을 가리킨다.
- MSI 의 루트 항목은 CLSID {000C1084-0000-0000-C000-000000000046} 을 가지며, 바이트로는 84 10 0C 00 00 00 00 00 C0 00 00 00 00 00 00 46 이다. [spec] [observed]
- 안 쓰는 항목은 링크 필드 셋이 FFFFFFFF 인 것을 빼고 모두 0 이다. [spec]

- 저장소의 지식들은 이름 길이, 그다음 대문자로 바꾼 UTF-16 이름의 비교 순으로 늘어선 레드-블랙 트리를 이룬다. [MS-CFB]는 가장 단순한 올바른 트리도 허용한다: 모든 노드가 검정이고 트리는 그냥 균형 잡힌 이진 탐색 트리. rubrapack 은 그것 - 정렬한 이름 위의 균형 트리, 모든 노드 검정 - 을 만들고, msi.dll 은 그것을 연다. [spec] [observed]
- 시각은 0 이어도 된다. 시각을 0 으로 쓰는 것(그리고 시계에서 오는 다른 입력을 쓰지 않는 것)이 CFB 작성기를 결정적으로 만든다. [spec]

30.5 단순하고 결정적인 작성기

rubrapack 은 파일을 다음 순서로, 모든 구간을 이어서 놓는다:

머리 | FAT 섹터 | DIFAT 섹터 | 디렉터리 | 미니 FAT | 미니 스트림 | 큰 스트림들을 차례로

1. 스트림 크기를 "미니"(< 4096)와 "보통"으로 나눈다.
2. 미니 스트림들은 64바이트 단위로 이어 붙인다. 미니 스트림 길이는 그 합이다.
3. 디렉터리, 미니 FAT, 미니 스트림 섹터 수를 세고, 그다음 FAT 와 DIFAT 섹터 수를 센다(되풀이).
4. 이어진 구간마다 FAT 체인을 채우고, 디렉터리 트리, 미니 FAT, FAT, DIFAT, 머리를 쓴다.

30.6 안전하게 읽기

판독기는 다음을 따라가지 말고 거부해야 한다: 도는 체인(파일의 섹터 수 안에 FFFFFFFE 에 닿지 않는 체인), 파일 밖의 섹터 번호, 스트림 크기가 요구하는 것보다 짧은 체인, 범위 밖의 디렉터리 링크, 한 노드를 두 번 지나는 형제 트리, 상식적인 한도를 넘는 수. 어느 것도 비용이 크지 않고, 그렇지 않으면 조작된 .msi 가 판독기를 멈추게 하거나 죽인다.

30.7 실제 예: 튜토리얼의 hello.msi

튜토리얼의 첫 패키지(2장)를 --reproducible 로 빌드하면 28672 바이트다: 머리 섹터와 4096 바이트 섹터 6 개. 머리는:

오프셋	바이트	필드	값
0x00	d0 cf 11 e0 a1 b1 1a e1	서명	Compound File
0x18	3e 00	부 판	0x003E
0x1A	04 00	주 판	4
0x1C	fe ff	바이트 순서	0xFFFF
0x1E	0c 00	섹터 시프트	12: $2^{12} = 4096$
0x20	06 00	미니 섹터 시프트	6: 64
0x28	01 00 00 00	디렉터리 섹터 수	1
0x2C	01 00 00 00	FAT 섹터 수	1
0x30	01 00 00 00	첫 디렉터리 섹터	1
0x38	00 10 00 00	미니 스트림 경계	4096
0x3C	02 00 00 00	첫 미니 FAT 섹터	2
0x40	01 00 00 00	미니 FAT 섹터 수	1
0x44	fe ff ff ff	첫 DIFAT 섹터	FFFFFFFE
0x4C	00 00 00 00	DIFAT[0]: 첫 FAT 섹터	0

섹터들과 각 섹터의 FAT 값(사슬의 다음 섹터):

섹터	파일 오프셋	FAT 값	다음은
0	0x1000	FFFFFFFD	FAT
1	0x2000	FFFFFFFE	디렉터리

섹터	파일 오프셋	FAT 값	담은 것
2	0x3000	FFFFFFFE	미니 FAT
3	0x4000	FFFFFFFE	미니 스트림
4	0x5000	5	cab1.cab
5	0x6000	FFFFFFFE	cab1.cab

뿌리 디렉터리 항목, 섹터 1 의 처음 128 바이트:

오프셋	바이트	필드	값
0	52 00 6f 00 6f 00 74 00 20 00 45 00 6e 00 74 00 72 00 79 00 ...	이름	Root Entry (UTF-16LE)
64	16 00	이름 길이	22
66	05	종류	5 (root)
67	01	색	1 (black)
76	02 00 00 00	자식	항목 2
80	84 10 0c 00 00 00 00 00 00 c0 00 00 00 00 00 00 46	CLSID	{000C1084-0000-0000-C000-000000000046}
116	03 00 00 00	시작	섹터 3
120	00 0f 00 00 00 00 00 00	크기	3840

뿌리의 크기가 곧 미니 스트림의 크기다: 3840 바이트에 스트림 20개 중 19개가 64 바이트 미니 섹터로 들어 있고, 보통 섹터를 쓸 만큼 큰 스트림은 1개다. 스트림의 이름과 내용은 복합 파일 속의 MSI 데이터베이스가 이어서 보인다.

31 복합 파일 속의 MSI 데이터베이스

이 장의 차례

31.1 스트림	124
스트림 이름	124
31.2 문자열 풀: <code>_StringPool</code> 과 <code>_StringData</code>	124
코드 페이지	125
31.3 시스템 표	125
열 형식 비트(<code>_Columns.Type</code>)	125
31.4 표 스트림	126
31.5 IDT 보관 파일(<code>MsiDatabaseExport</code> 가 쓰는 것)	126
31.6 안전하게 읽기	126
31.7 실제 예: 튜토리얼의 <code>hello.msi</code>	126

Microsoft 는 Windows Installer 표의 뜻은 문서로 내지만(Microsoft Learn, "Database Tables"), 데이터베이스가 복합 파일 안에 어떻게 놓이는지는 내지 않는다. 이 문서에서 배치에 관한 모든 것은 `msi.dll` 로 데이터베이스를 만들고, 스트림을 바이트 단위로 다시 읽고, 이 규칙으로 쓴 데이터베이스가 열리고 내보내지고 설치되는지 확인해서 알아냈다. 따로 표시하지 않으면 [observed] 다.

31.1 스트림

표 하나가 루트 저장소 바로 아래의 스트림 하나다. 보통 패키지에는 하위 저장소가 없다.

스트림 이름

T 라는 표는 U+4840 뒤에 T 를 UTF-16 단위 하나에 두 글자씩 **압축**해 붙인 이름으로 저장한다:

- 다음 64글자 알파벳의 각 글자를 0..63 으로 바꾼다: 0-9 -> 0..9, A-Z -> 10..35, a-z -> 36..61, . -> 62, _ -> 63.
- 알파벳 글자 둘이 이어지면 단위 하나 $0x3800 + \text{first} + (\text{second} \ll 6)$ 가 된다.
- 알파벳 글자 하나(마지막 글자, 또는 뒤에 알파벳 밖의 글자가 오는 글자)는 $0x4800 + \text{value}$ 가 된다.
- 그 밖의 글자는 그대로 쓴다.

압축한 이름(표는 U+4840 점두 포함)은 CFB 이름 한도 31단위 안에 들어야 하므로, 표 이름은 알파벳 글자 60 개까지 쓸 수 있다. 풀 때는 이 단계를 거꾸로 밟는다.

예: `_Columns` -> U+4840 U+3B3F U+43F2 U+4438 U+45B1.

다른 스트림:

스트림	이름
Binary/Icon 칸 (OBJECT 열)	<Table>.<key> 를 같은 방식으로 압축하되 U+4840 점두 없이 , 예 <code>Binary.Logo</code> (복합 키는 . 로 잇는다)
안에 넣은 캐비닛	<code>Media.Cabinet</code> 의 # 뒤 이름(예 <code>cab1.cab</code>), 압축, 점두 없음
요약 정보	<code>\x05SummaryInformation</code> - 압축하지 않는다(요약 정보 참고)

31.2 문자열 풀: `_StringPool` 과 `_StringData`

모든 표의 모든 문자열은 공유 풀에 한 번씩만 살고, 표의 칸은 문자열 id 를 담는다.

`_StringPool`:

u32 header 아래 31비트: 데이터베이스 코드 페이지; 비트 31: 문자열 참조가 3바이트
그다음 문자열 id 1, 2, 3, ... 마다(id 0 은 "문자열 없음"이고 저장하지 않는다):
u16 length _StringData 속 바이트 길이
u16 refs 비트 0-14: 참조 수; 비트 15: 문자열에 0x80 이상인 바이트가 있을 때만 켜짐

- 긴 문자열(64 KiB 이상의 바이트): 항목은 length = 0, refs 는 위와 같고(0 아님) 그 뒤에 전체 바이트 길이를 담은 u32 가 하나 더 온다. 이 둘이 여전히 id 하나를 기술한다. 본 예: 한 번 쓰인 70002바이트 문자열 - > 0000 8001 00011172.
- length = 0 이고 refs = 0 인 항목은 빈 id 다.
- 참조 수 = 그 문자열을 담은 칸의 수, 더하기 표마다 _Tables 행 몫 1, 열마다 _Columns 의 표 이름 몫 1 과 열 이름 몫 1. (msi.dll 이 만든 모든 데이터베이스를 다시 세어 확인했다.)
- _StringData 는 모든 문자열의 바이트를 id 순서대로 이어 붙인 것이고, 데이터베이스 코드 페이지로 되어 있다. 길이는 글자 수가 아니라 바이트 수다.
- 빈 문자열은 저장하지 않는다: 칸에 "" 를 쓰면 null 이 저장된다.
- 서로 다른 문자열이 65535개를 넘으면 머리의 비트 31 이 켜지고, 모든 표의 모든 문자열 참조가 3바이트(아래 16비트, 그다음 위 바이트)가 된다.

코드 페이지

데이터베이스 코드 페이지는 풀 머리의 아래 비트다: 한 번도 정하지 않았으면 0, 아니면 _ForceCodepage 가 쳐오기로 정한 값. 65001(UTF-8)은 요즘 Windows 에서 모든 표에 통한다: 한국어, 일본어, 보조 평면(예 U+20000, U+1F600)의 파일·폴더 이름, 레지스트리 키와 값, 바로가기 이름, 제품 이름이 모두 정확한 UTF-16 글자로 설치되고 대화창에도 보인다 - ANSI 코드 페이지가 한국어를 담지 못하는 시스템 로캘(en-US, 1252)에서도 그렇다. 그래서 따로 유니코드 설치 프로그램이 필요 없다. UTF-16 코드 페이지(1200)는 여기서 쓸 수 없다: 풀은 한 바이트 단위의 문자열을 담는다. [observed] Windows Installer 는 이름을 정규화하지 않는다: 분해형 한글(U+1100 U+1161)로 이름 붙인 폴더는 바로 그 코드 단위로 설치되고, U+AC00 으로 이름 붙인 폴더를 대신하지 않고 그 옆에 생긴다. [observed]

31.3 시스템 표

표	열	키
_Tables	Name(문자열)	Name
_Columns	Table(문자열), Number(i2, 1부터), Name(문자열), Type(i2)	Table, Number

_Tables 는 자기 자신과 _Columns 를 신지 않는다. 행이 없는 표는 스트림이 아예 없고 _Tables·_Columns 행만 있다. _Validation 은 있다면 보통 표다.

열 형식 비트(_Columns.Type)

비트	뜻
0x00FF	너비: 문자열 최대 길이(0 = 제한 없음), 또는 정수의 2 / 4
0x0100	유효
0x0200	지역화 가능
0x0400	비이진(글자면 켜짐, OBJECT 면 꺼짐)
0x0800	문자열(글자와 OBJECT 면 켜짐)
0x1000	null 허용
0x2000	기본 키의 일부

흔한 SQL 열 형식에 msi.dll 이 쓰는 값:

SQL	Type
CHAR(72) NOT NULL(키)	0x2D48

SQL	Type
CHAR(72)(null 허용)	0x1D48
CHAR(255) NOT NULL LOCALIZABLE	0x0FFF
LONGCHAR NOT NULL LOCALIZABLE	0x0F00
SHORT NOT NULL	0x0502
SHORT(null 허용)	0x1502
SHORT NOT NULL(키)	0x2502
LONG NOT NULL	0x0104
OBJECT NOT NULL	0x0900

정수 열에는 비이진 비트가 없다.

31.4 표 스트림

- 표 스트림은 **열 우선**이다: 모든 행의 1열 칸, 그다음 모든 행의 2열 칸, 이런 식이다. 행 수는 스트림 크기를 행 너비로 나눈 것이다.
- 칸 너비: 문자열 참조 2바이트(긴 참조면 3), 16비트 정수 2바이트, 32비트 정수 4바이트, OBJECT 2바이트.
- 정수는 부호 비트를 뒤집어 저장한다: $\text{stored} = \text{value} \wedge 0x8000$ (16비트) 또는 $\text{value} \wedge 0x80000000$ (32비트). 저장된 0 은 null 이다. 그래서 -32768 과 -2147483648 은 저장할 수 없다.
- OBJECT 칸은 스트림이 있으면 1, null 이면 0 을 담는다.
- 행은 기본 키 칸의 저장된 수 - 문자열 id, 뒤집은 정수 - 로, 키 열 순서대로 정렬한다. Windows Installer 는 이 순서에 기댄다.

문자열 id 를 문자열의 바이트 순서로 매기는 작성기는 행을 문자열 값 순으로 저절로 정렬하게 되고, 이것은 IDT 내보내기가 쓰는 순서이기도 하다.

31.5 IDT 보관 파일(MsiDatabaseExport 가 쓰는 것)

기준 출력으로 쓸모 있다: 바이트까지 같은 IDT 파일을 내보내는 판독기는 Windows 와 같은 방식으로 데이터베이스를 푸는 것이다.

1줄: TAB 으로 나눈 열 이름
 2줄: 열 형식: s/S(문자열), l/L(지역화 문자열), i/I(정수), v/V(이진),
 대문자 = null 허용, 뒤에 너비(0 = 제한 없음; 정수는 2 또는 4)
 3줄: [<코드 페이지> TAB] <표 이름> TAB <키 열> [TAB <키 열> ...]
 행 : TAB 으로 나눈 칸, 줄 끝은 CR LF

- 3줄의 코드 페이지 접두는 표에 ASCII 가 아닌 문자열이 있을 때만 나온다.
- 행은 기본 키 값(문자열은 바이트, 정수는 부호 있게)으로 정렬한다 - 저장 순서가 아니다.
- null 은 빈 필드로 찍는다. 이진 칸은 <key>.ibd 를 찍고, 바이트는 <Table>/<key>.ibd 로 간다.
- 값 속의 TAB, CR, LF 는 0x10, 0x11, 0x19 로 쓴다. [spec]
- _ForceCodepage.idt 는 빈 줄 둘 다음에 <코드 페이지> TAB _ForceCodepage 다.
- _SummaryInformation.idt 는 머리 줄 PropertyId TAB Value, i2 TAB 1255, _SummaryInformation TAB PropertyId 와 속성마다 한 행을 가진다. 날짜는 내보내는 컴퓨터의 지역 시각으로 yyyy/mm/dd hh:mm:ss 로 찍는다.

31.6 안전하게 읽기

풀 길이의 합이 _StringData 크기와 정확히 같은지, 표의 모든 문자열 참조가 쓰이는 id 를 가리키는지, 표 스트림 크기가 행 너비의 배수인지, _Columns 가 표마다 열을 1..n 으로 빈틈없이 번호 매기는지 확인한다.

31.7 실제 예: 튜토리얼의 hello.msi

튜토리얼 첫 패키지(복합 파일(Compound File Binary, CFB))의 표 File 은 이름이 U+4840 U+430F U+422F 인 스트림이다:

0x4840 (table)

F i: 0x3800 + 15 + (44 << 6) = 0x430F

l e: 0x3800 + 47 + (40 << 6) = 0x422F

_StringPool 은 e9 fd 00 00 로 시작한다: 코드 페이지 65001(UTF-8), 비트 31 은 꺼짐(2 바이트 문자열 참조).
그 뒤는 문자열 번호마다 u16 둘이다. 처음 번호들:

번호	바이트	길이	참조	문자열
1	09 00 01 00	9	1	#cab1.cab
2	01 00 01 00	1	1	.
3	01 00 01 00	1	1	1
4	05 00 03 00	5	3	1.0.0
5	07 00 01 00	7	1	1.2.3.4
6	04 00 02 00	4	2	1033

모두 111개이고, 그 바이트를 이어 붙인 것이 _StringData 의 1381 바이트다.

_Columns 가 표를 설명한다 - File 의 행들, 형식은 위의 비트 표로 풀었다:

#	열	Type	뜻
1	File	0x2D48	key, string, non-binary, valid, width 72
2	Component_	0x0D48	string, non-binary, valid, width 72
3	FileName	0x0FFF	string, non-binary, localizable, valid, width 255
4	FileSize	0x0104	valid, width 4
5	Version	0x1D48	nullable, string, non-binary, valid, width 72
6	Language	0x1D14	nullable, string, non-binary, valid, width 20
7	Attributes	0x1502	nullable, non-binary, valid, width 2
8	Sequence	0x0104	valid, width 4

File 스트림 자체는 20 바이트다: 행 하나를 열 순서로 저장했다:

32 00 0e 00 6c 00 00 46 00 80 05 00 06 00 00 82 01 00 00 80

열	저장값
File	32 00 -> id 50 Hello
Component_	0e 00 -> id 14 C_185f8db32271fe25f561
FileName	6c 00 -> id 108 hello.exe
FileSize	00 46 00 80 -> 0x80004600 ^ 0x80000000 = 17920
Version	05 00 -> id 5 1.2.3.4
Language	06 00 -> id 6 1033
Attributes	00 82 -> 0x8200 ^ 0x8000 = 512
Sequence	01 00 00 80 -> 0x80000001 ^ 0x80000000 = 1

문자열 칸은 문자열 번호이고, 정수는 맨 위 비트를 뒤집어 저장한다. 그래서 17920(0x00004600)은 0x80004600 으로 저장되고, 0 은 null 을 뜻하도록 비워 둔다.

32 요약 정보

이 장의 차례

32.1	배치 [spec] [observed]	128
32.2	설치 패키지가 정하는 속성	128
32.3	코드 페이지 함정 - 요약 문자열은 ASCII 로 [observed]	129
32.4	실제 예: 튜토리얼의 hello.msi	129

모든 패키지에는 \x05SummaryInformation 스트림이 있다(첫 글자는 U+0005, 이름은 압축하지 않는다). [MS-OLEPS]가 정한 OLE 속성 집합이고, Microsoft Learn("Summary Information Stream Property Set")이 설치 패키지가 어떤 속성을 쓰는지 알려 준다.

32.1 배치 [spec] [observed]

```

u16 바이트 순서      FFFE
u16 판               0 (msi.dll 은 0 을 쓴다)
u32 OS 필드         msi.dll 은 0x00020206 을 쓴다
16 CLSID             0
u32 구역 수         1
16 FMTID             F29F85E0-4FF9-1068-AB91-08002B27B3D9, 저장은 E0 85 9F F2 F9 4F 68 10 AB 91 08 00 2B 27 B3 D9
u32 구역 오프셋      48

```

구역:

u32 구역 크기

u32 속성 수

수만큼 (u32 속성 id, u32 구역 처음부터의 오프셋)

값들, 각각 4바이트 배수로 채움:

VT_I2 (2) u32 형식, i16 값, 채움 2바이트

VT_I4 (3) u32 형식, i32 값

VT_LPSTR (30) u32 형식, NUL 포함 u32 바이트 수, 바이트, NUL, 채움

VT_FILETIME (64) u32 형식, 1601-01-01 UTC 부터의 100ns 간격 수 u64

32.2 설치 패키지가 정하는 속성

Id	이름	형식	값
2	Title	LPSTR	예 Installation Database
3	Subject	LPSTR	제품 이름
4	Author	LPSTR	제조사
5	Keywords	LPSTR	예 Installer
7	Template	LPSTR	<플랫폼>;<언어 id>: x64;1033, Arm64;1042, Intel;1033
9	Revision Number	LPSTR	패키지 코드: GUID, 서로 다른 패키지 파일마다 새로
12, 13	Create / Last Save time	FILETIME	선택
14	Page Count	I4	최소 설치 엔진 판: x64 와 x86 은 200, Arm64 는 500
15	Word Count	I4	비트 1 = 파일이 (캐비닛에) 압축되어 있음; 보통 패키지는 2
18	Creating Application	LPSTR	아무것

32.3 코드 페이지 함정 - 요약 문자열은 ASCII 로 [observed]

속성 1(CodePage, VT_I2)은 LPSTR 바이트의 인코딩을 말한다. Windows 11 25H2 에서:

- **65001:** msi.dll 은 문자열을 UTF-8 로 제대로 쓰지만, 코드 페이지를 부호 있는 값 -535 로 다시 읽어 모 든 문자열 변환에 실패하고(설치 로그: "Failed to retrieve # 9 summary info property. GetLastError returned 87, the code page is -535") 오류 1620 으로 패키지를 거부한다.
- **1200(UTF-16):** 똑같이 거부한다(로그: "... the code page is 1200"). [MS-OLEPS]대로 완전한 UTF-16 문자열과 바이트 수를 써도 그렇다.
- **코드 페이지 속성 없음, ASCII 문자열:** 설치된다.

그러니 속성 1 은 빼고 요약 문자열은 모두 ASCII 로 쓴다. 지역화한 제품 이름은 데이터베이스 코드 페이지를 쓰는 Property 표(ProductName)에 둔다 - 거기서는 65001 이 통한다(복합 파일 속의 MSI 데이터베이스 참고).

32.4 실제 예: 튜토리얼의 hello.msi

튜토리얼의 첫 패키지: 스트림은 332 바이트다. 머리는:

오프셋	바이트	필드	값
0x00	fe ff	바이트 순서	0xFFFFE
0x04	06 02 02 00	OS	0x00020206
0x18	01 00 00 00	구역 수	1
0x1C	e0 85 9f f2 ...	FMTID	F29F85E0-...
0x2C	30 00 00 00	구역 오프셋	48
0x30	1c 01 00 00	구역 크기	284
0x34	09 00 00 00	속성 수	9

그 뒤에 (속성 번호, 오프셋) 짝 9개와 값들:

속성	오프셋	종류	원래 바이트	값
2	0x80	VT_LPSTR	1e 00 00 00 16 00 00 00 49 6e 73 74 61 6c 6c 61 ...	Installation Database
3	0xA0	VT_LPSTR	1e 00 00 00 06 00 00 00 48 65 6c 6c 6f 00	Hello
4	0xB0	VT_LPSTR	1e 00 00 00 11 00 00 00 45 78 61 6d 70 6c 65 20 ...	Example Software
5	0xCC	VT_LPSTR	1e 00 00 00 0a 00 00 00 49 6e 73 74 61 6c 6c 65 ...	Installer
7	0xE0	VT_LPSTR	1e 00 00 00 09 00 00 00 78 36 34 3b 31 30 33 33 ...	x64;1033
9	0xF4	VT_LPSTR	1e 00 00 00 27 00 00 00 7b 45 32 32 43 44 37 45 ...	{E22CD7E9-3AF1-82AB-811D-BCF61D7A0757}
14	0x124	VT_I4	03 00 00 00 c8 00 00 00	200
15	0x12C	VT_I4	03 00 00 00 02 00 00 00	2
18	0x134	VT_LPSTR	1e 00 00 00 10 00 00 00 72 75 62 72 61 70 61 63 ...	rubrapack 0.7.0

문자열 값은 형식(30), NUL 을 포함한 바이트 수, 바이트, NUL, 그리고 4 의 배수까지 채움이다. 속성 9 인 패키지 코드는 --reproducible 이면 내용에서 끌어내고, 없으면 빌드마다 새것이다.

33 설치·복구·업그레이드·제거가 되는 가장 작은 패키지

이 장의 차례

33.1	표와 열	130
33.2	Property	131
33.3	Directory	131
33.4	구성 요소, 기능, 파일	131
33.5	빈 폴더	131
33.6	Media 와 캐비닛	132
33.7	메이저 업그레이드와 다운그레이드 거부	132
33.8	설치한 프로그램으로 등록·해제하기	132
33.9	레지스트리 값	132
33.10	바로가기	133
33.11	파일 지우기와 복제	133
33.12	환경 변수	133
33.13	INI 파일	133
33.14	검색과 설치 조건	134
33.15	서비스, 글꼴, 권한	134
33.16	캐비닛, 관리 이미지, 광고	134
33.17	사용자별 패키지와 검용 패키지	134
33.18	사용 중인 파일	134
33.19	순서	135
33.20	대화창	135
33.21	이 조리법이 아직 다루지 않는 것	136

표의 뜻은 Microsoft Learn("Database Tables", "Standard Actions", "Suggested InstallExecuteSequence")에 있다. 이 문서는 시험을 거친 조리법이다: 정확히 이대로 지은 패키지는 Windows 11 에서 컴퓨터 전체로 설치되고, 복구를 견디고, 옛 판을 업그레이드하고, 다운그레이드를 거부하고, 깨끗이 제거된다. [observed]

33.1 표와 열

형식은 복합 파일 속의 MSI 데이터베이스의 표기를 쓴다(s72 = CHAR(72), 대문자 = null 허용, l = 지역화 가능, i2/i4 정수). 키는 굵게 쓴다.

표	열
Property	Property s72, Value l0
Directory	Directory s72, Directory_Parent S72, DefaultDir l255
Component	Component s72, ComponentId S38, Directory_ s72, Attributes i2, Condition S255, KeyPath S72
Feature	Feature s38, Feature_Parent S38, Title L64, Description L255, Display I2, Level i2, Directory_ S72, Attributes i2
FeatureComponent	Feature_ s38, Component_ s72
File	File s72, Component_ s72, FileName l255, FileSize i4, Version S72, Language S20, Attributes I2, Sequence i4
MsiFileHash	File_ s72, Options i2, HashPart1..HashPart4 i4

표	열
CreateFolder	Directory_ s72, Component_ s72
Media	DiskId i2, LastSequence i4, DiskPrompt L64, Cabinet S255, VolumeLabel S32, Source S72
Upgrade	UpgradeCode s38, VersionMin S20, VersionMax S20, Language S255, Attributes i4, Remove S255, ActionProperty s72
CustomAction	Action s72, Type i2, Source S72, Target S255
InstallExecuteSequence	Action s72, Condition S255, Sequence I2
InstallUISequence	

File.Sequence 와 Media.LastSequence 를 32비트 정수로 두면 패키지에 파일을 32767개 넘게 담을 수 있다.

33.2 Property

ProductCode, ProductName, ProductVersion(a.b.c, a 와 b 는 255 까지, c 는 65535 까지; 넷째 부분도 되지만 판을 비교할 때는 무시된다), Manufacturer, ProductLanguage(1033, 1042, ...), UpgradeCode, ALLUSERS=1(컴퓨터 전체), REBOOT=ReallySuppress, MSIRESTARTMANAGERCONTROL=Disable(아래 "사용 중인 파일" 참고), 그리고 아래 업그레이드 속성 둘을 나열한 SecureCustomProperties.

33.3 Directory

- 부모가 없고 DefaultDir = SourceDir 인 TARGETDIR.
- 쓰는 표준 폴더(ProgramFiles64Folder, ProgramFilesFolder, ...)는 저마다 DefaultDir = . 인 TARGETDIR 의 자식. 이 이름들은 Windows 가 스스로 푼다. x64·Arm64 패키지는 ...64Folder 이름을, x86 은 64 없는 이름을 쓴다.
- 그 아래의 자기 폴더는 DefaultDir = SHORT!Long name. 긴 이름은 어떤 유니코드 글이든 된다(코드 페이지 65001 일 때). 짧은 이름은 올바른 8.3 ASCII 이름이어야 하고, 같은 부모의 모든 파일·폴더 사이에서 달라야 한다. 긴 이름이 이미 올바른 8.3 이름이면 그것만 써도 된다.

33.4 구성 요소, 기능, 파일

- 파일 하나에 구성 요소 하나, 그 파일이 키 경로다. ComponentId 는 제품의 수명 동안 같은 자리의 같은 자원에 대해 변하지 않아야 하는 GUID 다(결정적인 정체와 재현 가능한 패키지 참고). Attributes 는 x64·Arm64 패키지면 256(64비트), x86 이면 0.
- Level 1 인 기능. Display 0 은 그것을 숨긴다. 모든 구성 요소를 FeatureComponents 에 짝짓는다. 자식 기능은 Feature_Parent 에 부모를 적는다. Level 이 INSTALLLEVEL(패키지가 정하지 않으면 1)보다 높은 기능은 기본으로 설치하지 않는다. 뒤에 MsiConfigureFeature 로 더할 수 있고, 그러면 그 파일들이 들어온다. [observed]
- File.FileName 은 DefaultDir 처럼 SHORT!Long 이다. Attributes 512 는 파일을 필수(vital)로 표시한다. Sequence 번호 1..n 은 캐비닛 속 파일의 순서다.
- 판 정보 리소스가 없는 파일에는 **MsiFileHash** 행이 필요하다: 파일의 MD5 를 리틀엔디언 32비트 정수 넷으로 읽은 것(부호 있는 값으로 저장). 설치된 파일은 MsiGetFileHash 로 해시하면 정확히 이 값이 나온다. 판 정보 리소스가 있는 파일은 대신 File.Version 에 판을 적는다(해시는 필요 없다).

33.5 빈 폴더

파일 없이 있어야 하는 폴더에는 제 Directory 행, Directory_ 가 그 폴더이고 KeyPath 가 null 인 구성 요소(폴더 자체가 키 경로), 둘을 가리키는 CreateFolder 행, FeatureComponents 행을 준다. CreateFolders 가 설치할 때 만들고, RemoveFolders 가 제거할 때 비어 있으면 지운다. [observed] 구성 요소 속성 16(영구)이면 다른 것은 다 지워져도 폴더는 남는다. [observed]

33.6 Media 와 캐비닛

행 하나: DiskId 1, LastSequence n, Cabinet #cab1.cab - # 는 "이 패키지 속 cab1.cab 이라는 스트림"이라는 뜻이다. 캐비닛은 파일을 File 키 이름으로 Sequence 순서대로 담는다 (캐비닛, MSZIP, deflate). 요약의 Word Count 2 는 파일이 캐비닛에 압축되어 있다는 뜻이다.

33.7 메이저 업그레이드와 다운그레이드 거부

패키지의 UpgradeCode 를 가진 Upgrade 행 둘:

VersionMin	VersionMax	Attributes	ActionProperty	뜻
이 판	-	0x102(찾기만, 최솟값 포함)	NEWER_FOUND	같거나 새로운 판이 설치됨
-	이 판	0x001(기능 옮기기)	OLDER_FOUND	옛 판이 설치됨: 지운다

Upgrade 표는 UpgradeCode, 판, 언어로만 맞춰 보고 아키텍처는 보지 않는다. x64 빌드와 x86 빌드가 UpgradeCode 를 같이 쓰면 하나를 설치할 때 다른 것이 "옛 판"으로 지워진다. 둘을 나란히 설치할 수 있어야 하면 아키텍처마다 제 UpgradeCode 를 준다(확인함: 코드가 다른 두 아키텍처는 서로 상관없이 설치·제거된다). [observed]

NEWER_FOUND 를 조건으로 한 오류 사용자 지정 동작(형식 19, Target = 메시지; 서식 문자열이라 [ProductName] 이 된다)은 같거나 새로운 판이 있을 때 오류 1603 으로 설치를 멈춘다. 판마다 새 ProductCode 가 필요하다(그리고 패키지 파일마다 새 패키지 코드).

33.8 설치한 프로그램으로 등록·해제하기

어떤 제품(입력기, 셀 확장)은 표의 행이 아니라 자기 프로그램으로 스스로를 등록해야 한다. 설치·복구·업그레이드·제거를 원자적으로 지키는 방법은 그 프로그램의 구성 요소 c 의 동작 상태(\$C = 이번 설치가 그것에 하는 일, ?C = 그 전의 상태; 2 없음, 3 로컬)를 쓰는 것이다 [spec]. 형식 18 동작(Source = 그 프로그램의 File 키, Target = 인수)을 모두 지연(0x400), 가장하지 않음(0x800)으로 둔다:

동작	형식	자리	조건	하는 일
...UndoRollback	18+0x100+0x400+0x800+0x40	RemoveFiles 앞	\$C=2 AND ?C=3	등록(다음 행의 되돌림)
...Undo	18+0x400+0x800	그 뒤, RemoveFiles 앞	\$C=2 AND ?C=3	해제
...DoRollback	18+0x100+0x400+0x800+0x40	InstallFiles 뒤	\$C>2 AND ?C<3	해제
...RedoRollback	같음	그 뒤	\$C>2 AND ?C=3	등록
...Do	18+0x400+0x800	둘 다 뒤	\$C>2	등록

되돌림 동작은 그것이 되돌리는 동작보다 앞에 놓아야 한다: 되돌림은 스크립트를 거꾸로 실행하고, 이미 스크립트에 든 되돌림 동작만 실행한다. 되돌림 동작은 종료 코드를 무시하고(0x40) 앞으로 가는 동작은 그러지 않으므로, 등록이나 해제가 실패하면 설치가 실패하고 되돌려진다. Windows 11 에서 본 것: 복구는 ...Do 를 다시 실행한다. 해제가 실패하면 제거가 되돌려지고 (아직 있는) 프로그램이 다시 등록된다. 업그레이드가 실패하면 새 패키지의 ...DoRollback 이 실행되고, 옛 판의 파일이 돌아오고, 옛 패키지 자신의 ...UndoRollback 이 옛 프로그램을 다시 등록한다. [observed]

InstallInitialize 와 RemoveExistingProducts 사이에는 스크립트에 쓰는 것이 아무것도 있어서는 안 된다: 거기 지연 동작이나 되돌림 동작이 하나라도 있으면 모든 업그레이드가 오류 2613("RemoveExistingProducts action sequenced incorrectly")으로 멈춘다. [observed]

33.9 레지스트리 값

Registry(Registry s72, Root i2, Key l255, Name L255, Value L0, Component_ s72) 는 WriteRegistryValues(5000)가 쓰고 제거할 때 되돌린다. RemoveRegistry(Value 없는 같은 열)는 RemoveRegistryValues(2600)가 처리한다. Root 0 = HKCR, 1 = HKCU, 2 = HKLM. 값의 첫 글자가 형식을 정한다: #x 이진(16진), %% 확장 문자열, # 뒤에 숫자는 DWORD(#4294967295 까지), 어디든 [~] 가 있으면 다중 문

자열([~]a[~]b[~]). # 으로 시작하는 보통 문자열은 ## 로 쓴다. RemoveRegistry 의 Name - 는 키 전체를 지운다. 설치가 실패하면 덮어쓴 값과 지운 키가 되살아난다. [observed]

키 경로가 레지스트리 값인 구성 요소는 속성 4 와 KeyPath = 그 Registry 행을 가진다. 구성 요소의 64비트 속성(256)이 레지스트리 보기를 고른다: 그것이 없으면 64비트 패키지 32비트 보기(WOW6432Node)에 쓴다. 64비트 패키지 속 32비트 구성 요소는 허용되고, 32비트 패키지 속 64비트 구성 요소는 허용되지 않는다.

REG_QWORD 는 Registry 표로 나타낼 수 없다. rubrapack 은 그것을 Binary 표의 DLL 사용자 지정 동작(형식 1)으로 쓴다: 즉시 동작이 구성 요소 동작 상태와 현재 값을 읽어 목록 둘을 CustomActionData 로 넘긴다 - 하나는 쓰거나 지우는 지연 동작에게, 하나는 원래 있던 것을 되살리는 그 되돌림 짝에게. DLL 의 비트 수는 패키지와 맞아야 한다(엔진은 그것을 패키지 아키텍처의 사용자 지정 동작 서버에 싣는다). [observed: x64 와 x86]

파일 형식과 URL 스킴은 Extension/Verb/ProgId 표(처음 쓸 때 광고와 복구를 끌고 온다)가 아니라 프로그램의 구성 요소에 든 Root 0(HKCR) 아래의 보통 Registry 행이다: .ext(기본값) = ProgId; ProgId(기본값) = 그 설명, ProgId\DefaultIcon = [#File], 0, ProgId\shell\open\command = "[#File]" "%1". 스킴은 scheme(기본값) = URL:<설명>, URL Protocol = 빈 값(Name 이 있고 Value 가 null 이면 빈 문자열을 쓴다), 그리고 같은 DefaultIcon 과 shell\open\command. Root 0 은 설치 방식을 따른다: 컴퓨터 전체면 HKLM\Software\Classes, 사용자별이면 HKCU\Software\Classes. 그러면 이 프로그램만 맡는 형식의 파일을 열 때 곧바로 이 프로그램이 시작된다. [observed]

33.10 바로가기

Shortcut(Shortcut s72, Directory_ s72, .lnk 없는 Name l128 SHORT!Long, Component_ s72, 광고 아닌 바로가기면 Target [#FileKey], Arguments(서식), Description(평문), Hotkey, Icon_, IconIndex, ShowCmd, WkDir = Directory 키)는 CreateShortcuts(4500)가 쓰고 RemoveShortcuts(3200)가 지운다. 바로가기는 대상 파일의 구성 요소에 둔다. 바로가기만을 위해 만든 폴더는 저절로 지워지지 않는다: 그 폴더와 표준 폴더 아래의 부모마다 RemoveFile 행(File Name null, DirProperty = 그 폴더, InstallMode 2)을 더한다. ALLUSERS=1 이면 ProgramMenuFolder 와 DesktopFolder 는 모든 사용자의 시작 메뉴와 공용 바탕화면이다. 설치가 실패하면 바로가기도 폴더도 남지 않고, 복구는 지운 바로가기를 다시 만든다. [observed]

33.11 파일 지우기와 복제

RemoveFile(FileKey s72, Component_ s72, */? 가 든 File Name L255 또는 폴더면 null, DirProperty s72, InstallMode i2: 1 설치, 2 제거, 3 둘 다)은 구성 요소가 설치될 때(1)나 제거될 때(2) RemoveFiles 에서 실행된다. 키 경로 파일이 없고 Directory_ 가 있는 구성 요소가 그 운반자 노릇을 한다. DuplicateFile(FileKey s72, Component_ s72 = 원본의 구성 요소, File_ s72, DestName L255 SHORT!Long, DestFolder S72)은 DuplicateFiles(4210)와 RemoveDuplicateFiles 에서 실행된다. 설치 때 지운 파일은 설치가 실패하면 돌아온다. 설치 전부터 있던 폴더는 비어 있어도 제거할 때 그대로 둔다. [observed]

33.12 환경 변수

Environment(Environment s72, Name l255, 서식 Value L255, Component_ s72)는 WriteEnvironmentStrings(5200)와 RemoveEnvironmentStrings 가 쓴다. Name 접두: * 시스템 변수(없으면 사용자 변수), = 만들거나 정함, - 구성 요소를 제거할 때 되돌림. Value [~];x 는 ;x 를 뒤에 붙이고 x;[~] 는 앞에 붙인다. - 가 있으면 제거할 때 정한 변수는 지우고, 원래 있던 변수에서는 붙인 부분만 떼어 낸다. - 가 없으면 아무것도 되돌리지 않는다. 설치가 실패하면 이전 값이 되살아난다. [observed]

33.13 INI 파일

IniFile(IniFile s72, File Name l255 SHORT!Long, DirProperty S72, Section l96, Key l128, 서식 Value l255, Action i2: 0 줄 더하기, 1 줄 만들기, 3 태그 더하기, Component_ s72)은 WriteIniValues(5100)가 적용하고 구성 요소를 제거할 때 되돌린다. RemoveIniFile(같은 열, Value null 허용, Action 2 줄 지우기, 4 태그 지우기)은 InstallFiles 전에 실행되는 RemoveIniValues 가 처리한다. 태그 더하기는 쉼표 목록에 ,value 를

붙이고, 제거하면 그 항목만 떴다. ASCII 가 아닌 파일·구역·키·값 이름도 된다. 설치가 실패하면 파일이 되살아난다. [observed]

33.14 검색과 설치 조건

두 순서 모두 50 에 있는 AppSearch(**Property, Signature**)는 같은 서명을 가진 찾개(locator)로 속성을 정한다: RegLocator(Root, Key, Name, Type 2 = 원 값, +16 = 64비트 보기), 파일이면 Signature 행 (FileName, MinVersion)을 결들인 DrLocator(Path 는 [System64Folder] 같은 폴더 속성으로 시작해도 된다), 또는 CompLocator(ComponentId, Type 1 = 키 파일). 그 속성은 공개 이름이어야 하고, 서버 쪽까지 가려면 SecureCustomProperties 에 있어야 한다. 두 순서 모두 100 에 있는 LaunchCondition(**Condition**, 서식 Description)은 조건이 거짓이면 그 설명으로 설치를 멈춘다. [observed]

33.15 서비스, 글꼴, 권한

ServiceInstall(Name, 서식 DisplayName 과 Description, ServiceType 0x10, StartType 2/3/4, ErrorControl 1, StartName null 또는 NT AUTHORITY\LocalService, 서식 Arguments, Component_ = exe 의 구성 요소)은 InstallServices(5800)와 함께 쓴다. ServiceControl 의 Event 플래그는 0x1 설치 때 시작, 0x2 설치 때 멈춤, 0x20 제거 때 멈춤, 0x80 제거 때 지움이고 Wait 1, StopServices(1900), DeleteServices(2000), StartServices(5900)와 함께 쓴다. Font(File_, FontTitle null = 글꼴에서 읽음)는 RegisterFonts/UnregisterFonts 와 함께 쓰고, 파일은 FontsFolder 바로 안에 있어야 한다. MsiLockPermissionsEx(MsiLockPermissionsEx, LockObject, Table = CreateFolder, File, Registry, ServiceInstall 가운데 하나, **SDDLText**(열 이름), Condition)는 Windows Installer 5.0 이 필요하다(요약 page count 500). 폴더는 CreateFolder 행을 거쳐 그것을 받는다. 설치가 실패하면 서비스도 글꼴도 남지 않는다. [observed]

33.16 캐비닛, 관리 이미지, 광고

캐비닛이 여럿이면 저마다 Media 행 하나(DiskId 1..n, LastSequence = 그 안 마지막 파일의 순번). 안에 넣은 것은 Cabinet 에 # 와 함께 이름 적은 스트림이고, 밖의 것은 패키지 옆에 있는 파일로 Cabinet 에 # 없이 이름을 적는다(긴 이름도 된다). AdminExecuteSequence(CostInitialize 800, FileCost 900, CostFinalize 1000, InstallValidate 1400, InstallInitialize 1500, InstallAdminPackage 3900, InstallFiles 4000, InstallFinalize 6600)와 AdminUISequence 는 msixexec /a 가 원본처럼 설치되는 압축 없는 이미지를 쓰게 한다. AdvtExecuteSequence(CostInitialize, CostFinalize, InstallValidate, InstallInitialize, PublishFeatures 6300, PublishProduct 6400, InstallFinalize)는 msixexec /jm 이 제품을 광고하게 한다. [observed]

33.17 사용자별 패키지와 겸용 패키지

패키지 하나로 하는 방법: ALLUSERS=2 와 MSIINSTALLPERUSER=1 은 사용자별로 설치한다 - 엔진이 ProgramFilesFolder/ProgramFiles64Folder 를 %LOCALAPPDATA%\Programs 로, 메뉴와 바탕화면 폴더를 그 사용자의 것으로 돌린다 - 그리고 ALLUSERS=1 MSIINSTALLPERUSER="" 는 컴퓨터 전체로 설치한다. 요약 Word Count 비트 8 은 권한 상승이 필요 없다는 뜻이다. Registry Root -1(HKMU)은 사용자별이면 HKCU, 컴퓨터 전체면 HKLM 이다. * 없는 Environment 이름은 사용자 변수다. 사용자별 제품이 설치되고 나면 엔진은 MSIINSTALLPERUSER 를 지우므로, 그것을 보는 설치 조건은 Installed OR ... 로 써야 한다. 그러지 않으면 제품을 더는 제거할 수 없다. 일반적으로 설치 조건은 모두 Installed OR (...) 로 쓴다. [observed]

33.18 사용 중인 파일

바꾸거나 지을 파일을 실행 중인 프로그램이 쥐고 있을 때(그 안에 DLL 이 실려 있을 때):

- 기본인 Restart Manager 를 쓰면 Windows Installer 는 **조용한 설치(/qn)**에서도 그런 파일을 **권 프로그램**을 모두 **끄러** 하고 뒤에 다시 시작한다. 하나라도 닫히지 않으면 설치 전체가 실패한다(1601). [observed]
- Property 표에 MSIRESTARTMANAGERCONTROL=Disable 이 있으면 엔진은 쥐인 파일을 옆으로 옮기고(c:\Config.Msi*.rbf) 새 파일을 곧바로 제자리에 두고 0 을 돌려준다. 프로그램은 옛 사본으로 계속 돌고, 그 뒤에 시작한 프로그램은 새 파일을 싣고, 옛 사본은 다음 재시작 때 지워진다. [observed]

- 명령줄에서 준 속성은 설치하고 있는 패키지에만 간다. 업그레이드 중 옛 판의 제거는 **옛 패키지의** Property 표를 따른다 - 그 속성은 첫 판부터 넣어 둔다. [observed]

33.19 순서

InstallExecuteSequence(대괄호 안은 조건):

```
FindRelatedProducts 25, <다운그레이드 거부 동작> 30 [NEWER_FOUND], CostInitialize 800,
FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, InstallValidate 1400,
InstallInitialize 1500, RemoveExistingProducts 1501, ProcessComponents 1600,
UnpublishFeatures 1800, RemoveFiles 3500, RemoveFolders 3600, CreateFolders 3700,
InstallFiles 4000, RegisterUser 6000, RegisterProduct 6100, PublishFeatures 6300,
PublishProduct 6400, InstallFinalize 6600
```

InstallUISequence: FindRelatedProducts 25, 다운그레이드 거부 동작 30 [NEWER_FOUND], CostInitialize 800, FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, ExecuteAction 1300(그리고 대화창, 대화창 참고).

InstallInitialize 바로 뒤의 RemoveExistingProducts 는 옛 판의 제거를 새 설치의 트랜잭션 안에 넣는다. 그래서 업그레이드가 실패하면 옛 판으로 되돌아간다. [observed: InstallFiles 바로 뒤에 실패하는 지연 사용자 지정 동작이 있으면 업그레이드는 1603 으로 끝나고 옛 판이 파일까지 바이트 그대로 다시 등록된다. 첫 설치가 실패하면 파일도 등록도 남지 않는다.]

33.20 대화창

대화창이 있는 패키지는 다음 표를 가진다(형식은 위와 같다):

표	열
Dialog	Dialog s72, HCentering i2, VCentering i2, Width i2, Height i2, Attributes I4, Title L128, Control_First s50, Control_Default S50, Control_Cancel S50
Control	Dialog_ s72, Control s50, Type s20, X i2, Y i2, Width i2, Height i2, Attributes I4, Property S72, Text L0, Control_Next S50, Help L50
ControlEvents	Dialog_ s72, Control_ s50, Event s50, Argument s255, Condition S255, Ordering I2
ControlCondition	Dialog_ s72, Control_ s50, Action s50, Condition s255
EventMapping	Dialog_ s72, Control_ s50, Event s50, Attribute s50
TextStyle	TextStyle s72, FaceName s32, Size i2, Color I4, StyleBits I2
UIText	Key s72, Text L255
Binary	Name s72, Data v0(스트림)
RadioButton	Property s72, Order i2, Value s64, X i2, Y i2, Width i2, Height i2, Text L0, Help L50
ComboBox	Property s72, Order i2, Value s64, Text L64

엔진이 대화창을 보일 때 검사하는 것(저마다 그 번호를 단 오류 대화창으로 보았다):

- **Tab 순서.** Control_Next 는 그것을 가진 모든 컨트롤을 Control_First 에서 시작해 한 바퀴 도는 고리를 이루어야 한다. 컨트롤 하나를 빠뜨리거나 Control_Next 가 빈 컨트롤에서 끝나는 사슬은 2810 이나 2809 로 설치를 멈춘다. 정적 글씨는 고리에 끼지 않는다. [observed] rubrapack 은 짓는 대화창마다 검사하고 깨진 것은 쓰지 않는다(RP0010).
- **오류 대화창.** ErrorDialog 속성이 가리키는 대화창에는 ErrorText 라는 Text 컨트롤과 ErrorIcon 이라는 Icon 컨트롤이 있어야 한다(없으면 2835). Attributes 에 오류 대화창 비트(0x10000)가 있고, EndDialog 인수 ErrorAbort .. ErrorYes 를 가진 A, C, I, N, O, R, Y 누름단추가 메시지에 필요한 대로 보인다. [observed]
- **문구 속 대괄호.** Text 는 서식 문자열이다: [Next] 는 속성으로 읽혀 아무것도 보이지 않는다. [observed] 본 문서에서 단추 이름은 대신 따옴표로 적는다.
- **사용 중인 파일.** Restart Manager 가 꺼져 있으면 엔진은 FilesInUse 라는 대화창(Retry, Ignore, Exit 로 끝나는 단추)을 FileInUseProcess 에 묶인 ListBox 와 함께 보인다. **ListBox** 표는 **비어 있어도 있어야 한다**: 없

으면 엔진은 2205 를 로그에 남기고 대화창을 건너뛰며, 설치하는 무시를 고른 것처럼 계속된다. 메이저 업그레이드에서는 대화창이 두 번 나온다(새 파일, 그다음 옛 판의 제거). 무시한 뒤에는 쥐인 파일을 옆으로 옮기고 설치는 0 으로 끝난다. [observed]

- **오류 아이콘.** 엔진은 컨트롤의 Binary 가 무엇을 담은 메시지 종류에 맞는 자기 아이콘(설치 조건이면 경고 삼각형)을 ErrorIcon 에 넣는다. [observed]
- **목록 순서.** ComboBox 는 컨트롤에 Sorted 속성(0x10000)이 없으면 항목을 가나다순으로 늘어놓고, 있으면 Order 열을 따른다. [observed] rubrapack 은 그것을 켜다.
- **권한 상승.** 권한 없는 사용자가 대화형으로 시작한 컴퓨터 전체 설치의 실행 순서가 시작될 때 보안 데스크톱에서 동의(UAC)를 묻는다. [observed]
- **코드 페이지.** 데이터베이스 코드 페이지가 65001 이면 한국어 제목, 문구, RTF 사용권 문구가 한국어 시스템 로캘에서도 영어(1252) 시스템 로캘에서도 제대로 보인다. observed, [msi-database.md 참고]

rubrapack 의 세트: 모든 대화창은 370 x 270 대화창 단위이고, 띠(그림이 Binary.RpBanner 인 Bitmap 컨트롤 0,0,370,44)에 굵은 제목({\RpTitle}, TextStyle)과 설명 한 줄이 있고, 44 와 234 에 Line, y 243 에 뒤로/다음/취소가 있다. InstallUISequence 는 환영(1230, NOT Installed)이나 유지 관리 대화창(1240, Installed AND NOT RESUME AND NOT Preselected), 진행 대화창(1280, 모델리스, EventMapping 으로 ActionText 와 SetProgress 를 구독), 그리고 -1(성공), -2(취소), -3(치명적 오류)의 끝 대화창 셋을 더한다. 설치 폴더 대화창은 폴더의 Directory 키를 PathEdit 에 넣고, 그 다음 단추는 NewDialog 전에 경로를 검사하는 SetTargetPath 를 실행한다. 사용권 문구는 RTF 를 담은 ScrollableText 컨트롤이다: 평문은 \uN? 이스케이프(U+FFFF 너머의 글자는 대리쌍)로, 줄마다 \par 하나로 바뀐다.

작성자가 만든 페이지는 같은 틀 속의 보통 Dialog 행이다. Control_Next 는 컨트롤들을 위치 순서대로 지나 뒤로, 다음, 취소로 간다. 라디오 묶음은 단추가 RadioButton 행(묶음 기준 위치)인 RadioButtonGroup 컨트롤이고, 드롭다운 목록은 ComboList(0x20000)와 Sorted 가 있는 ComboBox 컨트롤로 ComboBox 행에서 채운다. 그 속성들은 SecureCustomProperties 에 더해지므로, 대화창에서 고르거나 명령줄에서 준 값이 실행 순서까지 간다. [observed]

33.21 이 조리법이 아직 다루지 않는 것

위의 오류 형식, 등록 쌍, REG_QWORD 도우미 말고 다른 사용자 지정 동작, 그리고 _Validation(검증 도구에는 필요하지만 설치 엔진에는 필요 없다). rubrapack 이 구현하는 대로 이 문서들도 자란다.

34 프로그램 파일: 머신 형식과 판

이 장의 차례

34.1	머리 찾기 [spec]	137
34.2	판 정보 리소스 [spec]	137
34.3	패키지에 드는 것 [observed]	137
34.4	안전하게 읽기	138

패키지 작성기는 담는 PE 파일(.exe, .dll 등)마다 두 가지를 읽는다: 잘못된 아키텍처용 파일을 거부하기 위한 머신 형식, 그리고 Windows Installer 가 판이 있는 파일과 없는 파일을 다르게 다루기 때문에 판 정보 리소스. 배치는 Microsoft PE/COFF 명세와 VS_VERSIONINFO 문서를 따른다. [spec]

34.1 머리 찾기 [spec]

오프셋 0	"MZ"
오프셋 0x3C	u32 e_lfanew: PE 서명의 오프셋
e_lfanew	"PE\0\0"
+4	COFF 머리: u16 machine, u16 섹션 수, ..., u16 섹션 머리 크기(+16)
+24	선택 머리: u16 magic 0x10B(PE32) 또는 0x20B(PE32+); 데이터 디렉터리는 +96(PE32) 또는 +112(PE32+)에서 그 개수 다음에 시작
그다음	섹션 표: 섹션마다 40바이트(가상 크기 +8, 가상 주소 +12, 원 크기 +16, 원 오프셋 +20)

머신 값: 0x8664 x64, 0xAA64 Arm64, 0x014C x86. MZ 로 시작하지만 올바른 PE\0\0 머리가 없는 파일은 보통 데이터 파일이다.

34.2 판 정보 리소스 [spec]

- 데이터 디렉터리 2 가 리소스 표다(RVA 와 크기). RVA 는 섹션 표를 거쳐 파일 오프셋으로 바꾼다.
- 리소스 디렉터리는 세 단계다: 형식 -> 이름 -> 언어. 디렉터리마다 16바이트 머리(이름 항목 수와 ID 항목 수가 +12, +14)와 그 뒤 8바이트 항목(u32 이름 또는 ID, u32 오프셋; 오프셋의 최상위 비트는 하위 디렉터리 표시)이 있다.
- 형식 16(RT_VERSION), 그 첫 이름, 그 첫 언어: 앞은 데이터 항목(u32 RVA, u32 크기)이다.
- 데이터는 VS_VERSIONINFO 다: u16 길이, u16 값 길이, u16 형식, NUL 까지의 UTF-16 키 VS_VERSION_INFO, 4바이트 채움, 그다음 첫 u32 가 0xFEEF04BD 인 VS_FIXEDFILEINFO. dwFileVersionMS(+8)와 dwFileVersionLS(+12)가 판 HIWORD(MS).LOWORD(MS).HIWORD(LS).LOWORD(LS) 를 준다.

34.3 패키지에 드는 것 [observed]

- 판이 있는 파일: File.Version = 네 부분 판, File.Language = MsiGetFileVersion 이 알려 주는 언어, 곧 VarFileInfo\Translation 의 첫 언어(예 1033, 1042). MsiFileHash 행은 없다. 설치된 파일을 다시 읽으면 같은 판이다.
- 판이 없는 파일: 판 없음, MsiFileHash 행 있음(설치·복구·업그레이드·제거가 되는 가장 작은 패키지).
- 머신 형식: x64 패키지는 x64 파일을, Arm64 패키지는 Arm64 파일을, x86 패키지는 x86 파일을 받는다. 일부러 하는 예외(64비트 패키지 속 32비트 도우미)는 조용히 받지 말고 선언하게 한다. 구성 요소 자체의 비트 수는 바뀌지 않는다.

34.4 안전하게 읽기

오프셋과 크기는 쓰기 전에 모두 파일 길이와 대조하고, RVA 는 그 범위 전체를 담은 섹션을 거쳐서만 바꾸며, 기대한 세 디렉터리 단계에서 멈춘다 - 고리나 범위 밖 포인터를 가진 조작된 파일은 흔하다.

35 캐비닛, MSZIP, deflate

이 장의 차례

35.1	캐비닛 배치 [spec: MS-CAB]	139
	블록 체크섬 [spec]	139
35.2	MSZIP [spec: MS-MCI]	139
35.3	deflate 인코더 간단히 [spec: RFC 1951]	140
35.4	실제 예: 튜토리얼의 hello.msi	140

35.1 캐비닛 배치 [spec: MS-CAB]

CFHEADER (예약 필드가 없으면 36바이트):

"MSCF", u32 0, u32 캐비닛 크기, u32 0, u32 첫 CFFILE 오프셋, u32 0,
u8 부판 3, u8 주판 1, u16 폴더 수, u16 파일 수, u16 플래그(0),
u16 세트 id, u16 캐비닛 번호(0)

CFFOLDER (각 8바이트):

u32 폴더의 첫 CFDATA 오프셋, u16 CFDATA 블록 수, u16 압축
(0 = 없음, 1 = MSZIP)

CFFILE (파일마다):

u32 압축 전 크기, u32 폴더의 압축 전 데이터 속 오프셋, u16 폴더 번호,
u16 날짜, u16 시각, u16 속성, NUL 로 끝나는 이름

CFDATA (블록마다):

u32 체크섬, u16 압축 바이트 수, u16 압축 전 바이트 수(최대 32768), 데이터

- MSI 에서 캐비닛 속 파일 이름은 File 표의 키이고 ASCII 다.
- DOS 날짜 (year - 1980) << 9 | month << 5 | day, 시각 hour << 11 | minute << 5 | second / 2. 고정된 날짜(rubrapack 은 1980-01-01 00:00 을 쓴다)가 출력을 결정적으로 지킨다. 속성은 0x20.
- 폴더 하나에는 블록이 65535개(약 2 GiB)까지 든다. 데이터가 더 많으면 폴더나 캐비닛을 더 쓴다.

블록 체크섬 [spec]

checksum(bytes, seed):

바이트 속 온전한 리틀엔디언 u32 를 모두 seed 에 xor 한다;

남은 1~3바이트는 u32 하나를 이루되 남은 바이트 가운데 첫째가 쓰인 자리 가운데 가장 높은 곳에 온다

(3개 남음: b0<<16 | b1<<8 | b2; 2개: b0<<8 | b1; 1개: b0). 그것도 xor 한다.

블록 체크섬 = checksum(저장된 대로의 4바이트 "압축 크기, 압축 전 크기",
checksum(블록 데이터, 0))

체크섬 0 은 "계산하지 않음"을 뜻하고 판독기는 그것을 받는다.

35.2 MSZIP [spec: MS-MCI]

압축 형식이 1 인 폴더의 CFDATA 블록마다 다음이 든다:

"CK" (43 4B), 그다음 이 블록의 압축 전 바이트(최대 32768)에 대한 온전한 원시 deflate 스트림
(RFC 1951) 하나, 마지막 블록으로 끝남.

압축 해제기는 같은 폴더의 앞 블록들을 사전으로 간직하므로, 인코더는 앞 블록을 되돌아 가리켜도 된다. 그러지 않는 인코더(블록마다 따로 압축)도 올바른 MSZIP 을 만든다. rubrapack 은 블록마다 앞 블록의 압축 전 32 KiB 를 미리 준 사전으로 쓴다(섞인 자료에서 블록을 따로 압축할 때보다 약 9% 작다). 그 사전은 출력이 아니라 입력이므로 블록을 여러 스레드에서 동시에 압축할 수 있고, 스레드 수와 상관없이 바이트가 같다.

폴더는 65535블록이 되기 전에 파일 경계에서 새로 시작하고, 사전은 폴더를 넘지 않는다. Windows 는 이것을 모두 풀어낸다: 폴더 열두 개짜리 패키지가 설치·복구·제거되고 모든 파일이 바이트까지 같다. [observed]

35.3 deflate 인코더 간단히 [spec: RFC 1951]

1. **일치 찾기(LZ77):** 다음 3바이트를 해시하고, 32 KiB 안에서 같은 해시를 가진 앞 자리들의 체인을 따라가 가장 긴 일치(3..258바이트)를 고른다. 체인을 길게 따라갈수록 압축이 좋아진다. 다음 자리도 해 보고 그 일치가 더 길면 리터럴을 내보내는 "게으른" 단계가 눈에 띄게 도움이 된다.
2. **블록:** 기호를 수만 개까지 묶는다. 블록마다 세 가지 인코딩의 크기를 계산해 가장 작은 것을 쓴다:
 - 고정 허프만 부호(리터럴/길이 0-143: 8비트, 144-255: 9, 256-279: 7, 280-287: 8; 거리 5비트);
 - 동적 허프만 부호: 부호 길이는 15비트로 제한(부호 길이 알파벳은 7). package-merge 가 제한된 최적 길이를 준다. HLIT, HDIST, HLEN, 16 17 18 0 8 7 9 6 10 5 11 4 12 3 13 2 14 1 15 순서의 부호 길이 부호, 그다음 런LENGTH로 부호화한 길이들(16 = 앞 것 3-6번 반복, 17 = 0 을 3-10개, 18 = 0 을 11-138개)을 보낸다;
 - 저장(바이트 정렬한 LEN, NLEN, 원 바이트; 저장 블록 하나에 최대 65535).
3. 허프만 부호는 최상위 비트부터, 그 밖의 것은 최하위 비트부터 쓴다.
4. 늘 기호 둘 이상에 부호 길이를 준다(외톨이 기호는 가짜 기호와 짝지운다). 그래야 모든 부호가 완전하다 - 불완전한 부호를 거부하는 디코더가 있다.

인코더는 독립된 구현(예: zlib 라이브러리의 원시 deflate, window bits -15)으로 출력을 풀어 보고, 그것으로 지은 MSI 는 설치해서 설치된 파일의 해시를 비교해 확인한다(출력을 Windows 와 대조하기).

35.4 실제 예: 튜토리얼의 hello.msi

튜토리얼 첫 패키지의 캐비닛은 스트림 cab1.cab, 6776 바이트다. CFHEADER:

오프셋	바이트	필드	값
0x00	4d 53 43 46	서명	MSCF
0x08	78 1a 00 00	캐비닛 크기	6776
0x10	2c 00 00 00	첫 CFFILE	44
0x18	03 01	판	1.3
0x1A	01 00	폴더 수	1
0x1C	01 00	파일 수	1
0x1E	00 00	플래그	0
0x20	00 00	묶음 번호	0

0x24 의 CFFOLDER: 42 00 00 00 01 00 01 00 - 첫 CFDATA 는 66, 블록 1개, 압축 1(MSZIP).

CFFILE 항목 - 이름은 File 표의 키다:

오프셋	크기	폴더 안 오프셋	폴더	날짜	속성	이름
0x2C	17920	0	0	1980-01-01	0x20	Hello

66 의 첫 CFDATA: 체크섬 80 35 aa c0, 압축 6702 바이트가 풀면 17920 바이트. 데이터는 43 4b ed 5c 0d 74 로 시작한다: ck, 그리고 deflate. 첫 deflate 바이트 ed = 11101101 의 가장 낮은 비트가 BFINAL = 1, 다음 두 비트가 BTYPE = 2(동적 부호)다.

36 결정적인 정체와 재현 가능한 패키지

이 장의 차례

36.1	이름에서 만드는 GUID(UUID 8판) [spec: RFC 9562]	141
36.2	키	141
36.3	재현 가능한 패키지	141

Windows Installer 는 **변하지 않아야 하는** GUID 와 키로 것들을 식별한다: 같은 구성 요소는 모든 판에서 같은 ComponentId 를 지켜야 하고, 그러지 않으면 복구와 업그레이드가 어긋난다. 손으로 적어 두는 대신 내용에 서 만들어 내면 값이 변하지 않고 빌드도 재현 가능해진다. 이 문서는 rubrapack 의 규칙을 적는다. 변하지 않 기만 하면 다른 규칙도 된다.

36.1 이름에서 만드는 GUID(UUID 8판) [spec: RFC 9562]

```
digest = SHA-256( u32le(len(tag)) || tag
                  || u32le(1)                (형식 판)
                  || 필드마다: u32le(len(field)) || field ) (필드는 UTF-8)
uuid   = digest[0..16], 그다음 바이트 6 = (바이트 6 & 0x0F) | 0x80 (8판)
                  바이트 8 = (바이트 8 & 0x3F) | 0x80 (변종 10)
text   = "{" + hex(바이트 0-3) "-" hex(4-5) "-" hex(6-7) "-" hex(8-9) "-" hex(10-15) + "}"
          대문자, 바이트 순서대로
```

길이 접두는 ("ab","c") 와 ("a","bc") 를 가르고, 꼬리표(tag)는 서로 다른 종류의 정체를 가른다.

정체	꼬리표	필드
ComponentId	rubrapack.component	업그레이드 코드, 범위(machine), 구성 요소 아키텍처, 논리 대상 경로 (<dir id>/<파일 이름>), 키 경로 종류(file), 키 경로 이름(File 키)
ProductCode	rubrapack.product	업그레이드 코드, 아키텍처, 범위, 판의 앞 세 부분
패키지 코드(재현 빌드)	rubrapack.package	패키지 코드를 모두 0 으로 두고 지은 패키지의 SHA-256 16진

ProductCode 는 세 부분 판과 함께 바뀌고 - 메이저 업그레이드에 필요한 것 - ComponentId 는 바뀌지 않는다.

36.2 키

표 키(File, Component, Directory)는 식별자다: [A-Za-z_][A-Za-z0-9_]*, 최대 72자. rubrapack 은 파일과 폴 더에는 작성자의 ID 를 그대로 쓰고 나머지는 끌어낸다:

```
key = 접두 + "_" + SHA-256(논리 이름)의 소문자 16진 앞 20자리
      구성 요소는 접두 C(논리 이름: File 키), 중간 폴더는 D
      (논리 이름: "<표준 폴더>/<부분>/<부분>..."), 와일드카드에서 온 파일은 F
```

36.3 재현 가능한 패키지

시계, 컴퓨터, 해시 표의 순서에 기대는 것이 하나도 없으면 패키지는 바이트까지 재현된다: 고정된 CFB 디렉 터리 시각(0), 고정된 캐비닛 날짜, 문자열의 바이트 순서로 매긴 문자열 id, 키로 정렬한 행, 고정된 순서의 스트림, 무작위 GUID 없음. 패키지 파일마다 달라야 하는 정체는 패키지 코드 하나뿐이다. 재현이 중요하면 (위 처럼) 내용에서 끌어내고, 아니면 무작위로 쓴다.

37 Authenticode 서명

이 장의 차례

37.1	PE 파일에서 서명이 사는 곳	142
37.2	PE 다이제스트	142
37.3	CMS SignedData	142
37.4	MSI 패키지	143
37.5	타임스탬프	144
37.6	ECDSA	144
37.7	Windows 가 알려 주는 것	144
37.8	실제 예: 튜토리얼의 hello.msi	144

Windows 가 서명된 PE 파일(.exe, .dll)이나 MSI 패키지를 받아들이게 하려면 프로그램이 무엇을 써야 하는가. 출처 표시는 구현하는 사람을 위한 패키지 형식과 같다: **[spec]** 은 Microsoft 의 "Windows Authenticode Portable Executable Signature Format", PE/COFF 명세, RFC 5652(CMS), **[observed]** 는 Windows 자신의 서명기 (mssign32.dll 을 쓰는 PowerShell Set-AuthenticodeSignature)가 서명하고 Windows 11 에서 Get-AuthenticodeSignature / WinVerifyTrust 로 확인한 파일이다.

37.1 PE 파일에서 서명이 사는 곳

- 인증서 표: 선택 머리의 데이터 디렉터리 항목 4(항목은 8바이트: 파일 오프셋, 크기). 디렉터리는 PE32(magic 0x10B)면 선택 머리 + 96, PE32+(magic 0x20B)면 + 112 에서 시작하고, NumberOfRvaAndSizes 는 5 이상이어야 한다. 다른 모든 디렉터리 항목과 달리 그 "주소"는 RVA 가 아니라 **파일 오프셋**이다. **[spec]**
- 그 오프셋, 파일 끝에: WIN_CERTIFICATE { u32 dwLength, u16 wRevision = 0x0200, u16 wCertificateType = 0x0002 (PKCS_SIGNED_DATA), bCertificate[] }. bCertificate 는 DER CMS ContentInfo 다. dwLength 는 8바이트 머리를 포함하고 8의 배수로 올리며 늘어난 바이트는 0 이다. 디렉터리 항목의 크기는 dwLength 와 같다. **[spec]** **[observed]**
- 표를 붙이기 전에 파일을 8의 배수까지 0 으로 채운다. 채운 바이트도 해시한다(아래 "섹션 뒤의 데이터"). **[spec]**
- 선택 머리의 CheckSum(선택 머리 + 64)을 고친다. Windows 의 서명기는 다시 계산하지 않고 있던 값을 조정한다: 체크섬이 PE 알고리즘 값이 아니던 파일이면 Windows 는 그 차이를 지키고, 새로 계산하면 그렇지 않다. CheckSum 은 다이제스트 밖이라 둘 다 검증된다. **[observed]**

37.2 PE 다이제스트

SHA-256 으로 이 순서대로: **[spec]**

1. 머리, 0 부터 SizeOfHeaders 까지. CheckSum(4바이트)과 인증서 표 디렉터리 항목(8바이트)은 뺀다.
2. SizeOfRawData > 0 인 모든 섹션을 PointerToRawData 가 커지는 순서로, PointerToRawData 부터 SizeOfRawData 바이트.
3. 그 뒤의 데이터: SizeOfHeaders + sum(SizeOfRawData) 부터 파일 끝에서 인증서 표를 뺀 곳까지.

구멍 둘을 두고 파일을 앞에서부터 해시해도 섹션 사이에 틈이 없을 때만 같은 결과가 나온다. 위 단계가 정의다.

37.3 CMS SignedData

Windows 의 서명기가 쓰고 받아들이는 것: **[observed]**

```

ContentInfo { signedData (1.2.840.113549.1.7.2), [0] SignedData {
  version 1,
  digestAlgorithms SET { sha256 + NULL },
  encapContentInfo { SPC_INDIRECT_DATA (1.3.6.1.4.1.311.2.1.4),
    [0] SpcIndirectDataContent } -- SEQUENCE 그대로, OCTET STRING 없음
  certificates [0] { 서명자의 인증서, 그다음 중간 인증서; 자체 서명 루트 없음 },
  signerInfos SET { SignerInfo {
    version 1, 서명자의 IssuerAndSerialNumber,
    digestAlgorithm sha256 + NULL,
    signedAttrs [0] { SpcSpOpusInfo, contentType, SpcStatementType, messageDigest },
    signatureAlgorithm rsaEncryption + NULL,
    signature (RSA PKCS#1 v1.5) } } } }

```

- SpcIndirectDataContent ::= SEQUENCE { data, messageDigest DigestInfo }. PE 파일이면 data 는 SEQUENCE { SPC_PE_IMAGE_DATA (1.3.6.1.4.1.311.2.1.15), SpcPeImageData } 이고 Windows 는 SpcPeImageData 를 30 09 03 01 00 A0 04 A2 02 80 00 으로 쓴다: 플래그 없음, 그리고 빈 유니코드 문자열인 파일 링크. messageDigest 가 PE 다이제스트를 담는다. [observed]
- **messageDigest** 속성은 SpcIndirectDataContent 의 내용을 그 SEQUENCE 꼬리표와 길이 없이 해시한 것이다 - 요소 전체도 아니고 PE 파일도 아니다. [observed]
- 서명 속성은 꼬리표 [0](0xA0)로 저장하지만 서명은 같은 바이트를 꼬리표 SET(0x31)으로, DER SET OF 로 정렬해 계산한다. Windows 가 쓰는 속성 넷: SpcSpOpusInfo(1.3.6.1.4.1.311.2.1.12) = SEQUENCE { [0] { [0] "" }, [1] { [0] "" } }, contentType = SPC_INDIRECT_DATA, SpcStatementType(1.3.6.1.4.1.311.2.1.11) = SEQUENCE { individual code signing (1.3.6.1.4.1.311.2.1.21) }, messageDigest. 서명 시각은 없다. [observed]
- RSA PKCS#1 v1.5 는 결정적이다: 같은 키로 정확히 이 구조를 쓰는 프로그램은 Windows 와 같은 바이트를 만든다. rubrapack 이 그렇다(위의 CheckSum 차이만 빼고). [observed]

37.4 MSI 패키지

Microsoft 는 MSI 패키지를 어떻게 해시하는지 공개하지 않는다. 아래 규칙은 Windows 의 서명기로 패키지에 서명하고 비교해서 알아냈다: 같은 패키지와 키에 대해 rubrapack 의 서명과 Ex 스트림은 Windows 의 것과 바이트까지 같았고, 디렉터리 항목에 0 아닌 상태 비트·시각·CLSID 를 준 패키지들로 필드를 가려냈다. [observed]

- 서명은 복합 파일 루트의 \005DigitalSignature 스트림이다: PE 파일과 같은 CMS ContentInfo 에 data = SEQUENCE { SPC_SIPINFO (1.3.6.1.4.1.311.2.1.30), SpcSipInfo }. Windows 는 SpcSipInfo 를 SEQUENCE { 2, OCTET STRING <MSI SIP GUID {000C10F1-0000-0000-C000-000000000046} 을 리틀엔디언 바이트 순서로>, 0, 0, 0, 0, 0 } 으로 쓴다.
- Windows 는 \005MsiDigitalSignatureEx 도 쓴다: 32바이트, 복합 파일 디렉터리의 "사전 해시"에 대한 SHA-256: 루트 자신의 시각은 들지 않는다(Windows 는 서명한 파일을 저장할 때 루트의 고친 시각을 정한다).
 - ▶ 루트 항목의 CLSID(16바이트)와 상태 비트(4바이트, 리틀엔디언);
 - ▶ 그다음 루트의 모든 스트림 - 서명 스트림 둘은 빼고 - 을 이름의 UTF-16LE 바이트 비교 순서로: 이름 (UTF-16LE, 끝 NUL 없음), 스트림 크기 8바이트 리틀엔디언, 항목의 만든 시각과 고친 시각을 저장된 그대로(8 + 8바이트). 스트림의 CLSID 와 상태 비트는 들지 않는다.
- SpcIndirectDataContent 의 다이제스트는 다음의 해시다: 32바이트 MsiDigitalSignatureEx 값(그 스트림을 쓸 때), 그다음 같은 이름 순서의 모든 스트림 내용(역시 서명 스트림 둘은 빼고), 그다음 루트 CLSID.
- 루트의 스트림만 관찰했다. 저장소가 있는 패키지(안에 넣은 변환, 하위 데이터베이스)는 여기서 다루지 않고, rubrapack 은 그런 패키지에 서명하지 않는다.
- 서명은 .msi 안에 든 것만 덮는다: 밖의 캐비닛(# 으로 시작하지 않는 Media 표 Cabinet 값)은 다이제스트에 들지 않는다.

37.5 타임스탬프

타임스탬프 없는 서명은 서명 인증서가 만료되면 더는 유효하지 않다. RFC 3161 타임스탬프가 있으면 Windows 는 대신 찍힌 시각에 인증서를 확인한다. `SIGNER_TIMESTAMP_RFC3161` 을 준 `mssign32!SignerTimeStampEx2` 가 쓰는 것: [observed]

- 요청의 메시지 지문은 `SignerInfo` 의 `signature` 값 - OCTET STRING 의 내용, 꼬리표와 길이 없이 - 의 해시 (여기서는 SHA-256)다.
- 서버의 `TimeStampToken(TSTInfo` 위의 CMS SignedData, RFC 3161)은 서명 뒤에 `SignerInfo` 의 서명 안 된 속성으로 들어간다: [1] { SEQUENCE { 1.3.6.1.4.1.311.3.3.1, SET { TimeStampToken } } }.
- 서명의 다른 것은 바뀌지 않는다: 서명 속성, 서명, 바깥 인증서는 전과 같은 바이트이고, 서버의 인증서는 토큰 안에 실려 다닌다. MSI 패키지에서는 `\005DigitalSignature` 스트림이 그만큼 커지고(몇 킬로바이트 토큰이면 미니 스트림에서 보통 섹터로 옮겨 간다), `\005MsiDigitalSignatureEx` 와 다이제스트는 바뀌지 않는다 - 서명 스트림은 거기 들지 않기 때문이다.
- rubrapack 은 정확히 이것을 쓴다. 같은 키와 토큰이면 그 PE 파일은 Windows 의 것과 바이트까지 같고, MSI 서명 스트림도 그렇다.

`Set-AuthenticodeSignature -TimestampServer` 는 대신 옛 형태를 쓴다: PKCS#9 `counterSignature` 속성 (1.2.840.113549.1.9.6)과, 바깥 인증서에 더한 서버의 인증서. Windows 는 둘 다 받아들인다. [observed]

37.6 ECDSA

ECDSA P-256 서명자는 PE 파일, MSI 패키지, MSIX 패키지 모두에 통한다(Windows 11 26100 은 루트를 신뢰하면 셋 모두 Valid 라고 한다). Windows 의 서명기는 `SignerInfo` 의 `signatureAlgorithm` 을 `ecdsa-with-SHA256` 이 아니라 NULL 매개변수의 `id-ecPublicKey(1.2.840.10045.2.1)`로 쓰고, 서명 값은 DER SEQUENCE { INTEGER r, INTEGER s } 로 쓴다. rubrapack 도 결정적 nonce(RFC 6979)로 같게 쓰고, 두 OID 를 모두 읽는다. [observed]

37.7 Windows 가 알려 주는 것

`Get-AuthenticodeSignature` 는 서명하는 쪽에 필요한 경우들을 가려 준다(타임스탬프가 있으면 `TimeStampCertificate` 에 타임스탬프 인증서를 적는다): Valid(다이제스트, 서명, 신뢰하는 루트까지의 사슬), "the certificate chain ... not trusted" 를 단 `UnknownError`(다이제스트와 서명은 맞고, 루트를 이 컴퓨터가 신뢰하지 않음), `HashMismatch`(서명 뒤에 파일이 바뀐), `NotSigned`. 그러니 어떤 루트도 신뢰시키기 전에 서명이 맞는지 확인할 수 있다. [observed]

37.8 실제 예: 튜토리얼의 hello.msi

튜토리얼 첫 패키지의 다이제스트를 위의 규칙대로 키 없이 여기서 계산했다 - 어떤 키로 서명하든 서명에는 이 값이 그대로 들어간다. UTF-16LE 이름 순서로 놓은 스트림들:

#	이름(UTF-16LE)	이름	크기
1	05 00 53 00 75 00 ...	<code>\005SummaryInformation</code>	332
2	26 41 65 38 be 41 ...	<code>cab1.cab</code>	6776
3	40 48 0c 46 f6 45 ...	<code>CustomAction</code>	8
4	40 48 0d 43 35 42 ...	<code>Directory</code>	18
5	40 48 0f 42 e4 45 ...	<code>FeatureComponents</code>	4
6	40 48 0f 42 e4 45 ...	<code>Feature</code>	16
7	40 48 0f 43 2f 42	<code>File</code>	20
8	40 48 16 42 27 43 ...	<code>Media</code>	14
9	40 48 3f 3b f2 43 ...	<code>_Columns</code>	552
10	40 48 3f 3f 77 45 ...	<code>_StringData</code>	1381

#	이름(UTF-16LE)	이름	크기
11	40 48 3f 3f 77 45 ...	_StringPool	448
12	40 48 52 44 f6 45 ...	InstallExecuteSequence	120
13	40 48 52 44 f6 45 ...	InstallUISequence	42
14	40 48 59 45 f2 44 ...	Property	40
15	40 48 7f 3f 64 41 ...	_Tables	32
16	40 48 8c 44 f0 44 ...	Component	12
17	40 48 ca 41 30 43 ...	AdminExecuteSequence	48
18	40 48 ca 41 30 43 ...	AdminUISequence	24
19	40 48 ca 41 f9 45 ...	AdvExecuteSequence	42
20	40 48 de 44 6a 45 ...	Upgrade	32

프리해시는 뿌리의 CLSID 와 상태 비트(20 바이트)에, 스트림마다 이름, 8 바이트 크기, 0 인 시각 둘을 이은 것으로 모두 802 바이트다. 그 SHA-256 이 \005MsiDigitalSignatureEx 의 값이다:

```
99 02 fe fb ce eb 28 a8 61 dc 7d ce 9a a9 83 fe a1 b7 c0 e1 28 5e a6 80 ee e7 f2 fe 73 dd c6 0a
```

SpcIndirectDataContent 의 다이제스트 - 그 값, 같은 순서의 모든 스트림 바이트, 뿌리 CLSID 에 대한 SHA-256:

```
88 20 af 0f be 36 3d e2 1c fc 9d 96 5c f9 40 0d 2c 25 23 69 2f c0 2e ba 1a 17 25 8c bf 03 d8 a2
```

38 출력을 Windows 와 대조하기

이 장의 차례

38.1	데이터베이스가 열리고 풀린다	146
38.2	패키지가 설치된다	146
38.3	쓰기 전에 하는 검사	147
38.4	Microsoft 의 ICE 규칙 [관찰]	147
38.5	멀쩡해 보이지만 아닌 것	147

Microsoft 는 MSI 표 인코딩을 공개하지 않으므로 기준은 Windows 자신이다. 이 확인에는 Windows 컴퓨터와 Windows Installer API(msi.dll)만 있으면 된다. SDK 도구는 필요 없다.

38.1 데이터베이스가 열리고 풀린다

1. 파일을 MsiOpenDatabase 로 읽기 전용으로 연다 - 복합 파일과 문자열 풀이 받아들여진다는 뜻이다.
2. 모든 표(와 _ForceCodepage, _SummaryInformation)를 MsiDatabaseExport 로 IDT 파일로 내보낸다.
3. 자기 판독기가 내보낸 것과, 또는 msi.dll 로 만든 기준 데이터베이스(MsiOpenDatabase 만들기 모드, 65001 이 든 _ForceCodepage.idt 의 MsiDatabaseImport, 뷰를 통한 SQL CREATE TABLE/INSERT)의 IDT 와 비교한다. 바이트까지 같은 IDT 는 인코딩이 맞다는 뜻이다.

38.2 패키지가 설치된다

설치 엔진은 msixexec 보다 프로그램에서 부른다. 그래야 단계마다 동기로 돌고 오류 코드를 돌려준다:

단계	API	기대
조용한 UI	MsiSetInternalUI(INSTALLUILEVEL_NONE)	
자세한 로그	MsiEnableLog(모든 모드)	무엇이 실패하면 읽는다
설치	MsiInstallProduct(path, "")	0
이름	설치된 경로마다 FindFirstFile, 항목 자신의 UTF-16 이름을 정확히 비교	같음
내용	설치된 파일마다 MsiGetFileHash 와 자기 MsiFileHash 행	같음
등록	MsiGetProductInfo(ProductCode, InstalledProductName)	자기 ProductName
복구	설치된 파일 하나를 지우고 MsiReinstallProduct(ProductCode, ...)	파일이 같은 이름으로 돌아옴
업그레이드	판 N 위에 판 N+1 설치	옛 ProductCode 없어지고 새것이 설치됨
다운그레이드	판 N 을 다시 설치	거부(1603, 로그에 거부한 동작의 이름)
기능	MsiQueryFeatureState; MsiConfigureFeature(..., INSTALLSTATE_LOCAL)	INSTALLLEVEL 보다 높은 수준은 켜 때까지 없음
되돌림	InstallFiles 뒤에 실패하는 지연 사용자 지정 동작(형식 34 + 1024, "[SystemFolder]cmd.exe" /c exit 1)과 새 패키지 코드를 가진 판 N+1 의 사본을 N 위에 설치	1603(로그: 오류 1722); N 은 그대로 등록되어 있고 파일도 그대로
제거	MsiConfigureProduct(ProductCode, ..., INSTALLSTATE_ABSENT)	0; 폴더, 키, 등록이 사라짐

컴퓨터 전체 설치에는 관리자 권한 프로세스가 필요하다. 시험 제품은 제 UpgradeCode 를 쓰고 같은 실행 안에서 지운다.

38.3 쓰기 전에 하는 검사

msi.dll 은 깨진 데이터베이스를 많이 받아들이고, 실패하더라도 설치 때에야 실패한다(열 너비조차 강제하지 않는다). 작성기는 제 표를 먼저 검사해야 한다. rubrapack 의 build 는 다음 가운데 하나라도 여기는 패키지는 쓰지 않는다(진단 RP2001-RP2015):

- 칸마다 열의 종류가 맞고, null 을 허용하지 않는 열에 null 이 없고, 문자열은 UTF-16 단위로 센 열 너비보다 길지 않으며(BMP 밖 글자는 둘로 센다) 올바른 UTF-8 이다. 16비트 정수는 -32767..32767, 32비트 정수는 -2147483648 이 아니다(그 값들은 null 을 뜻한다);
- 기본 키가 겹치지 않고, 참조(Component.Directory_, File.Component_, FeatureComponents, CreateFolder, MsiFileHash.File_, Directory_Parent, Feature_Parent)가 있는 행을 가리키며, 순서 표의 동작은 모두 표준 동작, CustomAction, Dialog 가운데 하나다;
- ProductCode, ProductName, ProductVersion, Manufacturer, ProductLanguage 가 있다. GUID 는 중괄호 속 대문자, 구성 요소 GUID 는 서로 다르며, ProductCode 는 UpgradeCode 와 다르다;
- File.Sequence 는 저마다 다르고 어떤 Media.LastSequence 가 덮는다;
- 표준 동작의 순서를 지킨다(비용 계산은 InstallValidate 전, 파일 동작은 InstallInitialize 와 InstallFinalize 사이);
- 64비트 구성 요소 속성(256)이 요약 Template 의 플랫폼(x64/Arm64 대 Intel)과 맞는다;
- 파일 키 경로는 같은 구성 요소의 파일이다;
- Upgrade.ActionProperty 는 저마다 대문자(공개)이고 SecureCustomProperties 에 있다 - 그렇지 않으면 컴퓨터 전체 설치가 그것을 서버 쪽에 넘기지 않는다.

38.4 Microsoft 의 ICE 규칙 [관찰]

Windows SDK 의 "MSI Tools" 에는 MsiVal2 와 darice.cub 가 들어 있다. Microsoft 의 작성 도구들이 돌리는 내부 일관성 검사기(ICE)다: MsiVal2 package.msi darice.cub -f 가 실패한 것을 찍는다. Windows 에서만 돌고, 위의 검사보다 한 걸음 더 간다 - 표의 모양만이 아니라 뜻을 안다. rubrapack 의 시험 패키지(시험에 쓰는 모든 원본, 39개)에서 나오는 것과 그 까닭:

- **ICE03, "Missing specifications in _Validation"** 이 모든 열에: _Validation 표는 검사기만을 위해 열을 설명한다. Windows Installer 는 읽지 않고, rubrapack 은 쓰지 않는다.
- **ICE82**, 겹친 순서 번호: 대화창 언어가 여럿인 패키지의 언어 동작들이 번호 하나를 함께 쓴다. 서로 무관한 속성 설정 동작이라 순서가 상관없다.
- **ICE34**, 언어 페이지 라디오 속성에 Property 행이 없다: 일부러다. 명령줄(RPLANGUAGE=ko)이 감지보다 이겨야 하고, 감지는 NOT RPLANGUAGE 일 때만 돈다.
- **ICE43, ICE57**, 파일을 키 경로로 쓰는 컴포넌트의 광고되지 않은 바로가기: 이 규칙들은 바로가기 폴더를 사용자별로 가정한다. 컴퓨터 전체 패키지(ALLUSERS=1)에서 시작 메뉴와 바탕화면은 모든 사용자 폴더이고, 사용자별·점용 패키지는 사용자마다 따로 설치되므로, 이 규칙이 막으려는 "첫 사용자만" 문제는 생기지 않는다.
- **ICE52**, AppSearch 의 비공개 속성(RpFound_<ID>, 기억하는 폴더): 명령줄이 정하지 못하게 일부러 비공개다. AppSearch 는 두 순서에서 모두 돌므로 서버 쪽에 넘길 필요가 없다.

셋은 진짜 결함이었고 고쳤다(0.4.3): 완료·취소·실패 페이지가 AdminUISequence 에 없어 전체 UI 의 관리 설치(msiexec /a)가 그 페이지 없이 끝났다(ICE20). 폴더를 키 경로로 쓰는 컴포넌트(INI 항목, [remove])가 CreateFolder 에 없었다(ICE18). 대화창이 있는 패키지에 ControlCondition 표가 없었는데, ICE17 은 대화창마다 이 표를 읽는다(검사 대신 오류 2228 로 멈췄다).

38.5 멀쩡해 보이지만 아닌 것

- msi.dll 이 만든 데이터베이스가 완전한 명세는 아니다: msi.dll 은 65001 요약 문자열을 제대로 저장하고도 다시 읽지 못한다(요약 정보).
- MsiDatabaseExport 는 요약 날짜를 지역 시각으로 찍는다. 날짜는 UTC 로 비교한다.
- MsiOpenDatabase 가 성공했다고 행 순서나 참조 수가 맞다는 뜻은 아니다 - 그것은 설치만이 말해 준다. 열어 보기만 하지 말고 설치해서 시험한다.

39 MSIX 패키지

이 장의 차례

39.1	ZIP 압축 파일	148
39.2	블록 맵(AppxBlockMap.xml)	148
39.3	[Content_Types].xml	149
39.4	매니페스트(AppxManifest.xml)	149
39.5	가상 레지스트리(Registry.dat, User.dat)	149
39.6	가상 파일 시스템(VFS\...)	150
39.7	확장	150
39.8	묶음(.msixbundle)	151
39.9	서명(AppxSignature.p7x)	151
39.10	서명 없는 패키지 설치하기	152
39.11	실제 예: 튜토리얼의 hello.msix	152

Windows 가 MSIX 패키지를 읽고, 설치하고, 실행하게 하려면 프로그램이 무엇을 써야 하는가. 출처 표시는 구현하는 사람을 위한 패키지 형식과 같다: **[spec]** 은 Microsoft Learn(패키지 매니페스트와 블록 맵 스키마)과 ECMA-376 Part 2(Open Packaging Conventions), **[observed]** 는 Windows 자신의 패키징 API(Windows 의 일부인 AppxPackaging.dll 의 IAppxFactory/IAppxPackageWriter)가 쓴 패키지, 그리고 IAppxPackageReader 로 다시 읽고 Windows 11 에서 Add-AppxPackage 로 설치한 rubrapack 의 패키지다. rubrapack 은 여기 적은 것을 쓴다.

39.1 ZIP 압축 파일

- 페이로드 파일이 먼저, 그다음 AppxManifest.xml, AppxBlockMap.xml, [Content_Types].xml 이 그 순서로 온다. [observed]
- 모든 항목은 크기와 상관없이 ZIP64 다: 로컬 머리는 필요 판 4.5, 플래그 0x0008(데이터 서술자), CRC 와 크기 0, extra 필드 없음. 데이터 뒤에는 ZIP64 데이터 서술자(PK\7\8, CRC-32, 압축 크기와 원 크기 각 8 바이트)가 온다. 중앙 디렉터리 항목은 만든 판과 필요 판이 4.5(MS-DOS), 크기와 오프셋이 0xFFFFFFFF 이고 ZIP64 extra 필드(0x0001, 24바이트: 원 크기, 압축 크기, 로컬 머리 오프셋)를 가진다. 압축 파일은 ZIP64 끝 레코드(크기 44), 그 로케이터, 그리고 개수와 오프셋이 모두 0xFFFF / 0xFFFFFFFF 인 끝 레코드로 끝난다. [observed]
- 방식: 저장(0)과 deflate(8). AppxManifest.xml, AppxBlockMap.xml, [Content_Types].xml 은 페이로드가 저장되어도 deflate 한다. [observed]
- 항목 이름은 OPC 부분 이름이다: 폴더 사이는 /, A-Z a-z 0-9 - . _ ~ 밖의 모든 바이트는 UTF-8 바이트까지 퍼센트 인코딩한다 - data\<U+C790> %(1).txt(한글 음절, 공백, %, #, 괄호)는 data/%EC%9E%90%20%25%23%281%29.txt 로 저장한다 - 그리고 UTF-8 이름 플래그는 켜지 않는다. [Content_Types].xml 은 대괄호를 그대로 둔다. [observed]
- 머리의 날짜는 읽지 않는다. Windows 는 쓰는 시각을, rubrapack 은 1980-01-01 00:00 을 써서 빌드마다 패키지가 바뀌지 않게 한다. [observed]

39.2 블록 맵(AppxBlockMap.xml)

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<BlockMap xmlns="http://schemas.microsoft.com/appx/2010/blockmap"
  xmlns:b4="http://schemas.microsoft.com/appx/2021/blockmap" IgnorableNamespaces="b4"
  HashMethod="http://www.w3.org/2001/04/xmlenc#sha256">
```

```

<File Name="data\text.txt" Size="218890" LfhSize="43">
  <Block Hash="(원 바이트 65536개의 base64 SHA-256)" Size="(이 블록의 압축 바이트 수)"/>
  ...
  <b4:FileHash Hash="(파일 전체의 base64 SHA-256)"/>
</File>
...
</BlockMap>

```

- 페이로드 파일마다 File 하나, AppxManifest.xml 에도 하나. 블록 맵과 [Content_Types].xml 은 신지 않는다. Name 은 적은 그대로의 경로로 \ 를 쓰고 퍼센트 인코딩하지 않는다. Size 는 원 크기, LfhSize 는 로컬 머리의 크기(30 + ZIP 이름 길이)다. 빈 파일에는 Block 이 없다. [spec] [observed]
- 원 데이터 65536바이트마다 Block 하나. Hash 는 그 원 블록의 base64 SHA-256 이다. [spec]
- deflate 한 파일은 블록마다 독립된 원시 deflate 조각 하나다: 조각마다 새 디코더로 풀리고(앞 블록을 되돌아 가리키는 것이 없다) 빈 저장 블록(00 00 FF FF)으로 끝나며, 마지막 조각 뒤에는 빈 마지막 블록(03 00)이 온다. 블록의 Size 는 그 조각의 바이트 수라서, 파일의 압축 크기는 Size 합 더하기 2 다(빈 파일은 2바이트, 블록 없음). 저장 파일의 블록에는 Size 가 없다. 압축되지 않는 데이터도 (저장 블록으로) deflate 한다. [observed]
- b4:FileHash 는 블록이 둘 이상인 파일에만 나온다. [observed]
- Windows 의 판독기는 파일을 읽으면서 블록마다 해시를 확인하고, 파일이 블록 맵과 다른 패키지는 읽을 때도 설치할 때도 거부한다. [observed]

39.3 [Content_Types].xml

한 줄: 처음 쓰인 순서대로의 파일 확장자(소문자)마다 Default 하나, xml 은 application/vnd.ms-appx.manifest+xml, 그리고 application/vnd.ms-appx.blockmap+xml 인 Override PartName="/AppxBlockMap.xml". rubrapack 은 확장자 없는 파일마다 Override 를, 그리고 페이로드의 .xml 파일이 xml 기본을 차지했으면 / AppxManifest.xml 에 Override 를 더한다. [observed]

39.4 매니페스트(AppxManifest.xml)

Windows 가 설치하고 시작하는 가장 작은 데스크톱 앱(네임스페이스 foundation/windows10, uap/windows10, restrictedcapabilities): [spec] [observed]

- Identity: Name(A-Z a-z 0-9 . - 로 3~50자), Publisher(서명 인증서의 주체), Version(네 부분, 각 65535 이하), ProcessorArchitecture(x64, arm64, x86).
- Properties: DisplayName, PublisherDisplayName, Logo(패키지 속 PNG).
- MinVersion 과 MaxVersionTested 를 가진 Dependencies/TargetDeviceFamily Name="Windows.Desktop".
- Resources/Resource Language.
- uap:VisualElements(DisplayName, Description, BackgroundColor, Square150x150Logo, Square44x44Logo)를 가진 Application Id Executable EntryPoint="Windows.FullTrustApplication", 그리고 제한된 기능 runFullTrust.
- 매니페스트 속 경로는 \ 를 쓰고 패키지의 파일을 가리킨다.

39.5 가상 레지스트리(Registry.dat, User.dat)

패키지 뿌리의 레지스트리 하이브(REGF 형식: 레지스트리 하이브 파일(REGF)). Windows 11(26100)에서 패키징된 앱으로詹 것: [observed]

- Registry.dat: 그 REGISTRY\MACHINE\SOFTWARE 키가 HKLM\Software 를 나타낸다(하이브의 뿌리 자체는 아니다). REGISTRY\MACHINE\SOFTWARE\WOW6432Node\... 는 앱이 32비트 보기에서 보는 것이다. Microsoft 의 설명("registry.dat serves as the logical equivalent of HKLM\Software")은 앱이 보는 것에 대한 말이지 하이브의 배치에 대한 말이 아니다.
- User.dat: 그 뿌리가 HKCU 를 나타낸다(그 아래 Software\... 가 HKCU\Software\...).
- 앱은 패키지의 키를 실제 레지스트리와 합쳐 읽는다. 컴퓨터의 레지스트리에는 아무것도 쓰지 않고, 제거한 뒤에는 아무것도 남지 않는다.

- Windows의 패키징 도구는 이 하이브를 오프라인 레지스트리 라이브러리로 쓴다(그 표시 "OfRg"가 기본 블록에 있다).

39.6 가상 파일 시스템(VFS\...)

VFS\<폴더> 아래의 파일은 앱에게 실제 자리에 있는 것으로 보이고, 실제 폴더는 바뀌지 않는다. 켄 것: ProgramFilesX64(%ProgramFiles%), SystemX64(System32), Common AppData(%ProgramData%) - 그리고 Microsoft Learn이 말하는 대로 AppData나 Local AppData에는 없다. rubrapack이 쓰는 다른 이름(ProgramFilesX86, ProgramFilesCommonX64/X86, SystemX86, Windows)은 Microsoft Learn이 늘어놓은 것이다. [spec] [observed]

39.7 확장

rubrapack이 앱의 <Extensions>(uap:VisualElements 뒤)에 쓰는 것, 그리고 이것뿐이다: 여기 행이 없는 원본 기능은 오류이고, 필요한 Windows 빌드가 패키지의 MinVersion보다 높은 것은 min-version을 올리라고 요구한다. [spec] Microsoft Learn, "Integrate your desktop app with Windows using packaging extensions"와 요소 문서들. 마지막 열은 Windows 11(26100)에 패키지를 설치해 확인한 것이다. [observed]

원본	요소(범주)	네임스페이스	최소 빌드	기능	Windows 11에서 확인한 것
[assoc]	uap:Extension windows. fileTypeAssociation > uap3: FileTypeAssociation(Name, Parameters) > uap:DisplayName, uap: SupportedFileTypes > uap:FileType	uap, uap3	14393	runFullTrust	.rpx/.rpy 파일을 열면 프로그램이 그 파일과 함께 시작된다
[protocol]	uap3:Extension windows.protocol > uap3:Protocol(Name, Parameters)	uap3	14393	runFullTrust	scheme: URI를 시작하면 프로그램이 그것과 함께 시작된다
[msix-extension] 별칭	uap3:Extension windows. appExecutionAlias(Executable, EntryPoint) > uap3: AppExecutionAlias > desktop: ExecutionAlias(Alias)	uap3, desktop	14393	runFullTrust	별칭이 %LOCALAPPDATA%\Microsoft\WindowsApps에 생기고 프로그램을 시작한다
[msix-extension] 시작 작업	desktop:Extension windows. startupTask(Executable, EntryPoint) > desktop: StartupTask(TaskId, Enabled, DisplayName)	desktop	14393	runFullTrust	첫 시작 뒤에 등록된다
[shortcut] Desktop	desktop7:Extension windows.shortcut > desktop7:Shortcut(File \$(Desktop)\<이름>.lnk, Icon, Arguments, Description)	desktop7	19645	runFullTrust	사용자의 바탕화면에 바로가기 생성하고 프로그램을 시작한다
[shortcut] Programs, StartMenu	없음: 앱 자신의 시작 메뉴 항목	-	-	-	시작 메뉴에 앱이 있다
[font]	uap4:Extension windows.sharedFonts > uap4:SharedFonts > uap4:Font(File Fonts\<이름>), 첫 앱에	uap4	15063	-	패키지가 설치되어 있는 동안 다른 프로그램이 글꼴을 보고, 제거하면 보지 못한다

- 네임스페이스는 쓸 때만 Package 에 선언하고 무시 가능(ignorable)으로 둔다. 그래서 확장 없는 패키지는 위의 매니페스트를 그대로 지킨다.
- Parameters, Arguments: 리터럴 글(MSI 의 [...]) 는 거부). Windows 는 %1 자리에 파일이나 URI 를 넣는다.
- FileTypeAssociation 의 Name: prog-id 를 소문자로. prog-id 를 같이 쓰는 [assoc] 표들은 FileType 이 여러 인 연결 하나가 된다.
- 모든 것은 원본이 대상으로 적은 실행 파일을 가진 앱에 들어간다. 앱의 실행 파일이 아닌 대상은 오류다.

39.8 묶음(.msixbundle)

Windows 의 묶음 작성기(IAppxBundleWriter)가 만드는 대로, 패키지처럼 놓인 ZIP 압축 파일(데이터 서술자를 가진 ZIP64 항목): [observed]

- 패키지들이 먼저, 저장(방식 0)으로, 그 파일 이름으로, 더한 순서대로. 그다음 AppxMetadata/AppxBundleManifest.xml, AppxBlockMap.xml, [Content_Types].xml 을 deflate 로.
- 블록 맵은 묶음 매니페스트(AppxMetadata\AppxBundleManifest.xml)만 싣는다. 패키지들은 제 블록 맵을 가진다.
- [Content_Types].xml: Default msix = application/vnd.ms-appx, Default xml = application/vnd.ms-appx.bundlemanifest+xml, 그리고 /AppxBlockMap.xml 의 Override.
- 묶음 매니페스트, 줄 끝은 CRLF, 들여쓰기는 탭, 마지막 꼬리표 뒤에는 줄 끝이 없다:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<Bundle xmlns="http://schemas.microsoft.com/appx/2013/bundle" SchemaVersion="5.0" xmlns:b4="http://schemas.microsoft.com/appx/2018/bundle" xmlns:b5="http://schemas.microsoft.com/appx/2019/bundle" IgnorableNamespaces="b4 b5">
  <Identity Name="Example.App" Publisher="CN=Example" Version="1.0.0.0"/>
  <Packages>
    <Package Type="application" Version="1.0.0.0" Architecture="x64" FileName="Example.App_1.0.0.0_x64.msix" Offset="66" Size="61340">
      <Resources>
        <Resource Language="en-US"/>
      </Resources>
      <b4:Dependencies>
        <b4:TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.17763.0" MaxVersionTested="10.0.26100.0"/>
      </b4:Dependencies>
    </Package>
  </Packages>
</Bundle>
```

- Offset 은 묶음에서 패키지의 바이트가 시작하는 곳(로컬 머리 다음), Size 는 그 길이다. Resources 와 Dependencies 는 패키지 매니페스트의 Resource 와 TargetDeviceFamily 요소를 되풀이한다.
- 묶음의 Version 은 작성기의 인수(부분마다 16비트인 64비트 수)다. rubrapack 은 패키지들의 판을 준다. 모든 패키지는 묶음의 Name 과 Publisher, 한 가지 판, 그리고 저마다 다른 아키텍처를 가져야 한다.
- 같은 패키지들로 만든 rubrapack 의 묶음은 같은 항목을 같은 오프셋에 가지고, 매니페스트·블록 맵·콘텐츠 형식이 바이트까지 같다. ZIP 날짜만 다르다(rubrapack 은 1980). Windows 는 그 가운데 자기 아키텍처의 패키지를 설치한다: x64 컴퓨터는 x86 과 arm64 가 있어도 x64 를, x86 만 든 묶음에서는 x86 을. [observed]

39.9 서명(AppxSignature.p7x)

Windows 의 서명기(APPX_SIP_CLIENT_DATA 를 준 mssign32!SignerSignEx2; PowerShell 명령은 패키지에 서명하지 못한다)와 대조해 알아내고, rubrapack 이 서명한 패키지를 설치해 확인했다. [observed]

- 매니페스트의 Publisher 는 Windows 가 보여 주는 방식으로 쓴 서명 인증서의 주체와 같아야 한다: RDN 을 마지막부터 처음으로, A=value 를 , 로 이어서 - /CN=Example/O=Example Ltd 로 만든 인증서는 O=Example Ltd, CN=Example 이 필요하다. 그렇지 않으면 서명이 0x8007000B 로 실패한다.
- 서명은 서명기가 하는 대로 압축 파일을 다시 쓴다: 페이로드의 로컬 레코드, 매니페스트, 블록 맵은 그대로 두고, [Content_Types].xml 에 <Override PartName="/AppxSignature.p7x" ContentType="application/vnd.ms-

appx.signature"/> 를 더하고, 서명을 맨 끝에 deflate 로, 크기를 로컬 머리에 담아(필요 판 2.0, 데이터 서술자 없음)더한다. 중앙 디렉터리는 크기와 오프셋이 허락하는 한 ZIP32 방식(ZIP64 extra 필드 없음)으로 쓰고, 끝 레코드의 디스크 번호는 0 이다.

- AppxSignature.p7x 는 PKCX 뒤에 Authenticode 서명과 같은 CMS SignedData 가 오되, 다음이 다르다: data 는 SEQUENCE { SpcSipInfo (1.3.6.1.4.1.311.2.1.30), SEQUENCE { INTEGER 0x01010000, OCTET STRING <SIP GUID>, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0 } } 이고 패키지 SIP GUID 바이트는 4B DF C5 0A 07 CE E2 4D B7 6E 23 C8 39 A0 9F D1, 묶음 SIP GUID 바이트는 B3 58 5F 0F DE AA 9A 4B A4 34 95 74 2D 92 EC EB 다. 서명 속성은 contentType 과 messageDigest 뿐이다.
- DigestInfo 의 다이제스트는 해시 하나가 아니라 APPX 뒤에 4바이트 꼬리표와 SHA-256 의 기록들이 오는 것이다:
 - ▶ AXPC: 압축 파일 처음부터 서명 항목의 로컬 머리까지;
 - ▶ AXCD: 서명 항목을 뺀 중앙 디렉터리, 그다음 ZIP64 끝 레코드, 그 로케이터, 끝 레코드 - 모두 서명 항목이 없는 것처럼(중앙 디렉터리가 서명의 로컬 머리 자리에서 시작하는 것처럼). Windows 는 서명 항목의 자리를 그 ZIP32 필드에서 읽는다: ZIP64 방식으로 기술한 서명 항목(거기 0xFFFFFFFF)은 HashMismatch 이고, 서명한 뒤 중앙 디렉터리를 그렇게 다시 쓴 패키지도 그렇다;
 - ▶ AXCT: [Content_Types].xml(원 바이트); AXBM: AppxBlobMap.xml;
 - ▶ AXCI: AppxMetadata/CodeIntegrity.cat, 패키지에 그것이 있을 때만.
- Windows 의 서명기는 패키지에 프로그램 파일이 있으면 AppxMetadata/CodeIntegrity.cat - 같은 키로 서명한, 패키지 프로그램 파일의 카탈로그 - 도 더한다. 그것은 선택이다: 그것 없이 서명한 패키지도 설치되고 실행된다. rubrapack 은 그것을 쓰지 않고, 있으면 AXCI 를 확인한다.
- 묶음: 안의 모든 패키지를 먼저 서명하고(저마다 제 AppxSignature.p7x), 그 둘레에 묶음을 쓰고, 묶음 SIP GUID 로 AXCI 없이 서명한다.
- AllowUnsigned 와 publisher OID(아래)가 필요한 서명 없는 패키지는 서명하지 않는다: 서명은 그 OID 를 가진 publisher 를 거부한다.

39.10 서명 없는 패키지 설치하기

- Publisher 가 OID.2.25.311729368913984317654407730594956997722=1 로 끝나야 한다. 그러면 Add-AppxPackage -AllowUnsigned 가 Windows 11 에 설치한다. [spec]
- 실행 파일이 든 패키지는 관리자 권한 프로세스가 필요하다(아니면 0x80073D2B: 서명 없는 패키지는 실행 파일 활성화를 가질 수 없다). 개발자 모드는 필요 없다. [observed]
- 네트워크 로그온(SSH 세션)에서 Add-AppxPackage 는 "PLM initialization" 에서 0x80070005 로 실패하고, 대화형 세션에서는 된다. [observed]
- 파일 형식이 든 패키지를 제거하면 HKCU\Software\Classes\.<ext> 아래에 빈 OpenWithProgids 키가 남는다 - 패키지가 아니라 Windows 가 하는 일이다. [observed]

39.11 실제 예: 튜토리얼의 hello.msix

튜토리얼 17장의 MSIX(--unsigned-test, x64)는 11009 바이트다. 첫 파일 hello.exe 의 로컬 머리로 시작한다:

오프셋	바이트	필드	값
0x00	50 4b 03 04	서명	PK\3\4
0x04	2d 00	필요한 판	45 (4.5, ZIP64)
0x06	08 00	플래그	0x0008: 크기는 데이터 뒤에
0x08	08 00	방법	8 (deflate)
0x0A	00 00 21 00	시각, 날짜	1980-01-01 00:00
0x0E	00 00 00 00 00 00 00 00 00 00 00 00	CRC, 크기	0 (데이터 기술자에)
0x1A	09 00 00 00	이름, 추가 필드 길이	9, 0
0x1E	68 65 6c 6c 6f 2e 65 78 65	이름	hello.exe

hello.exe 의 deflate 된 6706 바이트가 뒤따르고, 이어서 데이터 기술자가 온다: 50 4b 07 08 ac a3 c6 2e 32 1a 00 00 00 00 00 00 46 00 00 00 00 00 00 00 - PK\7\8, CRC-32 2EC6A3AC, 그리고 압축된 크기와 원래 크기를 8 바이트씩(6706, 17920).

AppxBlockMap.xml 에 있는 그 항목:

```
<File Name="hello.exe" Size="17920" LfhSize="39">
  <Block Hash="K7HbHzXLkhwTo3dcUr0vs35DLYX27qM0exD0TR8e2k=" Size="6704"/>
```

파일이 64 KiB 보다 작아 블록은 하나다. Hash 는 원래 17920 바이트의 SHA-256 을 base64 로 쓴 것(여기서 계산한 값 K7HbHzXLkhwTo3dcUr0vs35DLYX27qM0exD0TR8e2k=), Size 는 그 deflate 부분의 6704 바이트 - 압축된 크기 6706 에서 2 바이트 마지막 블록을 뺀 것 - 이고, LfhSize 는 로컬 머리의 39 바이트(30 + 이름 9 바이트)다.

40 레지스트리 하이브 파일(REGF)

이 장의 차례

40.1	배치	154
40.2	기본 블록	154
40.3	키(nk)	155
40.4	하위 키 목록(lh, ri)	155
40.5	값(vk)과 값 목록	155
40.6	보안 칸(sk)	155
40.7	재현 가능한 하이브	155

MSIX 패키지 속 Registry.dat 와 User.dat 의 파일 형식이다. Microsoft 는 이것을 공개하지 않는다. 여기의 모든 사실은 **[observed]** 다: 고른 키/값 집합으로 Windows 의 오프라인 레지스트리 라이브러리(offreg.dll, 10.0.26100, Windows 패키징 도구가 쓰는 작성기 - 그 표시가 기본 블록에 있다)가 쓴 하이브, 그리고 같은 집합으로 rubrapack 이 쓴 하이브를 Windows 에서 OROpenHive 와 RegLoadAppKey 로 읽어 Windows 가 자기 하이브에서 늘어놓는 것과 정확히 같은 키와 값이 나오는 것을 확인했다. 새로 쓴 하이브에 필요한 것만 적는다: 트랜잭션 로그, 휘발성 키, 빈 칸 재사용은 없다.

40.1 배치

- 4096바이트 기본 블록, 그다음 하이브 bin 들. 칸 속의 오프셋("칸 오프셋")은 기본 블록 끝에서부터 센다 (파일 오프셋 - 4096). "없음"은 0xFFFFFFFF 다.
- bin: hbin, u32 자신의 칸 오프셋, u32 크기(4096 이나 그 배수), 0 인 8바이트, u64 시작(0), u32 여분(0); 모두 32바이트. 그 뒤에 칸이 온다. 지금 bin 에 들어가지 않는 칸은 그것을 담을 크기의 새 bin 을 시작하고, bin 의 나머지는 빈 칸 하나다.
- 칸: i32 크기(쓰는 중이면 음수, 8의 배수, 크기 필드 포함), 그다음 내용. 칸은 제 bin 의 끝을 넘지 않는다.

40.2 기본 블록

오프셋	필드	값
0	서명	regf
4, 8	순서 번호	1, 1 (다르면 = 깨끗이 닫히지 않음; 판독기는 거부하거나 로그를 다시 적용)
12	u64 시작	0 도 받는다
20, 24	판	1, 5
28, 32	종류, 형식	0, 1
36	루트 키의 칸 오프셋	
40	모든 bin 의 크기	
44	클러스터링	1
0xB0	offreg 의 표시	0fRg, 그다음 u32 1
508	체크섬	앞의 dword 127개의 XOR; 0 은 1 로, 0xFFFFFFFF 는 0xFFFFFFFFE 로 쓴다

나머지는 모두 0 이다.

40.3 키(nk)

nk, u16 플래그, u64 시각, u32 접근 비트(0), u32 부모(루트면 없음), u32 하위 키 수, u32 휘발성 하위 키 수(0), u32 하위 키 목록, u32 휘발성 목록(없음), u32 값 수, u32 값 목록, u32 보안 칸, u32 클래스(없음), u32 가장 긴 하위 키 이름, u32 가장 긴 클래스(0), u32 가장 긴 값 이름, u32 가장 큰 값 데이터, u32 작업 변수(0), u16 이름 길이, u16 클래스 길이(0), 그다음 이름.

- 플래그: 이름을 Latin-1 로 저장하면("압축": 모든 UTF-16 단위가 256 미만) 0x20, 아니면 이름은 UTF-16LE 다. offreg 하이브의 루트 키 이름은 ROOT 이고 플래그는 0x20 뿐이다.
- 가장 긴 이름들은 저장 형태와 상관없이 UTF-16 바이트로 센다.
- 이름은 대소문자 없이 비교한다: UTF-16 단위마다 유니코드 단순 대문자로 바꾼다.

40.4 하위 키 목록(lh, ri)

- lh, u16 수, 그다음 하위 키마다: u32 키 칸, u32 해시. 대문자로 바꾼 이름순 정렬.
- 해시: $h = 0$; 이름의 UTF-16 단위마다 $h = h * 37 + \text{upper}(\text{unit})(32\text{비트에서 넘침})$.
- 하위 키가 507개보다 많으면: 507개까지 담은 lh 목록들의 ri(u16 수, lh 목록마다 u32).

40.5 값(vk)과 값 목록

- 값 목록은 u32 값 칸 오프셋을 값을 정한 순서대로 담은 보통 칸이다.
- vk, u16 이름 길이, u32 데이터 크기, u32 데이터 오프셋, u32 형식, u16 플래그, u16 여분, 그다음 이름. 플래그 1 = Latin-1 이름. 이름 없는 기본값은 이름이 비고 플래그가 0 이다.
- 4바이트 이하의 데이터(빈 값 포함)는 데이터 오프셋 필드에 두고 크기의 최상위 비트를 켜다.
- 16344바이트까지: 데이터 칸 하나. 16345바이트부터: db 칸(db, u16 조각 수, u32 조각 목록), u32 조각 칸들의 목록 칸, 저마다 16344바이트인 조각들 - 마지막 것까지 모든 조각 칸을 온 크기로 잡는다: 마지막 조각 칸이 더 짧으면 Windows 는 그 값을 읽다 멈춘다.

40.6 보안 칸(sk)

모든 키가 나눠 쓰는 sk 칸 하나: sk, u16 0, u32 다음과 u32 이전(둘 다 자기 자신), u32 참조 수(키의 수), u32 서술자 길이, 그다음 자기 상대(self-relative) 보안 서술자. offreg 의 것: 소유자와 그룹은 Administrators; SYSTEM 과 Administrators 모든 권한, Everyone 과 RESTRICTED 읽기, 모두 컨테이너 상속.

40.7 재현 가능한 하이브

시각 필드를 모두 0 으로 두고 칸을 깊이 우선으로 놓으면(키; 루트 키 다음에는 보안 칸; 키의 하위 키 목록, 값 목록, 값마다 그 뒤에 데이터; 그다음 하위 키들을 정렬 순서로) 같은 키와 값은 어느 호스트에서나 같은 바이트가 된다. Windows 는 그런 하이브를 아무 불평 없이 읽는다.